

ACERVATOR

User Manual and Feature Design Intention Guide

Turbator aequilibrii dissolvendus reformandusque.

"The disturber of the balance is to be dissolved and reformed."

"You are either gaining more territory cheaper or you are selling for higher than you bought it in direct response to the market's moment to moment volatility." - Ekthelius

Contents

Part 1 Frontmatter & Overview	8
Copyright and Attribution	8
Contact	8
Legal Disclaimers	8
Risk Factors Specific to Algorithmic Trading	9
Executive Summary	9
What Acervator Is and What It Is Aspiring To Be...	9
Product Manual Parts	11
Part 2 Novel Concepts and Patent Candidate Portfolio	13
Novel Concepts and Patent Candidate Catalogue	13
1 - Harvest-Fold / Scrumming	13
2 - The Phantom Bot	16
3 - The Landing Strip	17
4 - Bollinger Band Travel Detection	19
5 - Position-Aware Technical Analysis	19
6 - Market Inspector	20
7 - Bot Swarm and Smart Wire Network w/ Provenance Tracking	21
8 - Hunger and Satiety Indices	22
9 - Accumulation Shadows	23
10 - Charge Up Permission Gate	24
11 - Fold, Stack, Extractor, and Shadow Tranches / Provenance Queue	25
12 - Technical Analysis Indicator Confidence Tiers	26
13 - Initial-Purchase-Price Floor on Folds	27
14 - Position Ceiling and Detonation	28
15 - Proof of Accumulation (PoA)	29
16 - Technical Analysis, Goodness of Fit, and the Confidence Index	30
17 - Tiered Strategy Validation and Workflow	31
Part 3 System Architecture and Features Catalogue	33
System Architecture and Features Catalogue	33
Trading Tab	34
Indicator Voting Panel	82
The twelve voters	88
Indicator formulae	90
Departures from the published maths	102
Trading gate logic chain	102
Subsystem Tabs	110
Main Window	113
Simulator Tab (Hot Mess; Complete Rebuild In Progress)	115
Paper Trader Tab (To Be Built)	121

Proof of Accumulation (Anonymized Trading Tournaments Via Blockchain)	123
Market Inspector Tab	124
Bot Swarm Tab	127
Asset Charts	133
History Tab	135
Console	140
System Status Tab (To Be Built)	144
Settings	146
Converting the Interface to React	182
Subsystem Detail	198
Contents	199
The live tab set	199
How a screen reaches its renderer	200
Portfolio Information Panels	200
What builds it	200
Left: the spendable columns	201
Right: the five counter cards	202
Privacy	203
Where the numbers come from	203
Hiding	203
Bridge	204
Simulator Tab	204
The clone the tab draws now	204
What the tab holds today	209
What builds it	209
Fleet Replay	210
The candles	213
Portfolio Battery history	214
Portfolio Battery coverage	215
The validation criterion	217
Validation Mode	219
Back Test Mode	220
Portfolio Battery Mode	222
Nuclear Mode	225
Bridge	227
Paper Trader Tab	227
The empty tab it replaced	228
What the step is	228
Where the platform once offered it	228
The screen as it stands now	229
The two presses	230
Downstream only	230

One gate chain	231
The fake balance	231
The fleet ledger	231
The Paper Trader log	232
Real time	233
Paper Trade History	233
Proof of Accumulation	234
The skeleton	234
Identity	234
The trade log	235
The competition lifecycle	235
Tournaments	235
The token	236
The token contract	237
Head to head	238
Trophies	238
The chain	239
The two shelved tabs	239
Market Inspector Tab	240
What builds it	240
Left half: the scanner	240
Where the analysis happens	240
Fetching	241
What a scan reports	241
Right half: topology proposals	242
Dismissal	243
Adopt	243
Exchange comparison arbitrage	244
Bridge	244
Bot Swarm Tab	244
What builds it	244
The nodes	244
Drawing a wire	245
Routing the profit	246
Landing the profit	247
The tranche book	248
Where the fleet comes from	249
The list view	249
Capital claims	249
Bridge	250
Asset Charts	250
What builds it	250

The chart widget	251
Indicators on the chart	252
Where the candles come from	253
The other chart	253
Bridge	254
History Tab	254
What builds it	254
Fetching	254
Where the trade record is kept	255
Filtering, paging, export	255
Where the cell values come from	256
Grading	257
Gate analysis	257
The renderer	259
The Simulator front-load	259
Paper History	259
Bridge	259
Console	259
What builds it	260
Upper pane: the Python log	260
Pausing	262
Lower pane: the Emitter Network	262
Where the lower pane goes	262
Bridge	262
System Status Tab	263
The skeleton	263
Half one: the Emitter Network	263
Half two: the Watchdog	265
Until the tab lands	266
Settings	266
The eleven pages	266
What each page persists	266
Settings, User page	268
Settings, Exchanges page	269
Bridge	270
The Promotion Pipeline	270
What each step contributes	271
One logic, three data sources	271
The dashed edge	271
Where the chain breaks today	272
Part 4 Live and Supporting Evidence	273
The Year-to-Date Record and Exchange Test Coverage	273

The venue holds the authority	273
Readers of the record	274
The connectors	275
What the exchange tests exercise	276
How a test stands in for a venue	277
The four refusals around an exchange call	278
The simulation battery	279
Where a figure and a run carry less than they read	280
Part 5 Conceptual Hopscotch: The HOP Protocol, Rules Registry, and SADP v1 and v2	282
Conceptual Hopscotch	282
The handoff file	282
What the drift check measures	282
The rules registry	283
Version one of the protocol	284
Version two of the protocol	284
Related parts	288
Part 6 ADR Index & Glossary	289
Decision Records and Glossary	289
Where a decision is recorded	289
Glossary	289
The operator's own words	292
Part 7 Recent Updates and Version Tracking	293
Version Tracking	293
Recent updates	293
Where the number comes from	293
What the string says	294
The six readers	294
What a frozen bundle carries	294
A version that moves while the suite runs	295
The release gate	295
Part 8 The Development Chronicle As Told By Claude	296
The Development Chronicle	296
Part 9 Live Trade History and Supporting Evidence	297
VWAP Charts and Trade Grading	297
The record behind the charts	297
Subtotal is the price	297
How a VWAP is computed here	298
What the combined chart plots	298
The buy and sell ratio	300
Traps in the export	302
Where the figures live	302
How to read a per-asset chart	302

What this repository holds for these charts	303
The 38 per-asset charts	304
Trade grading	344
Gate logs behind each trade	345

Frontmatter & Overview

Copyright and Attribution

Copyright © 2025–2026 Ekthelius the Accumulator. All rights reserved. Unauthorized reproduction, distribution, modification, or derivative work is prohibited.

This document describes systems, algorithms, and methods developed by the author over the course of a self-teaching period starting in 2017.

Produced using the SADP (Structured Assistant Development Protocol) discipline framework designed by Ekthelius and deployed within Claude Code as a Skills-based and python-driven harness.

Contact

Acervator is the personal project of Zavyrian Productions and Ekthelius the Accumulator. Commercial inquiries, licensing requests, and networking requests are available via the channels listed at publication time:

ekthelius_the_accumulator@protonmail.com

Legal Disclaimers

NOT INVESTMENT ADVICE Nothing in this document constitutes investment advice, a solicitation to buy or sell any security or financial instrument, a recommendation of any trading strategy, or an offer to engage in any financial transaction. Acervator is a software system, and software cannot give investment advice. Trading cryptocurrencies and digital assets involves substantial risk of loss. Acervator is a highly complex, professional, and stable tool for automating digital asset trades. It does not and cannot guarantee the ultimate state of a portfolio given any number of transactions executed through or by it based upon a given user's configuration.

NOT A REGISTERED INVESTMENT ADVISOR The author is not a registered investment advisor, broker-dealer, or fiduciary. No fiduciary relationship is created by distribution of this document. Consult a qualified financial professional before making any investment decision. Ekthelius is a self-taught Technical Analyst with no official documentation to certify or desire to certify his skillset beyond his own personal portfolio results. They have been developed and achieved the hard way.

SOFTWARE PROVIDED AS-IS

The Acervator software system is provided as-is without warranty of any kind, express or implied, including merchantability, fitness for a particular purpose, and noninfringement. Users assume all risk associated with deployment. The author is not liable for financial losses, system failures, or downstream damages resulting from use of the software. Training, guidance, and general set up assistance can be provided but the vast majority of help one needs will be found in the eight part documentation so read it and understand what you are trying to use.

JURISDICTIONAL COMPLIANCE

Users are solely responsible for compliance with all applicable laws and regulations in their jurisdiction, including but not limited to: securities regulations, cryptocurrency-specific rules, KYC/AML requirements, tax reporting obligations, and exchange terms of service. Acervator does not assess or enforce jurisdictional compliance.

PATENT AND INTELLECTUAL PROPERTY NOTICE

The mechanisms catalogued in Part 2 are claimed as novel inventions by the author. No patent has been granted or filed for any of them. Research, academic, and personal study use is permitted. Commercial or competitive use requires written license.

Risk Factors Specific to Algorithmic Trading

Algorithmic trading systems can fail in ways manual trading cannot. Known risk categories include: (1) exchange API failures and rate limits, (2) network latency and partial fills, (3) flash-crash and liquidity-gap events that bypass stop conditions, (4) stuck positions during exchange downtime, (5) adverse selection on thinly-traded pairs, (6) slippage exceeding model assumptions, (7) parameter drift as market microstructure evolves, (8) bugs in the trading code itself. Acervator includes explicit mitigations for these failure modes (see Part 4 · Risk Controls), but no mitigation is complete and Acervator is a platform that is being actively developed. A trader using any algorithmic system must understand these risks before deploying capital.

Executive Summary

What Acervator Is and What It Is Aspiring To Be...

Acervator is a desktop trading platform whose innovation is the platform itself. The breakthrough is not a feature, an indicator, or a bot — it is the integrated system. A harvest-fold accumulation engine, a twelve-voter Indicator Voting Panel across up to five timeframes, a seventeen-class declarative GateChain safety framework, multi-bot coordination, and a live evidence trail proving the whole operates on real capital — coherent as one machine that converts price oscillation into asset accumulation and monotonic target growth without forecasting direction.

The platform will ultimately be designed to trade both cryptocurrency and stocks but is being developed to focus on the blockchain first given its current and immediate future impact on global financial systems. The crypto trading side is being prototyped via Coinbase Advanced and is the production surface today — live operator capital, fourteen weeks of execution, 5,055 trades with all assets successfully fending off the current bear trend or flat out beating it.

Harvest-Fold is the method and a mechanism. A position opens against a base currency with an explicit USD target (or a target that is translated to a USD or other local equivalent) with the initial size of the position being used as a value set point about which the mechanism of the platform allow volatility to “orbit” by persistently re-zeroing to it in response to local market structures and price ranges. The method itself stands apart from traditional Technical Analysis trading strategy or systems by not speculating or attempting to guess what the price will be within any given amount of time but instead persistently remolds a portfolio as its positions fluctuate and refusing to leave profit on the table by saying “It might be worth more later.” .

The primary “bot” or strategy that executes the Harvest-Fold method is called a Scrumming Bot and can be supported by, and this is exclusive to the cryptocurrency side of the platform, an Extractor Bot which performs more traditional scalp-style trades using isolated chunks of the parent Scrumming Bot's Target Asset with the Extractor's primary limitation being that it only works for established Base Currencies on a given exchange that has multiple asset pairs to exchange them against. The platform is designed and optimized in such a manner to allow “swarms” of Scrumming Bots (with or without Extractors) to be deployed to support dozens of positions across multiple exchanges simultaneously and thereby allowing the average digital asset investor to experience full spectrum digital asset trading that has previously been inaccessible or inefficient to execute properly.

The term “scrum” was initially encountered while I was learning about how a barrel of oil's profit is split or distributed. The scrum of the barrel, according to the tutorial, was what the gas station gets as its share of profit and its insultingly small, thus requiring huge sums of fuel to be sold for any respectable amount of money to be made by the facilities hosting the gas pumps and their infrastructure. The concepts of small, repeating, and accumulating were found to rest inside on one term and so it was readily adopted. The extractor, however, is more straightforward in origin and it rests on the idea that BTC and ETH trading became the primary means of acquisition after direct mining became unaffordable for average blockchain participants. It has always been apparent that the potential for faster wealth acquisition existed via asset trading but the skill and technology gaps are what prevent this door of opportunity from opening for just about everyone.

Prior to the invention of the Harvest-Fold mechanism, I had also conceptualized grid bots via a now deprecated cryptocurrency auto-trader called Margin. When this application first appeared and was used, it only featured a simple “ping-pong” and a more speculative / adaptive “margin” bot. It should also be noted that it was never a stable product throughout all of the versions attempted and this was what led to its failure while its company became a consultant firm of some sort. I am not sure how producing an unstable, feature-limited auto-trader qualifies you as a fintech advisor but maybe qualifications remain loose or have become looser. Regardless, and in spite of the instability experienced immediately during basic use, an attempt was made to stack dozens of ping bots of a large range on a given market. This may have been the first “grid bot” used in cryptocurrency trading. The resulting and constant crashing led to my seeking help from the company via their Telegram channel where he was told, basically, that he was using it incorrectly which was an unacceptable and stupid excuse for not fixing their product. Instead, they stole said concept and added a “stack bot” in a later version of Margin that performed the same basic behavior under one bot instance but because of the fundamentally buggy quality of the software, this was not used much before a boycott was necessary because the company chose to ban people criticizing the unending stability issues. After this, Margin supposedly partnered with KuCoin to bring their technology to said exchange but this seems to have dissolved at some point with KuCoin releasing its own Grid Bots with which I was able to win 3rd place and win three Lomen NFTs as a result. Said Lomens still reside on a certain Metamask to this day should this or any other part of this tale be doubted. Whether I am a co-inventor or discoverer of modern automated cryptocurrency trading is a point for irrelevant debate. What matters is that Acervator is my newest invention and it is infinitely more powerful than anything before it.

After the United States blocked access to KuCoin, I continued to develop my trading strategies further and executed them manually while learning more Technical Analysis. In the end, high volume trading methodologies were seen as the only method for maximizing profits in volatile markets. The entire idea of ‘HODL’ being pushed by people who were obviously continuing to accumulate was and remains disgusting but it is just how the game is played. No one is clicking anyone else's mouse so it is permissible. This is just a variation of what is generally called “Wall Street Investment” as well or at least fits the meta-pattern well enough. The average person, and their precious money, is expected to trust and be exposed to dangerous, unstable market environments while gaining no real benefits beyond a promise that breaks every few years.

There is not an entry level for market volatility and, with the advent of Acervator, there does not have to be.

More than anything else perhaps, the creation of Acervator represents an explicit effort to essentially debunk the approach and hamstrung interaction methods most investors are expected to utilize or, more accurately, tolerate. The average person who puts money into the market surrenders most or all control of it based on a hope that increasingly has no rational basis due to the lack of protections or proportionate, timely reward. The investor gets a secondary value abstraction that they have to wait to sell for more later (and are attacked for doing so if it's anything other than a stock!) assuming the market moves favorably. I see this as an over-exposed "bait and switch" where too often people are left holding worthless bags of tokens or stocks or bonds. It is an upgraded lottery system in some regard but it is only gambling to those that attempt to play blindly or passively and accept the 'HODL' advice from whales who just need their next meal to sit still.

The highest goal is for Acervator to be a self-contained retirement platform that provides a powerful option for anyone seeking to take a direct, elegant, self-contained approach to their digital asset portfolio management. It aspires to be the lens, the manager, and the fiduciary while only requiring users to test it, suggest new features, and / or donate to its continued development. Regardless, this will be my last attempt to help humanity see past the wiggly chart illusions but, honestly, I do not think anything else will be required.

If Acervator has helped you accumulate or you wish to support development directly (I am a one man show...), consider sending a tip to the inventor.

USDT 0x6Eb5bc78f43b71F5f09B87ED54e7FF3bD8a3428B

ETH 0x8b1022d836d19aA848Ebc9E09A6FC2921d9EB7d41

BTC 3HutPgxaMTftWQY3KyRkC7Dckp6A1zdfdp0

PayPalUSD 0x52D75aD65a98f98fA2d1D3675Bd5781ee547426D

BTC 2 bc1qn3mkssem0k2tyhwI5yeea4hj222k8j9rr6ruff

ETH 2 0x56d1Ee9F683d7535D2637F242a1A2B338aC5aC12

LAST NOTE: This manual, like the software it discusses, is a work in progress and it will be updated on a regular basis as features are added and finalized..

Product Manual Parts

This manual is an attempt to clarify all parts and functions for what might be considered a somewhat esoteric product given its innovative approaches and the sheer number of novel features it contains. More cynical SaaS providers might disagree but I look forward to all feedback regardless. Currently there are nine such sections with each focusing on ever smaller components of the total system.

Part Title and Area of Focus

1 Frontmatter & Overview

2 Novel Concepts and Patent Candidate Portfolio

3 System Architecture and Features Catalogue

- Main Window

- Trading Tab - Active Bot Lists Under Exchange Tabs - Indicator Voter Panel - Trade Logic Activity - API Activity - Top-Level Portfolio Metrics
- Simulator Tab - Validation Runs vs. Historical Trade Data - Acervator Strategy Back Testing - Runs Historical Data Through Simulated Bots - Bot Swarm Simulation - Deploy As Paper Trading - Deploy As Live Trading
- Paper Trader Tab - Runs Live Exchange Data Through Simulated Bots - Executes and Documents Paper Trades - Collects Paper Trade Data - Pushes Trade Data to Paper Trade History Log - Paper Trade History Tab Reads and Displays
- Proof of Accumulation (PoA) Tab - PoA Blockchain and Trading Tournament Interface - Hosts User Competition Performance and Data - Displays Tournament Monsters and Targets
- Market Inspector Tab - Synthesizes and Proposes Bot Swarm Topologies and Oppositional Trades - Pushes Proposals to Sim, Paper, or Live As Directed
- Bot Swarm Tab - Displays and Allows Configuration of Capital Reinforcement Networks
- Asset Charts Tab - Displays Price Charts for All Active Bots
- History Tab - Loads and Displays Live Trade History Via Exchange API - Merges Live Trade History w/ Acervator Trade Log for Grading and Gate Analysis - Paper History Tab - Loads and Displays Paper Trade History w/ Grading and Gate Analysis
- Console Tab - Displays internal codebase activity, warnings and faults
- System Status Tab - Displays Platform Status by Subsystem - Fed by the Emitter Network
- Settings - System and High-Level User Settings

4 Live and Supporting Evidence

- YTD Exchange Data Covering the Entire Development Arc and Exchange Test Coverage

5 Conceptual Hopscotch: The HOP Protocol, Rules Registry, and SADP v1 and v2

6 ADR Index & Glossary

7 Recent Updates and Version Tracking

8 The Development Chronicle As Told By Claude

9 Live Trade History and Supporting Evidence

- VWAP Charts - YTD Data Analysis and Trade Grading - Deep Indicator Function Investigations Via Trade Logic Gate Logs

Novel Concepts and Patent Candidate Portfolio

Novel Concepts and Patent Candidate Catalogue

Active development on Acervator began in early April of 2026. It began after realizing that the AI-assisted software development tools had achieved an inflection point with their capabilities. Prior to this, efforts were being made to learn traditional coding with little success that was mainly due to an inability to get into a grinding effort for whatever reasons. This has been particularly frustrating since other subjects seem to grab and others deflect. With the advent of Claude Code however, it feels like some of us have been given a decently priced door man and well-informed guide through this and other labyrinths.

Each entry below carries an anchor. The anchor is the code that runs the concept today, quoted from the module that holds it. Where a concept has no code, the entry says so and gives the query that measured the absence, and then proposes the build. Anchors describe the present state of the platform, not the plan.

1 - Harvest-Fold / Scrumming

At the core of Acervator is what might not first appear as a multi-faceted concept designed to address or alleviate many of the mental pitfalls that can afflict a new trader. After having observed a number of people via Telegram and web approach the challenge of investment differently, I have come away convinced that all it only becomes as complex as the individual allows it to be. This is not to be reductive about economics or any other connected discipline but it is to say there are reliable, stable, consistent means of reading price charts and too many people are simply doing it too many different ways and this only compounds the complexity illusion further. Much of the uneasiness inflicted on the new trader surrounds the emotional desire or even personal need for the value of their positions to increase as soon as possible or show the midterm potential of becoming profitable. Those that adopt the HODL meme are, in my experience, quite crippled in their ability to sell positions even when the time is right, which is usually when most people are happiest about the market. The ironic truth is that however good a trade is, there is an equally bad one on the other side of it. Acervator attempts to respect the market by not taking profit all at once and absorbing or claiming ownership of its positions' variance strictly based on the market local, organic price ranges. In other words, its trade actions are always proportionate to whatever the market is actually doing at the time the trade is executed. Over time, this 'molds' the position to the long term truth of the market and avoids the need to speculate entirely. The more bold hope for this approach is that it might add another tool for stabilizing macro-economic phenomena such as underpinning currency exchange rates between countries.

Acervator addresses 'fundamental market unpredictability' by allowing volatility to "orbit" around a set point we call the Target Balance. This element is what performs the first mental trick. It forces the trader to say and establish that "my position is worth X and I will support X with an equal amount of liquidity" i.e. a Target Balance of \$50 is set and a total mental position of \$100 is established. Trades will be executed to maintain the truth of the Target Balance always being X. After the initial entry is executed and filled, which Acervator will only do in the lower extent of the Bollinger Band on the chosen timeframe, the Scrumming Bot will persistently shave off and "Fold" back in Y% of profits through repeating Harvest-Fold operations that continue until the bot is

deactivated or the market suffers a catastrophic collapse which can and do happen so don't not leave it unmonitored. It should also be noted that the term 'harvest' came to replace 'scrum' for the sake of clarity but they may be used interchangeably throughout this text. Essentially a Scrum or Harvest is a sale of a position's increased value over the Target Balance. The amount that the Scrumming Bot harvests or folds is pinned to the Target Delta or the amount of variation (%delta) from the most recent trade event. Much of the finer details of trade execution can be tuned by the user as expected for any professional application but the primary point here is that the amount of profit folded back is capped by the Maximum Target Growth Per Cycle setting while the ability of Acervator to prevent inverted (profit losing: buying higher than sold / selling lower than bought) trades we also have the Minimum Opposing Trade Distance rule which dictates the next trade must be Z distance from the most recent trade event. In addition, sells are not allowed to occur or be pushed below the Bollinger Band midline while buys are not allowed to occur or be pushed above the midline. As a result of just these few component features, we already have multiple soft, non-TA or non-traditional rules protecting the investor before we even start optimizing the trades themselves (pushing them as close to volatility extents as possible) via Technical Analysis and other inventions.

The harvest-fold method itself was manually executed and tested on a multiple cryptocurrency pairs throughout 2023~2026 but it has always been the aspiration to automate it and have it executed at a high rate of frequency with the operating theory that this approach to investment, being a hybrid a HODL and active trading, creates frontline area of support for whatever markets in which it participates long term. It does this by maintaining and, utilizing as directed, all earned profits. It turns any user into an active participant and liquidity provider.

After entering the space full time in 2017, I immediately began to parse and breakdown how price charts were being interpreted. This also allowed me to perform ever deeper introspections on my own response to how my mind was responding to the value of my portfolio fluctuating. I did myself absolutely no psychological favors in this pursuit but, as you will see, I had plenty of reasons to take this reckless approach. By this point, my undiagnosed and untreated military injuries had worn me down to the point where I realized I was running out of time to continue working as normal. I had, by then, been walking a torsioned pelvis for 17 years that had seized in place and, by 2020, would be causing enough pain to induce panic attacks. I could have joined the pill-mill club but I declined. In combination with the pain, the forced TA self-training, physical job, and broken marriage, I also assembled an entire GPU-based mining farm...alone. I had some business partners but their participation was basically nonexistent. The VA was providing no support and would not until 2022. The Navy and Pentagon had spit in my face by sending me PDBR paperwork that they had no intention of ever honoring no matter what I wrote down. My family more or less remained ignorant of all my turmoil while I kept the paychecks coming. So really the story of Acervator's and Harvest-Fold is one of betrayal to a degree that allowed me to see through things more clearly than would otherwise have been possible. It has been invented out of necessity and it is my intuition that Acervator can become that necessary tool for other people as well.

The target and the two sides

Functional. The bot holds one number, and that number is the dollar value you told it the position is worth. On every tick it compares the position against that number. Above it by more than a dust band, the bot has surplus to sell. Below it by more than the band, the bot has a deficit to buy. Inside the band it does nothing at all. One function answers that question for the whole engine, and the dust band is a tenth of a percent of the target with a floor of one cent. At a target of \$50 the band is \$0.05, so a position at \$60 answers scrum and a position at \$40 answers fold.

src/trading/target_bands.py — target_territory

```

delta = float(position_value) - float(target_balance)
band = at_target_dust_band(target_balance)
if delta > band:
    return "scrum"
if delta < -band:
    return "fold"
return "at_target"

```

Design intention. The Target Balance is the set point the whole method orbits. It is declared once in the bot's config and the tick re-zeroes to it. The bot brain holds the tick, the scrumming package fires the two halves of the cycle, and the execution module places the orders.

src/trading/container/config.py — BotConfig

```

target_balance: float = 200.0 # Balance the bot trades relative to
scrumming_interval_pct: float = 1.0 # % market move between actions
max_target_growth_pct: float = 1.0

```

The distance rule

Functional. An opposing trade has to travel far enough to clear both legs of the round trip. That is the operator's scrumming interval plus the exchange fee. A one percent interval against a six tenths fee gives 1.60 percent. One function owns the arithmetic, and the clamp is applied to the sum rather than to the interval, because the bound exists to keep the rebuy factor sane.

src/trading/otd_math.py — minimum_opposing_trade_distance_pct

```

total = float(interval_pct) + float(fee_pct)
return max(OTD_MIN_PCT, min(OTD_MAX_PCT, total))

```

Design intention. Without this rule the bot can sell low and buy back higher, which is the one outcome the method exists to prevent. The gate that enforces it refuses the trade until price has travelled that distance from the pivot, and it rebuilds the required price in its own refusal message so the operator can read why it held.

src/trading/gate_chain.py — HysteresisGate.evaluate

```

ok = ctx.hyst_ok_scrum_side if self.side == "scrum" else ctx.hyst_ok_fold_side
if ok:
    return GateResult(passed=True)
eff_pct = ctx.scrumming_interval_pct + ctx.trading_fee_pct

```

The midline rule and the growth cap

Functional. A sell is not allowed below the Bollinger midline and a buy is not allowed above it. The tick sets one flag for each side off the band position, and a gate refuses the wrong side of the band. The scrum flag also carries the phantom lock, so a locked bot cannot sell whatever the band says.

src/trading/scrumming_bot.py — ScrummingBot.tick

```
scrum_ok = (bb_pos > 0.50) and not self._phantom_locked
fold_ok_midline = bb_pos < 0.50
```

Design intention. Profit folded back into the target is capped for one market cycle, so a single violent move cannot inflate the set point beyond what the next cycle can support. The cap is a percentage of the target at the moment the cycle opened, and it shrinks as the cycle consumes it.

src/trading/scrumming_bot.py — ScrummingBot.cycle_growth_cap_usd

```
_pct = float(getattr(self.config, "max_target_growth_pct", 1.0) or 0.0)
```

The same key answers differently in different places. Nine read sites fall back to 1.0 and three fall back to 0.0, so a bot whose stored config lacks the key compounds or freezes depending on which site read it first. Issue #409 carries it.

The gate set the bot writes is named in one place: ten scrum labels and nine fold labels, nineteen lights per bot.

src/trading/gate_vocabulary.py — _GATE_ORDER_SCRUM

```
_GATE_ORDER_SCRUM: tuple[str, ...] = (
    "TGT",
    "INT",
    "BB",
    "FIRE",
    "TA",
    "LS",
    "TRND",
    "HTF",
    "CB",
    "OTD",
)
```

2 - The Phantom Bot

While the Harvest-Fold method housed with the Scrumming Bot addresses several persistent and nagging questions around any given investment position (Should I buy more? Should I sell some? When? How much?), it does have the weakness of operating on a single timeframe. The Phantom Bot is another feature intended to optimize trade execution position. In this specific case, it allows a single Scrumming Bot to be inhibited by a higher timeframe signal with the intention of capturing alignments or disagreements so that trades are not being executed prematurely. The design intention here is that, for example, a 5m TF bot can be inhibited by a 15m, 30m or 1hr signal while maintaining a volatility watch on its primary TF. The intention is to build out this feature so that it obviates or reduces the need for multi-screen monitoring of individual assets.

Functional. A phantom is a copy of the position read on a slower chart. It computes technical analysis on its own timeframe, stores the result, and constructs no order. Six timeframes are the default set.

src/trading/scrumming_bot.py — ScrummingBot.DEFAULT_PHANTOM_TIMEFRAMES


```
DEFAULT_PHANTOM_TIMEFRAMES = ["5m", "15m", "30m", "1h", "4h", "1d"]
```

Design intention. The inhibition the entry describes runs today. A bullish higher timeframe stops the scrum and a bearish one stops the fold, and each writes its own label into the blocker list so the operator can see which timeframe held the trade. The coordinator weights the phantom summaries by their rank before the gate reads them.

src/trading/gate_chain.py — HTFDeferGate.evaluate

```
blocks = (
    ctx.eff_htf_blocks_scrum
    if self.side == "scrum"
    else ctx.eff_htf_blocks_fold
)
if not blocks:
    return GateResult(passed=True)
label = "HTF-bullish" if self.side == "scrum" else "HTF-bearish"
return GateResult(passed=False, blocker_message=label)
```

Three parts of the feature do not work. The phantom lock guard is dead, so a lock never holds anything (issue #398). Every profit and loss figure on the Phantom Bots tab prints as zero, because the key the tab reads is never written (issue #197). The tab draws a refused change as a success and can drop a timeframe (issue #313). Issue #155 tracks the remaining build-out. Replacing multi-screen monitoring is an intention, not current behaviour.

3 - The Landing Strip

Perhaps the earliest assumption that hit me the most and really presented the catalyzing challenge to creating all of my alternate trading methods, is that you cannot predict what the market will do and after only a short while of actively trading I asked the questions: “But what if I don’t have to?” “What if I decide that I do not have to speculate at all?” I do not remember exactly when I first used the term but I think it was when I was trading ADA and XLM some years ago while participating in a Telegram group with some psytrance friends and associations called the Better Bitcoin Bureau. The name changed a few times but this is the one I recall.

The concept emerged from my persistent use of Heikin-Aishii candles. I had decided on this candle style early just by intuition. I could say it was because I “felt it looked nicer” but that does not quite capture the magnetism. To me, they just structurally make more sense and are less sloppy than traditional candles. What I began to observe over and over was the propensity for crypto markets to form long, tapering structures from these candles and that the likelihood of market reversal could be pinned to 1) the landing strip length and 2) its proximity to the Bollinger Band. Further, I found that those markets that formed these on 1D or higher timeframes would often have explosive breakouts like a volcano that had charged up to blow out. These findings can be historically verified and back tested. I manually performed this observation all the way down to 1 minute trading and found that it was only at this lowest TF that it started to exhibit sub-50% reliability. Given this, the concept has been transposed into Acervator as a core component and trading logic gate with the number of candle strip candles being a user configurable setting.

This is also where I should mention that I am not at all a quant or mathematical trader. I consider myself a visual or tactical trader that focuses on market reversals. I have taught myself to visually interpret price chart structure

using a given set of indicators used by countless other technical analysts. When looking at some of the labeling within Acervator, however, one will readily see military references and hints at how I view asset accumulation as something akin to progressive territory capture on a perpetual gameboard. The less skilled traders are surrendering it with every cycle of the market.

Functional. Two detectors build the Landing Strip and both run on every tick. The first places price inside the Bollinger channel and counts trailing Heikin Ashi bodies under a tightness threshold. It reads the operator's control directly.

src/trading/scrumming_bot.py — the first detector's call

```
min_pattern_candles=self.config.bb_landing_strip_candles,
```

Design intention. The second detector is the one that scores the taper. It counts a run of shrinking Heikin Ashi bodies, names the band the run reached, and returns a confidence boost the tick adds to its own. On a tapering rally into the upper band it answers detected, side upper, length three, boost 0.1437. The same tape without the taper answers not detected, and a five-candle tape answers not detected. A detected strip can force a direction, which is why the gate vocabulary marks it an override rather than a gate.

src/trading/indicators/landing_strip.py — detect_landing_strip_v2

```
def detect_landing_strip_v2(
    candles: list[Candle],
    min_consecutive: int = 3,
    shrink_threshold: float = 0.90,
    bb_tolerance_pct: float = 3.0,
    use_ha: bool = True,
    bb_period: int = 20,
    bb_std: float = 2.0,
) -> TighteningResult:
```

The second detector does not read the setting. The tick hands it a literal three, so the wizard's Landing Strip Candles control moves the first detector and leaves the second where it is. Both defaults sit at three, so the two agree until the operator changes the control.

src/trading/scrumming_bot.py — the second detector's call

```
tightening = detect_landing_strip_v2(
    candles,
    min_consecutive=3,
    shrink_threshold=0.90,
    bb_tolerance_pct=3.0,
)
```

Issue #432 carries it.

4 - Bollinger Band Travel Detection

Allowing a strategy to “see” where it is in relation to local width of the Bollinger Band is another aspect of treating the immediate market as a region on a map through which a stack of tokens is traveling. Along the route, we are able to offload surplus at the peaks and pick up additional supplies in the next valley. This is used in combination with Minimum Opposing Trade Distance. The complete question then becomes “Has the market traveled 70% (default), is current price X% distant from the most recent trade event, and is Target Delta not zero? When combining these gates with the (Technical Analysis) Indicator Voting Panel and Landing Strip Detection, you have a full market structure detection suite that is able to incrementally adjust a position’s size and generate recurring profits or growth with each market cycle.

Functional. The tick measures how far price has moved since the last trade and divides that by the current band width. When the fraction reaches the operator’s percentage and the target delta is positive, the travel flag is set. The default is seventy percent and the wizard offers zero to a hundred. That is the compound question the entry states, and the code asks it in one expression.

src/trading/scrumming_bot.py — ScrummingBot.tick

```
_bb_width = max(bb_result.upper - bb_result.lower, 1e-12)
band_travel_frac = abs(ticker.last - self._last_trade_price) / _bb_width
if (
    abs(ticker.last - self._last_trade_price)
    >= self.config.band_travel_pct / 100.0 * _bb_width
    and delta > 0
):
    band_travel_triggered = True
```

Design intention. A fixed percentage means two different things in a quiet market and a violent one. Measuring the move against the band width makes the threshold follow the volatility instead. The flag then clears a standing trend hold and lets the harvest through. Travel is no gate of its own: the ten scrum labels carry no travel light.

src/trading/scrumming_bot.py — what the flag releases

```
trend_override = abs(delta) >= _interval_usd * 2.0 or band_travel_triggered
```

5 - Position-Aware Technical Analysis

Traditional TA treats a signal (e.g., RSI > 70) as context-independent. Position-Aware TA encodes the current Bollinger Band position as context: RSI > 70 near the upper band confirms overbought (sell bias), but RSI > 70 near the lower band is a failed oversold — a trap for mean-reversion buyers. The system applies this inversion systematically across all oscillator signals. In other words, it attempts to strengthen the price to Bollinger Band relationship by reading additional indicators with relation to it. If you have the landing strip, band travel, opposing trade distance, and a bullish or bearish confidence score from the Indicator Voting Panel, it is likely a trading event will occur. In the end, many of these functions were layered in one after the other until I saw the platform take most trades that I would myself manually execute most of the time.

Functional. The band context enters at the bot, not inside the voters. The tick reads the band position once, marks the upper quarter and the middle band, and accumulates a skew from the votes that arrive. A bullish

vortex adds 0.12 at the upper band and takes 0.05 back in the middle. A bullish MACD adds 0.08 and takes 0.03 on the same test. Ichimoku and Stochastic RSI move the skew on direction alone, with no band term at all.

src/trading/scrumming_bot.py — ScrummingBot.tick

```
at_upper_bb = bb_pos > 0.75
in_bb_middle = 0.35 <= bb_pos <= 0.65

if (
    sig.indicator == "vortex"
    and sig.direction == SignalDirection.BULLISH
):
    if at_upper_bb and sig.confidence > 0.5:
        position_boost += 0.12
    elif in_bb_middle:
        position_boost -= 0.05
```

Design intention. The skew relaxes or tightens the confidence floor the vote has to clear. A floor of 0.30 becomes 0.2679 at a skew of plus 0.12 and 0.3158 at a skew of minus 0.05. A skew of minus one returns infinity, which no confidence can clear, so a fully contradicted reading is refused outright rather than discounted.

src/trading/scrumming_bot.py — the two terms the floor reads

```
_ta_conf_skew = position_boost + bb_confidence_boost
```

The inversion does not reach the oscillator modules. Each indicator computes from candles alone, and the band position appears in six indicator modules only: Bollinger, band proximity, landing strip, M top, spring and W bottom. Neither the RSI module nor the Stochastic RSI module takes a band argument. Today the band context re-weights the vote after the fact. It does not re-read the oscillator.

6 - Market Inspector

This addition was, like the Phantom Bot above, a move towards making Acervator “hyper-vigilant” of hypothetical opportunities that take the form of oppositional trades (assets that move inversely to one another) and bot swarm network topologies. The bot swarm itself will be covered in a moment but basically this feature allows for the creation of custom capital reinforcement networks where profits are configured to flow between positions in a manner that can potentially strengthen a portfolio faster than just focusing on single, isolated bot instances.

The Market Inspector is designed to analyze the broader market for groups of assets from which to build topologies and identify non-active assets that could be traded in opposition to active positions. Its findings can be pushed to the Simulator for historical back testing, to the Paper Trader for live market and real time validation, and, of course, Live Trading should the user choose to do so.

Functional. The Inspector walks daily, weekly and monthly candles, scores each market, and keeps the result. It then pairs a long-signal market with a short-signal market when the correlation of their returns over a rolling window sits inside a negative window. Those are the oppositional trades the entry describes, and the ranking puts the most negative correlation first.

src/trading/market_inspector.py — MarketInspector._find_opposing_pairs

```

corr = self._pearson(r_l[-m:], r_s[-m:])
if self._corr_min <= corr <= self._corr_max:
    pairs.append(
        OpposingPair(long_side=lo, short_side=sh, correlation_30d=corr)
    )

```

Design intention. The findings are meant to leave the screen. A separate module turns those signals into swarm proposals with the bots and the wires already written, and one push target exists today: the Simulator asks the Inspector for its current proposals at build time.

src/gui/main_tabs/simulator_tab.py — _build_simulator_tab The Simulator rebuild removed this file; it is not in the tree.

```

if hasattr(self._simulator, "set_topology_getter"):
    self._simulator.set_topology_getter(
        lambda: (
            self._market_inspector.current_topology_proposals()
            if hasattr(
                getattr(self, "_market_inspector", None),
                "current_topology_proposals",
            )
            else []
        )
    )

```

No Paper Trader exists to push to; entry 17 gives the measurement. Issue #23 tracks the tab's build-out and the topology and oppositional pushes, and issue #290 a live connector defect on Refresh.

7 - Bot Swarm and Smart Wire Network w/ Provenance Tracking

The Bot Swarm, which is how I presented the concept to Claude, is my version of a capital reinforcement network. At the Exchange level (swarms cannot be connected across multi-API boundaries) profit from individual bots can be routed to others with such outputs being termed Wire Credits since they are sent and received along Smart Wires which connect the bots. Each such wire can have its own flow rate. The network nodes are designed to protect a given bot from excessive outflows which would cripple its own growth capacity. The other piece of this is that Wire Credits are Provenance tracked which allows outflows to eventually start sending profits back to the originating bot with the over-arching philosophy being long-term investment with accumulation and position strengthening rather than just sitting around waiting for a good outcome that might not arrive.

In combination with the Market Inspector, the Bot Swarm can receive and deploy groups of pre-configured and pre-connected bot clusters that immediately begin participating in the market. This layer, along with future portfolio-level algorithms, will enable the fully automated and persistent execution of complex trading strategies from a single terminal window.

Functional. The wire manager holds the topology and one ledger for each bot. A wire records a source, a target and a flow percentage, and a realised fold moves the share along it. The outflow is capped so a funder keeps its own growth capacity: a ten dollar scrum profit against a hundred dollar target releases ninety percent, and a zero profit releases nothing.

src/trading/smart_wire.py — compute_safe_outflow_pct

```
if scrum_profit_usd <= 0:
    return 0.0

if band_upper <= band_lower:
    distance_to_fold_pct = 0.0
else:
    distance_to_fold_pct = max(
        0.0, min(1.0, (current_price - band_lower) / (band_upper - band_lower))
    )
```

Design intention. Provenance is what makes the network more than a pipe. Each credit that lands on a tranche keeps the id of the bot that supplied it, so an outflow can eventually route profit back to the bot that funded it. The detail list is bounded at twenty entries per tranche and everything older folds into a rolled total, which keeps the sums exact without letting the record grow without limit.

src/trading/scrumming/wire_routing.py — _WIRE_CREDIT_CAP

```
_WIRE_CREDIT_CAP = 20
if not isinstance(wc, list) or len(wc) <= _WIRE_CREDIT_CAP:
    return 0
overflow = wc[: -_WIRE_CREDIT_CAP]
del wc[: -_WIRE_CREDIT_CAP]
```

How much of an arriving inflow a bot stacks onto an open position is its own setting, and it defaults to one percent.

src/trading/container/config.py — BotConfig.wire_inflow_stack_pct

```
wire_inflow_stack_pct: float = 1.0
```

One routing path is dead. The two spawn-and-route functions have no caller anywhere in the repository, while the fold distribution and wire registration beside them are called throughout. Issue #404 carries it, and issue #239 carries the Bot Swarm tab's live rows. Deploying a pre-connected cluster from the Market Inspector is an intention.

8 - Hunger and Satiety Indices

These elements were implemented as part of an effort to boost accumulation and scrumming performance in longer term bearish or bullish conditions that might not latch all logic gates to trigger a fold or scrum despite the Target Delta not being zero. This piece adds a factor to the bot that makes it more aggressively evaluate subsequent candles and will increase its likelihood of executing the re-zeroing action. This allows for trades to be executed in the hypothetical 'dead zone' between Minimum Opposing Trade Distance and the lower or upper Bollinger Band extent if the candle structure begins to float or cruise within it for too long.

Not built. Neither index exists in code today. No aggression factor shortens the wait between candles, and the bot has no term that widens its willingness to fire inside the dead zone between the Minimum Opposing Trade Distance and the band extreme.

The build this specifies. The engine already carries a skew term that relaxes the confidence floor, and hunger is that term driven by time rather than by band position. A counter grows while the target delta is non-zero and nothing fires, and it resets to zero on a fill. That reset is satiety. Two new config fields set the rate and the ceiling, and the counter joins the skew the floor already reads, so no new gate and no new call site are needed.

Proposed, not present, in src/trading/container/config.py:

```
hunger_gain_per_candle: float = 0.0
hunger_skew_max: float = 0.20
```

Proposed, not present, in ScrummingBot.tick:

```
if territory != "at_target" and not fired_this_tick:
    self._hunger_candles += 1
else:
    self._hunger_candles = 0
hunger_skew = min(
    float(self.config.hunger_skew_max),
    self._hunger_candles * float(self.config.hunger_gain_per_candle),
)
_ta_conf_skew = position_boost + bb_confidence_boost + hunger_skew
```

The default gain of zero keeps present behaviour exactly, so the feature ships switched off and the operator turns it on per bot. Issue #433 carries the build.

9 - Accumulation Shadows

The Accumulation Shadow was invented as a means of taking maximum advantage of long term and bearish conditions. It allows for a Scrumming to create a Shadow of itself that focuses on accumulating a smaller chunk of the target asset during a bearish regime. The shadow is allowed to accumulate downward but does not scrum. It was a bullish regime to be dedicated and then detonates thereby claiming all of its profits at once. Shadows operate in parallel with their parent bots and will be listed as Shadow Tranches once they are fully implemented.

Not built. The Accumulation Shadow has no code. No shadow bot, shadow tranche or shadow regime exists today.

Three functions collapse or remove a tranche, and none of them knows a shadow kind. One merges two rungs, one removes an aged row, and one discards a queue. Despawn removes a tranche; it does not delist one. The entry's own wording, "once they are fully implemented", matches the code.

src/trading/stack_math.py — `_apply_merge_rule`

```
def _apply_merge_rule(tranches: list[Tranche]) -> list[Tranche]:
    """Combine adjacent tranches whose prices are within MERGE_THRESHOLD.

    A merged Tranche keeps the higher `price` and the summed `size`, and the
    survivors are re-indexed from 0.
    """
```

The build this specifies. A shadow is a tranche kind, not a second bot. It accumulates downward, never sells, and exits in one move when the regime turns. The tranche dict gains a kind key, the three collapse functions skip a row that carries it, and the exit reuses the detonation executor that already sells a surplus and reseeds the lots.

Proposed, not present, in src/trading/scrumming/fold_tranches.py:

```
def _despawn_aged_tranches(self, now: Optional[float] = None) -> dict:
    for t in list(self._fold_tranches):
        if t.get("kind") == "shadow":
            continue
```

A row with no kind key behaves exactly as it does today, so the change is invisible until a shadow is spawned. Issue #433 carries the build.

10 - Charge Up Permission Gate

This is client-side trade batching. It operates in conjunction with the Fold and Stack Tranches (explained later) to allow these individual positions to be combined into a single trade action in response to a given market trend.

Not built. The Charge Up Permission Gate has no code today. Fold and Stack Tranches do run, and entry 11 anchors them. Nothing combines them into one batched order.

The build this specifies. The fold chain already carries a gate that refuses when no tranche is queued, and the context it reads already counts them. Charge Up is the same test with a threshold: hold the fold until enough tranches are eligible, then release them together. It is a new gate class in the existing chain, and it needs no new context field.

Proposed, not present, in src/trading/gate_chain.py:

```
class ChargeUpGate(Gate):
    """FOLD-only: hold until ``min_tranches`` rows are queued, then release."""

    name = "charge_up"
    side = "fold"

    def __init__(self, min_tranches: int = 1) -> None:
        self.min_tranches = int(min_tranches)

    def evaluate(self, ctx: GateContext) -> GateResult:
        if ctx.n_fold_tranches >= self.min_tranches:
            return GateResult(passed=True)
        return GateResult(passed=False, blocker_message="charging")
```

The gate enters the fold chain before the hysteresis gate, and the gate vocabulary gains one fold label, taking the row from nineteen lights to twenty. A threshold of one reproduces present behaviour exactly. Issue #433 carries the build.

11 - Fold, Stack, Extractor, and Shadow Tranches / Provenance Queue

The term tranche emerged in Acervator as the concept of placing soft boundaries around where an opposing trade 1) can initially happen in terms of price level, 2) denote the amount of the position expected to be transacted once the price thresholds are met or exceeded, 3) allows wire credits to be fed into Fold Tranches (Buys) and Extractor Tranche profits to be distributed across Stack Tranches (Scrums / Sells) for additional accumulation and profit optimization. For the sake of clarity, Fold Tranches spawn after a Scrum (Sell) occurs; Stack Tranches spawn after a Fold (Buy) occurs; and Extractor Tranches spawn when an Extractor Bot finds a favorable Alternate Currency entry on its chosen timeframe and these tranches remain until the acquired Alternate Currency stack is sold by either the Extractor Bot (Sibling) or its corresponding Scrumming Bot (Parent). All Tranches will persist under a bot's Details > Tranches Tab.

Functional. Three of the four tranche kinds run. Fold Tranches spawn on a scrum and hold the price of the sale that made them. Stack Tranches spawn on a fold and are priced as a ladder above the buy. A three-unit scrum at 100 with a one percent split and linear spacing returns rungs at 101, 102 and 103 carrying one unit each. Two rungs closer than a tenth of a percent merge, so 100, 100.05 and 105 come back as 100.05 carrying two units, then 105 carrying one.

src/trading/stack_math.py — split_scrum_into_tranches

```
def split_scrum_into_tranches(
    scrum_price: float,
    scrum_size: float,
    n_target: int,
    split_distance_pct: float = DEFAULT_INITIAL_GAP_PCT,
    spacing_mode: str = DEFAULT_SPACING_MODE,
    min_opposing_pct: float = 0.0,
    min_order_size: float = 0.0,
) -> list[Tranche]:
```

Design intention. Extractor Tranches are the sibling-and-parent relationship the entry describes. The Extractor opens a position on an alternate pair, names the tranche with an id that survives a restart, and hands the base currency back to the parent Scrumming Bot on exit. The parent raises its own target by the amount that arrived rather than selling it straight back out as surplus.

src/trading/extractor_bot.py — ExtractorBot.tranche_id_for_position

```
"""Build the stable Extractor Tranche id: ``bot_id|pair|opened_at``.

The row emitter and `set_tranche_arbiter` both format
`opened_at` through this one method, so they can never format
it differently and disagree on the same id.
"""
```

Shadow Tranches do not exist; entry 9 gives the measurement and the proposal.

The queue sorts the fold tranches by their reference price, highest first, and drops any row whose reference or unit count is not a finite number. Wire credits ride on the tranche, which is the provenance the entry names.

Three defects stand against the Tranches tab and the rows it draws: spawned tranches do not match the scrum that spawned them (issue #367), the list is not visible and lifetime counters reset (issue #133), and the Units

column prints a quantity that does not exist (issue #201). One more sits behind them. The capital reservation registry answers before a sale is placed, and of the five bot construction sites only the two Simulator controllers hand the bot a registry (issue #427).

12 - Technical Analysis Indicator Confidence Tiers

Most TA systems stack indicators additively. The Tier-4 suite organizes signals into four confidence tiers with cumulative weighting: Tier-1 (structural, highest weight), Tier-2 (confirmation, medium weight), Tier-3 (momentum), Tier-4 (context). Entries require Tier-1 + Tier-2 alignment; Tier-3/4 scale confidence but cannot override. This tier system arose out of queries with Claude Code while it was roleplaying as a TA Archetype and is only one such example where most of the other such contributions deal with bug fixes.

Not built as tiers. The four-tier scheme is a design. The engine has no tier. The signal record carries an indicator name, a timeframe, a direction, a confidence, a weight, a details map, a timestamp and an abstention flag, and nothing else. No field names a tier, no code groups the voters into tiers, and no rule requires a Tier-1 and Tier-2 alignment before an entry.

src/trading/indicators/types.py — Signal

```
@dataclass
class Signal:
    """One indicator's output at a point in time."""

    indicator: str
    timeframe: str
    direction: SignalDirection
    confidence: float
    weight: float = 1.0
    details: dict = field(default_factory=dict)
    timestamp: float = field(default_factory=time.time)
    abstained: bool = False
```

What runs instead. A flat per-indicator weight. Each of the twelve voters carries one number between 0.8 and 1.2, and a signal's score is its direction times its confidence times that weight. Every vote can move the total, and none can veto another.

src/trading/ta_engine.py — DEFAULT_WEIGHTS

```

DEFAULT_WEIGHTS = {
    "bollinger_bands": 1.0,
    "vortex": 0.9,
    "macd": 1.2,
    "stochastic_rsi": 1.0,
    "ichimoku": 1.1,
    "volume": 0.8,
    "slingshot": 1.0,
    "adx": 1.0,
    "kaufman_er": 1.0,
    "supertrend": 1.0,
    "zscore": 0.9,
    "rsi": 0.8,
}

```

The build this specifies. A tier is a second map beside the weights, and the veto is one test in the aggregator. The signal record grows one field, and it has to go last: the record is built positionally in this tree, so inserting a field earlier would re-bind an existing argument.

Proposed, not present, in src/trading/ta_engine.py:

```

DEFAULT_TIERS = {
    "bollinger_bands": 1,
    "ichimoku": 1,
    "supertrend": 1,
    "macd": 2,
    "vortex": 2,
    "adx": 2,
    "stochastic_rsi": 3,
    "rsi": 3,
    "slingshot": 3,
    "volume": 4,
    "kaufman_er": 4,
    "zscore": 4,
}

```

Proposed, not present, in VotingEngine._aggregate:

```

_lead = {s.tier for s in signals if not s.abstained and s.direction is not NEUTRAL}
if require_tier_alignment and not {1, 2} <= _lead:
    consensus_conf = 0.0

```

Issue #433 carries the build.

13 - Initial-Purchase-Price Floor on Folds

This rule asserts that a subsequent Fold occurring after a Scrum **MUST** be less than sell. This, of course, is asserted by Bollinger Band Travel, Minimum Opposing Trade Distance, Target Balance Not Zero, etc.

Functional. The floor runs, and it lives on the tranche. Every fold tranche stores the price of the scrum that spawned it. The row is released only when price has fallen to that reference multiplied by the rebuy factor, and

the factor is one minus the opposing trade distance. A one percent interval with a six tenths fee gives a distance of 1.60 percent and a factor of 0.984, so a tranche spawned by a sale at \$100.00 folds back at \$98.40 or lower.

src/trading/scrumming/fold_tranches.py — `_fold_eligible_tranches`

```
return [
    t
    for t in self._fold_tranches
    if ticker_last <= float(t.get("ref", 0)) * otd_factor
]
```

Design intention. The same distance is held in three places so the rule cannot be true for the bot and false for a row. The bot-wide hysteresis gate holds it, the per-tranche filter above holds it, and each new ladder rung is priced against it.

src/trading/otd_math.py — `fold_rebuy_factor_from_pct`

```
def fold_rebuy_factor_from_pct(otd_pct: float) -> float:
    """Convert an already-clamped OTD percentage to a FOLD rebuy factor.

    Multiply a tranche's ``ref`` by this to get the HIGHEST price at
    which that tranche is still eligible to be re-bought:

        eligible <=> ticker.last <= ref * factor
    """
    return 1.0 - (float(otd_pct) / 100.0)
```

The lot carries its own cost basis as well as the scrum reference, and the tick counts the rows eligible against that basis.

src/trading/scrumming/tick_phases.py — the lot's own basis

```
_patent_only_eligible = sum(
    1
    for _t in self._fold_tranches
    if ticker.last
    <= float(_t.get("initial_buy_price", _t.get("ref", 0)))
)
```

One gap sits in the same path, and the function names it in its own docstring. The fold gate wraps the distance call and falls back to a distance of zero when a config value is not a number. At that fallback the rule is not active.

14 - Position Ceiling and Detonation

This setting allows a total profit cap to be set for a given position before it sells its entire or a large portion of its total value.

Functional. The entry covers two mechanisms. They act on opposite sides of the cycle and both start disabled. The ceiling brakes buying. It is the anchor target multiplied by a factor clamped between one and ten, and the fold rate tapers as the position approaches it: full rate below half the ceiling, a linear taper from full to a tenth

across the upper half, and a hard stop at the ceiling. The ceiling starts no sale.

src/trading/scrumming_bot.py — ScrummingBot.fold_rate_taper

```
ratio = self.ceiling_ratio
if ratio is None:
    return 1.0
if ratio >= 1.0:
    return 0.0
if ratio < 0.5:
    return 1.0
return 1.0 - (ratio - 0.5) / 0.5 * 0.9
```

Design intention. Detonation sells. It requires the feature to be enabled, a position above the anchor, and a bullish consensus at or above the confidence minimum on the detonation timeframe, which defaults to one day. The check is edge triggered, so one bull reversal fires it once, and a rate limit spaces the checks. The executor then market-sells the surplus above the anchor, clears the fold tranches and reseeds the lots. It harvests the excess above the anchor, not the whole position.

src/trading/scrumming_bot.py — ScrummingBot._check_detonation_trigger

```
conf_min = float(getattr(self.config, "detonation_confidence_min", 0.75))
is_bullish = (
    summary.consensus_direction == SignalDirection.BULLISH
    and summary.consensus_confidence >= conf_min
)

fired = is_bullish and not self._detonation_last_signal_bullish
self._detonation_last_signal_bullish = is_bullish
```

Both features default to off, and the ceiling multiple defaults to five.

src/trading/container/config.py — BotConfig

```
position_ceiling_enabled: bool = False
position_ceiling_multiple: float = 5.0 # Range [1.0, 10.0]
```

Issue #336 records both pairs among the wizard settings that never reach the bot. Issue #107 records a detonation that can fire twice across a restart. Any earlier description reading "bearish" is wrong: the code compares against the bullish direction.

15 - Proof of Accumulation (PoA)

This is another one of the larger and more distinguishing features of the platform. At this point, it is a design proposal for a blockchain that provides certified trade performance between competitors without revealing their specific strategy tunings within Acervator or their identity. When this blockchain is initiated, ZERO tokens for it will exist in the world. I intend it to be, much like BTC to 1) have a fixed supply, 2) mean something through its generation and acquisition. Tokens are only created through the PoA tournaments and defeating Token Beasts or inflicting the most damage to them in a colosseum format. Additional functions would allow PoA tokens to be socketed into the platform to change its appearance and identify a user as a victor.

Functional. The competition package is the Proof of Accumulation package, and its engine runs without a screen. The engine walks a competition through registration, active trading, submission and adjudication, and hashes each entrant's config so a strategy proves consistency without publishing itself. A Merkle log produces the root that stands in for the trade record.

The token ledger is append-only and hard-capped. Balances come from replaying the log, and no operation edits one. An award that would cross the cap raises rather than trimming, and an award id repeats at most once.

src/competition/token_ledger.py — TokenLedger.award

```
event_id = self._event_id(bot_id, competition_id, tier.name)
if event_id in self._seen:
    return None

if self.total_minted() + tier.base_value > TOTAL_SUPPLY_CAP:
    raise OverflowError(
        f"Supply cap {TOTAL_SUPPLY_CAP:,} ACRV would be exceeded. "
        f"Only {self.remaining_ever()} tokens remain mintable."
    )
```

Design intention. The fixed supply is the whole point of the token, so the cap is a constant the ledger reads rather than a policy a caller can pass. It reads ten million.

src/competition/season_schedule.py — TOTAL_SUPPLY_CAP

```
TOTAL_SUPPLY_CAP = 10_000_000 # Hard cap – immutable
```

Nothing in the running platform starts a competition. Every module that would request one sits outside the running application, and the retired-tab sentinels assign nothing to the competition and testnet tabs. The canonical tab order now lists an Accumulation tab, and that tab is empty: it draws its name, one sentence saying it is not built, and issue #147, which covers the build-out.

src/competition/local_testnet.py — the in-platform chain, built every session

```
MAX_SUPPLY_WEI = TOTAL_SUPPLY_CAP * (10**TOKEN_DECIMALS)
```

The trophy generator draws the trophy as vector art and letters SOLVE ET COAGULA around its rim. No module imports it. Token Beasts and the colosseum appear in no Python file at all.

16 - Technical Analysis, Goodness of Fit, and the Confidence Index

The Confidence Index is the collated result of the 12-indicator Voting Panel and emulates the concept of a set of features (market structures / indicators) being read by a system having similar measurements for a given or expected or past action i.e. a Fold will always have a certain number of indicators voting bearish before its released for execution.

Functional. The Confidence Index runs. The voting engine builds twelve voters and holds twelve weights, and both count twelve when driven. The net is the bullish score minus the bearish score, and the confidence is the absolute net divided by the summed weight of the voters that did not abstain. Two bulls at weights one and two

against one bear at weight one give a net of 2.0 and a confidence of 0.5. A single bull beside an abstainer of weight three gives 1.0, not 0.25, which shows the abstainer leaving the denominator.

src/trading/ta_engine.py — VotingEngine._aggregate

```
net = bull_score - bear_score

voted_weight = sum(s.weight for s in signals if not s.abstained)
consensus_conf = abs(net) / voted_weight if voted_weight > 0.0 else 0.0
```

Design intention. Confidence measures the breadth of agreement across the panel, not the strength of one reading. That is why an abstention has to be declared rather than inferred: an indicator with too little history returns a fabricated middle reading, and counting it would dilute a real consensus.

src/trading/indicators/types.py — Signal.weighted_score

```
@property
def weighted_score(self) -> float:
    """Signed score: direction * confidence * weight."""
    return self.direction.value * self.confidence * self.weight
```

Goodness of fit is not built. The phrase names the work of comparing the Simulator against the live platform, and the criterion for that comparison is gates latching identically on the same data. The measure has no settled definition yet and nothing here to attach it to, so nothing is proposed. Issue #117 carries it.

In development.

Two standing defects touch the panel. The API Interaction Log still tells the operator that seven indicators read the candles (issue #417), and eight indicator implementations depart from their published formulae (issue #414).

17 - Tiered Strategy Validation and Workflow

Acervator has many unique characteristics and many of these are rooted in attempting to protect an investor from themselves. As such, I have designed a means of self-contained strategy testing that is meant to educate and validate before any attempt at using the platform against actual personal funds is ever attempted. Some may be confident or skilled enough to skip these protective steps. That is an individual user choice. I did not follow this workflow while developing it but I know exactly how my strategy works and when it is not. Given this, the intended workflow for a new user of Acervator should be Simulator > Paper Trader > Live. The system is configured so that the relevant operational elements, such as the Market Inspector, can inject Simulator and Paper equivalents into the appropriate Bot Swarm layers or bot fleet under the respective tab.

Functional. All four stages exist. The Inspector, Sim, Paper and Live tabs are all built and inserted. The Inspector no longer hands its proposals to the Sim tab: the rebuild removed that hook, and Sim and Paper each load a fleet from the stored bot record instead. Live is the tab the part list calls Trading, and it is third in the canonical order.

src/gui/main_tabs/main_window_surface.py — CANONICAL_TAB_ORDER

```
CANONICAL_TAB_ORDER = (  
    SIM_TAB,  
    PAPER_TAB,  
    LIVE_TAB,  
    CHARTS_TAB,  
    INSPECTOR_TAB,  
    SWARM_TAB,  
    ACCUMULATION_TAB,  
    HISTORY_TAB,  
    STATUS_TAB,  
    CONSOLE_TAB,  
)
```

The Paper stage is built. Issue #19 added `src/paper/`, the Qt tab and the React panel. Before that no file named for it existed in the tree and none had ever been committed on any branch, which is what the query below measured. One default still records the older state: the shelved stock window imports a module name that does not exist, so it adds no tab of its own.

`src/gui/main_tabs/stock_main_window_surface.py` — the declared default

```
paper_trader_available: bool = False,  
paper_trader_error: Any = "No module named 'src.gui.paper_trader_tab'",
```

Design intention. Paper is the stage that proves a strategy against the real market in real time before real money reaches it, and its defining property is that it runs at the market's own pace. What it shares with Live and what it forks is settled: one trading logic, three data sources. What it does not do is import the live stateful shells. That was a build, and issue #19 landed it after the Simulator rebuild.

Issue #19 covers the build-out. Two surfaces that told the operator a Paper Trader was present were issues #422 and #426, and both folded into #19. Issue #117 tracks the Simulator rebuild that gated it.

Today the workflow runs Market Inspector, then Simulator, then Paper, then Live. Every stage has a screen.

System Architecture and Features Catalogue

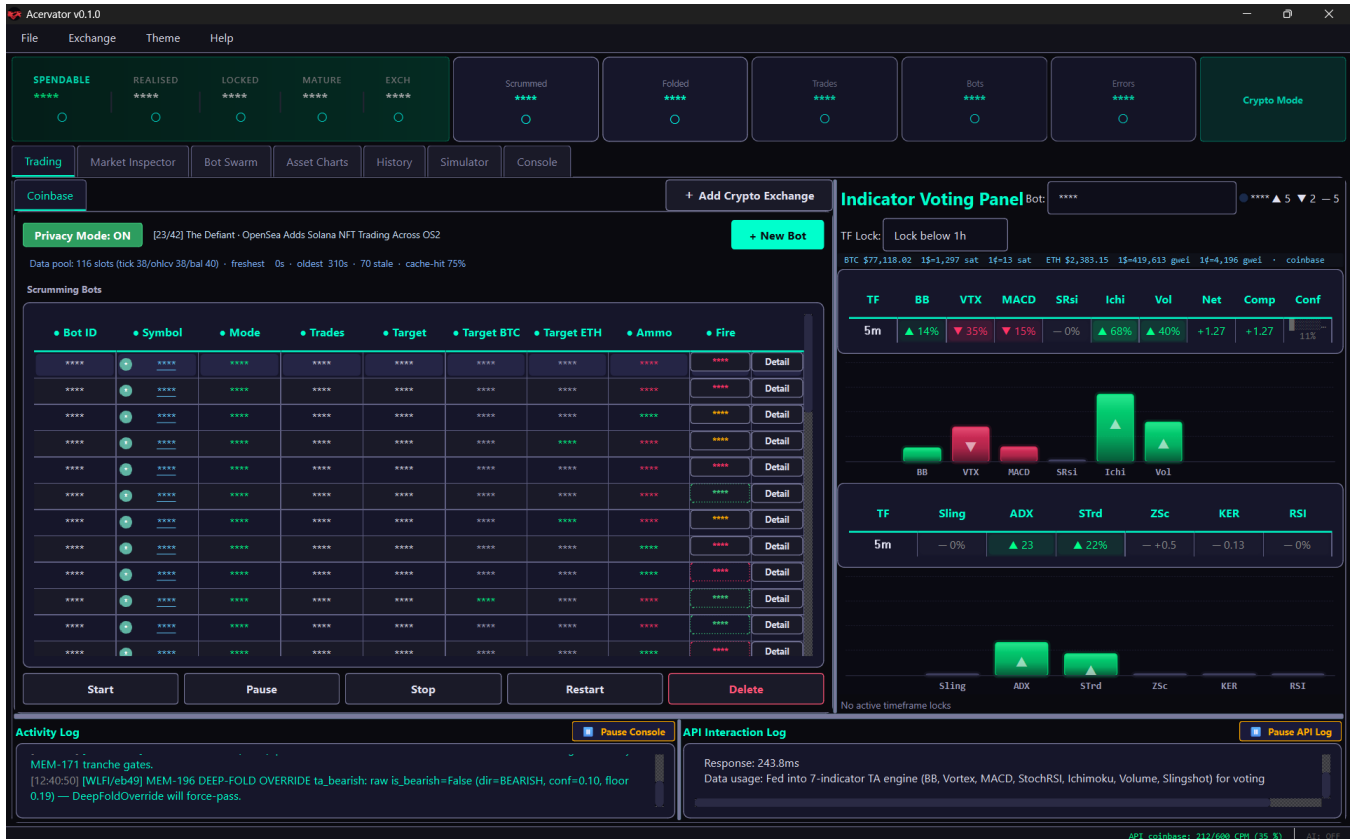
- Main Window
- Trading Tab - Active Bot Lists Under Exchange Tabs - Indicator Voter Panel - Trade Logic Activity - API Activity - Top-Level Portfolio Metrics
- Simulator Tab - Validation Runs vs. Historical Trade Data - Acervator Strategy Back Testing - Runs Historical Data Through Simulated Bots - Bot Swarm Simulation - Deploy As Paper Trading - Deploy As Live Trading
- Paper Trader Tab - Runs Live Exchange Data Through Simulated Bots - Executes and Documents Paper Trades - Collects Paper Trade Data - Pushes Trade Data to Paper Trade History Log - Paper Trade History Tab Reads and Displays
- Proof of Accumulation (PoA) Tab - PoA Blockchain and Trading Tournament Interface - Hosts User Competition Performance and Data - Displays Tournament Monsters and Targets
- Market Inspector Tab - Synthesizes and Proposes Bot Swarm Topologies and Oppositional Trades - Pushes Proposals to Sim, Paper, or Live As Directed
- Bot Swarm Tab - Displays and Allows Configuration of Capital Reinforcement Networks
- Asset Charts Tab - Displays Price Charts for All Active Bots
- History Tab - Loads and Displays Live Trade History Via Exchange API - Merges Live Trade History w/ Acervator Trade Log for Grading and Gate Analysis - Paper History Tab - Loads and Displays Paper Trade History w/ Grading and Gate Analysis
- Console Tab - Displays internal codebase activity, warnings and faults
- System Status Tab - Displays Platform Status by Subsystem - Fed by the Emitter Network
- Settings - System and High-Level User Settings

System Architecture and Features Catalogue

Acervator features a semi-modular / layered design that nests a hyper-vigilant trading engine capable of operating in multiple investment domains simultaneously and all from one terminal. The Main Window contains all of the various subsections found in each subsystem tab. As it stands the existing and planned subsystem tabs are: Simulator, Paper Trading, Trading (Live), Market Inspector, Bot Swarm, Asset Charts, History, Console, Proof of Accumulation (PoA), and System Status. Most of these subsystems interact with each other to some degree with key isolations existing between the three trading tabs and their wiring to Market Inspector, Bot Swarm, and History.

Trading Tab

Here you see the first subsystem we are going to cover and this will primarily be due to familiarizing the prospective investor or user with the core trading philosophy and strategies that drive Acervator. To begin, the platform runs locally on whichever hardware is selected. A single authorization phase requiring a purchased license key will be the only non-exchange communication the application will ever need. The user's keys and secrets are stored locally and encrypted after API handshake verification passes which allows the chosen exchange to be initialized. At a later development stage, I have plans for dedicated hardware that uses a hardware key for quick boot into a given user's account and also allows the platform to run in isolation under Linux.



The Trading tab, with Privacy Mode on.

This is the whole tab in one capture. The title bar and the menu row open it. The header strip and the tab row run under them. Below those the exchange sub-tab fills the left with the Scrumming Bots table and the command bar, the Indicator Voting Panel fills the right, and the Activity Log and the API Interaction Log close the foot. The status bar carries the API load pill and the AI state label. The parts below take that screen in that order.

Privacy Mode is on, so every masked field draws four asterisks and the bot table reads as ten columns of them. The title bar is the one reading no part below names. It carries the version the tree answered with on the day of the capture, v0.1.0. Nothing types that string out. Git answers for a source checkout and the baked file answers for a frozen bundle, so the title cannot name a release the build is not.

src/gui/main_window.py — the window title

```
self.setWindowTitle("Acervator v" + __version__ + "")
```

Press the mode button once and the title is rewritten to name the wing. The version leaves the title for the rest of the session, and only a restart brings it back. Issue #426 carries that handler.

src/gui/main_window.py — `_toggle_trading_mode`, the title after a swap

```
self.setWindowTitle("Acervator – CRYPTO WING")
```

The screen itself

Functional. One method builds this whole screen. It makes two layers, one for crypto and one for equities, and stacks them so only one is on show at a time. The crypto layer opens first. A second method builds the strip along the top, and that strip stays put on every tab. Privacy Mode is on in the figure, so each masked field draws four asterisks where its number would be.

src/gui/main_tabs/trading_tab.py — `TradingTabMixin._build_trading_tab`

```
self._trading_stack.addWidget(crypto_page) # index 0
self._trading_stack.addWidget(stock_page)  # index 1
self._trading_stack.setCurrentIndex(0)     # start in crypto
```

Design intention. The two-layer stack is the part of the multi-domain aim you can use today. Pressing the mode button swaps the whole wing, tables and Paper Trader together.

src/gui/main_window.py — `_toggle_trading_mode`

```
self._trading_mode = "stock"
self._mode_btn.setText("Stock Mode")
self._mode_btn.setChecked(True)
self._trading_stack.setCurrentIndex(1)
```

The Modulus Bot and the Paper Trading layer this section names have no module behind them. This manual marks them unbuilt where it reaches them.

The header strip

Functional. Five columns and five counter cards run along the top. The columns read SPENDABLE, REALISED, LOCKED, MATURE and EXCH. One call to `get_aggregate_stats` fills all of them once a tick. Spendable takes the wallet cash, Locked takes the value tied up in crypto, and Exch counts the open exchange sub-tabs. Realised and Mature are handed nothing at all, and the widget draws an em dash for each. Privacy Mode then masks that em dash to asterisks, which reads on screen as a hidden number rather than a missing one.

The five cards are Scrummed, Folded, Trades, Bots and Errors. Scrummed and Folded total the fleet's sold and bought dollars, Bots counts the bots that are running, and Errors totals the lifetime error count. Click Errors and the rolling error log opens. The small circle under every column and every card is a privacy dot, and it masks that one field on its own.

src/gui/main_window.py — `_refresh_dashboard`

```

if _wallet_cash > 0 or _crypto_value > 0:
    self._spendable_widget.update_profits(
        {
            "spendable": _wallet_cash,
            "total_realised": None,
            "locked": _crypto_value,
            "mature": None,
            "exchange_count": exchanges,
        }
    )

```

Design intention. The strip should answer one question at a glance: what the fleet holds, what it has earned, and what it has spent. Two of the five columns do not answer it. Realised profit and matured profit are the numbers those columns were built for, and nothing computes either one. The aggregate already carries a realised total, so the first half is a short change at the one call site.

Proposed, not present:

```

self._spendable_widget.update_profits(
    {
        "spendable": _wallet_cash,
        "total_realised": float(agg.get("total_realised_pnl", 0.0) or 0.0),
        "locked": _crypto_value,
        "mature": None,
        "exchange_count": exchanges,
    }
)

```

`total_realised_pnl` is already read two lines above this call, for the Scrummed card. Mature has no source yet and stays an em dash. Issue #418 carries this.

08-tabs/portfolio-panels.md covers the strip in full.

Both columns are now fed, and both are fed from the exchange. The window no longer writes the payload itself. It calls the same builder the React strip calls, so one function decides what the five columns hold and neither host can drift from the other.

src/gui/main_window.py — `_refresh_dashboard`

```

self._spendable_widget.update_profits(
    header_strip_surface.profits_payload(agg, exchanges)
)

```

Realised profit is the exchange's own figure, matched buy against sell. Mature profit is the profit on positions that have grown past two hundred per cent over what they cost. Neither is computed from the platform's internal running totals, because the venue is the authority on money.

src/gui/main_tabs/header_strip_surface.py — `profits_payload`

```
"total_realised": exchange_amount(data, "total_realized_exchange"),
"mature": exchange_amount(data, "total_mature_exchange"),
```

Where the exchange has answered for no bot, both columns stay empty and say so. The aggregate counts the bots the venue has answered for, and at zero the two columns are handed nothing rather than a computed zero, so an empty column is never a figure the platform made up.

src/gui/main_tabs/header_strip_surface.py — exchange_amount

```
answered = int(data.get(EXCHANGE_FRESHNESS_KEY, 0) or 0)
if answered <= 0:
    return None
```

An empty column stays empty under Privacy Mode. The mask replaces a number with four asterisks and leaves the empty marker alone, so the operator can always tell a hidden figure from a missing one.

src/core/privacy_mask_registry.py — mask_or

```
text = str(value)
if field_id not in ALL_FIELD_IDS or text == ABSENT_TEXT:
    return text
```

The tab row

Functional. The row of main tabs takes its order from one list. Each label in the list is moved to its own index at build time. Any tab the list does not name keeps the position it was added at.

src/gui/main_tabs/main_window_surface.py — the order, declared once

```
CANONICAL_TAB_ORDER = (
    SIM_TAB,
    PAPER_TAB,
    LIVE_TAB,
    CHARTS_TAB,
    INSPECTOR_TAB,
    SWARM_TAB,
    ACCUMULATION_TAB,
    HISTORY_TAB,
    STATUS_TAB,
    CONSOLE_TAB,
)
```

Each of those ten names is a constant holding the label the tab bar shows, and the main window applies the order once, after the last builder has run.

src/gui/main_window.py — where the order is applied

```
self._reorder_main_tabs(list(CANONICAL_TAB_ORDER))
```

Design intention. The order should read as the promotion order. Practise first, then paper, then real money, then the screens that inspect the trade. One list decides it, so the order cannot drift as tabs are added.

src/gui/main_window.py — the move loop

```
tab_bar = self._main_tabs.tabBar()
for target_idx, name in enumerate(desired):
    for cur_idx in range(self._main_tabs.count()):
        if self._main_tabs.tabText(cur_idx) == name:
            if cur_idx != target_idx:
                tab_bar.moveTab(cur_idx, target_idx)
            break
```

One sub-tab per exchange

Functional. One sub-tab holds one exchange. Along its header sit Privacy Mode, a news headline, and the + New Bot button that opens the wizard for this venue. Privacy Mode toggles all eighteen masks at once. Under the header is a data pool line: how many slots are held by kind, the age of the freshest and the oldest, how many have run past their time to live, and the cache hit ratio. At the right of the sub-tab row sits the button that adds another venue.

src/gui/widgets/exchange_tab.py — ExchangeTab.__init__

```
self._privacy_mode_btn = QPushButton("Privacy Mode: OFF")
self._privacy_mode_btn.setToolTip(
    "Toggle ALL 18 privacy masks at once. When ON, every "
```

Design intention. One venue, one tab, one swarm. Add a second exchange and it arrives beside the first with its own bots and its own data pool, and nothing about the first changes.

src/gui/main_window.py — _sync_exchange_tabs

```
def _sync_exchange_tabs(self) -> None:
    """Add a tab for each configured exchange missing one, in its own layer."""
```

The news line

Functional. The headline between Privacy Mode and + New Bot is one item from the crypto news ticker. The counter in front of it gives that item's place in the batch the ticker holds, so a reading of 23 of 42 marks the twenty-third headline of forty-two. The item itself carries the feed name, a middle dot, then the story title. A timer steps to the next item and wraps at the end of the batch.

src/gui/crypto_news_ticker.py — the line the strip draws

```
_prefix = f"[{self._index + 1}/{len(self._headlines)}] "
self._label.setText(_prefix + h.display_text())
```

Design intention. The counter tells you the batch is whole. A feed that came back short shows a smaller second number rather than a strip that looks the same and carries less. With nothing reachable the strip says

so in words instead of going blank.

src/gui/crypto_news_ticker.py — the empty state

```
self._label.setText("(no crypto news feeds reachable)")
```

Functional. In the React build the strip is drawn by `crypto_news_ticker.js` into the header space the exchange screen keeps between the Privacy Mode button and + New Bot. The exchange screen names that space only when the strip is there; where the strip would not build, the header takes plain space instead and the two buttons keep their positions.

src/gui/main_tabs/exchange_tab_surface.py — `react_news_ticker`

```
def react_news_ticker() -> str:
    """The module that draws the news strip, as ``build_news_ticker`` reads it.

    ``ExchangeTabModel`` calls this as its ``news_ticker_factory``, so
    ``news_ticker`` is set and the header keeps the strip's space.
    """
    return NEWS_TICKER_MODULE
```

Functional. Hovering the headline pauses the step timer and moving away starts it again. Clicking the headline opens that story. Each of the three sends one request and redraws the strip from the answer, so the strip on screen matches what the backend holds.

src/gui/main_tabs/crypto_news_ticker_surface.py —
`CryptoNewsTickerModel.handle_event`

```
if event_type == EVENT_ENTER:
    self.paused = True
    self.calls.append([HOVER_PAUSED])
    return False
if event_type == EVENT_LEAVE:
    self.paused = False
    self.calls.append([HOVER_RELEASED])
    return False
```

The bot tables

Functional. The Scrumming Bots table carries ten columns. Nine are named and the tenth is blank, because that one holds the Detail button. Mode is the coloured cell: green while running, amber while paused, grey while idle or stopped, red on error, orange in cooldown, cyan while starting. The Ammo cell draws green above the target, red below it, and neutral grey inside the dust band. Target BTC and Target ETH restate the Target in those two assets, and go blank when the pair is unlisted or when the target already is that asset. The Extractor table sits underneath. Both tables start hidden and appear when their own list gains a row.

src/gui/widgets/bot_status_table.py — `BotStatusTable.SCRUMMING_COLUMNS`

```

SCRUMMING_COLUMNS = ColumnSpec(
    labels=(
        "Bot ID",
        "Symbol",
        "Mode",
        "Trades",
        "Target",
        "Target BTC",
        "Target ETH",
        "Ammo",
        "Fire",
        "",
    ),

```

Functional. Column 2 is Current Position Value. It shows what the bot's holdings are worth at the exchange's own price. It is blank whenever no fresh exchange price exists, and a blank cell names the missing thing in its tooltip: no position held, no exchange price for the pair yet, no exchange price this tick, or a price older than twenty seconds. The cell never falls back to a last-known figure, to a stand-in, or to a value read out of the bot's own ledger. The state colour now sits on the Bot ID cell, which is green while running, amber while paused, grey while idle or stopped, red on error, orange in cooldown and cyan while starting. That cell's tooltip names the mode and the state.

src/gui/main_tabs/table_cells_surface.py — the one multiplication both priced cells read, so the Position Value cell and the Ammo cell can never disagree

```

def priced_position(holdings: float, price: float, quote_rate: float) -> float:
    """The position value at one price: ``holdings`` times ``price`` times
    ``quote_rate``."""
    return holdings * price * quote_rate

```

Functional. The price both cells read comes from the shared exchange price cache, and it carries its own age. An age of None means the cache held nothing and the bot's own last reading was used instead, which is why the Position Value cell goes blank on that path.

src/gui/main_tabs/table_cells_surface.py — the four blank paths and the one priced path

```

POSITION_PATH_PRICED = "priced"
POSITION_PATH_NO_HOLDINGS = "no_holdings"
POSITION_PATH_NO_PRICE = "no_price"
POSITION_PATH_OFF_EXCHANGE = "off_exchange"
POSITION_PATH_AGED = "aged"

```

Design intention. The Ammo cell measures against the live target, not the frozen number typed into the wizard, so the reading follows the grown balance the engine re-zeroes to.

src/gui/widgets/bot_status_table.py — the target the Ammo cell measures against


```

target_val = float(
    status.get("live_target_balance", status.get("target_balance", 0.0))
    or status.get("target_balance", 0.0)
    or 0.0
)

```

Functional. In the React build the Scrumming Bots table is drawn by `bot_status_table.js`. The exchange screen keeps a named empty space for it and fills that space when it draws itself. The rows come from the backend, not from the exchange screen: the renderer asks the bridge for the table's own state and names the exchange it wants rows for. Each exchange gets its own table state, so two exchange screens on one page never share rows or a highlight.

`src/gui/web/exchange_tab.js` — `mountScrumTable`

```

function mountScrumTable(target, model) {
    var loader = global[LOAD_BOT_TABLE];
    var wait =
        typeof loader === "function" ? loader(askFor(model)) : Promise.resolve(null);
    return Promise.resolve(wait).then(function () {
        return renderScrumTable(target, exchangeOf(model)) === null
            ? null
            : BOT_TABLE_MODULE;
    });
}

```

Functional. The four things the operator can press on the table each send one request and redraw from the answer. A column header toggles that column's privacy mask, a Symbol cell opens the chart address, Fire hands the bot to Manual Fire, and Detail selects the row and opens the bot. `on_detail` is the handler behind the Detail button.

`src/gui/main_tabs/bot_status_table_surface.py` — `BotStatusTableModel.on_detail`

```

def on_detail(self, bot_id: str) -> None:
    """Select the row, then hand the bot to whatever opens the detail."""
    self.select_row_for_bot(bot_id)
    self.detail_clicks.append(bot_id)
    self.calls.append([DETAIL_CLICKED, bot_id])
    if self.on_bot_clicked:
        self.on_bot_clicked(bot_id)

```

Design intention. The exchange screen redraws itself once a second to keep the data-pool line fresh. Each redraw re-fills the table space, so the table is drawn from the state the renderer holds for that exchange rather than from the answer captured at first paint. A row the operator selected therefore survives the next redraw.

`src/gui/web/bot_status_table.js` — `modelFor`

```

function modelFor(exchangeId) {
    return owns(models, String(exchangeId)) ? models[String(exchangeId)] : null;
}

```

Functional. The Extractor table underneath is drawn the same way, by `extractor_bot_table.js` into the second named space the exchange screen keeps. Its rows come from the same fleet list, kept to the records whose mode is extractor. The Detail button is the one control the Extractor table offers; Fire stays disabled on this screen, because Manual Fire is per position and lives in the Positions Held tab of the bot's own window.

`src/gui/main_tabs/extractor_bot_table_surface.py` — `extractor_statuses`

```
def extractor_statuses(statuses: Any) -> list:
    """The records of ``statuses`` whose mode is ``MODE_TEXT``.

    ``update_bots`` builds a row for every record it is handed, so only the
    Extractor bots reach the Extractor table.
    """
    return [
        found
        for found in statuses or []
        if isinstance(found, dict) and found.get("mode") == MODE_TEXT
    ]
```

Functional. The exchange screen mounts its three children in one step, each into its own space and each asked for its own view model.

`src/gui/web/exchange_tab.js` — `mountChildren`

```
function mountChildren(target, model) {
    return Promise.all([
        mountScrumTable(target, model),
        mountExtractorTable(target, model),
        mountNewsTicker(target, model)
    ]).then(function (drawn) {
        return drawn.filter(function (name) {
            return name !== null;
        });
    });
}
```

The command bar

Functional. Start, Pause, Stop, Restart and Delete all act on one bot. Two tables share the bar, so the bar takes its bot from whichever table you clicked last. If that table holds no selection it tries the other one. If neither holds a selection it says "Select a bot first." and does nothing.

`src/gui/widgets/exchange_tab.py` — `ExchangeTab._cmd`

```
def _cmd(self, command: str) -> None:
    if self._last_clicked_table == "extractor":
        bot_id = self._extractor_table.get_selected_bot_id()
        if not bot_id:
            bot_id = self._bot_table.get_selected_bot_id()
    else:
        bot_id = self._bot_table.get_selected_bot_id()
        if not bot_id:
            bot_id = self._extractor_table.get_selected_bot_id()
```

Design intention. One command bar for two tables means the bar has to guess which bot you meant. The last table you clicked is the tie-breaker, and the fallback keeps a stray click from swallowing the command. With nothing selected anywhere it refuses out loud rather than acting on a guess.

src/gui/widgets/exchange_tab.py — the refusal

```
if not bot_id:
    if self._status_log:
        self._status_log.log("Select a bot first.", "warning")
    return
```

The rest of the screen

The right half is the Indicator Voting Panel, described at the end of this section. The two panes at the foot are the Activity Log and the API Interaction Log. The status bar carries the API load pill written by `_refresh_api_load_pill` and the AI: state label.

The pill names the venue, the calls that venue took in the last minute, the ceiling for it and the two as a percentage. It reads the worst-loaded connected venue, so one busy exchange cannot hide behind a quiet one.

src/gui/main_window.py — the pill text

```
text = (
    f"API {worst.exchange}: "
    f"{worst.calls_per_minute:.0f}/"
    f"{worst.ceiling_cpm:.0f} CPM ({pct} %)"
)
```

Its colour is the warning. Green under half load, amber above that, red past the venue's safety percentage. With no venue connected the pill draws an em dash and no number.

src/gui/main_window.py — the three colours

```
if worst.load_score > mon.safety_pct:
    colour = ds.ERROR
elif worst.load_score > 0.5:
    colour = ds.FOLD_RATIO_AMBER
else:
    colour = ds.SUCCESS
```

1 - Add Exchange - User provides valid API key and secret for target exchange - Platforms validates with an API handshake - Exchange Initializes

After an exchange is connected to the platform, bots can be added to target entire or portions of existing positions as well as using standing liquidity to enter positions for the first time. The two primary bots used by Acervator are known as Scrumming and Extractor with the conceptual Modulus Bot to be added later. The first two are the active traders that continuously interact with the market whereas the Modulus Bot will be a macro-strategic interface designed to trigger portfolio-scale shifts in response to custom market signals and it can best be visualized as a modular synthesizer with investment and market-specific functions.

Scrumming Bot

If Acervator has a crown jewel, this is it. The scrumming bot is what houses and executes the harvest-fold method and the harvest-fold method is what leads to the creation of the platform itself. All else stems from this origin point and the simple revelation that allowed the method to be discovered (or probably rediscovered) by nonmathematical eyes. Instead of paying attention to what my entire portfolio was doing, I opted to start focusing on a single position until I had found a strategy that was more reliable than Grid Bots or standard speculation. I was convinced such a method existed and thought it absurd that price charts could not be played better when there was obviously so much room for improvement. You do not have to predict the market. You flow and mold your portfolio to it as time goes. Scrumming allows profits to be shaved off, held and re-investment in a cyclical, reliable manner that sizes its trades in direct proportion to actual market movement as it relates to position value drift. Trading in this manner allows a fixed balance for a position to be maintained and asserts the truth that "Position X is Y value. Any deviation from Y is a Target Delta and is subject to a Scrum (Sell) or Fold (Buy)." The primary weaknesses to this strategy are not have an equal amount of liquidity to the position value (a \$500 Scrumming Bot should be supported by \$500 of liquidity when initialized) and severe market downturns will little or no short term upside which can lock most liquidity for the position into the Target Asset. Scrumming is a survival strategy and is directionally agnostic but also depends on the market to cycle. I also refer to this simply as Accumulation Trading.

Scrumming Bot Terms

Target Balance - The initial value of the position to be taken or controlled by the Scrumming Bot. The bot will monitor the market for bullish or bearish conditions, check for Target Balance deviations (Target Delta), and re-zero back to the set point. It will repeat this until stopped by the user or some other market condition.

The figure is one field on the bot's own configuration. It is carried in from the wizard and held for the life of the bot, and every other term on this page is measured against it.

src/trading/container/config.py — BotConfig.target_balance

```
target_balance: float = 200.0 # Balance the bot trades relative to
```

Target Delta - The amount by which a position value has drifted from the Target Balance. The Target Delta is denoted by the Ammo column under the Scrumming Bot list of the Trading Tab. This is the amount of value that will be fired during the appropriate market conditions.

One subtraction, and the platform does it in two places with opposite signs. The Ammo cell takes the position value less the target, so a surplus reads positive. The fold path takes the target less the position, so a deficit reads positive. Both measure the same drift.

src/gui/table_cells.py — ammo_cell, the figure the Ammo column shows

```
delta = position_val - target_val
territory = target_territory(position_val, target_val)
```

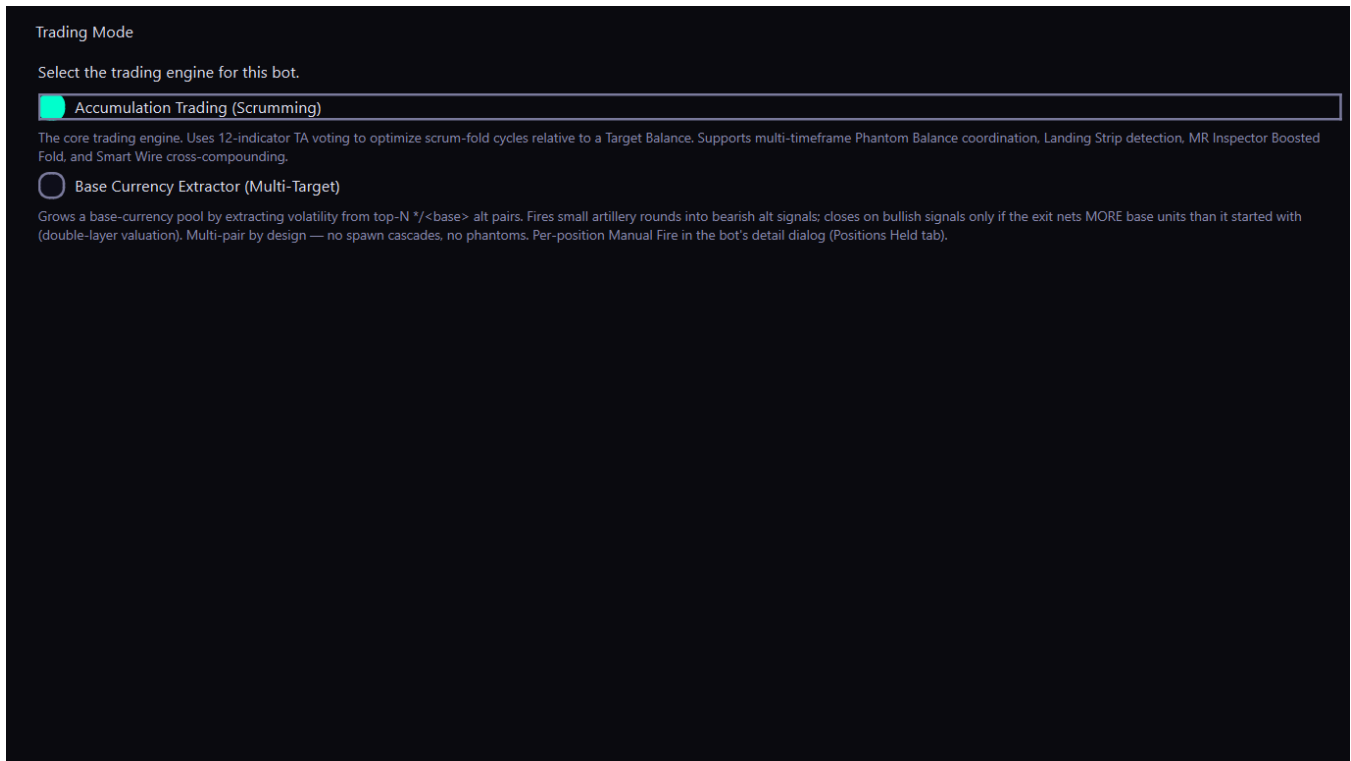
Scrum - To sell an amount from an investment position that allows it to return to its initial price level. This never closes the position. This is the first half of the infinitely divisible circle that can persist for such positions in a healthy market.

One shared helper decides which half of the cycle a position sits in. It answers scrum above the target and fold below it, and it answers at target inside a dust band, so a position that has barely moved is left alone.

src/trading/target_bands.py — target_territory

```
delta = float(position_value) - float(target_balance)
band = at_target_dust_band(target_balance)
if delta > band:
    return "scrum"
if delta < -band:
    return "fold"
return "at_target"
```

Fold - To buy an amount for an investment position that allows it to return to its initial price level. This also drives Compounding Growth based on Local Volatility and is restricted by the Maximum Growth Per Cycle setting which has been set to a conservative 1% globally for Acervator's live test and development run.



The wizard's first page: Trading Mode, with Scrumming selected.

When the +New Bot button is pressed, the above window appears. Currently there are two “trading mode”

options (this will be change to Strategies). We are only interested in the Scrumming Bot at this point.

Functional. This is the wizard's first page, and the wizard opens on it. Two radio buttons, one description under each. Accumulation Trading is already selected when the page opens, and its description promises 12-indicator voting, which is what the engine builds. Choosing Base Currency Extractor sends you to the Extractor Pool page instead of the asset page. One question decides that branch, and the rest of the wizard asks the same question whenever it needs to know which kind of bot it is building.

A third mode, Grid, is retired. Its test returns False without looking at a widget, and the branch that reads it is marked unreachable in the source.

src/gui/bot_wizard.py — BotCreationWizard.nextId

```
def nextId(self):
    current = self.currentId()
    if current == PAGE_MODE:
        if self._mode_page.is_extractor():
            return PAGE_EXTRACTOR_POOL
        return PAGE_ASSET
```

Functional. In the React build the wizard is drawn by bot_wizard.js into a space the exchange screen keeps for it. The screen keeps that space only after + New Bot is pressed. The press asks the surface, and the surface answers with the name of the module that draws the wizard.

src/gui/main_tabs/exchange_tab_surface.py — react_bot_wizard

```
def react_bot_wizard(exchange_id: str) -> str:
    """The module that draws the Create Auto Trader wizard for ``exchange_id``.

    ``ExchangeTabModel`` calls this as its ``on_new_bot``, so
    ``on_new_bot_clicked`` puts the module name in ``bot_wizard``.
    """
    return BOT_WIZARD_MODULE
```

Functional. Cancel closes the wizard. Finish closes it from the last page and refuses from any other. Either close drops the module the screen holds, so the space goes with the wizard and the exchange screen underneath it is whole again.

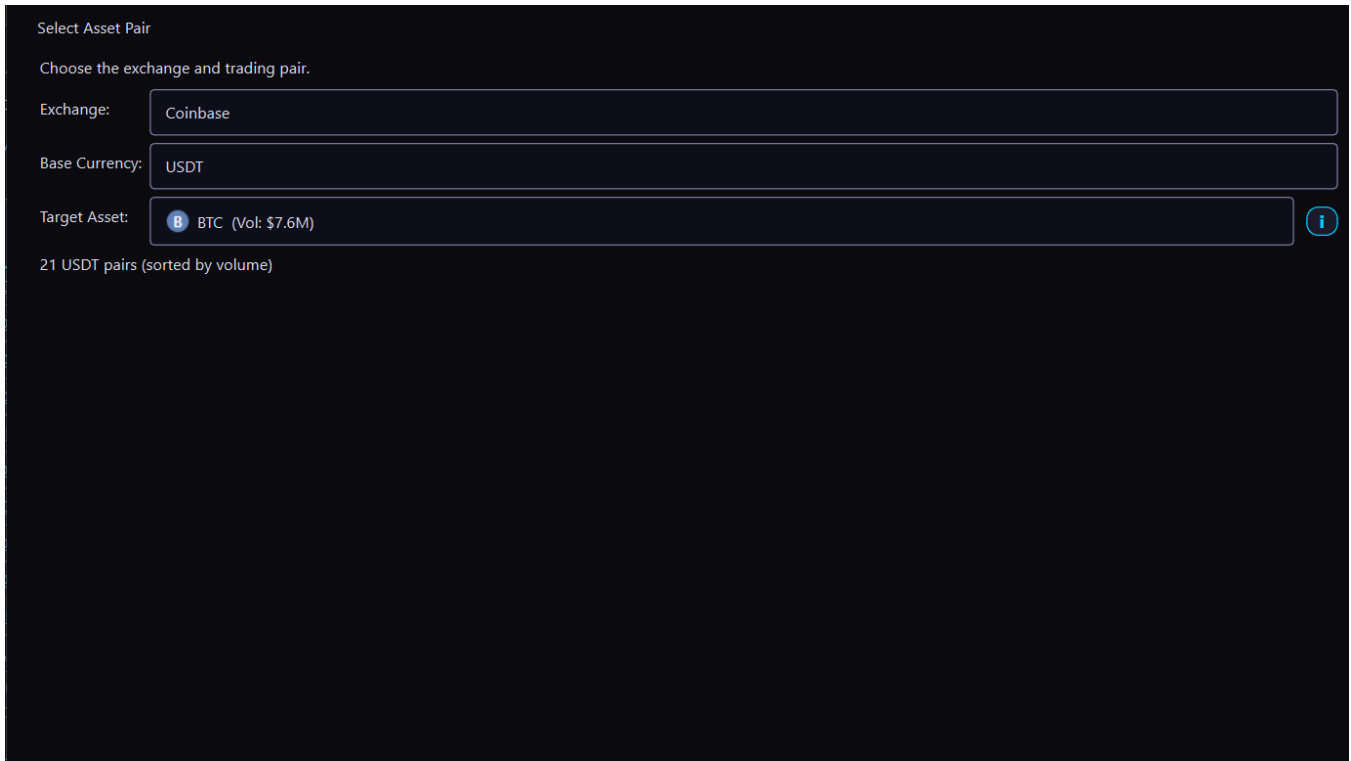
src/gui/main_tabs/exchange_tab_surface.py — ExchangeTabModel.close_bot_wizard

```
def close_bot_wizard(self) -> None:
    """Drop the wizard, so the screen keeps no space for it."""
    if self.bot_wizard is None:
        return
    self.bot_wizard = None
    self.calls.append([BOT_WIZARD_CLOSED, self.exchange_id])
```

Functional. Every control on the page sends the steps taken so far, not the one just pressed. The surface lays out fresh pages on each call and keeps nothing between them, so the page holds the walk and sends the whole of it. The answer replaces the payload the page holds and the wizard draws again.

src/gui/web/bot_wizard.js — press

```
function press(name, step) {
  lastPress = { name: name, step: step };
  dispatched.push(lastPress);
  remember(step);
  lastAnswer = hasBridge()
    ? global.acervator.call(METHOD, copyOf(heldSteps)).then(take)
    : Promise.resolve(null);
  return lastPress;
}
```



Select Asset Pair

Choose the exchange and trading pair.

Exchange:

Base Currency:

Target Asset: ⓘ

21 USDT pairs (sorted by volume)

The wizard's asset page: exchange, base currency and target asset.

After Scrumming / Accumulation is selected, next we are presented with Exchange, Base Currency, and Target Asset options.

Exchange - A privately and federally licensed financial platform that allows API interfacing for remote or automated trade execution.

The row lists the venues already connected. Picking one re-scans that venue and keeps only the spot markets the venue itself marks active.

src/gui/bot_wizard.py — the market filter inside `_fetch_markets`

```

for sym, info in exch.markets.items():
    if (
        not info.get("active", True)
        or info.get("type", "spot") != "spot"
    ):
        continue

```

Base Currency - This will be a National Currency, Stable Coin, or High Volume Crypto (BTC, ETH, BNB) for which multiple trading pairs are available on the selected exchange.

The page offers a fixed list of seven, and it holds exactly the national currencies, stable coins and high-volume crypto named above.

src/gui/bot_wizard.py — AssetSelectionPage, the base list

```

self._base.addItem(["USDT", "USDC", "BTC", "ETH", "BNB", "EUR", "USD"])

```

Target Asset - This is the asset in which the scrumming bot bases its position. If \$50 and ZEC are selected, it will strive to maintain and compound against a balance of \$50 in ZEC until it is shut down or some other adverse market condition occurs.

The list holds every asset trading against the base you chose, each with a cached coin icon and, where the venue reported one, a 24-hour volume figure. The round information button opens a written description of the selected asset, and the line under the rows counts the pairs found. Sorting by volume puts the tradeable pairs at the top of a list that runs to several hundred entries on a large venue, and the sort reads a cached figure rather than fetching one.

src/gui/bot_wizard.py — the volume sort

```

# Cached volume only, since a network fetch here blocks the GUI thread.
filtered.sort(
    key=lambda m: _market_number(m.get("volume")) or 0.0, reverse=True
)

```

Trading Parameters

Next we come to the combined Scrumming Bot configuration page which has several sections for fine tuning how the specific instance behaves. Given that Acervator, at the time of this writing, is still in active development the description for each setting should be seen as a design intention should any issues be encountered.

Trading Parameters

Configure target balance, scrumming interval, and compounding.

Trading Parameters

Order Visibility: Order Book (Visible)

☐ Aggressive Trading (force IOC-limit takers)

☐ Stack Mode (split SCRUM across upward tranches)

Split Distance: 1.00 %

Tranche Count: 3

Spacing: Linear (1, 2, 3, 4...)

Personal Hold (units): 0.0000000000

The Trading Parameters group of the wizard's parameter page.

Order Visibility - Trades are listed on the order books or tracked internally to the platform.

Two entries, Order Book (Visible) and Internal (Invisible). ScrummingBot reads the choice once at construction and holds it as its invisible flag. The label a reader sees and the value the bot stores are two different strings on the same row.

src/gui/bot_wizard.py — the Order Visibility row

```
self._visibility = QComboBox()
self._visibility.addItem("Order Book (Visible)", "orderbook")
self._visibility.addItem("Internal (Invisible)", "internal")
self._visibility.setToolTip("How orders appear on the exchange.")
self._visibility.currentIndexChanged.connect(self._on_visibility_changed)
mf.addRow("Order Visibility:", self._visibility)
```

Aggressive Trading - Trades are priced so that they fill immediately. Trading like this is a bit like guerilla warfare. In and out before anyone notices.

A checkbox, off at the start. Every engine-initiated order then leaves as an immediate-or-cancel limit priced through the spread, so it pays the taker fee for an immediate fill. Manual Fire is unaffected.

src/gui/bot_wizard.py — the Aggressive Trading row

```
self._aggressive = QCheckBox("Aggressive Trading (force IOC-limit takers)")
```

Stack Mode - Stack Mode enables Stack Tranches which operate on the Sell or Scrum side. This forms the “upside” of the organic ladder structure whereas Fold Tranches form its “downside”.

The box takes its start state from the engine rather than from a second literal typed onto the page.

src/gui/bot_wizard.py — TradingParamsPage, the Stack Mode default

```

from ..trading.bot_container import STACK_MODE_DEFAULT

self._stack_mode = QCheckBox(
    "Stack Mode (split SCRUM across upward tranches)"
)
# One declaration, so the box and the config cannot disagree.
self._stack_mode.setChecked(STACK_MODE_DEFAULT)

```

Split Distance - This setting determines the spacing between tranches if Tranche Spread (not available yet) is being used.

A percentage from 0.10 to 20.00, at 1.00 % to start. The bot hands it to the stack maths as the gap between one tranche and the next, and the Spacing row below decides how that gap grows across the ladder.

src/gui/bot_wizard.py — the Split Distance row

```

self._split_distance = QDoubleSpinBox()
self._split_distance.setRange(0.1, 20.0)
self._split_distance.setDecimals(2)
self._split_distance.setSuffix(" %")
self._split_distance.setValue(1.0)

```

Tranche Spread - This allows a given Scrum or Fold to divide its result X# of times across multiple incremented (as dictated by Split Distance) positions.

The page carries no control of that name, and no setting named `tranche_spread` reaches the engine.

The same sweep run for `split_distance` returns a declaration, a restore entry and a reader, so the sweep itself finds a setting when one is there.

In development.

Tranche Count - This can also be referred to as Spread Count. It determines how pieces a given Scrum or Fold is split into and distributed across incremented tranches as opposed to just one.

A whole number from 2 to 20, at 3 to start, written out as `stack_tranche_count_target`. It is a target rather than a promise: the runtime count drops when a tranche would fall under the venue minimum, or when two computed prices land within a tenth of a percent of each other and merge.

src/gui/bot_wizard.py — the Tranche Count row

```

self._stack_count = QSpinBox()
self._stack_count.setRange(2, 20)
self._stack_count.setValue(3)

```

Spacing - This adds a scaling factor Split Distance and works in conjunction with Tranche Spread and Tranche Count to induce curves and more aggressive growth within the ladder structure.

Three entries: Linear, Quadratic and Exponential. The sequence beside each name is the cumulative distance from the anchor in units of Split Distance.

src/gui/bot_wizard.py — the Spacing row

```

self._stack_spacing = QComboBox()
self._stack_spacing.addItem("Linear (1, 2, 3, 4...)", "linear")
self._stack_spacing.addItem("Quadratic (1, 2, 4, 7...)", "quadratic")
self._stack_spacing.addItem("Exponential (1, 2, 4, 8...)", "exponential")

```

Personal Hold - Setting to be removed.

The control is still on the page. `TradingParamsPage.get_config` still emits `personal_hold_qty`, and the capital reservation mixin still reads it, adding it to the units the bot claims against its siblings. A removal has that reader to retire with it.

`src/trading/scrumming/capital_reservation_mixin.py` — `_compute_reservation_qty`

```

_hold = float(getattr(self.config, "personal_hold_qty", 0.0) or 0.0)

```

`src/gui/bot_wizard.py` — `TradingParamsPage.get_config`, the settings this group emits, in the order of the rows above

```

"stack_mode": self._stack_mode.isChecked(),
"split_distance": self._split_distance.value(),
"stack_tranche_count_target": int(self._stack_count.value()),
"stack_spacing_mode": self._stack_spacing.currentData(),
"personal_hold_qty": float(self._personal_hold_qty.value()),

```

The same method writes `visibility` and `aggressive_trading` before it branches on the kind of bot, so an `Extractor` carries those two as well.

Trading Parameters

Configure target balance, scrumming interval, and compounding.

Scrumming Settings

Opposing Trade Interval:	1.00 %
BB Tolerance:	1.00 %
Landing Strip Candles:	3 candles
TA Timeframe:	1h
Target Balance:	\$ 200.00
Max Entry Price:	\$ 0.00000000
Min Entry Price:	\$ 0.00000000
Trading Fee %:	0.60 %
Max Target Growth %:	1.00 %
Scrum Fold Ratio:	100 %

The Scrumming Settings group.

Scrolling down we next find the first block of Scrumming Settings. These are the core or basic metrics for a Scrumming Bot.

Opposing Trade Interval - Establishes the minimum travel distance required by price action from the point a given trade in order the next trade of the opposite type to occur.

A percentage from 0.10 to 20.00, at 1.00 % to start. The Trading Fee below is added to it, so the real distance a reversal must travel is the two together.

src/gui/bot_wizard.py — the Opposing Trade Interval row

```
self._scrumming_interval = QDoubleSpinBox()
self._scrumming_interval.setRange(0.1, 20.0)
self._scrumming_interval.setDecimals(2)
self._scrumming_interval.setSuffix(" %")
self._scrumming_interval.setValue(1.0)
```

BB Tolerance - Determines the minimum distance of the Bollinger Band extent price action must be in order for a trade action to occur.

A percentage from 0.25 to 5.00, at 1.00 % to start. The band-proximity detector takes it as its tolerance. This is the narrowest range on the page, and it cannot be set to zero.

src/gui/bot_wizard.py — the BB Tolerance row

```
self._bb_tolerance = QDoubleSpinBox()
self._bb_tolerance.setRange(0.25, 5.0)
self._bb_tolerance.setDecimals(2)
self._bb_tolerance.setSuffix(" %")
self._bb_tolerance.setValue(1.0)
```

Landing Strip Candles - Determines the strictness of Landing Strip detection. Minimum is three candles. Longer Landing Strips are historically more likely to indicate an impending market reversal than shorter ones assuming the taper remains intact or grows tighter.

A whole number of candles from 2 to 10, at 3 to start. The widget accepts 2, one below the minimum of three the description names, and the number reaches only one of the two landing-strip detectors. Issue #432 carries this.

src/gui/bot_wizard.py — the Landing Strip Candles row

```
self._ls_candles = QSpinBox()
self._ls_candles.setRange(2, 10)
self._ls_candles.setValue(3)
self._ls_candles.setSuffix(" candles")
```

TA Timeframe - This is the timeframe at which the bot operates and denotes the price chart it will monitor for trade decisions.

Seven entries to start, with 1h chosen. Pick an exchange and the list is rebuilt from the timeframes that venue carries, so a venue without 4h does not offer it. The starting choice is an index into the list rather than a named

timeframe.

src/gui/bot_wizard.py — the TA Timeframe row

```
self._ta_timeframe = QComboBox()
```

```
self._ta_timeframe.setCurrentIndex(4) # Default 1h
```

Target Balance - This is the intended starting and locked value for the investment position that the Scrumming Bot is controlling.

From \$1.00 to \$1,000,000.00. Its start value is whatever default the wizard was handed, which is the figure the Settings Trading page stores.

src/gui/bot_wizard.py — the Target Balance row

```
self._target_balance = QDoubleSpinBox()  
self._target_balance.setRange(1.0, 1000000.0)  
self._target_balance.setDecimals(2)  
self._target_balance.setPrefix("$ ")  
self._target_balance.setValue(defaults.get("default_target_balance", 200.0))
```

Max Entry Price - If the new bot does not detect the requisite amount (as dictated by Target Balance) of the Target Asset, this price threshold sets a limit at which it will attempt to perform the initiating Fold.

Eight decimal places, at \$0.00000000, where zero means no ceiling. The buy path is the only reader, and it refuses a buy above the ceiling. Manual Fire goes around it.

src/gui/bot_wizard.py — the Max Entry Price row

```
self._max_entry_px = QDoubleSpinBox()  
self._max_entry_px.setRange(0.0, 10_000_000.0)  
self._max_entry_px.setDecimals(8)  
self._max_entry_px.setPrefix("$ ")  
self._max_entry_px.setValue(0.0)
```

Min Entry (Should Be Exit) Price - If the new bot detects a requisite amount (as dictated by the Target Balance) of the Target Asset, this price threshold sets a limit at which it will attempt to perform the initiating Scrum.

The same shape, where zero means no floor. The buy path is the only reader here too, so the number refuses a buy below the floor and never reaches a sell. The row is labelled Min Entry Price on the page, without the parenthesis his line carries.

src/gui/bot_wizard.py — the Min Entry Price row

```
self._min_entry_px = QDoubleSpinBox()  
self._min_entry_px.setRange(0.0, 10_000_000.0)  
self._min_entry_px.setDecimals(8)  
self._min_entry_px.setPrefix("$ ")  
self._min_entry_px.setValue(0.0)
```

Trading Fee % - This allows the trading fee for the target exchange to be set. This is added to Minimum Opposing Trade Distance to further ensure buys / sells are properly distant and that a given bot is not losing an excessive amount to fee chop in volatile but overly tight market regimes.

A percentage from 0.00 to 5.00, at 0.60 % to start, which is the Coinbase Advanced Trade maximum tier. It steps in twentieths of a percent, and it is a per-side figure.

src/gui/bot_wizard.py — the Trading Fee row

```
self._trading_fee = QDoubleSpinBox()
self._trading_fee.setRange(0.0, 5.0)
self._trading_fee.setSuffix(" %")
self._trading_fee.setDecimals(2)
self._trading_fee.setSingleStep(0.05)
self._trading_fee.setValue(0.6)
```

Max Target Growth % - This determines the maximum amount of growth the Target Balance can increase in a given Market Cycle with a cycle being a Fold / Scrum / Fold sequence. Essentially any Fold preceded by a Scrum will be allowed to Fold an amount of profit back in and, if Surplus remains after the Target Delta is re-zero'd, it can be used to increase Target Balance up to this hard limit for that cycle. This is the organic compounding mechanic.

A percentage from 0.00 to 100.00, at 1.00 % to start. Setting it to zero freezes Target Balance. It steps a quarter of a percent at a time, and it is the only mechanism allowed to raise the figure.

src/gui/bot_wizard.py — the Max Target Growth row

```
self._max_target_growth_pct = QDoubleSpinBox()
self._max_target_growth_pct.setRange(0.0, 100.0)
self._max_target_growth_pct.setSuffix(" %")
self._max_target_growth_pct.setDecimals(2)
self._max_target_growth_pct.setSingleStep(0.25)
self._max_target_growth_pct.setValue(1.0)
```

Scrum Fold Ratio - This precedes Surplus calculation as described under Max Target Growth %. It determines how much of a given trade's profits will be redistributed to directly contribute to its own organic compounding. Note that this does not interfere with normal Target Delta re-zeroing and is intended to only serve as a "cushion" to slow runaway compounding when Wire Credits are being received from multiple sources.

A whole percentage from 1 to 100, at 100 % to start.

src/gui/bot_wizard.py — TradingParamsPage.get_config, the settings this group emits, in the order of the rows above

```

"scrumping_interval_pct": self._scrumping_interval.value(),
"bb_tolerance_pct": self._bb_tolerance.value(),
"bb_landing_strip_candles": self._ls_candles.value(),
"ta_timeframe": self._ta_timeframe.currentData(),
"target_balance": self._target_balance.value(),
"max_entry_price": (float(_max_ep) if _max_ep > 0 else None),
"min_entry_price": (float(_min_ep) if _min_ep > 0 else None),
"trading_fee_pct": self._trading_fee.value(),
"max_target_growth_pct": self._max_target_growth_pct.value(),
"scrum_fold_pct": self._scrum_fold_pct.value(),

```

Where the code departs. Two of these rows read differently in the engine than the entries above say.

Min Entry Price is the first. The entry above writes the correction into its own heading, and the code follows the label on screen rather than that correction. The buy path is the only place either entry-price field is read; it refuses a buy above the ceiling and refuses a buy below the floor. Neither number reaches the sell path, so a bot that holds the asset and reaches that price sells nothing. Manual Fire runs its own rebalance and reads neither.

src/trading/scrumping/execution.py — `_execute_buy`

```

_max_ep = getattr(self.config, "max_entry_price", None)
_min_ep = getattr(self.config, "min_entry_price", None)
try:
    _px = float(price) if price is not None else 0.0
except (TypeError, ValueError):
    _px = 0.0
if _max_ep is not None and _px > 0 and _px > float(_max_ep):

```

The sell path already takes the price, so the floor check has somewhere to attach.

Proposed, not present, in `_execute_sell`:

```

_min_ep = getattr(self.config, "min_entry_price", None)
if _min_ep is not None and price > 0 and price < float(_min_ep):
    self._bus.emit(
        "bot.log",
        bot_id=self.bot_id,
        message=(
            f"SELL REFUSED (min_entry_price floor): current price "
            f"${price:.8f} < ${float(_min_ep):.8f}."
        ),
    )
    return None

```

`min_entry_price` is declared in `src/trading/container/config.py` and reaches the bot through the wizard block above, so the proposal adds no new setting.

Max Target Growth % is the second. The engine holds two answers for a bot whose stored config lacks the key. Ten read sites take it with a fallback: seven fall back to 1.0 and three fall back to 0.0. A bot compounds or freezes depending on which site read it first. The declared default is 1.0, and the three outliers should say the

same.

Proposed, not present, at each of the three sites:

```
_growth = float(getattr(self.config, "max_target_growth_pct", 1.0) or 0.0)
```

The three outliers are the manual rebalance, the compounding snapshot and the SWOS inputs. Issue #409 carries this.

```
src/trading/scrumming/execution.py    _execute_manual_rebalance
src/trading/scrumming/snapshots.py    _compounding_snapshot
src/trading/scrumming_bot.py          get_swos_inputs
```

The screenshot shows a dark-themed web interface titled "Trading Parameters". Below the title is a subtitle: "Configure target balance, scrumming interval, and compounding." The interface is divided into two main sections: "Advanced Scrumming (P1.9)" and "Hedge Rebalance".

Advanced Scrumming (P1.9)

- Detect Threshold:** A slider set to 75 %.
- Fire Threshold:** A slider set to 0.50 %.
- BB Midline Gate:** A toggle switch that is turned on (indicated by a red square).
- Read Rate:** A slider set to 5 min.
- Band Travel:** A slider set to 70 %.
- BB Bullseye Check:** A toggle switch that is turned on (indicated by a red square).
- Wire Inflow Stack:** A slider set to 1.00 %.

Hedge Rebalance

- Hedge Rebalance Active:** A toggle switch that is turned on (indicated by a red square).
- Hedge Balance:** A slider set to \$ 200.00.

The Advanced Scrumming and Hedge Rebalance groups.

Next are the “advanced” Scrumming settings which primarily affect when the bot is allowed to fire a trade. These can be thought of as “calibrating the scope”.

Detect Threshold - Intended as the point at which the bot “takes the safety off” and starts looking for a shot. To be re-evaluated.

A whole percentage from 10 to 90, at 75 % to start. The engine reads 75 as a lower mark of 0.125 and an upper mark of 0.875, measured across the band rather than in dollars.

src/trading/scrumming/circuit_breakers.py — _bb_detect_thresholds

```
detect_frac = max(0.0, min(1.0, detect_pct / 100.0))
half = detect_frac * 0.5
return (0.5 - half, 0.5 + half)
```


Fire Threshold - The final Bollinger Band approach metric. Once satisfied, the bot can fire a trade.

A percentage from 0.10 to 10.00, at 0.50 % to start. It measures distance from the band, where the Detect Threshold above it measures distance from the midline.

src/gui/bot_wizard.py — the Fire Threshold row

```
self._scrum_fire_pct = QDoubleSpinBox()
self._scrum_fire_pct.setRange(0.1, 10.0)
self._scrum_fire_pct.setDecimals(2)
self._scrum_fire_pct.setSuffix(" %")
self._scrum_fire_pct.setValue(0.5)
```

BB Midline Gate - This is another, perhaps redundant layer, of Bollinger Band travel protection. It is different in that it is concerned with distance from the midline instead of the entire local width.

A checkbox, on at the start. While it is on, a scrum fires only above the midline and a fold only below it.

src/gui/bot_wizard.py — the BB Midline Gate row

```
self._bb_midline_gate = QCheckBox("BB Midline Gate")
self._bb_midline_gate.setChecked(True)
```

Read Rate - To be re-evaluated.

From 1 to 60 minutes, at 5 minutes to start. It sets the search-mode read rate; track mode reads ten times faster. Fire mode reads at that same faster rate.

The wizard writes it as `scrum_read_rate_min`, bot creation passes it through, and the bot reads it at the top of every tick. The range starts at 1, so this control cannot switch the throttle off.

src/trading/scrumming_bot.py — `ScrummingBot.tick`, the throttle

```
if self.config.scrum_read_rate_min > 0 and not self._manual_fire_pending:
    _tick_sec = max(self.tick_interval, 0.1)
    _base_skip = max(1, int((self.config.scrum_read_rate_min * 60) / _tick_sec))
    self._tick_skip_search = _base_skip
    if self._scrum_target_mode in ("track", "fire"):
        self._tick_skip = max(1, _base_skip // 10)
    else:
        self._tick_skip = _base_skip
```

Band Travel - Previously described. To be re-evaluated.

A whole percentage from 0 to 100, at 70 % to start. Zero switches it off. The wizard writes it as `band_travel_pct` and bot creation passes it through.

The engine reads it once per evaluation. It measures how far price has moved since the last trade, as a share of the current band width, and raises a second trigger once price covers that share. The same trigger overrides a trend hold, so Band Travel can release a trade the trend gates were holding.

src/trading/scrumming_bot.py — the band-travel trigger

```

_bb_width = max(bb_result.upper - bb_result.lower, 1e-12)
band_travel_frac = abs(ticker.last - self._last_trade_price) / _bb_width
if (
    abs(ticker.last - self._last_trade_price)
    >= self.config.band_travel_pct / 100.0 * _bb_width
    and delta > 0
):
    band_travel_triggered = True

```

BB Bullseye Check - If current price and Bollinger Band thresholds are equal, the user can opt to perform a double-sized trade.

A checkbox, on at the start.

src/gui/bot_wizard.py — the BB Bullseye row

```

self._bb_bullseye = QCheckBox("BB Bullseye Check")
self._bb_bullseye.setChecked(True)

```

Wire Inflow Stack - To be re-evaluated.

A percentage from 0.00 to 100.00, at 1.00 % to start. The wizard writes it as wire_inflow_stack_pct, and one method in the engine reads it. Above zero, wire income arriving while the position sits within that band of its target and within that band of its entry price goes onto the target and queues an aggressive buy, rather than spreading over the fold queue.

Bot creation does not pass this setting, so a new bot takes the declared default of 1.00 whatever you type here. The declared default and the wizard's start value are the same number, so the loss shows only once you change it. Bot Settings can set it on a bot that is already running. Issue #336 carries this.

src/trading/scrumping/wire_routing.py — WireRoutingMixin.apply_wire_income

```

try:
    stack_pct = float(getattr(self.config, "wire_inflow_stack_pct", 1.0) or 0)
except (TypeError, ValueError):
    stack_pct = 0.0
_stack_eligible = False
_stack_reason = ""
_target = 0.0
_entry_px = 0.0
if stack_pct > 0:

```

Hedge Rebalance Active - Determines if Current Price drifting below Initial Entry Price will have a limited amount of funds that can be used to keep re-zeroing the Target Delta at key bearish thresholds or areas of possible reversal.

A checkbox, on at the start. It opens the one group on the page that holds a reserve outside Target Balance.

src/gui/bot_wizard.py — the Hedge Rebalance row

```
self._hedge_rebalance = QCheckBox("Hedge Rebalance Active")
self._hedge_rebalance.setChecked(True)
```

Hedge Balance - Sets a limit on the amount of additional liquidity a given bot is allowed to absorb when Current Price drifts below Initial Entry Price.

At \$200.00 to start, a reserve the bot holds apart from Target Balance.

src/gui/bot_wizard.py — TradingParamsPage.get_config, the settings these two groups emit, in the order of the rows above

```
"scrum_detect_pct": self._scrum_detect_pct.value(),
"scrum_fire_pct": self._scrum_fire_pct.value(),
"bb_midline_gate": self._bb_midline_gate.isChecked(),
"scrum_read_rate_min": self._scrum_read_rate.value(),
"band_travel_pct": self._band_travel_pct.value(),
"bb_bullseye_check": self._bb_bullseye.isChecked(),
"wire_inflow_stack_pct": self._wire_inflow_stack_pct.value(),
"hedge_rebalance_active": self._hedge_rebalance.isChecked(),
"hedge_balance": self._hedge_amount.value(),
```

The group title on screen carries an internal release identifier after the words Advanced Scrumming. Issue #420 carries that, and four more group titles with it.



The Circuit Breakers group.

Now we arrive at some safety controls. Circuit Breakers are designed to fully inhibit trade actions for a given period should an extreme volatility (pump and dump) event occur. Soft Circuit Breakers have a candle-count based timer whereas Hard Circuit Breakers require the user to clear the bot to continue trading.

Soft CB Threshold - The amount of instantaneous, single-candle price action required for the bot to pause trading for a number of candles denoted by Soft CB Cooldown.

A percentage from 0.0 to 100.0, at 25.0 % to start. Zero switches it off. It interrupts only the side of the market that moved, so an upward candle stops scrums and a downward one stops folds.

src/gui/bot_wizard.py — the Soft CB Threshold row

```
self._cb_soft_pct.setSuffix(" %")
self._cb_soft_pct.setValue(25.0)
```

Hard CB Threshold - The amount of instantaneous, single-candle price action required for the bot to be hard stopped at which point the user must re-authorize trading.

The same range, at 35.0 % to start. Zero switches it off. Where the soft breaker interrupts one side, this one pauses the bot outright and the pause survives a restart.

src/gui/bot_wizard.py — the Hard CB Threshold row

```
self._cb_hard_pct = QDoubleSpinBox()
self._cb_hard_pct.setRange(0.0, 100.0)
self._cb_hard_pct.setDecimals(1)
self._cb_hard_pct.setSuffix(" %")
self._cb_hard_pct.setValue(35.0)
```

Soft CB Cooldown - This is the number of candles that must close before the Soft Circuit Breaker opens again.

From 1 to 100 candles, at 3 to start. It is counted in closed candles, so its real length follows the bot's own timeframe.

src/gui/bot_wizard.py — the Soft CB Cooldown row

```
self._cb_cooldown = QSpinBox()
self._cb_cooldown.setRange(1, 100)
self._cb_cooldown.setValue(3)
```

Max Cartridge Size - This is the maximum amount of deviation allowed for the Target Delta. At this threshold the bot is actively and aggressively looking for a trade opportunity.

A percentage from 0.0 to 200.0, at 10.0 % to start. Crossing it fires an aggressive rebalance that bypasses the detection, hysteresis and soft-breaker checks. It is the one figure on the page whose range runs past 100.

src/gui/bot_wizard.py — the Max Cartridge Size row

```
self._max_cartridge_pct = QDoubleSpinBox()
self._max_cartridge_pct.setRange(0.0, 200.0)
self._max_cartridge_pct.setDecimals(1)
self._max_cartridge_pct.setSuffix(" %")
self._max_cartridge_pct.setValue(10.0)
```

Smart Cartridge - This allows the Max Cartridge Size to organically resize in response to current price range as defined by the current-candle Bollinger Band reading.

One checkbox labelled Calibrate to BB range, off at the start. While it is on, the cartridge size comes from the current band range instead of the fixed percentage above it.

src/gui/bot_wizard.py — the Smart Cartridge row

```
self._cartridge_smart_chk = QCheckBox("Calibrate to BB range")
self._cartridge_smart_chk.setChecked(False)
```

Smart Ceiling - To be re-evaluated.

A percentage from 1.0 to 100.0, at 30.0 % to start. It caps the cartridge threshold while Smart Cartridge is on.

src/gui/bot_wizard.py — TradingParamsPage.get_config, the settings this group emits, in the order of the rows above

```
"circuit_breaker_soft_pct": self._cb_soft_pct.value(),
"circuit_breaker_hard_pct": self._cb_hard_pct.value(),
"circuit_breaker_cooldown_candles": self._cb_cooldown.value(),
"max_cartridge_size_pct": self._max_cartridge_pct.value(),
"max_cartridge_smart": self._cartridge_smart_chk.isChecked(),
"max_cartridge_smart_ceiling_pct": self._cartridge_smart_ceiling.value(),
```

A trip stops a trade at one gate on each side of the chain, and 07-indicators.md lists both chains in full.

src/trading/gate_chain.py — the two breaker gates in the built chains

```
CircuitBreakerGate(side="scrum"),
```

src/trading/gate_chain.py — and the fold chain

```
CircuitBreakerGate(side="fold"),
```

The screenshot shows a dark-themed GUI titled "Trading Parameters". Below the title is a subtitle: "Configure target balance, scrumming interval, and compounding." There are two main sections:

- Risk Controls (MEM-244)**: This section contains four controls:
 - ☐ Enable Position Ceiling
 - Ceiling Multiple: A dropdown menu showing "5.0x anchor".
 - ☐ Enable Detonation (auto-harvest on bullish TF)
 - Detonation TF: A text input field showing "1d".
 - Min Confidence: A dropdown menu showing "0.75".
- Strategy Gate Flags (v3.16.15)**: This section contains five checkboxes, all of which are checked:
 - SCRUM requires bullish TA
 - SCRUM holds in sustained uptrend
 - SCRUM defers to higher-TF bullish
 - FOLD requires bearish TA
 - FOLD defers to higher-TF bearish

The Risk Controls and Strategy Gate Flags groups.

After Circuit Breakers, which help defend against extreme volatility, we come to Risk Controls. These are designed to cap the amount of profit or growth a given bot can earn before performing a full position exit.

Enable Position Ceiling - Enables a growth cap for a given position.

A checkbox, off at the start. Nothing in the Risk Controls group acts until it is on.

src/gui/bot_wizard.py — the Position Ceiling row

```
self._position_ceiling_enabled = QCheckBox("Enable Position Ceiling")
self._position_ceiling_enabled.setChecked(False)
```

Ceiling Multiple - This setting caps the maximum amount of growth a position at a multiple of the Target Balance (anchor) and, once reached (and under higher timeframe bullish conditions with Detonation enabled) will allow the entire position to be sold and the corresponding bot will pause all further operations. Without Detonation enabled, this becomes a user notification.

From 1.0 to 10.0, at 5.0x anchor to start. The anchor is the target balance the bot was created with, not the balance it has grown to. It steps half a multiple at a time and carries its unit in the box.

src/gui/bot_wizard.py — the Ceiling Multiple row

```
self._position_ceiling_multiple = QDoubleSpinBox()
self._position_ceiling_multiple.setRange(1.0, 10.0)
self._position_ceiling_multiple.setDecimals(1)
self._position_ceiling_multiple.setSingleStep(0.5)
self._position_ceiling_multiple.setSuffix("x anchor")
self._position_ceiling_multiple.setValue(5.0)
```

Enable Detonation - Enables an entire remaining position to be sold after the Ceiling Multiple growth threshold is crossed.

A checkbox, off at the start. Detonation needs the box ticked, a position worth more than its anchor, and a bullish reading at or above the confidence you set. It also needs the reading to have just turned bullish, so a tape that was already bullish last time fires nothing.

src/trading/scrumming_bot.py — ScrummingBot._check_detonation_trigger

```
conf_min = float(getattr(self.config, "detonation_confidence_min", 0.75))
is_bullish = (
    summary.consensus_direction == SignalDirection.BULLISH
    and summary.consensus_confidence >= conf_min
)

fired = is_bullish and not self._detonation_last_signal_bullish
```

Detonation TF - Selects the timeframe for the chart that is being evaluated for bullish conditions that will allow the detonation to occur.

Two entries, 1d and 1w. The label and the stored value are the same string on both rows, which is not true of every picker in the wizard.

src/gui/bot_wizard.py — the Detonation TF row

```
self._detonation_timeframe = QComboBox()  
self._detonation_timeframe.addItem("1d", "1d")  
self._detonation_timeframe.addItem("1w", "1w")
```

Min Confidence - This is the minimum technical analysis confidence index (via the Indicator Voting Panel) that will allow the detonation to occur.

From 0.50 to 1.00, at 0.75 to start.

src/gui/bot_wizard.py — TradingParamsPage.get_config, the settings this group emits, in the order of the rows above

```
"position_ceiling_enabled": self._position_ceiling_enabled.isChecked(),  
"position_ceiling_multiple": self._position_ceiling_multiple.value(),  
"detonation_enabled": self._detonation_enabled.isChecked(),  
"detonation_timeframe": self._detonation_timeframe.currentData(),  
"detonation_confidence_min": self._detonation_confidence_min.value(),
```

Next we use the Strategy Gate Flags which allows top-level trade restrictions to be enabled or disabled thus relaxing or restricting the conditions under which a trade action can occur. The settings, in this case, are self-descriptive.

Functional. The box draws five checkboxes, all ticked at the start: SCRUM requires bullish TA, SCRUM holds in sustained uptrend, SCRUM defers to higher-TF bullish, FOLD requires bearish TA, and FOLD defers to higher-TF bearish.

src/gui/bot_wizard.py — TradingParamsPage.get_config, the six flags this group emits

```
"scrum_require_ta_bullish": self._gate_scrum_ta_chk.isChecked(),  
"scrum_hold_in_uptrend": self._gate_scrum_uptrend_chk.isChecked(),  
"scrum_defer_to_htf": self._gate_scrum_htf_chk.isChecked(),  
"fold_require_ta_bearish": self._gate_fold_ta_chk.isChecked(),  
"fold_hold_in_downtrend": True, # reserved, no gate  
"fold_defer_to_htf": self._gate_fold_htf_chk.isChecked(),
```

Design intention. Six flags are declared, the box shows five, and the missing one is the fold-side twin of a scrum gate that works. It goes out as a hard True, and nothing in the engine reads it. Its scrum mirror holds a sell during a sustained uptrend, and the fold twin would hold a buy during a sustained downtrend.

Proposed, not present, in src/trading/gate_chain.py:

```

class FoldTrendHoldGate(Gate):
    """FOLD-only: hold the buy while a sustained downtrend runs."""

    name = "trend_hold_fold"
    side = "fold"

    def evaluate(self, ctx: GateContext) -> GateResult:
        if not ctx.eff_trend_hold:
            return GateResult(passed=True)
        return GateResult(
            passed=False,
            blocker_message=f"trend_hold_fold({ctx.trend_strength:.0%})",
        )

```

The gate would join the fold chain beside its scrum twin, and `fold_hold_in_downtrend` in `src/trading/container/config.py` would arm it. A checkbox has to arrive with it, or the flag stays a hard True. Issue #419 carries this.



Profit Routing (v3.20.85)

Route: Fold back to target balance

Target bot ID: leave blank unless route = cross_bot

The Profit Routing group.

Moving onto the final section, we have Profit Routing which was intended to allow profits to be routed differently during initial set-up. This will be re-evaluated and potentially removed.

The group writes two settings and stores them with the bot. No module under `src/trading/` reads either one. The wizard records the route, Bot Settings can change it, and a restart restores it. None of that reaches a trade: `_route_scrum_proceeds_via_wires` moves the scrum proceeds, and it never asks what the route says.

The bot config declares both fields with a default, so the destination a bot holds is always the first entry in the list. Issue #336 carries this, with the rest of the settings the wizard writes and bot creation drops.

`src/trading/container/restore.py` — the round-trip that puts both fields back on a restarted bot

```

"profit_route": cfg.get("profit_route", "fold_to_target"),
"profit_route_bot_id": cfg.get("profit_route_bot_id", ""),

```

Route - Destination for profits.

Four destinations: fold back to target balance, send to spendable, split fold and spendable by percentage, and route to another bot. Only the last of the four makes the Target bot ID field below it mean anything.

`src/gui/bot_wizard.py` — the Route row


```

self._profit_route = QComboBox()
self._profit_route.addItem("Fold back to target balance", "fold_to_target")
self._profit_route.addItem("Send to spendable", "spendable")
self._profit_route.addItem("Split fold/spendable per %", "split")
self._profit_route.addItem("Route to another bot (cross-bot)", "cross_bot")

```

Target bot ID - Field for manually a bot ID which was intended to create a Smart Wire under the Bot Swarm tab.

A free-text field. Its placeholder tells you to leave it blank unless you chose the cross-bot route.

src/gui/bot_wizard.py — TradingParamsPage.get_config, the settings this group emits, in the order of the rows above

```

"profit_route": self._profit_route.currentData(),
"profit_route_bot_id": self._profit_route_bot_id.text().strip(),

```



The Phantom Balance Bots page.

Lastly we have the selection for the Phantom (Balance) Bots. These are intended to provide trade action overrides from higher timeframe charts and indicator sets which, in turn, may result in an improved trade or prevent a premature one.

Functional. This is the last page on the scrumming path:

- Enable Phantom Balance Bots, a checkbox, off at the start.
- Active Timeframes, eleven checkboxes: 1m, 5m, 15m, 30m, 1h, 2h, 4h, 6h, 12h, 1d and 1w. All start clear. A timeframe the chosen venue does not carry is greyed out and cleared, with the reason written into its tooltip.
- Candles to lock, 1 to 10, at 2, inside the Higher-TF Lock Duration group.

The page hands on only the boxes that are both ticked and available. Before it lets you leave, it asks the API load monitor whether that many phantoms would breach the safety threshold for the venue, and offers you Back to adjust or Continue anyway.

src/gui/bot_wizard.py — PhantomConfigPage.get_config

```

def get_config(self):
    checked = [
        tf
        for tf, cb in self._tf_checks.items()
        if cb.isChecked() and cb.isEnabled()
    ]
    return {
        "enable_phantoms": self._enable.isChecked(),
        "phantom_timeframes": checked,
        "lock_candle_count": self._lock_candles.value(),
    }

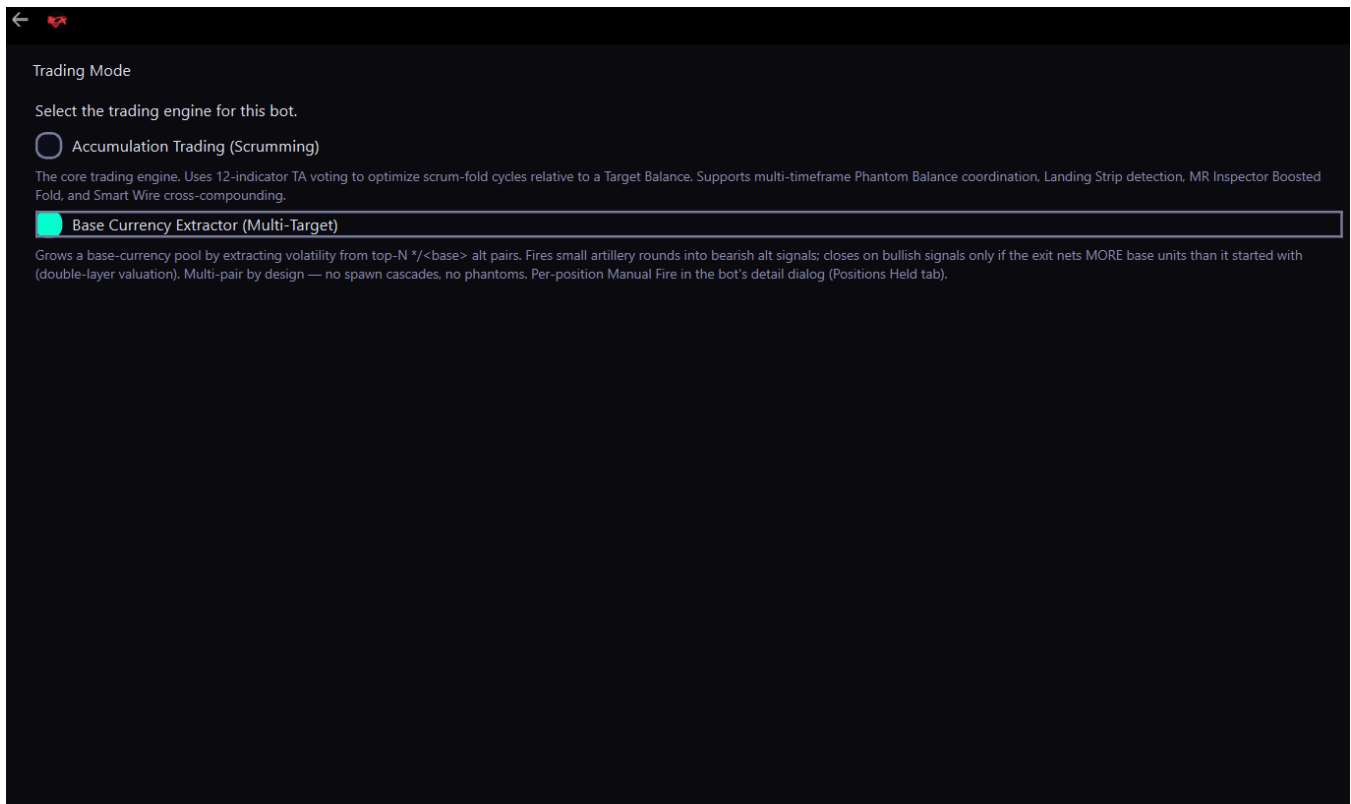
```

Design intention. The engine behind this page is unfinished. The Indicator Voting Panel section of this manual records that phantom bots are still in active development and that related features do not yet work, so nothing is proposed for the page.

In development.

Extractor Bot (Partially Built; Untested)

The second type of bot offered within Acervator is the Extractor. These operate quite differently from Scrumming Bots and actually operate as Siblings of them. In fact, an Extractor Bot cannot even be called unless a corresponding Base Currency Scrumming Bot (i.e. BTC:USD or ETH:USD) is already active. This is due to the core operating principle of the Extractor bot to acquire more of these base currencies by performing trades against available alternate currency pairings. It does this by using and blocking off a portion of the Parent's position within an Extractor Tranche that represents an active position taken against one of the available alternate pairs. The Extractor Tranche remains open until its opposing accumulating (or Short Position if preferred by the user) or profit taking trade is filled. Extractor Tranches can be of any size but should generally be a relatively small fraction of the Parent's total position which will allow the Extractor to take multiple positions if available and allowed by the specific user.



The Trading Mode page with the Extractor selected.

Functional. The same first page as before, with the other radio chosen. From here the wizard goes to the Extractor Pool page instead of the asset page. Phantom bots and profit folding are both switched off for this bot as it is built, and an Extractor never sees the Phantom page at all.

src/gui/bot_wizard.py — BotCreationWizard.get_bot_config

```
config.update(self._params_page.get_config())
if config["mode"] == "extractor":
    config["enable_phantoms"] = False
    config["profit_folding_active"] = False
else:
    config.update(self._phantom_page.get_config())
```

Design intention. Phantom overrides and profit folding belong to the parent, and the two hard False values above are that decision written down. The mode is stamped onto the config at the very top of the same method, so nothing downstream has to guess which kind of bot it received.

src/gui/bot_wizard.py — the mode stamp

```
if self._mode_page.is_extractor():
    config["mode"] = "extractor"
    config.update(self._extractor_pool_page.get_config())
else:
    config["mode"] = "scrumming"
    config.update(self._asset_page.get_config())
```

Extractor Pool

Choose the base currency the pool accumulates and select target alt pairs from the exchange scan. Leave all unchecked to use auto-scan (top-N by volume).

Exchange:

Pool Base Currency:

24 */BTC pairs available (sorted by 24h volume). Leave all unchecked for auto-scan (top-N by volume).

Target alt pairs (multi-select):

☐ ETH
☐ SOL
☐ LINK
☐ ATOM
☐ UNI
☐ AAVE
☐ BCH
☐ DOGE
☐ AVAX
☐ DOT
☐ ...

The Extractor Pool page.

Functional. This page draws:

- Exchange, the connected venues. Changing it re-scans that venue.
- Pool Base Currency, a fixed list of five. It names the asset the pool accumulates.
- A line counting the pairs available against that base.
- Target alt pairs, a check list of every alt that trades against the base. Leave every box clear and the bot picks its own pairs by volume at run time.
- Select all and Clear act on the whole list.

The page hands on a single asterisk where a Scrumming Bot would hand on one target asset. Its own docstring calls that the pool sigil. An Extractor holds a pool of alts rather than one target, and the sigil keeps the field's shape for the code downstream that builds a trading symbol.

src/gui/bot_wizard.py — ExtractorPoolPage._POOL_BASES

```
_POOL_BASES = ["BTC", "ETH", "USDT", "USDC", "BNB"]
```

src/gui/bot_wizard.py — ExtractorPoolPage.get_config

```
return {
    "exchange_id": self._exchange.currentData(),
    "base_currency": base,
    "target_asset": "*", # pool sigil – multi-pair indicator
    "extractor_alt_targets": checked,
}
```

Design intention. The five pool bases are exactly the assets named as base currencies above. Leaving the list empty hands the choice back to the bot, and only a box that is both ticked and available reaches the config.

src/gui/bot_wizard.py — how a ticked alt is collected

```
for i in range(self._alt_list.count()):
    item = self._alt_list.item(i)
    if item.checkState() == Qt.Checked:
        sym = item.data(Qt.UserRole)
        if sym:
            checked.append(sym)
```

Trading Parameters

Configure base-currency chunk, artillery sizing, and compounding-tier policy.

Extractor — Pool Artillery

Chunk size (USD):	\$100.00	▲ ▼
Artillery size (USD):	\$5.00	▲ ▼
Watch list top-N:	8	▲ ▼
Watch list refresh:	60 candles	▲ ▼
Pool reserve:	50.0%	▲ ▼
Exit %:	100.0%	▲ ▼
Max compounding tier:	3	▲ ▼
Max cost-basis multiple:	2.0×	▲ ▼
Direction:	Normal (base → alt: buy first)	

The Extractor group of the parameter page, first nine rows.

Functional. Choosing the Extractor hides the eight scrumming groups and shows this one. The two sides never appear together.

src/gui/bot_wizard.py — TradingParamsPage.set_mode, the group swap

```
scrum_visible = not is_grid and not is_extractor
for g in (
    self._mode_group,
    self._scrum_group,
    self._adv_group,
    self._hedge_group,
    self._cb_group,
    self._risk_group,
    self._gates_group,
    self._routing_group,
):
    g.setVisible(scrum_visible)
self._extractor_group.setVisible(is_extractor)
```

The group title in the source reads `Extractor`, an em dash, `Pool`, an ampersand, then `Artillery`. Qt reads that ampersand as a keyboard-mnemonic marker, so the rendered title drops it and underlines the `A` of `Artillery`. The figure shows the gap the dropped character leaves. Issue #421 carries this.

Chunk size (USD) - This determines the maximum amount of the parent's pool that the `Extractor` can use.

From \$10.00 to \$10,000,000.00, at \$100.00 to start. It is the pool every artillery round below is drawn from.

`src/gui/bot_wizard.py` — the `Chunk size` row

```
self._ext_chunk_size_usd = QDoubleSpinBox()
self._ext_chunk_size_usd.setRange(10.0, 10_000_000.0)
self._ext_chunk_size_usd.setPrefix("$")
self._ext_chunk_size_usd.setDecimals(2)
self._ext_chunk_size_usd.setValue(100.0)
```

Artillery size (USD) - This determines the individual size of `Extractor` Tranches.

From \$0.50 to \$100,000.00, at \$5.00 to start. The default is set small enough to fire often and still clear a venue's minimum order cost.

`src/gui/bot_wizard.py` — the `Artillery size` row

```
self._ext_artillery_size_usd = QDoubleSpinBox()
self._ext_artillery_size_usd.setRange(0.5, 100_000.0)
self._ext_artillery_size_usd.setPrefix("$")
self._ext_artillery_size_usd.setDecimals(2)
self._ext_artillery_size_usd.setValue(5.0)
```

Watch list top-N - The determines the number of Alternate Currency pairs the bot will scan for potential extraction.

From 5 to 10, at 8 to start. The pairs are ranked by twenty-four hour volume, so the list holds the most liquid alternates against the chosen base.

`src/gui/bot_wizard.py` — the `watch list size` row

```
self._ext_scan_top_n = QSpinBox()
self._ext_scan_top_n.setRange(5, 10)
self._ext_scan_top_n.setValue(8)
```

Watch list refresh - This determines the rate at which the `Extractor` will scan its watched markets. This is the equivalent of a `Timeframe` for the `Extractor` but covers multiple pairs.

From 10 to 240 candles, at 60 to start, which is once an hour at a one-minute cadence. It re-ranks the list rather than re-reading one pair.

`src/gui/bot_wizard.py` — the `watch list refresh` row

```

self._ext_scan_refresh = QSpinBox()
self._ext_scan_refresh.setRange(10, 240)
self._ext_scan_refresh.setValue(60)
self._ext_scan_refresh.setSuffix(" candles")

```

Pool Reserve - To be re-evaluated.

A percentage from 0.0 to 90.0, at 50.0 % to start. Bot creation passes it through as `extractor_pool_reserve_pct`, and one method reads it: the capacity check the artillery path runs before it fires a round. The reserve is that share of the pool, and the check refuses any round that would take the free pool below it.

The share counts against the pool the operator allocated, not against what is left of it. Corrections that have already drained the pool therefore cannot hide inside the reserve arithmetic.

`src/trading/extractor_bot.py` — `ExtractorBot._has_chunk_capacity`

```

reserve = self._chunk_size_base * (
    self.config.extractor_pool_reserve_pct / 100.0
)
return (self._chunk_free_base - artillery_base) >= reserve

```

Exit % - To be re-evaluated.

A percentage from 10.0 to 100.0, at 100.0 % to start. Bot creation passes it through as `extractor_exit_pct`. It sets the share of a position's alt units an exit sells, and the Extractor reads it nine times across three methods: the profitability test, the bullish exit, and the per-position Manual Fire.

The profitability test is the one that can refuse. It prices the proportional sell in base units, takes the trading fee off, and compares the result against the same share of the cost basis. An exit that would gain dollars but lose base units does not happen.

`src/trading/extractor_bot.py` — `ExtractorBot._exit_is_profitable_in_base`

```

units_to_sell = pos.alt_units * (self.config.extractor_exit_pct / 100.0)
base_back = units_to_sell * alt_price_in_base
fee_pct = float(getattr(self.config, "trading_fee_pct", 0.6))
base_back_after_fee = base_back * (1.0 - fee_pct / 100.0)
base_in_proportional = pos.cost_basis_base * (
    self.config.extractor_exit_pct / 100.0
)
return base_back_after_fee > base_in_proportional

```

Max compounding tier - Allows the Extractor to attempt a number of compounding Swing Trades with a given Extractor Tranche with subsequent re-entries based upon the Parent Scrumming Bot's Minimum Opposing Trade Distance + Trade Fee + Bollinger Band extension settings.

From 1 to 10, at 3 to start. The first tier always locks its gain back to the pool, and the counter is held by the position rather than by the bot, so it ends when the position does.

`src/gui/bot_wizard.py` — the compounding tier row

```

self._ext_max_tier = QSpinBox()
self._ext_max_tier.setRange(1, 10)
self._ext_max_tier.setValue(3)

```

Max cost-basis multiple - To be re-evaluated.

From 1.0x to 10.0x, at 2.0x to start. Setting it to 1.0 stops averaging down. Bot creation passes it through as `extractor_max_cost_basis_multiple`, and the correction path reads it twice. The first read is the ceiling: the bot may average a position down until its cost basis reaches this multiple of its opening round, then it holds and waits for the exit.

The second read writes a log line, and it prints this multiple where a correction count belongs. The line reads `corrections=1/2x cap`, and nothing anywhere compares the correction counter against a cap. Issue #438 carries this.

`src/trading/extractor_bot.py` — `ExtractorBot._maybe_fire_correction`, the ceiling

```

max_basis = pos.artillery_size_base * float(
    self.config.extractor_max_cost_basis_multiple
)
headroom = max_basis - pos.cost_basis_base
if headroom <= 0:
    return # hard floor reached; wait for bullish exit

```

Direction - To be re-evaluated.

Two entries: Normal, which runs base to alt and buys first, and Inverted, which runs standing alt to base and sells first.

`src/gui/bot_wizard.py` — `TradingParamsPage.get_config`, the settings this group emits, in the order of the rows above

```

"extractor_chunk_size_usd": self._ext_chunk_size_usd.value(),
"extractor_artillery_size_usd": self._ext_artillery_size_usd.value(),
"extractor_scan_top_n": int(self._ext_scan_top_n.value()),
"extractor_scan_refresh_candles": int(
    self._ext_scan_refresh.value()
),
"extractor_pool_reserve_pct": self._ext_pool_reserve.value(),
"extractor_exit_pct": self._ext_exit_pct.value(),
"extractor_max_compounding_tier": int(
    self._ext_max_tier.value()
),
"extractor_max_cost_basis_multiple": self._ext_max_cost_basis.value(),
"extractor_direction": self._ext_direction.currentData(),

```


Standing alt units (Inverted):	0.00000000	▲ ▼
Correction skip candles:	4 candles	▲ ▼
Drawdown threshold:	3.00%	▲ ▼
Hedge budget (USD):	\$0.00	▲ ▼
Trend strength threshold:	0.650	▲ ▼

The Extractor group, remaining five rows.

Standing alt units (inverted) - To be re-evaluated.

Eight decimal places, at 0 to start. The Inverted direction reads it; the Normal direction ignores it. The field takes up to a billion units. The wizard writes it as `inverted_extractor_standing_alt_units`.

Bot creation passes neither this number nor the Direction beside it, so a new Extractor runs Normal with a standing position of zero whatever you enter. The one method that reads the number, `set_initial_chunk_rate`, has no caller in the product source either. Only tests call it. Issue #437 carries the method, and issue #336 the settings creation drops.

`src/trading/extractor_bot.py` — `ExtractorBot.set_initial_chunk_rate`, the inverted branch

```
_standing = float(
    getattr(self.config, "inverted_extractor_standing_alt_units", 0) or 0
)
if self._is_inverted and _standing > 0:
    self._chunk_size_base = _standing
    self._chunk_size_usd = _standing * usd_per_base
```

Correction skip candles - To be re-evaluated.

From 0 to 100 candles, at 4 to start. It throttles averaging down: the Extractor will not correct the same position again until this many have passed. One method reads it, the correction path, and it counts ticks rather than candles. The Extractor ticks every five seconds, so 4 is twenty seconds on any timeframe.

Bot creation does not pass it, so a new bot takes the declared default of 4. Issue #438 carries the counting, and issue #336 the drop.

`src/trading/extractor_bot.py` — `ExtractorBot._maybe_fire_correction`, the throttle

```
last_tick = self._last_correction_tick.get(pos.pair, -(10**9))
if self._tick_counter - last_tick < int(
    self.config.extractor_correction_skip_candles
):
    return # skip-candles throttle
```

Drawdown threshold - To be re-evaluated.

A percentage from 0.00 to 50.00, at 3.00 % to start. One method reads it, and that method decides when a position counts as down: the current dollar value against the dollar value snapshotted at firing, which never

changes afterwards. Crossing the threshold puts a position in front of the correction path.

The wizard writes it as `extractor_drawdown_threshold_pct`, and bot creation does not pass it, so a new bot takes the declared default of 3.00. Issue #336 carries this.

`src/trading/extractor_bot.py` — `ExtractorBot._is_in_drawdown`

```
current_usd = self._position_value_usd(pos, alt_price_in_base)
threshold = pos.artillery_size_usd_at_entry * (
    1.0 - self.config.extractor_drawdown_threshold_pct / 100.0
)
return current_usd < threshold
```

Trend Strength Threshold - To be re-evaluated.

From 0.000 to 1.000, at 0.650 to start.

A sixth control sits in this part of the group and no entry above names it. Hedge budget (USD) starts at \$0.00, which switches it off. Above zero, the bot converts it into a base-currency reserve held out of artillery rotation. This manual states that the control exists and claims nothing about what it is for.

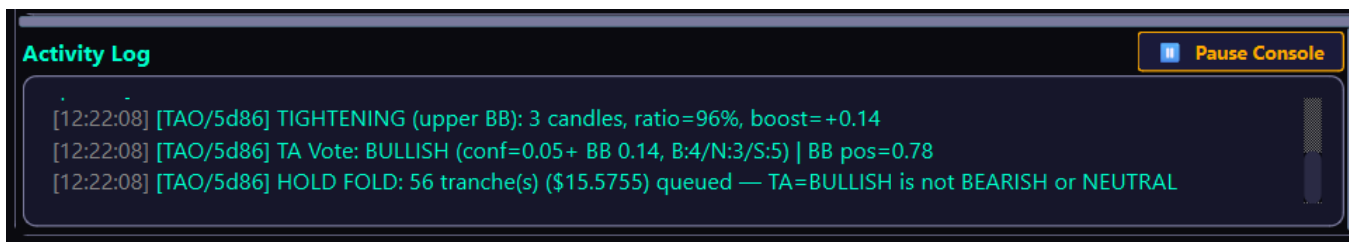
`src/gui/bot_wizard.py` — `TradingParamsPage.get_config`, the remaining Extractor settings

```
"inverted_extractor_standing_alt_units": self._ext_standing_alt_units.value(),
"extractor_correction_skip_candles": int(
    self._ext_correction_skip.value()
),
"extractor_drawdown_threshold_pct": self._ext_drawdown_threshold.value(),
"extractor_hedge_budget_usd": self._ext_hedge_budget.value(),
"extractor_trend_strength_threshold": self._ext_trend_strength.value(),
```

Additional Main Window > Trading Tab Features

Trade Logic and Gate Activity

This pool displays the trading logic and gate activity.



The Activity Log pane.

Functional. The pane is read-only. Every line opens with a timestamp in a muted colour, and the message takes its colour from its level: one colour for info, one for success, one for warning, one for error. Three kinds of message get a shape of their own. A trade notification draws larger and bold and takes its colour from the stage of the trade. A wire flow or wire income line draws in magenta behind a bolt character. A wire stack line draws in the pending colour behind the same character. The pane holds 5,000 lines and drops the oldest past that.

Pause Console is a toggle. While it is down, each new line goes into a buffer of 2,000 instead of the screen, and a full buffer drops the newest rather than the oldest. Resume replays the buffer with the original timestamps and adds a line counting what it replayed. One writer goes through the pause regardless, for a message you must not miss. A timer checks the pane's health every sixty seconds and writes a warning into the pane itself when the render-error count rises, or when nothing has rendered for ten minutes while bots are running.

src/gui/widgets/status_log.py — StatusLog.log, the pause branch

```
if len(self._pause_buffer) < self._pause_buffer_cap:
    self._pause_buffer.append((ts, message, level))
return
```

Design intention. Two decisions follow from the pane's job. The buffer drops the newest line rather than the oldest, so the lines around the moment you hit pause are the ones that survive. And a pane that has gone quiet says so in the pane, because silence otherwise reads as calm.

src/gui/main_tabs/trading_tab.py — the silence check

```
bots_active = self._bot_manager and any(
    b.state.value == "running"
    for b in getattr(self._bot_manager, "_bots", {}).values()
)
if bots_active and age > 600:
```

What the lines say. Every tick message follows one shape: the event in capitals, the side in square brackets where the message has one, then a short sentence and the numbers behind it. A message names the gate that refused and the reading that made it refuse. It never names a direction or a strength the data behind it cannot give.

The gate snapshot is the longest of them. It writes the twelve-voter panel as a count and three groups, strongest confidence first, with an indicator's own raw reading in brackets where it publishes one. A neutral vote carries no confidence, so only its name prints.

```
RISK GATE [SCRUM] blocked by hysteresis_scrum. Price $0.00000317.
Panel 6 bullish, 3 bearish, 3 neutral.
bullish vortex 1.00, kaufman_er 0.70 (er 1.0), macd 0.60, adx 0.36 (adx 25.52),
volume 0.35, supertrend 0.26.
bearish bollinger_bands 0.82, stochastic_rsi 0.50, slingshot 0.35.
neutral ichimoku, rsi, zscore (z 0.143).
```

A hold line names the half of the gate that failed. Two conditions must both hold before the TA gate lets a scam through: the direction has to be bullish or neutral, and the confidence has to clear the floor. The line says which one refused, so a bullish reading held back by its confidence is never reported as a wait for a bullish reading.

src/trading/scrumming_bot.py — the scam hold reason

```

if eff_direction not in (
    SignalDirection.BULLISH,
    SignalDirection.NEUTRAL,
):
    _scrum_why = f"TA={eff_direction.name} is not BULLISH or NEUTRAL"
else:
    _scrum_why = (
        f"TA={eff_direction.name} but confidence "
        f"{eff_confidence:.4f} < {_eff_conf_floor:.4f} floor"
    )

```

Design intention. A confidence and the floor it is measured against print at four decimals wherever a message compares them. At two decimals they collided, and the pane showed a comparison of a number against itself, which is not a statement anybody can act on.

Both snapshot lines read one renderer, so the blocked half and the fired half of a decision cannot drift into two shapes.

src/trading/scrumming/snapshots.py — the one panel renderer

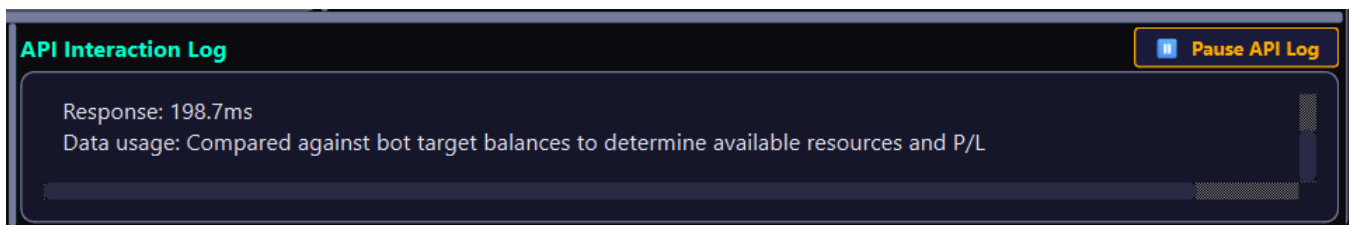
```

def _panel_line(summary: Optional[VotingSummary]) -> str:
    panel = _build_panel_snapshot(summary)
    if not panel:
        return PANEL_ABSENT_TEXT

```

API Interaction Log

This spool displays API handshake information and data transfer speeds for these messages.



The API Interaction Log pane.

Functional. The pane is read-only, holds 2,000 lines, and never wraps. One entry is written per API call. A call that arrives on any thread but the GUI thread is refused outright and written to a thread-violation file under the runtime log directory instead, because touching a widget from a background thread ends the process. An entry carries a timestamp, the exchange name, the action and a Reason line, then Endpoint, Result, Response time and Data usage wherever the record holds them. Pause API Log buffers up to 2,000 lines and flushes them on resume with a count.

src/gui/main_window.py — _on_api_event, the thread check

```

current = _threading.current_thread().name
origin = entry.get("_thread_name", "unknown")
if current != "MainThread":

```

Design intention. The pane should tell you what the platform did with the bytes it just paid for. One line does the opposite. The candle fetch writes a Data usage note naming a seven-indicator engine and lists seven names. The engine builds twelve and the Voting Panel shows all twelve, so the log tells you something the screen next to it contradicts.

src/exchange/ccxt_connector.py — get_ohlcv, what it writes today

```
data_usage="Fed into 7-indicator TA engine (BB, Vortex, MACD, StochRSI, Ichimoku, Volume, Slingshot) for voting",
```

Proposed, not present:

```
data_usage=(
    "Fed into the "
    f"{len(DEFAULT_WEIGHTS)}-indicator TA engine for voting"
),
```

DEFAULT_WEIGHTS in src/trading/ta_engine.py is the one declaration of the voter set, so a count taken from it cannot drift again. Issue #417 carries this.

Two mechanisms with no control on this tab

Two more mechanisms sit inside the engine and the wizard offers no switch for either. Neither appears anywhere else in this manual, and their state differs: one runs, one is declared and dormant.

Fair Value Gap. Three candles that leave a gap between the first and the third mark a price band the market tends to come back to. The indicator scans the last 24 candles for those bands, reports whether price sits inside one or is approaching one, counts how many are open, and gives the bounds of the nearest. It is not one of the twelve voters. It adjusts the confidence of a side that already has a case: a fold gains ten points near a bullish band, a scrum gains eight inside a bearish one.

src/trading/indicators/fvg.py — the four constants that set its reach

```
FVG_LOOKBACK = 24 # candles to scan for 3-candle gap patterns

FVG_PROXIMITY_PCT = 0.005 # "approaching from above": within 0.5% of FVG top

FVG_BULL_BOOST = 0.10 # fold confidence boost when in/near bullish FVG

FVG_BEAR_BOOST = 0.08 # scrum confidence boost when inside bearish FVG
```

The engine imports the indicator and its four constants together, so the adjustment travels with the reading rather than being applied somewhere else.

src/trading/ta_engine.py — what it takes from that module

```

from .indicators.fvg import (
    FVG_BEAR_BOOST,
    FVG_BULL_BOOST,
    FVG_LOOKBACK,
    FVG_PROXIMITY_PCT,
    FVGIndicator,
)

```

Boost Fold. The intent is a pair: sell a slice of holdings when the mean reversion reading goes extreme, then buy that slice back when price returns to its moving average, and let the difference raise the target. The Market Inspector states the pair at the head of its own module.

`src/trading/mr_inspector.py` — the pair, in the module's own words

```

# ■ When all 3 gates pass → Boost Sell (sell 2.5% of holdings) ■
# ■ When price returns to SMA → Boost Fold (buy back cheaper) ■
# ■ Profit from boost fold increments target (compound growth) ■

```

Where it runs is the stock accumulation bot, which carries the queued amount, the reference price, the moving average and the two counters, and works all of them. The crypto Scrumming Bot declares the same four fields, sets them to zero and never reads them again: across the whole trading package the only lines naming them are those four assignments. A crypto bot therefore takes no boost fold, whatever the Market Inspector reads.

`src/trading/scrumming_bot.py` — the four fields, written once and never read

```

self._boost_fold_q: float = 0.0
self._boost_fold_ref: float = 0.0
self._boost_fold_sma: float = 0.0

self._boost_folds: int = 0

```

Proposed, not present, in `src/trading/scrumming_bot.py`: the crypto side needs the read half of the pair, taking the queued amount back at the moving average the way the stock bot does, guarded so a bot with an empty queue takes no action.

```

if self._boost_fold_q > 0 and self._boost_fold_sma > 0:
    if price <= self._boost_fold_sma and price < self._boost_fold_ref:
        buy_usd = self._boost_fold_q
        self._boost_folds += 1
        self._boost_fold_q = 0

```

Add Crypto Exchange Button (to be changed - Add Exchange)

While the initial set up of Acervator requires at least one valid exchange API, additional exchanges can be added and have their own bot swarms. The current upper operational limit of Acervator is unknown.

Multi-exchange testing has yet to be attempted as of 8/25/26 with API compatibility work pending. This button also currently opens the Settings Panel but on the incorrect 'User' Tab when it should be 'Exchanges'.

[+ Add Crypto Exchange](#)

The Add Crypto Exchange button.

Functional. The button sits at the right of the exchange sub-tab row, as the corner widget of that row. Each layer builds its own and labels it for that layer: Add Crypto Exchange on the crypto page, Add Stock Exchange on the stock page. Both run the same handler as the button on the empty card a layer shows while it holds no exchange at all.

src/gui/main_tabs/trading_tab.py — _make_layer

```
tab_w = QTabWidget()
add_btn = QPushButton(f"■ Add {label_text} Exchange")
add_btn.setMinimumWidth(140)
add_btn.setToolTip(f"Add a {label_text} exchange connection")
add_btn.clicked.connect(self._add_exchange)
tab_w.setCornerWidget(add_btn)
```

Design intention. The handler already knows which wing you are in and passes that to the dialog, so it has somewhere to say which tab to open.

src/gui/main_window.py — _add_exchange today

```
dlg = SettingsDialog(self._settings, self._status_log, self, wing=wing)
dlg.exec()
```

Proposed, not present:

```
dlg = SettingsDialog(self._settings, self._status_log, self, wing=wing)
dlg.show_exchanges_tab()
dlg.exec()
```

The dialog in src/gui/settings_dialog.py has no such method yet. The proposal adds one rather than changing the call shape, so the wing argument already passed here keeps working exactly as it does.

The exchange row in the Electron shell. The shell draws one tab button per configured exchange, and the screen under the button is drawn by exchange_tab.js. A layer shows its empty card only while it holds no exchange, which is what the Qt tab row does once it drops the Get Started tab.

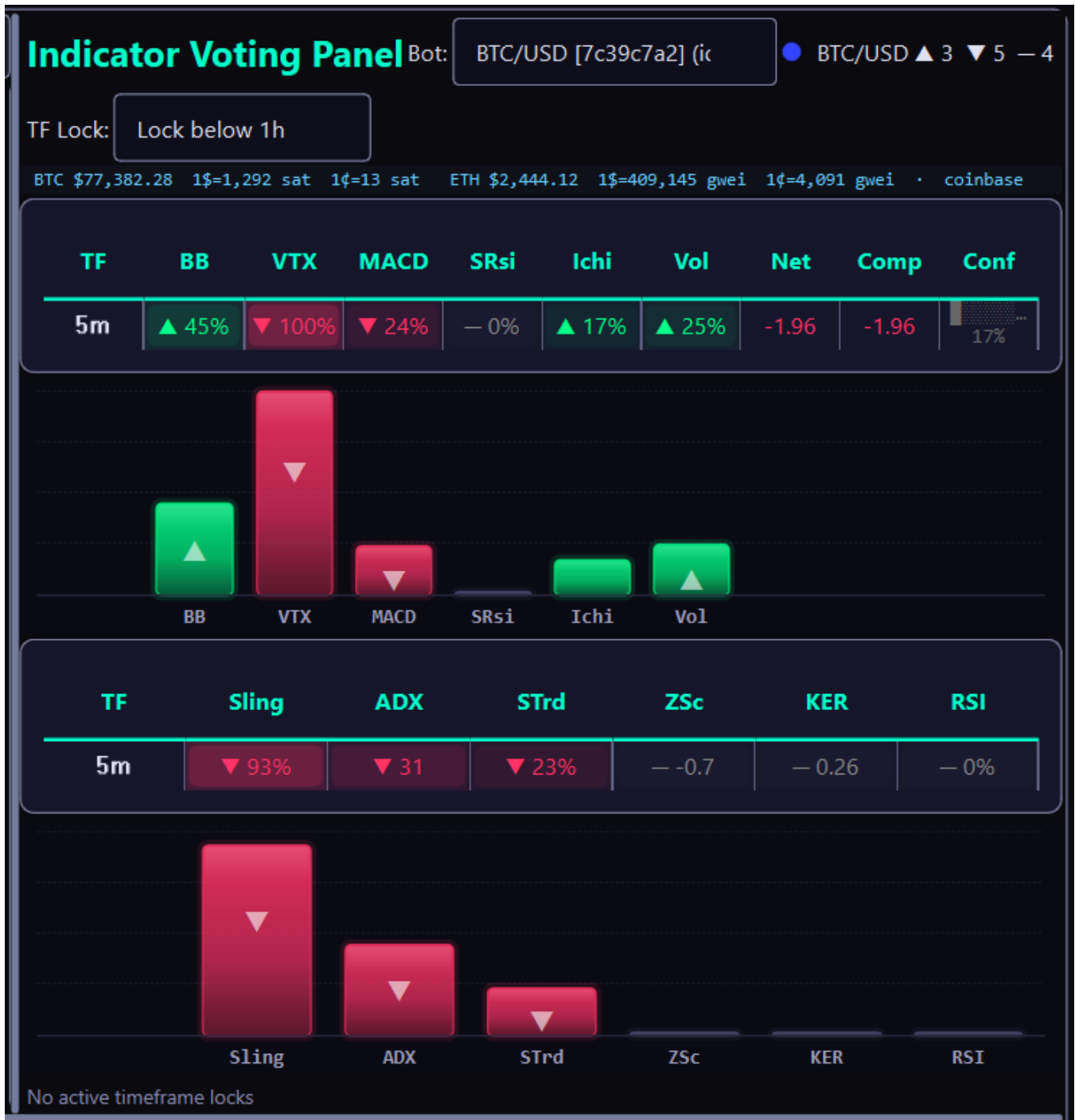
src/gui/main_tabs/trading_tab_surface.py — layer_exchanges

```
for entry in entries or []:
    holder = entry if isinstance(entry, dict) else {}
    exchange_id = str(holder.get("exchange_id") or "")
    if not exchange_id:
        continue
    layer = "stock" if is_equity_exchange(exchange_id) else "crypto"
    split[layer][exchange_id] = exchange_display_name(holder)
```

One caption for one exchange. The caption on a tab comes from a single helper, so the start-up path and the settings path cannot label the same exchange two different ways. An exchange saved with no name is captioned from its own id, and an entry with no id gets no tab at all.

```
def exchange_display_name(entry: Any) -> str:
    holder = entry if isinstance(entry, dict) else {}
    exchange_id = str(holder.get("exchange_id") or "")
    named = str(holder.get("display_name") or "")
    return named or exchange_id.capitalize()
```

Indicator Voting Panel



The Indicator Voting Panel, at the right of the Trading tab.

Functional. The panel fills the right half of the Trading tab.

- The Bot selector at the top names the bot whose votes the panel draws. The list refills every tick. The badge beside it counts the bullish, bearish and neutral voters.
- TF Lock chooses a timeframe below which an opposing trade is refused. The list opens on "None (no lock)".
- The rate line under it shows the BTC and ETH prices with their satoshi and gwei equivalents, and the venue name at the end.
- Two tables of six voters each. A green up triangle is a bullish vote, a red down triangle a bearish one, an em dash a neutral one. ADX, ZSc and KER print a raw value; the other nine print a percentage.
- Each voter belongs to one of three groups, trend, momentum or structure, and the group decides its header colour.
- Net, Comp and Conf close the first table. Conf draws as a small filled bar with its percentage beside it.
- A bar chart under each table draws that table's six confidences at the width of the column above.
- The line at the foot names any timeframe lock that is active.

src/gui/indicator_panel.py — IndicatorVotingPanel.INDICATOR_COLS

```
INDICATOR_COLS = [  
    ("bollinger_bands", "BB", "S"),  
    ("vortex", "VTX", "T"),  
    ("macd", "MACD", "M"),  
    ("stochastic_rsi", "SRsi", "M"),  
    ("ichimoku", "Ichi", "T"),  
    ("volume", "Vol", "S"),  
    ("slingshot", "Sling", "S"),  
    ("adx", "ADX", "T"),  
    ("supertrend", "STrd", "T"),  
    ("zscore", "ZSc", "M"),  
    ("kaufman_er", "KER", "M"),  
    ("rsi", "RSI", "M"),  
]
```

Design intention. One place that shows what every voter thinks and how far they agree. The names, the split into two rows and the three colour groups all come out of the one list above, so the panel cannot fall out of step with itself as voters are added or reordered.

src/gui/indicator_panel.py — the two rows, both cut from that list

```
_ROW_A_INDICATOR_COLS = INDICATOR_COLS[:6] # BB VTX MACD SRsi Ichi Vol  
_ROW_B_INDICATOR_COLS = INDICATOR_COLS[6:] # Sling ADX STrd ZSc KER RSI
```

The operator's own reading of each column follows in the next section.

Where the maths lives. 07-indicators.md is the single home for all twelve. Each voter is described once there and once only: the operator's own line, then what the cell prints and what makes the vote bullish, bearish or

neutral, then the published formula with the code that computes it. Nine voters print a direction arrow and a confidence percentage. Three print a raw value instead, and those three are ADX, Z-Score and Kaufman ER.

Voter	Its formula and its vote
BB	Bollinger Bands
VTX	Vortex Indicator
MACD	MACD
SRsi	Stochastic RSI
Ichi	Ichimoku Cloud
Vol	Volume
Sling	Slingshot
ADX	ADX and DMI — raw value
STrd	Supertrend
ZSc	Z-Score — raw value
KER	Kaufman Efficiency Ratio — raw value
RSI	RSI

One decision picks the text a cell carries. Three voters are named in it and every other voter falls through to the percentage.

src/gui/indicator_panel.py — _populate_indicator_cell

```

if ind_key == "adx":
    adx_v = details.get("adx", 0)
    if details.get("ranging"):
        cell_text = f"Rng {adx_v:.0f}"
    else:
        cell_text = f"{sym} {adx_v:.0f}"
elif ind_key == "zscore":
    cell_text = f"{sym} {details.get('z', 0):+.1f}"
elif ind_key == "kaufman_er":
    cell_text = f"{sym} {details.get('er', 0):.2f}"
else:
    cell_text = f"{sym} {confidence:.0%}"

```

The same file carries the gate logic chain that turns those twelve votes into a trade decision.

Indicator Voting Panel

This panel shows the readings for each of the 12 as well the collated indices and confidence score.

TF Lock - To be re-evaluated.

A drop-down of eleven entries: None, then Lock below each of ten timeframes from 5m to 1w. It opens on the fifth entry, Lock below 1h. Choosing one writes a status line beside the box and puts the timeframe on the event bus.

The main window takes that event and sets `_lock_timeframe` on the coordinator of every running Scrumming Bot. Nothing declares or reads that name, so the choice reaches the coordinator and stops there. The tooltip on the box says the lock feeds directly into active Scrumming Bots. Issue #155 carries this.

src/gui/main_window.py — `_on_tf_lock_changed`

```
for bot in self._bot_manager._bots.values():
    if isinstance(bot, ScrummingBot) and hasattr(bot, "_coordinator"):
        bot._coordinator._lock_timeframe = tf
```

Bot - Drop down menu for selecting which bot's TA signals are displayed.

The list refills every tick and carries one entry per running bot, opening on a placeholder until one is chosen. Choosing a bot is what makes the twelve columns below read anything at all.

src/gui/indicator_panel.py — the selector

```
self._bot_selector = QComboBox()
self._bot_selector.setMinimumWidth(180)
self._bot_selector.addItem("(select a bot)", "")
self._bot_selector.currentIndexChanged.connect(self._on_bot_selected)
```

TF - Timeframe for the selected bot.

One row per timeframe, each row a separate verdict from the same twelve voters. The rows are the bot's own timeframes and any phantom timeframes it runs.

In development.

BB - Bollinger Bands - https://en.wikipedia.org/wiki/Bollinger_Bands

In Acervator, Bollinger Bands are used as the thresholds or boundaries at which trades are allowed to fire. Multiple market structure pieces are intertwined with our readings of the bands such as Landing Strip Detection (Heikin-Aishii Candle Consolidation Pattern) and Minimum Opposing Trade Distance. From a tactical standpoint, these represent the area or zone through which an investment position is passing.

The cell prints a direction arrow and a confidence percentage. Direction comes from the band position alone: under 0.15 or under 0.35 votes bullish, over 0.85 or over 0.65 votes bearish, and anything between the two middle figures votes neutral. Bollinger Bands carries the formula.

src/trading/indicators/bollinger.py — the vote

```

if bb_pos < 0.15:
    direction = SignalDirection.BULLISH
    confidence = max(0.0, min(1.0, (0.15 - bb_pos) / 0.15 * 0.8 + 0.3))
elif bb_pos > 0.85:
    direction = SignalDirection.BEARISH
    confidence = max(0.0, min(1.0, (bb_pos - 0.85) / 0.15 * 0.8 + 0.3))
elif bb_pos < 0.35:
    direction = SignalDirection.BULLISH
    confidence = 0.2
elif bb_pos > 0.65:
    direction = SignalDirection.BEARISH
    confidence = 0.2

```

VTX - Vortex Indicator - https://en.wikipedia.org/wiki/Vortex_indicator

In my experience (which is entirely subjective), this particular indicator is unusually strong at flagging early trend reversals. For Acervator, we are generally looking for signal saturation or near-saturation in either polarity which, under optimum conditions, will align with Bollinger Band thresholds being hit.

The cell prints an arrow and a percentage. The crossover of the two lines decides the vote, and then Acervator adds one reading of its own: the direction is overwritten to bullish whenever the downward line reaches 1.30, whatever the crossover said. Vortex Indicator carries the formula.

src/trading/indicators/vortex.py — the ceiling that overwrites the vote

```
VX_CEILING_PCT = 130.0
```

MACD - Moving Average Convergence Divergence - <https://en.wikipedia.org/wiki/MACD>

This is one of the oscillators used and, with it, we are looking for “top of the hill” formations in either polarity and these, under optimum conditions, will align with Bollinger Band thresholds being hit. Keep in mind that I do not approach any of these indicators from a mathematical standpoint as I am a purely visual trader without a quant or certified technical analysis background.

The cell prints an arrow and a percentage. A signal-line crossover decides the vote first. Failing that, a histogram growing away from zero votes with its own sign, and a histogram merely on one side of zero votes the same way with less conviction. MACD carries the formula.

src/trading/indicators/macd.py — the crossover, tested first

```

if prev_macd <= prev_signal and curr_macd > curr_signal:
    direction = SignalDirection.BULLISH

```

SRsi - Stochastic RSI - https://en.wikipedia.org/wiki/Stochastic_oscillator

Our second oscillator and, like the vortex indicator, we are looking at initial maximizing or prolonged periods of saturation of the signal at either polarity and this will, under optimum conditions, align with other indicators to trigger a trade.

The cell prints an arrow and a percentage. The vote needs a crossover of the two smoothed lines, and how deep in the range that crossover happens sets the confidence: a bullish cross under 30 scores 0.8, under 50

scores 0.5, and higher still scores less. Stochastic RSI carries the formula.

src/trading/indicators/stochastic_rsi.py — the bullish crossover

```
if prev_k <= prev_d and k > d:
    crossover = "bullish"
    if k < 30:
        direction = SignalDirection.BULLISH
        confidence = 0.8
```

Ichi - Ichimoku Cloud - https://en.wikipedia.org/wiki/Ichimoku_Kink%C5%8D_Hy%C5%8D

This particular indicator is one of the few 'predicative' indicators. In Acervator, we are focused on its additional ability to strengthen market reversal zone detection which is where we prefer to shave off or fold in funds.

The cell prints an arrow and a percentage. Several cloud readings add into one score, and only a score past 0.08 either way casts a vote. The score doubles as the confidence. Ichimoku Cloud carries the formula.

src/trading/indicators/ichimoku.py — the vote

```
if score > 0.08:
    direction = SignalDirection.BULLISH
    confidence = max(0.0, min(1.0, score))
elif score < -0.08:
    direction = SignalDirection.BEARISH
    confidence = max(0.0, min(1.0, abs(score)))
else:
    direction = SignalDirection.NEUTRAL
    confidence = 0.0
```

Net - Primary TF panel index collated from all 12 indicators.

The cell prints a signed figure to two decimals, green above zero and red below. It is the weighted bullish total less the weighted bearish total, and an abstaining voter adds nothing to either side. Net, Comp and Conf carries the arithmetic.

src/gui/indicator_panel.py — the Net cell

```
net = tf_data.get("net_score", 0)
item = QTableWidgetItem(f"{net:+.2f}")
```

Comp - Composite TF panel index provided by the highest TF Phantom Bot that is active. Phantom bots are still in active development so any related features are not yet working.

The cell prints an em dash when no composite figure is present, which is what a fleet with no active phantom shows. Net, Comp and Conf carries the arithmetic.

In development.

Conf - Confidence reading for the panel.

This is the 'breadth' of agreement between all 12 indicators.

The cell draws a ten-block bar and the percentage beside it. The bar fills one block per tenth. Green from 0.6, amber from 0.3, grey below that.

src/gui/indicator_panel.py — the Conf cell

```
conf = tf_data.get("confidence", 0)
conf_bar = "■" * int(conf * 10) + "■" * (10 - int(conf * 10))
item = QTableWidgetItem(f"{conf_bar} {conf:.0%}")
```

Sling - CM (Chris Moody) Slingshot -

<https://www.tradingview.com/script/GE7tSQK1-CM-Sling-Shot-System/>

Another excellent indicator for reversal detection and here it adds to our laser precision detection of such zones and structures.

The cell prints an arrow and a percentage. A squeeze breaking either way votes first, and a band snapback votes when no squeeze is breaking. Slingshot carries the formula and the attribution, which is shared between three authors rather than one.

src/trading/indicators/slingshot.py — the squeeze, tested first

```
if squeeze_bull:
    direction = SignalDirection.BULLISH
    confidence = squeeze_conf
    active_type = "squeeze_bull"
```

ADX - https://en.wikipedia.org/wiki/Average_directional_movement_index

This is a trend strength indicator but, as with the other indicators, we are using it to detect reversal by way of signal decreasing.

The cell prints the raw ADX value, 0 to 100, not a percentage. Below 20 the cell reads Rng and the figure, with no arrow. The vote itself is decided by which directional line is on top, at any ADX, so a ranging market still casts a directional vote even though the cell shows no arrow. ADX and DMI carries the formula.

src/trading/indicators/adx.py — the vote

```

if bull_dominant and strong_trend:
    direction = SignalDirection.BULLISH
    confidence = max(0.0, min(1.0, (adx - 35) / 30 + 0.5))
elif bear_dominant and strong_trend:
    direction = SignalDirection.BEARISH
    confidence = max(0.0, min(1.0, (adx - 35) / 30 + 0.5))
elif bull_dominant:
    direction = SignalDirection.BULLISH
    confidence = max(0.0, min(0.5, adx / 70))
elif bear_dominant:
    direction = SignalDirection.BEARISH
    confidence = max(0.0, min(0.5, adx / 70))
else:
    direction = SignalDirection.NEUTRAL
    confidence = 0.0

```

STrd - <https://www.tradingview.com/support/solutions/43000634738-supertrend/>

Supertrend is based upon Average True Range and, unsurprisingly further assists in detecting reversals.

The cell prints an arrow and a percentage. The vote is simply which side of the sticky band the price closed on, so this voter is never neutral once it reads at all. A band that has fallen to zero makes it abstain instead.

Supertrend carries the formula.

src/trading/indicators/supertrend.py — the vote

```

direction = SignalDirection.BULLISH if curr_bull else SignalDirection.BEARISH

```

ZSc - Z Score - <https://www.tradingview.com/script/KSMvIkvh-Z-Score-Predictive-Zones-AlgoPoint/>

Our current indicator is the basic or classic Z Score indicator but this will be upgraded. But, as before, we are after more data to confirm reversals.

The cell prints the raw signed z-value to one decimal, not a percentage. Past two deviations either way the vote is strong and contrarian: high votes bearish, low votes bullish. Between 1.5 and 2 it votes the same way at a fixed 0.25 confidence, and inside 1.5 it casts no vote. Z-Score carries the formula.

src/trading/indicators/zscore.py — the thresholds

```

extreme_high = z > 3.0
strong_high = z > 2.0
mild_high = z > 1.5
extreme_low = z < -3.0
strong_low = z < -2.0
mild_low = z < -1.5

```

The cell now prints a smoothed z, and the vote compares it against levels the market itself set. The indicator remembers where the z-score last turned around, averages those turning points, and uses each average as the level that decides a vote. Until it has remembered a turn, the fixed level of two deviations decides, exactly as before. Each average is also projected back into a price, which is the upper and lower zone the hover text carries.

src/trading/indicators/zscore.py — the level that decides, and its price

```
target_z_high = self.reversal_threshold if peak_z is None else peak_z
target_z_low = -self.reversal_threshold if trough_z is None else trough_z

resistance_price = sma + target_z_high * std
support_price = sma + target_z_low * std
```

KER -

Kaufman Efficiency Ratio. It measures how much of the distance the price travelled became progress in one direction. A reading near 1.0 is a straight move and a reading near 0.0 is noise. A low ratio marks the market Acervator prefers. The Efficiency Ratio Regime gate refuses a SCRUM at 0.70 and above, and refuses one again at 0.05 and below.

RSI -

Relative Strength Index. It reports the share of recent movement that ran upward, on a 0 to 100 scale. Above 70 votes bearish, below 30 votes bullish, and between the two it casts no vote. It runs the same published Wilder recursion as the Stochastic RSI above, computed separately inside its own module. Its weight is 0.8, the lightest in the panel, which it shares with Volume.

The twelve voters

Reference. The operator's own reading of each column opens this part. What follows is the code behind those columns.

Functional. One weight map names the twelve indicators that vote, and one method builds exactly those twelve from it. The engine builds the list once and runs it in order on every tick. "Candles needed" in the table below is the tape length each indicator wants before it will read anything. Give it less and it abstains outright rather than guessing.

src/trading/ta_engine.py — VotingEngine._create_indicators

```
def _create_indicators(self) -> list:
    """Instantiate all indicator instances with configured weights."""
    return [
        BollingerBands(weight=self.weights.get("bollinger_bands", 1.0)),
        VortexIndicator(weight=self.weights.get("vortex", 0.9)),
        MACD(weight=self.weights.get("macd", 1.2)),
        StochasticRSI(weight=self.weights.get("stochastic_rsi", 1.0)),
        IchimokuCloud(weight=self.weights.get("ichimoku", 1.1)),
        VolumeAnalysis(weight=self.weights.get("volume", 0.8)),
        SlingshotIndicator(weight=self.weights.get("slingshot", 1.0)),
        ADXIndicator(weight=self.weights.get("adx", 1.0)),
        KaufmanERIndicator(weight=self.weights.get("kaufman_er", 1.0)),
        SupertrendIndicator(weight=self.weights.get("supertrend", 1.0)),
        ZScoreIndicator(weight=self.weights.get("zscore", 0.9)),
        RSIIndicator(weight=self.weights.get("rsi", 0.8)),
    ]
```


Design intention. The weights rank the voters by how much they are trusted. MACD is heaviest at 1.2 and Ichimoku next at 1.1. RSI and Volume are lightest at 0.8, RSI because it overlaps the Stochastic RSI and Volume because it is meant to confirm rather than lead. Hand the engine a different map and the panel retunes without an indicator being touched.

src/trading/ta_engine.py — VotingEngine.__init__

```
self.weights = weights or DEFAULT_WEIGHTS.copy()
self._indicators = self._create_indicators()
```

Panel column	Voter	Weight	Module under src/trading/indicators/	Candles needed
BB	bollinger_bands	1.0	bollinger.py	20
VTX	vortex	0.9	vortex.py	15
MACD	macd	1.2	macd.py	35
SRsi	stochastic_rsi	1.0	stochastic_rsi.py	36
Ichi	ichimoku	1.1	ichimoku.py	79
Vol	volume	0.8	volume.py	25
Sling	slingshot	1.0	slingshot.py	52
ADX	adx	1.0	adx.py	30
STrd	supertrend	1.0	supertrend.py	12
ZSc	zscore	0.9	zscore.py	51
KER	kaufman_er	1.0	kaufman_er.py	12
RSI	rsi	0.8	rsi.py	15

Functional. Count the files in the indicator folder and you get 23, not 12. Eleven of them cast no vote. Two carry the shared maths and the signal vocabulary. Three are measurements other code reads. Six are structural detectors that feed the bot's own pattern reads rather than the panel. Twelve is the count of voters, and twelve is the right count.

```
shared      helpers.py, types.py
measurements atr.py, bb_proximity.py, heikin_ashi.py
detectors   fvg.py, landing_strip.py, m_top.py, w_bottom.py,
            spring.py, macd_taper.py
```

Design intention. One module holds one indicator, and a module that casts no vote still lives here so the voters can read it. Nothing borrows another indicator's arithmetic, and a test in tests/ fails the moment one starts to.

tests/test_one_indicator_per_module.py — the rule, held as tests

```
def test_the_module_holds_exactly_one(self, module):
```

tests/test_one_indicator_per_module.py — and the name check beside it

```
def test_no_two_modules_define_the_same_name(self):
```

Indicator formulae

Each block below carries the published formula for reference, then the code that computes it, then what the code does today and what the reading is meant to give the platform. The operator's own reading of each indicator opens this part and is not repeated here.

Every indicator computes its own maths from candles alone. No indicator borrows another's arithmetic, and tests/test_one_indicator_per_module.py fails when one starts to.

Where the code departs from the published formula, the code is the defect. Departures lists the ones found.

Shared terms

Every formula below uses these. All five live in src/trading/indicators/helpers.py.

SMA(v, n)	mean of the last n values	_sma_tail
sigma(v, n)	population deviation of the last n values	_stdev_tail
EMA(v, n)	seed = SMA(v, n) at index n-1, then EMA_t = (v_t - EMA_prev) * k + EMA_prev k = 2 / (n + 1) no value exists before the seed	_ema
TR_t	max(H_t - L_t, H_t - C_prev , L_t - C_prev) the first bar's TR is H - L	_true_range
Wilder(v, n)	seed = mean of the first n, then W_t = (W_prev * (n - 1) + v_t) / n	(per module)

Functional. True Range gives one bar per candle. The first bar is simply high minus low, because the other two ways of measuring it need a previous close and the first bar has none. Each indicator then takes the slice its own formula covers. ATR and Supertrend take the whole series. ADX and Vortex drop the first bar, because directional movement needs a bar before it. The moving average helper answers nothing at all below its seed, rather than zero, so reading it too early fails loudly instead of returning a plausible wrong number.

src/trading/indicators/helpers.py — _true_range

```
tr = [candles[0].high - candles[0].low]
for i in range(1, len(candles)):
    c = candles[i]
    prev_close = candles[i - 1].close
    tr.append(
        max(c.high - c.low, abs(c.high - prev_close), abs(c.low - prev_close))
    )
return tr
```

Design intention. One definition per shared term, held in one place, so five indicators cannot quietly drift apart on the same arithmetic. Slingshot is the one deliberate exception. It computes its own true range, and its first bar agrees with this one.

src/trading/indicators/slingshot.py — the exception, written out

```
# Carter's Keltner leg needs True Range; `tr_all` computes it
# without `ATRIndicator`.
tr_all = [candles[0].high - candles[0].low]
```

Bollinger Bands

Published formula, for reference:

```
Middle    = SMA(close, 20)
Upper     = Middle + 2.0 * sigma(close, 20)
Lower     = Middle - 2.0 * sigma(close, 20)
%B        = (close - Lower) / (Upper - Lower)
Bandwidth = (Upper - Lower) / Middle
```

Functional. The vote is a reading of where the last close sits between the two bands. Zero is the lower band, one is the upper, and the panel and the gates both call that number the band position. A second method returns all three bands for every candle so a chart can draw them, and answers nothing for the first nineteen. Give it fewer than twenty closes, or a window with no range at all, and it abstains.

src/trading/indicators/bollinger.py — BollingerBands.compute

```
sma = _sma_tail(closes, self.period, tail=self.period)
std = _stdev_tail(closes, self.period, tail=self.period)

mid = sma[-1]
upper = mid + self.std_dev * std[-1]
lower = mid - self.std_dev * std[-1]
price = closes[-1]
# BandWidth = (Upper - Lower) / Middle. `candles_from_raw` admits
# only positive closes, so `mid` cannot be zero.
band_width = (upper - lower) / mid
```

Design intention. The bands are the boundary a trade fires at. That is the operator's own reading, and the band position is the one number that carries it into the engine. Four gates read it directly and both overrides arm on it, so this reading has more reach into a trade decision than any other voter.

src/trading/gate_chain.py — two of the four, in the scrum chain

```
MidlineGate(side="scrum"),
TargetFiresGate(),
BBProximityGate(side="scrum"),
```

src/trading/gate_chain.py — and the other two, in the fold chain

```
MidlineGate(side="fold"),
SmartCeilingGate(),
BBProximityGate(side="fold"),
```

Vortex Indicator

Published formula, for reference:

```
VM+_t = |H_t - L_(t-1)|  
VM-_t = |L_t - H_(t-1)|  
VI+   = sum(VM+, 14) / sum(TR, 14)  
VI-   = sum(VM-, 14) / sum(TR, 14)
```

Functional. Three totals are taken over each fourteen-bar window: upward movement, downward movement, and true range. A window whose true range totals zero, which is what a halted market gives, is skipped rather than divided by. The two movement totals are then divided by the true-range total, so both lines are dimensionless and 1.0 is dead parity. The vote reads the crossover of the two lines.

src/trading/indicators/vortex.py — `VortexIndicator.window_sums`

```
for i in range(1, n_c):  
    vm_plus.append(abs(candles[i].high - candles[i - 1].low))  
    vm_minus.append(abs(candles[i].low - candles[i - 1].high))
```

src/trading/indicators/vortex.py — `VortexIndicator.lines`

```
plus_series[i] = sum_vp / sum_tr_window  
minus_series[i] = sum_vm / sum_tr_window
```

Design intention. The operator uses the Vortex to catch a reversal early, and watches for the signal to saturate at one polarity or the other. Acervator adds one reading of its own on top of the published lines: the vote is overwritten to bullish whenever the downward line reaches 1.30, after the crossover has already decided. That overwrite is not part of the indicator, and Departures records it.

In development. The overwrite belongs to issue #414, which owns every indicator departure, and no replacement reading has been chosen.

MACD

Published formula, for reference:

```
MACD Line = EMA(close, 12) - EMA(close, 26)  
Signal    = EMA(MACD Line, 9)  
Histogram = MACD Line - Signal
```

Functional. One method builds all three series and the vote reads the last two bars of them. What you are looking at is the gap between a fast and a slow average of the price, and how quickly that gap is changing. The signal line is an average of the MACD line rather than of the candles, so its first real value does not arrive until 33 bars into the tape. A series with a hole in the middle raises rather than being handed on.

src/trading/indicators/macd.py — `MACD._lines`

```

ema_fast = _ema(closes, self.fast)
ema_slow = _ema(closes, self.slow)
macd_line: _Line = [
    None if (f is None or s is None) else f - s
    for f, s in zip(ema_fast, ema_slow)
]

```

src/trading/indicators/macd.py — the signal line and the histogram

```

signal_line: _Line = [None] * first
signal_line.extend(_ema(real, self.signal_period))
histogram: _Line = [
    None if (m is None or g is None) else m - g
    for m, g in zip(macd_line, signal_line)
]

```

Design intention. The operator watches for a top-of-the-hill shape in either polarity, timed against a Bollinger band threshold. The histogram is the series that shape reads from, and the vote reads the last two of its bars, so a turn is caught on the bar it happens rather than a bar later.

src/trading/indicators/macd.py — the two bars the vote reads

```

macd_line, signal_line, histogram = self._lines(closes)

curr_hist = histogram[-1]
prev_hist = histogram[-2]
curr_macd = macd_line[-1]

```

The confidence divides the histogram by a share of the last close, so the same crossover scores the same on a four-figure asset and on one worth a few millionths of a dollar.

Stochastic RSI

Published formula, for reference:

```

StochRSI = (RSI - min(RSI, 14)) / (max(RSI, 14) - min(RSI, 14))
%K       = SMA(StochRSI * 100, 3)
%D       = SMA(%K, 3)

```

Functional. The module runs Wilder's recursion over the closes to build its own RSI series, then places each RSI value inside its own fourteen-value range. The vote reads the crossover of the two smoothed lines. The result saturates at 0 and at 100 far more often than plain RSI does, and that saturation is what the panel is watching for. A window whose RSI never moved is caught before the division, and the vote abstains on it rather than dividing by that spread.

src/trading/indicators/stochastic_rsi.py — StochasticRSI.rsi_values

```

avg_gain = (avg_gain * (self.rsi_period - 1) + gains[i]) / self.rsi_period
avg_loss = (avg_loss * (self.rsi_period - 1) + losses[i]) / self.rsi_period

```

src/trading/indicators/stochastic_rsi.py — StochasticRSI.stoch_ratios

```
window = rsi_values[i - self.stoch_period + 1 : i + 1]
low = min(window)
ratios.append((rsi_values[i] - low) / (max(window) - low))
```

Design intention. This is the operator's second oscillator, and he looks for a long run at one polarity to line up with the other voters. Rescaling RSI against its own recent range is what makes those runs stand out at all. A window that never moved is reported as flat rather than hidden, so the caller can tell a real zero from a market that stood still.

src/trading/indicators/stochastic_rsi.py — what the caller is handed

```
return start, ratios, flat
```

Ichimoku Cloud

Published formula, for reference:

```
Tenkan = (highest high 9 + lowest low 9) / 2
Kijun = (highest high 26 + lowest low 26) / 2
Senkou A = (Tenkan + Kijun) / 2          plotted 26 bars forward
Senkou B = (highest high 52 + lowest low 52) / 2 plotted 26 forward
Chikou = close                          plotted 26 bars back
```

Functional. Every line here is a midpoint: the highest high and the lowest low of a window, averaged. A window that has not closed yet answers nothing rather than zero. Five values come back for each candle. Two of them draw the cloud twenty-six bars ahead of the last candle, and that is the only forward-looking reading anywhere in the panel. The lines are returned at their own bar, so the twenty-six bar shift belongs to whatever draws them.

src/trading/indicators/ichimoku.py — IchimokuCloud.lines

```
tenkan = self._span_mid(candles, i, self.tenkan)
kijun = self._span_mid(candles, i, self.kijun)
senkou_b = self._span_mid(candles, i, self.senkou_b)
senkou_a = (
    (tenkan + kijun) / 2.0
    if (tenkan is not None and kijun is not None)
    else None
)
out.append((tenkan, kijun, senkou_a, senkou_b, candles[i].close))
```

Design intention. The operator calls this one of the few predictive indicators and uses it to sharpen reversal-zone detection. The forward cloud is the half of the indicator that does that work. A window that has not closed answers nothing rather than zero, so an unfinished cloud can never be mistaken for a flat one.

src/trading/indicators/ichimoku.py — IchimokuCloud._span_mid

```

if end < period - 1:
    return None
return self._mid(candles, end - period + 1, period)

```

The cloud thickness divides by the price bare. Every candle reaching this point has already been refused unless its price is above zero, so the division needs no guard of its own.

Volume

Published formulae, for reference:

```

OBV_t  = OBV_prev + V_t    if C_t > C_(t-1)
        OBV_prev - V_t    if C_t < C_(t-1)
        OBV_prev          otherwise
TP_t   = (H_t + L_t + C_t) / 3
MF_t   = TP_t * V_t
MFI    = 100 - 100 / (1 + sum(MF where TP rose, 14)
                    / sum(MF where TP fell, 14))
CLV_t  = ((C_t - L_t) - (H_t - C_t)) / (H_t - L_t)
CMF    = sum(CLV * V, 20) / sum(V, 20)
A/D_t  = A/D_prev + CLV_t * V_t

```

Functional. The Volume column is the one voter the operator does not describe in his list above, so this is its only description in the manual. Three published measures are computed separately and scored together. The vote answers one question: does the volume traded agree with the direction the price moved. A price low that On-Balance Volume refuses to match is the divergence this module scores highest. Where neither money-flow bucket was ever credited, the Money Flow Index answers 50 rather than 100, so a market that never moved does not read as maximum overbought.

src/trading/indicators/volume.py — VolumeAnalysis._obv

```

obv = [0.0]
for i in range(1, len(candles)):
    if candles[i].close > candles[i - 1].close:
        obv.append(obv[-1] + candles[i].volume)
    elif candles[i].close < candles[i - 1].close:
        obv.append(obv[-1] - candles[i].volume)
    else:
        obv.append(obv[-1])

```

src/trading/indicators/volume.py — VolumeAnalysis._mfi

```

if pos_mf <= 0.0 and neg_mf <= 0.0:
    return 50.0
if neg_mf <= 0.0:
    return 100.0
return 100.0 - 100.0 / (1.0 + pos_mf / neg_mf)

```

Design intention. Volume confirms or contradicts the other eleven. It does not lead them. Its weight is 0.8, the lightest in the panel alongside RSI, and that number is the intention written down.

src/trading/ta_engine.py — the weight that says so

```
"volume": 0.8,
```

Both tests in the block above compare a money flow against zero. Money flow is a price times a volume and carries no fixed scale, so only an exact zero can stand as the test.

Slingshot

Published formula, for reference:

```
UpperBB = SMA(close, 20) + 2.0 * sigma(close, 20)
LowerBB = SMA(close, 20) - 2.0 * sigma(close, 20)
UpperKC = SMA(close, 20) + 1.5 * SMA(TR, 20)
LowerKC = SMA(close, 20) - 1.5 * SMA(TR, 20)
sqzOn = (LowerBB > LowerKC) and (UpperBB < UpperKC)
delta_t = C_t - (donchian_mid_20 + SMA(close, 20)) / 2
val = linreg(delta, 20, 0)
```

Functional. The squeeze is on while the Bollinger band sits entirely inside the Keltner Channel. When it opens again, a fitted momentum line gives the sign of the release, and that sign is the vote. The module computes its own true range rather than calling the shared helper, and its first bar agrees with the shared one anyway.

src/trading/indicators/slingshot.py — SlingshotIndicator.compute

```
rangema = trma_v[i] if trma_v[i] is not None else 0.0
up_kc = mid + rangema * self.kc_mult
lo_kc = mid - rangema * self.kc_mult
sqz_on = (lo > lo_kc) and (up < up_kc)
seg = [
    deltas[k] for k in range(max(delta_lo, i - self.bb_period + 1), i + 1)
]
val = self._linreg_endpoint(seg)
```

Design intention. The operator wants a precise reading of a reversal zone, and he names Chris Moody's Slingshot system as the source. The code does not implement that system. What is in the module is John Carter's TTM Squeeze, with Bollinger's own rules 6 and 8 for the snapback. Moody's method is an EMA trend and pullback with no band, no channel and no squeeze at all. The module says so itself, in its own docstring. The column keeps the name he gave it, and the name and the code are two different indicators.

src/trading/indicators/slingshot.py — the module stating it

```
# LazyBear's KC shares the BB basis; rangema is the SMA of
# True Range over the same period.
```

In development. Which of the two indicators the column should hold is the operator's decision, not a coding one, so nothing is proposed here.

ADX and DMI

Published formula, for reference:


```

up_t    = H_t - H_(t-1)
down_t  = L_(t-1) - L_t
+DM_t   = up_t   if up_t > down_t and up_t > 0      else 0
-DM_t   = down_t if down_t > up_t and down_t > 0    else 0
+DI      = 100 * Wilder(+DM, 14) / Wilder(TR, 14)
-DI      = 100 * Wilder(-DM, 14) / Wilder(TR, 14)
DX       = 100 * |+DI - -DI| / (+DI + -DI)
ADX      = Wilder(DX, 14)

```

Functional. One smoothing recursion carries every line in this indicator. The reading tells you how committed a move is, and says nothing about which way it points. Below 20 the market is ranging. Above 35 it is trending hard. The indicator abstains when its smoothed true range is not above zero, so a market that moved at all still gets a vote however little the asset costs.

src/trading/indicators/adx.py — ADXIndicator._wilder_smooth

```

if len(values) < period:
    return [0.0] * len(values)
result = [0.0] * (period - 1)
result.append(sum(values[:period]) / period)
for v in values[period:]:
    result.append((result[-1] * (period - 1) + v) / period)
return result

```

Design intention. The operator reads ADX for trend strength, and he watches it fall rather than rise, because a falling reading marks a trend running out of road. The engine carries the other half of that: a strong trend holds a scrum back rather than letting it fire into a move that is still running.

src/trading/gate_chain.py — ADXTrendSuppressionGate.evaluate

```

if ctx.adx < self.adx_threshold:
    return GateResult(passed=True)

```

Issue #414 owns the abstain threshold and no correction is proposed here.

Supertrend

Published formula, for reference:

```

ATR      = Wilder(TR, 10)
hl2_t    = (H_t + L_t) / 2
rawUp    = hl2 + 3.0 * ATR
rawLow    = hl2 - 3.0 * ATR
Upper_t  = rawUp if rawUp < Upper_prev or C_(t-1) > Upper_prev else Upper_prev
Lower_t  = rawLow if rawLow > Lower_prev or C_(t-1) < Lower_prev else Lower_prev
bullish while C_t >= Lower_t; turns bullish again when C_t > Upper_t

```

Functional. A band here is sticky. It moves toward the price and refuses to move away from it, unless the price closed past it last bar. What comes out is a line that trails under the price in an uptrend and over it in a downtrend, and the bar where price crosses that line is the flip.

src/trading/indicators/supertrend.py — SupertrendIndicator.compute

```
final_ub = (
    raw_ub
    if raw_ub < prev_ub or candles[i - 1].close > prev_ub
    else prev_ub
)
final_lb = (
    raw_lb
    if raw_lb > prev_lb or candles[i - 1].close < prev_lb
    else prev_lb
)
```

Design intention. Supertrend rests on Average True Range and helps the operator spot a reversal. The flip is the event that does that work, and the line either side of it is context. The bullish test and the bearish test differ on purpose, so a trend has to be broken outright before the line turns.

src/trading/indicators/supertrend.py — the asymmetric flip test

```
prev_bull = st[-1]
if prev_bull:
    curr_bull = candles[i].close >= final_lb
else:
    curr_bull = candles[i].close > final_ub
```

The band this indicator measures distance from is derived rather than read off the tape, and it can genuinely reach zero once the average true range passes the midpoint over the multiplier. The indicator abstains outright on that bar instead of dividing by it.

src/trading/indicators/supertrend.py — the abstention

```
if st_line <= 0.0:
    return Signal(
        "supertrend",
        timeframe,
        SignalDirection.NEUTRAL,
        0.0,
        self.weight,
        abstained=True,
    )
```

Z-Score

Published formula, for reference:

```
mean = SMA(close, 50)
sigma = population deviation of the same 50 closes
Z = (close - mean) / sigma
```

Functional. The reading is how many standard deviations the price sits from its own fifty-bar mean, and the vote reads the extremes of that. Fifty bars is longer than the twenty the Bollinger Bands use, so a stretch those bands have already absorbed still shows up here. A window with almost no spread at all abstains rather than dividing by it.

src/trading/indicators/zscore.py — ZScoreIndicator.compute

```
closes = [c.close for c in candles[-self.period :]]
sma = sum(closes) / self.period
variance = sum((c - sma) ** 2 for c in closes) / self.period
std = variance**0.5
```

Design intention. The operator calls this the classic Z-Score, says an upgrade is coming, and uses it to confirm a reversal. The longer window is why it earns a column of its own beside the Bollinger Bands.

One formula, written once, for both bars. The current bar and the previous bar each divide by their own deviation bare, and a guard above each refuses a deviation under the same threshold before the division runs.

src/trading/indicators/zscore.py — both divisions

```
z = (candles[-1].close - sma) / std
```

```
z_prev = (candles[-2].close - s2) / std2
```

Published method. The source the operator named publishes two formulae. The first is the classic score above. The second turns a score back into a price, so a level the market has reversed at can be drawn as a zone that widens and narrows with the deviation.

```
Projected Price = Mean + (Target Z * Standard Deviation)
```

The same page names the noise reduction: the raw score is smoothed by a volume weighted average before anything reads it. Turning points are then found in that smoothed series, kept only when they reach a minimum score, and the last few of each side are averaged into the two levels the vote uses.

Both bars now read one window through one expression, so the two spellings that disagreed cannot come back. A window whose prices never moved returns nothing at all, which is how the reading abstains rather than shifting.

src/trading/indicators/zscore.py — one window, one expression, every reader

```
def _window_zscore(closes: list, close: float) -> Optional[tuple]:
    if _window_has_no_range(closes):
        return None
    count = len(closes)
    mean = sum(closes) / count
    deviation = (sum((c - mean) ** 2 for c in closes) / count) ** 0.5
    return mean, deviation, (close - mean) / deviation
```

What the page does not publish. It names four settings and gives a default value for none of them. Three take a number this file already used or the page itself quotes, and one is chosen here and marked as such.

Setting	Value	Where the value comes from
Z-Score Length	50	the period this file already used
Reversal Threshold	2.0	the strong level this file already used, and the level the page quotes as the classic one
Pivot bars	1	a turn against the bar each side; the page gives noise reduction to the smoothing instead
Lookback Depth	5	chosen here, because the page publishes no value and this file had none

Kaufman Efficiency Ratio

Published formula, for reference:

$$ER = |C_t - C_{(t-10)}| / \text{sum}(|C_i - C_{(i-1)}|, 10)$$

Functional. Net displacement divided by the distance travelled to get there. 1.0 is a straight line, 0.0 is pure noise, and both halves read the same eleven closes. A window where the price never moved travels exactly zero distance, and the indicator abstains on that rather than reporting a regime it never measured.

src/trading/indicators/kaufman_er.py — `KaufmanERIndicator.compute`

```
closes = [c.close for c in candles[-(self.period + 1) :]]
net_change = abs(closes[-1] - closes[0])
price_travel = sum(
    abs(closes[i] - closes[i - 1]) for i in range(1, len(closes))
)
```

Design intention. A low ratio marks the market Acervator prefers, in the operator's own words above. The engine acts on that at both ends: a scrum is refused when the market is running too straight to mean-revert, and refused again when it is too noisy to hold an edge.

The division that produces the ratio is bare. A guard four lines above returns early when the distance travelled is not above zero, so the denominator is already proved positive by the time the division runs.

src/trading/indicators/kaufman_er.py — the division

```
er = net_change / price_travel
```

RSI

Published formula, for reference:

```

gain_t = max(C_t - C_(t-1), 0)
loss_t = max(C_(t-1) - C_t, 0)
AvgGain = Wilder(gain, 14)
AvgLoss = Wilder(loss, 14)
RS      = AvgGain / AvgLoss
RSI     = 100 - 100 / (1 + RS)

AvgLoss = 0 and AvgGain > 0   -> RSI 100
AvgLoss = 0 and AvgGain = 0   -> no reading, the indicator abstains

```

Functional. The reading is the share of recent movement that ran upward, on a 0 to 100 scale. Above 70 votes bearish, below 30 votes bullish, and anything between casts no vote. A bar with neither a gain nor a loss abstains outright.

src/trading/indicators/rsi.py — RSIIndicator.compute

```

if rsi > 70:
    direction = SignalDirection.BEARISH
elif rsi < 30:
    direction = SignalDirection.BULLISH
else:
    direction = SignalDirection.NEUTRAL

```

Design intention. RSI is the plainest voter in the panel, and it overlaps the Stochastic RSI on purpose. Its weight is 0.8, which stops the same evidence being counted twice at full strength.

One number is read two ways here. The value is rounded to two decimals before the vote tests it, while the overbought and oversold flags in the same dictionary test the unrounded number. An RSI of 70.003 therefore sets overbought and still votes neutral.

src/trading/indicators/rsi.py — the rounding today

```

return {
    "rsi": round(rsi, 2),
    "rs_indeterminate": rs_indeterminate,
    "overbought": rsi > 70,
    "oversold": rsi < 30,
}

```

Proposed, not present:

```

return {
    "rsi": rsi,
    "rs_indeterminate": rs_indeterminate,
    "overbought": rsi > 70,
    "oversold": rsi < 30,
}

```

The rounding belongs to display, not to a comparison. Issue #414 carries this.

Departures from the published maths

Functional. Two departures remain, and neither is a matter of scale. Every voter now reads the same on an asset worth four figures and on one worth a few millionths of a dollar, because no division and no threshold in the twelve carries an absolute constant against a quantity measured in the asset's own price units. Where a denominator can genuinely reach zero the indicator abstains on that bar rather than adding a constant to it.

- The Vortex overwrites its own direction to BULLISH whenever the downward line reaches 1.30, after the crossover has already decided. The two lines themselves match Botes and Siepmann exactly. The overwrite is a reading Acervator adds on top of them.
- The RSI is rounded to two decimals before the vote tests it against 70 and 30, while the overbought and oversold flags beside it test the unrounded number. An RSI of 70.003 sets overbought and still votes NEUTRAL.

Supertrend is the one place an abstention stands in for a division. Its lower band is derived from the average true range rather than read off the tape, so it can reach zero on a real market, and the indicator refuses to vote on that bar.

Design intention. An indicator has to read the same on every asset, because one threshold serves the whole fleet. Four test files hold that property for the twelve, one shape at a time, by running the same tape at several price scales and comparing the readings.

```
tests/test_bollinger_and_slingshot_are_scale_invariant.py
tests/test_vortex_and_landing_strip_are_scale_invariant.py
tests/test_macd_and_kaufman_er_are_scale_invariant.py
tests/test_the_last_eight_indicators_are_scale_invariant.py
```

Issue #414 owns the two readings above. Both are deliberate choices in Acervator's own reading of a published indicator, not arithmetic errors, and neither has a replacement chosen yet.

In development.

Trading gate logic chain

The epigraph on the title page reads *dissolvendus reformatusque*, dissolved and reformed. The chain below does exactly that, once per tick: twelve separate readings dissolve into one signed number, and the gates re-form that number into an order or a refusal.

The chain

Diagram: flowchart, top to bottom

- OHLCV candles -> **The 12 indicator modules**
- The 12 indicator modules -> **VotingEngine.compute_all** (*Signal: direction, confidence, weight*)
- VotingEngine.compute_all -> **VotingEngine._aggregate**
- VotingEngine._aggregate -> **VotingSummary.net_score**
- VotingEngine._aggregate -> **VotingSummary.consensus_confidence**
- VotingSummary.net_score -> **consensus_direction: deadband +/- 0.1**
- consensus_direction: deadband +/- 0.1 -> **BULLISH / BEARISH / NEUTRAL**

- BULLISH / BEARISH / NEUTRAL -> **ScrummingBot.tick: is_bullish, is_bearish**
- VotingSummary.consensus_confidence -> **ScrummingBot.tick: is_bullish, is_bearish**
- _skewed_confidence_floor: 0.25 relaxed by the skew -> **ScrummingBot.tick: is_bullish, is_bearish**
- OHLCV candles -> **detect_bb_proximity**
- detect_bb_proximity -> **bb_position**
- bb_position -> **bb_pos against _bb_detect_thresholds**
- ScrummingBot.tick: is_bullish, is_bearish -> **build_scrumming_scrum_chain**
- bb_pos against _bb_detect_thresholds -> **build_scrumming_scrum_chain**
- adx signal detail 'adx' -> **build_scrumming_scrum_chain**
- kaufman_er signal detail 'er' -> **build_scrumming_scrum_chain**
- zscore signal detail 'z' -> **build_scrumming_scrum_chain**
- build_scrumming_scrum_chain -> **ChainResult.should_fire**
- ChainResult.should_fire -> **_tick_execute_scrum, then _execute_sell (true)**
- ChainResult.should_fire -> **ChainResult.blocked, one message per gate (false)**
- _tick_execute_scrum, then _execute_sell -> **guarded_place_order**

Net, Comp and Conf

VotingEngine._aggregate in src/trading/ta_engine.py produces all three panel aggregates from the twelve signals.

```
weighted_score = direction * confidence * weight          one per voter
bull_score     = sum of |weighted_score| over the BULLISH voters
bear_score     = sum of |weighted_score| over the BEARISH voters
net_score      = bull_score - bear_score                  the Net column
voted_weight   = sum of weight over the voters that did not abstain
confidence     = min(1.0, |net_score| / voted_weight)     the Conf column
```

Functional. Net is a direction bias, not a reversal signal. Positive means the bullish voters outweighed the bearish ones. Conf is the breadth of that agreement, not its force. Six voters agreeing strongly while six abstain reads lower than twelve agreeing mildly, and the panel says the same thing in the Conf column's own tooltip. An abstaining indicator is left out of the divisor, so a short tape cannot water down the reading of the voters that did read. Net is rounded to four decimals and Conf is capped at 1.0.

src/trading/ta_engine.py — VotingEngine._aggregate

```
net = bull_score - bear_score
```

src/trading/ta_engine.py — and the divisor beneath it

```
voted_weight = sum(s.weight for s in signals if not s.abstained)
consensus_conf = abs(net) / voted_weight if voted_weight > 0.0 else 0.0
```

Design intention. Conf answers how many of the voters that could read agree, not how loud they were. Dividing by the weight of the voters that actually read, rather than by all twelve, is the arithmetic that says so. The panel repeats the same rule to the reader in its own tooltip, so the screen and the engine cannot tell two different stories.

src/trading/ta_engine.py — the rule, stated where it is applied

```
# A weighted mean divides by the weight of the points included, and an
# abstention is not one; it is NEUTRAL, so it adds nothing to `net`.
```

The deadband

VotingSummary.consensus_direction in src/trading/indicators/types.py turns Net into a direction, and it holds a deadband of plus and minus 0.1.

```
net_score > +0.1    -> BULLISH
net_score < -0.1    -> BEARISH
otherwise           -> NEUTRAL
```

ScrummingBot.tick then folds the direction and the Conf reading into one boolean per side.

```
is_bullish = direction in (BULLISH, NEUTRAL) and confidence >= floor
is_bearish = direction in (BEARISH, NEUTRAL) and confidence >= floor
```

Functional. The confidence floor starts at 0.25 and the bot relaxes it when the vote is skewed. A landing strip at the upper band sets the bullish flag outright, and one at the lower band sets the bearish flag. A neutral direction passes both sides, so a panel with no consensus does not on its own block a trade. The floor and the gates below decide that.

src/trading/indicators/types.py — VotingSummary.consensus_direction

```
def consensus_direction(self) -> SignalDirection:
    if self.net_score > 0.1:
        return SignalDirection.BULLISH
    elif self.net_score < -0.1:
        return SignalDirection.BEARISH
    return SignalDirection.NEUTRAL
```

Design intention. The deadband stops a near-zero Net reading as a direction. Plus or minus 0.1 against a full voted weight of 11.7 is a very small bias, and the band treats it as none at all. The bot then folds the direction and the confidence into one boolean per side, and a neutral panel passes both.

src/trading/scrumming_bot.py — the two booleans, in plain terms

```
is_bullish = direction in (BULLISH, NEUTRAL) and confidence >= floor
is_bearish = direction in (BEARISH, NEUTRAL) and confidence >= floor
```


The band position

Functional. The band position is %B: zero at the lower band, one at the upper. The two thresholds a scrum and a fold must clear come back in the same terms, so both sides of every comparison are positions and neither side is a price. The fraction is clamped to the 0 to 1 range, so a stored setting outside 0 to 100 cannot push a threshold outside the band. A missing setting falls back to 75 through an explicit test for absence, so a stored zero stays zero.

The relationship in plain terms:

```
detect_frac = scrum_detect_pct / 100          75% by default
upper_detect = 0.5 + detect_frac * 0.5        0.875
lower_detect = 0.5 - detect_frac * 0.5        0.125

SCRUM requires bb_pos >= upper_detect
FOLD  requires bb_pos <= lower_detect
```

src/trading/scrumming/circuit_breakers.py — `_bb_detect_thresholds`

```
detect_frac = max(0.0, min(1.0, detect_pct / 100.0))
half = detect_frac * 0.5
return (0.5 - half, 0.5 + half)
```

Design intention. Working in band positions rather than in prices is what lets one threshold serve the whole fleet. A bot on a four-figure asset and a bot on an asset priced in millionths both read 0.875 and mean the same thing by it. This is the pattern the constants listed above break. A missing setting is tested for absence rather than for truthiness, so a deliberate zero survives instead of being turned back into 75.

src/trading/scrumming/circuit_breakers.py — the absence test

```
# None-sentinel default so a legitimate 0 isn't coerced to 75 by
# `or` truthiness.
_raw = getattr(self.config, "scrum_detect_pct", None)
```

The scrum chain gates

src/trading/gate_chain.py — `build_scrumming_scrum_chain`

```

return GateChain(
    gates=[
        DeltaPositiveGate(),
        IntervalGate(),
        TADirectionGate(side="scrum"),
        TrendHoldGate(),
        MidlineGate(side="scrum"),
        TargetFiresGate(),
        BBProximityGate(side="scrum"),
        CircuitBreakerGate(side="scrum"),
        HTFDeferGate(side="scrum"),
        HysteresisGate(side="scrum"),
        ADXTrendSuppressionGate(),
        EfficiencyRatioRegimeGate(),
        ZScoreExtremityGate(side="scrum"),
        RipeHarvestScrumOverride(), # runs in pass 2 regardless of list position
    ],
    side="scrum",
)

```

Functional. Every gate in the list runs, and every refusal is collected. The trade fires only when the refusal list is empty, so one blocked gate is enough to hold it. The order of the list fixes the order the refusals appear in, which is the order the log prints them.

src/trading/gate_chain.py — the verdict

```

return ChainResult(
    should_fire=(not final_blocked),
    passed=passed,
    blocked=final_blocked,
    overrides_applied=overrides_applied,
)

```

Design intention. One gate holds one question, and each writes its own refusal in its own words. Reading back a held trade is then a matter of reading which name refused, rather than re-deriving the whole decision.

src/trading/gate_chain.py — `DeltaPositiveGate.evaluate`, the pattern every gate follows

```

def evaluate(self, ctx: GateContext) -> GateResult:
    if ctx.delta > 0:
        return GateResult(passed=True)
    return GateResult(passed=False, blocker_message="delta≤0")

```

Gate	Passes when	Reads
delta_positive	the holdings sit above target	the target delta
interval	the delta clears the scrumming interval	the delta
ta_bullish	is_bullish	Net, Conf and the floor
trend_hold	no sustained uptrend	more than 13 of the last 20 candles closing above their open

Gate	Passes when	Reads
midline_scrum	bb_pos above 0.50 and no phantom lock	bb_position
target_fires	the detect and fire ramp is in FIRE	the target ramp
bb_proximity_scrum	bb_pos >= upper_detect	bb_position
circuit_breaker_scrum	no soft trip on the scrum side	the breaker state
htf_defer_scrum	the higher timeframe is not bullish	the coordinator
hysteresis_scrum	the price cleared the last pivot plus fees	the pivot
adx_trend_suppression	ADX under 30.0	the adx signal detail
efficiency_ratio_regime	ER between 0.05 and 0.70	the er signal detail
zscore_extremity	Z above -2.0	the z signal detail

Functional. Three of those gates skip the collated vote and read one indicator's own number: ADX, the Efficiency Ratio and the Z-Score. The bot fills each field from that indicator's signal on the tick. A reading of zero or below is the not-populated marker, and all three pass on it.

src/trading/gate_chain.py — EfficiencyRatioRegimeGate.evaluate, one of the three

```
def evaluate(self, ctx: GateContext) -> GateResult:
    if ctx.efficiency_ratio <= 0.0:
        return GateResult(passed=True)
    if ctx.efficiency_ratio >= self.upper_threshold:
```

Design intention. These three describe the market regime rather than its direction, which is why they reach the chain on their own instead of through the vote. Passing on the marker keeps a bot that never filled the field from being frozen by it. The cost is real and worth stating: a genuine zero reads exactly the same as a missing one, and the gate cannot tell them apart.

In development. Separating the two would need a marker the field cannot produce, and no shape for that has been chosen.

The fold chain gates

src/trading/gate_chain.py — build_scrumming_fold_chain

```

return GateChain(
    gates=[
        TranchesQueuedGate(),
        TADirectionGate(side="fold"),
        MidlineGate(side="fold"),
        SmartCeilingGate(),
        BBProximityGate(side="fold"),
        CircuitBreakerGate(side="fold"),
        HTFDeferGate(side="fold"),
        HysteresisGate(side="fold"),
        ZScoreExtremityGate(side="fold"),
        DeepFoldOverride(),
    ],
    side="fold",
)

```

Functional. The buy side runs nine gates against the sell side's thirteen. Six of the sell-side gates have no twin here, and the buy side adds two of its own.

absent on the fold side	delta_positive, interval, trend_hold, target_fires, adx_trend_suppression, efficiency_ratio_regime
present only here	tranches_queued, smart_ceiling

Design intention. The two chains mirror each other, and two of the six absences carry a written reason. The interval gate says in its own source that it needs no fold twin, because the bot returns early on the same condition. The trend-hold gate is declared sell-side only, and the config field that would arm a buy-side twin is declared and unread. 06-trading-tab.md carries the proposal for that one, at the Strategy Gate Flags group.

src/trading/gate_chain.py — IntervalGate, stating its own case

```

# No FOLD twin; ScrummingBot.tick returns early on below_interval when both
# its fold queue and distribution accumulator are empty.

```

The other four absences carry no stated reason in the code at all. That is worth knowing before anyone assumes the asymmetry is deliberate.

Gate	Passes when
tranches_queued	at least one fold tranche is waiting
ta_bearish	is_bearish
midline_fold	bb_pos below 0.50
smart_ceiling	the position sits under the smart ceiling
bb_proximity_fold	bb_pos <= lower_detect
circuit_breaker_fold	no soft trip on the fold side
htf_defer_fold	the higher timeframe is not bearish
hysteresis_fold	the price cleared the last pivot plus fees
zscore_extremity	Z below +2.0

The two overrides

Functional. An override runs in a second pass and force-passes gates that already refused. Its own verdict never appears in the pass list or the block list. Each refusal it lifts has its message rewritten to name the override, and the name is recorded separately, so the log tells you which gate was overridden and by what.

src/trading/gate_chain.py — GateChain.evaluate, the second pass

```
overrides_applied: list[str] = []
for gate in self._override_gates:
    result = gate.evaluate(ctx)
    if not result.passed:
        continue
    for target_name in result.override_gates:
        if target_name in blocked_index:
            idx = blocked_index.pop(target_name)
            _name, _msg = blocked[idx]
            blocked[idx] = (_name, f"OVERRIDDEN_BY:{gate.name}({_msg})")
            overrides_applied.append(target_name)
```

Override	Arms on	Force-passes
ripe_harvest_override	positive delta, interval cleared, bb_pos >= upper_detect	midline_scrum, target_fires, trend_hold, ta_bullish
deep_fold_override	negative delta, interval cleared, bb_pos <= lower_detect	midline_fold, ta_bearish

Design intention. At the band extreme, the band position outranks the vote. That comes straight from the operator's own reading of the Bollinger Bands: they are the boundary a trade fires at, and a position sitting on one is the condition he wants to trade. Neither override touches the circuit breaker, the hysteresis or the three indicator gates, so the safety gates keep their veto.

src/trading/gate_chain.py — RipeHarvestScrumOverride, and what it may lift

```
name = "ripe_harvest_override"
side = "scrum"
overrides = ("midline_scrum", "target_fires", "trend_hold", "ta_bullish")
```

Reading the chain after the fact

Functional. A trade already on the books can have its reading rebuilt. The whole voting engine runs again over the candles that stood before the trade, and the direction, the Net, the Conf, the three vote counts and every indicator's own vote are recorded beside it. The band position is recomputed too. A note goes with the record saying the market half was rebuilt and the bot-state fields are empty on purpose.

src/trading/gate_healer.py — reconstruct_market_gate

```

parsed = candles_from_raw(window)
summary = VotingEngine().compute_all(parsed, timeframe)
out.ta_direction = _direction_name(summary)
out.ta_net_score = _safe_float(getattr(summary, "net_score", None))
out.ta_confidence = _safe_float(getattr(summary, "consensus_confidence", None))
out.ta_bullish_count = getattr(summary, "bullish_count", None)
out.ta_bearish_count = getattr(summary, "bearish_count", None)
out.ta_neutral_count = getattr(summary, "neutral_count", None)
out.indicator_votes = _indicator_votes(summary)

```

Design intention. The record should be the panel exactly as it stood the moment the order left, so a trade can be read back later without the bot that placed it. Only the market half can be rebuilt. The bot's own state at that moment is gone, and the note says so rather than filling the fields with a guess.

src/trading/gate_healer.py — the note it writes

```

out.reconstruction_note = (
    f"market half rebuilt from {len(window)} candles; "
    "bot-state fields null by design"
)

```

Subsystem Tabs

Reference. Each heading below names a screen. The manual's own text for it comes first, and under that is what the code does today. Longer descriptions live one file per screen in 08-tabs/.

One list names the ten tabs the window builds, and every screen below that the window does not build says as much in its own entry. Two are skeletons now: Status and Accumulation each draw a name, one sentence saying the tab is not built, and the issue that carries the build-out. Sim and Paper were skeletons until issues #117 and #19 built them, and both are full screens today.

src/gui/main_tabs/main_window_surface.py — the order before issue #450

```

CANONICAL_TAB_ORDER = (
    TRADING_TAB,
    MARKET_INSPECTOR_TAB,
    BOT_SWARM_TAB,
    ASSET_CHARTS_TAB,
    HISTORY_TAB,
    SIMULATOR_TAB,
    CONSOLE_TAB,
    PAPER_TRADER_TAB,
    SYSTEM_STATUS_TAB,
    PROOF_OF_ACCUMULATION_TAB,
)

```

Each of those names is a constant holding the label the tab bar shows. The skeletons take their label from their own surface, so the bar and the screen cannot carry two spellings of one name. Issue #450 renamed every one of those constants and reordered the tuple, and the section under this one holds what the file carries today.

src/gui/main_tabs/main_window_surface.py — the labels before issue #450

```
TRADING_TAB = "Trading"
MARKET_INSPECTOR_TAB = "Market Inspector"
BOT_SWARM_TAB = "Bot Swarm"
ASSET_CHARTS_TAB = "Asset Charts"
HISTORY_TAB = "History"
SIMULATOR_TAB = "Simulator"
CONSOLE_TAB = "Console"

PAPER_TRADER_TAB = paper_trader_tab_surface.HEADING
SYSTEM_STATUS_TAB = system_status_tab_surface.HEADING
PROOF_OF_ACCUMULATION_TAB = proof_of_accumulation_tab_surface.HEADING
```

The order, the names and the colours today

The bar now opens on Sim and every tab carries a single word. The order below is what the window ends with, and it is decided in the one place named above.

src/gui/main_tabs/main_window_surface.py — CANONICAL_TAB_ORDER

```
CANONICAL_TAB_ORDER = (
    SIM_TAB,
    PAPER_TAB,
    LIVE_TAB,
    CHARTS_TAB,
    INSPECTOR_TAB,
    SWARM_TAB,
    ACCUMULATION_TAB,
    HISTORY_TAB,
    STATUS_TAB,
    CONSOLE_TAB,
)
```

Eight of the ten labels are new words for screens that already existed. History and Console were already one word and did not change. Each constant was renamed to match the word it now holds.

src/gui/main_tabs/main_window_surface.py — the labels

```
LIVE_TAB = "Live"
INSPECTOR_TAB = "Inspector"
SWARM_TAB = "Swarm"
CHARTS_TAB = "Charts"
HISTORY_TAB = "History"
SIM_TAB = "Sim"
CONSOLE_TAB = "Console"

PAPER_TAB = paper_trader_tab_surface.HEADING
STATUS_TAB = system_status_tab_surface.HEADING
ACCUMULATION_TAB = proof_of_accumulation_tab_surface.HEADING
```

The eight renames, in the order the bar reads them:

was	is now
Simulator	Sim
Paper Trader	Paper
Trading	Live
Asset Charts	Charts
Market Inspector	Inspector
Bot Swarm	Swarm
Proof of Accumulation	Accumulation
System Status	Status

Sim, Paper and Live carry the promotion pipeline, and three different grounds separate them at a glance. The other seven tabs share the gold ground.

src/gui/main_tabs/main_window_surface.py — TAB_GROUNDS

```
TAB_GROUNDS = {
    SIM_TAB: BLACK_GROUND,
    PAPER_TAB: WHITE_GROUND,
    LIVE_TAB: GOLD_GROUND,
    CHARTS_TAB: GOLD_GROUND,
    INSPECTOR_TAB: GOLD_GROUND,
    SWARM_TAB: GOLD_GROUND,
    ACCUMULATION_TAB: GOLD_GROUND,
    HISTORY_TAB: GOLD_GROUND,
    STATUS_TAB: GOLD_GROUND,
    CONSOLE_TAB: GOLD_GROUND,
}
```

No tab holds a colour of its own. A ground names two theme tokens, and every theme fills those tokens with its own values, so gold is the yellow that suits each theme rather than one hex value everywhere.

src/gui/theme_engine.py — the six tab tokens each theme carries

theme	tab_gold_bg	tab_gold_text	tab_black_bg	tab_black_text	tab_white_bg	tab_white_text
cyberpunk_dark	#fcee0a	#8c0018	#0a0a0f	#ff5577	#f5f5fa	#0a0a0f
neon_light	#f0cf1f	#99001f	#1a1a2e	#ff6b8a	ffffff	#1a1a2e
classic_terminal	ffffff00	#990000	#0a0a0a	#ff3333	#e8e8e8	#0a0a0a
minimal_modern	#eab308	#7f1d1d	#1a1a1a	#ff6b6b	ffffff	#1a1a1a
glass_metal	#e8b34a	#6b1020	#1c1c24	#ff6688	#e8e8f0	#1c1c24

Every pair above clears WCAG 2.2 AA at 4.5 to 1. The narrowest margin is red on gold under Minimal Modern, at 5.22 to 1.

The Qt bar paints those colours itself, because a Qt style sheet cannot colour one tab by its position. The theme's own style sheet sets the six colours on the bar, so switching theme repaints it.

src/gui/main_tabs/main_tab_bar.py — MainWindowTabBar.paintEvent


```
painter.fillRect(self.tabRect(index), ground)
painter.setPen(text)
painter.drawText(self.tabRect(index), int(Qt.AlignCenter), self.tabText(index))
```

The React bar draws the same ten buttons with the same two colours each. The window publishes the colours beside the labels, so the page holds no palette of its own.

src/gui/main_tabs/main_window_surface.py — what the frontend reads

```
"tab_labels": list(self.tab_labels),
"tab_methods": dict(self.tab_methods),
"tab_colours": {tab: dict(pair) for tab, pair in self.tab_colours.items()},
"theme": self.theme,
```

The Live tab, which the manual's part list calls the Trading Tab, has its own section, 06-trading-tab.md, and the Indicator Voting Panel has 07-indicators.md.

A strategy reaches real money through four steps: Market Inspector, Simulator, Paper Trader, Live. Each step is a gate, and all four screens are built. 08-tabs/promotion-pipeline.md draws the chain and marks where a step still hands nothing to the next.

The sections below run in the order 04-manual-parts.md lists the tabs.

Main Window

Portfolio Information Panels

Found at the top of the Main Window at all times, this provides metrics for total performance and activity of the platform.

The window is built in three parts: the menu bar, the header strip, and the tab row. The panels above fill that strip, and each tab below sits in that row.

The strip is one row. Five labelled columns run down the left — SPENDABLE, REALISED, LOCKED, MATURE and EXCH — and five counter cards close it on the right.

src/gui/main_tabs/header_strip.py — HeaderStripMixin._build_header_strip

```
self._spendable_widget = SpendableProfitsWidget()
top_row.addWidget(self._spendable_widget, stretch=3)
```

The five cards are Scrummed, Folded, Trades, Bots and Errors. One aggregate call fills every field on the strip once a tick, which keeps two cards from disagreeing about the same fleet.

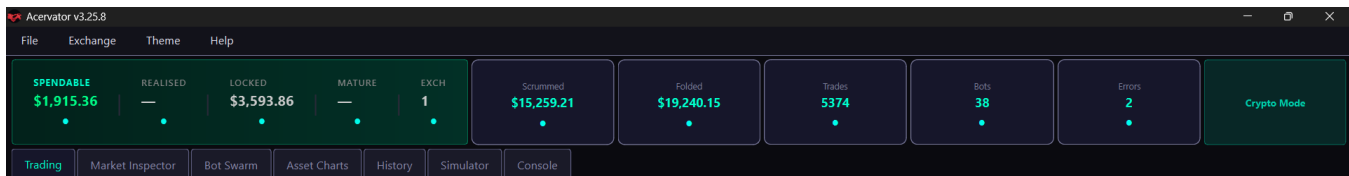
src/gui/main_window.py — _refresh_dashboard

```

agg = self._bot_manager.get_aggregate_stats()
_scr = float(agg.get("total_scrummed_usd", 0.0) or 0.0)
_fld = float(agg.get("total_folded_usd", 0.0) or 0.0)
self._stat_scrummed.set_value(f"${_scr:,.2f}")
self._stat_folded.set_value(f"${_fld:,.2f}")

```

Each field carries a privacy dot that masks the value through the registry the Swarm tab shares. The strip hides itself while the Sim tab or the Paper tab is active, and the Paper tab draws four figures of its own in place of it. The absent live strip is itself the signal that the screen is not live trading.



The header strip and the tab row, with Privacy Mode off.

The chrome above the strip carries four menus. File holds Settings, Reset All Settings and Exit. Exchange holds Add Exchange. Theme lists the five display names the theme engine declares, and Help holds About.

src/gui/main_window.py — the menu bar

```

file_menu = menu_bar.addMenu("&File")
file_menu.addAction("&Settings", self._open_settings)
file_menu.addAction("&Reset All Settings", self._reset_settings)

```

Privacy Mode is off in the figure, so each field draws its own value rather than four asterisks. SPENDABLE and LOCKED carry dollar figures. EXCH counts the open exchange sub-tabs. Crypto Mode at the right swaps the window between the crypto layer and the stock layer.

REALISED and MATURE draw an em dash. Both take a literal absence at the one call site that fills them, on every tick.

src/gui/main_window.py — _refresh_dashboard

```

self._spendable_widget.update_profits(
    {
        "spendable": _wallet_cash,
        "total_realised": None,
        "locked": _crypto_value,
        "mature": None,
        "exchange_count": exchanges,
    }
)

```

Turning Privacy Mode on masks that em dash to asterisks, which reads as a hidden number rather than an empty column. 06-trading-tab.md carries the proposal for the first of the two. Issue #428 carries a second disagreement on this strip, between what the counter tooltips promise and what the cards draw.

Both columns now carry an exchange figure. The window hands the strip the same payload the React strip receives, built by one function, so the two hosts cannot show different numbers.

src/gui/main_window.py — _refresh_dashboard

```
self._spendable_widget.update_profits(
    header_strip_surface.profits_payload(agg, exchanges)
)
```

REALISED is the venue's matched profit and loss across the fleet. MATURE is the profit held in positions worth more than three times what they cost. Where the venue has answered for no bot, both stay empty, and the mask now leaves an empty column empty instead of drawing four asterisks over it.

src/gui/main_tabs/header_strip_surface.py — exchange_amount

```
answered = int(data.get(EXCHANGE_FRESHNESS_KEY, 0) or 0)
if answered <= 0:
    return None
```

Detail: 08-tabs/portfolio-panels.md.

Simulator Tab (Hot Mess; Complete Rebuild In Progress)

The old Simulator was removed and the rebuild replaced it. The tab is named Sim, it opens first on the bar on a black ground, and it draws a clone of the Live tab reading Stone Tablets. It carries three modes — Validation, Back Test and Portfolio Battery — and Nuclear Mode is cancelled. The rest of this section describes the screen that was removed and is kept as the record of what the rebuild replaces.

```
def _build_simulator_tab(self) -> None:
    """Insert the Sim tab at ``SIMULATOR_BUILD_INDEX``."""
    self._add_empty_tab(simulator, index=SIMULATOR_BUILD_INDEX)
```

The tab stacks two panels. Fleet Replay loads every bot from the operator's own state file, builds one real bot per config, and plays Stone Tablet candles through them against a fake exchange. The sim uses the bot class body unchanged, which is the parity guarantee: it runs live's code against a fake exchange rather than a second implementation.

The tab is now called Sim. It sits first on the bar, on a black ground with red text.

src/gui/simulator_tab/fleet/fleet_replay_panel.py — _spawn_sim_fleet The Simulator rebuild removed this file; it is not in the tree.

```
def _spawn_sim_fleet(self) -> int:
    """Construct a real sim bot for every loaded config.

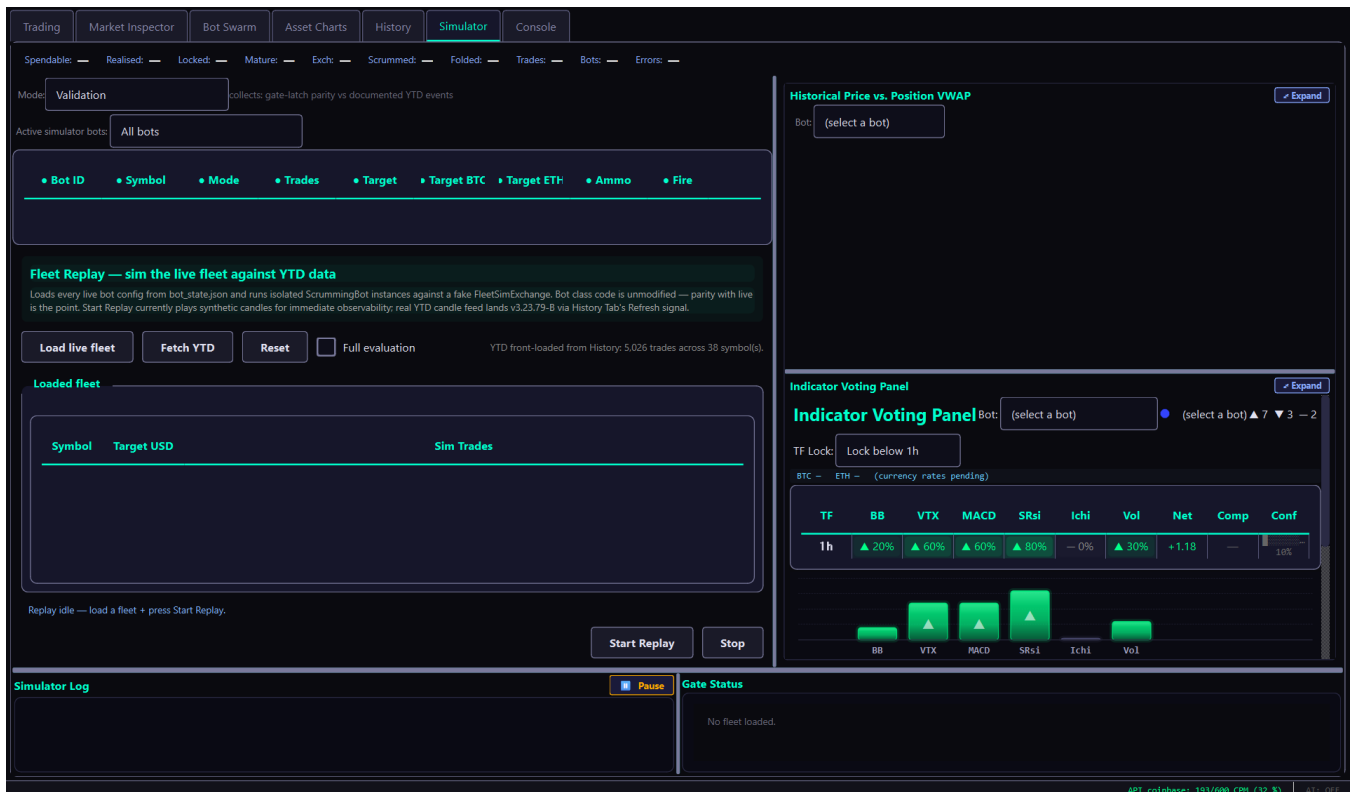
    Returns the number spawned. Reads the Stone Tablet registry
    through the same call Start Replay uses, rather than adding a
    second candle path.
```

The criterion is the gates latching identically on the same data. Not profit and loss, and not the trade count. The sim's gate cell reads the same vocabulary the History table reads, which stops the two surfaces drifting apart.

src/gui/simulator_tab/fleet/sim_visuals.py — GateLightsCell The Simulator rebuild removed this file; it is not in the tree.

```
class GateLightsCell(QWidget):  
    """One linear labelled row of trading gates.
```

Nuclear Mode looped the same fleet over the tablet window with per-cycle market noise and a load pulse, writing over no tablet. It stood as a soak test, judged on coverage and survival, and it compared nothing to live. The operator cancelled it, its code is deleted, and the Sim tab has three modes, not four.



The Simulator tab, with no fleet loaded.

The strip along the top replaces the window's own while this tab is active. It names the same ten readings against sim balances, and every one draws an em dash in the figure, because no fleet is loaded.

React draws that strip. The tab asks variant_surface which of the two strips to build, and both answer the same set and clear calls, so the fleet panel and Nuclear Mode write to either without knowing which they hold.

src/gui/simulator_tab/simulator_tab.py — SimulatorTab._build_stat_strip The Simulator rebuild removed this file; it is not in the tree.

```
from ..variant_surface import SIM_STAT_STRIP, surface_class

return surface_class(SIM_STAT_STRIP)(self)
```

SimStatStripWebStrip holds SimStatStripModel where the Qt strip held ten label pairs, and draws it with `sim_stat_strip.js` in one `QWebEngineView`. A field name the strip does not carry is still ignored, and an empty value still falls back to the em dash. Setting `ACERVATOR_VARIANT` to `qt` builds the Qt strip instead, unchanged.

React drew the Nuclear Mode page too, before the rebuild deleted both. The tab asked the same seam for the panel class that it asked for the strip.

`src/gui/simulator_tab/simulator_tab.py` — `SimulatorTab._nuclear_panel_class` The Simulator rebuild removed this file; it is not in the tree.

```
from ..variant_surface import NUCLEAR_MODE, surface_class

return surface_class(NUCLEAR_MODE)
```

`NuclearModeReactPanel` inherits the Qt panel, so the fleet preview, Start, Stop and the half-second status tick are one piece of Python on both sides. Every widget write moved behind a named accessor. The Qt panel answers those with labels, spin boxes and tick boxes; the React panel answers them by writing into its view model and redrawing the page.

`src/gui/react_nuclear_mode_panel.py` — `NuclearModeReactPanel._set_status_text` The Simulator rebuild removed this file; it is not in the tree.

```
def _set_status_text(self, key: str, text: str) -> None:
    """Show ``text`` on the live-status row ``key`` names."""
    if key in self._panel.status_text:
        self._panel.status_text[key] = text
        self.push()
```

The page draws the header card, the fleet readout, the four run settings, the Start and Stop buttons and the seventeen live-status rows. A press on the page runs the inherited Python and comes back as a redraw. Setting the variant to `qt` builds the Qt panel instead, unchanged.

The two drawn panels follow the same route. `SimPriceVwapChartReact` inherits the price and VWAP chart, so the ticks, the thinning and the trade markers stay one piece of Python, and `GateStatusPanelReact` inherits the gate pane.

`src/gui/simulator_tab/simulator_tab.py` — `SimulatorTab._price_chart_class` The Simulator rebuild removed this file; it is not in the tree.

```
from ..variant_surface import SIM_PRICE_CHART, surface_class

return surface_class(SIM_PRICE_CHART)
```

Neither panel is redrawn in JavaScript. Python builds the whole draw program — every line, box, dot and label, with its colour and its position — and `sim_visuals.js` runs that program on one canvas. The picture is decided on the Python side, so the two builds cannot draw different charts from the same ticks.

`src/gui/main_tabs/sim_visuals_surface.py` — `chart_program` The Simulator rebuild removed this file; it is not in the tree.

```
if not model.symbols:
    return []
if model.focus:
    return focused_program(model, width_px, height_px)
return band_program(model, width_px)
```

The gate pane draws one labelled row per bot and nineteen lights on each, and a gate reading written from Python changes what those lights show. An empty pane still says "No fleet loaded."

The Fleet Replay page is the last of the four. `FleetReplayReactPanel` inherits the Qt panel, so Load live fleet, Fetch YTD, Reset, Start Replay, Stop and both timers are the same Python. Every widget write the panel used to make moved behind a named accessor, and the React panel answers those by writing into its view model.

`src/gui/simulator_tab/simulator_tab.py` — `SimulatorTab._fleet_replay_class` The Simulator rebuild removed this file; it is not in the tree.

```
from ..variant_surface import FLEET_REPLAY, surface_class

return surface_class(FLEET_REPLAY)
```

A press on the page names its own action. The panel maps that name to the method the Qt button was wired to, looked up on the panel itself, so the two sides run one piece of code.

`src/gui/react_fleet_replay_panel.py` — `FleetReplayReactPanel.run_action` The Simulator rebuild removed this file; it is not in the tree.

```
step = surface.ACTIONS.get(str(request.get(ACTION_KEY) or ""))
named = self.STEP_RUNNERS.get(step)
if named is not None:
    getattr(self, named)()
```

Everything else on this tab is still drawn by Qt.

Mode is the picker beside it. Three modes, each with its own line saying what it collects.

`src/gui/simulator_tab/simulator_tab.py` — `SimulatorTab.SIM_MODES` The Simulator rebuild removed this file; it is not in the tree.

```

SIM_MODES = (
    (
        "Validation",
        "validation",
        "Stone Tablets paired with YTD data. Verifies trade-gate "
        "parity at documented events.",
    ),
    (
        "Looping Back Test",
        "looping",
        "Loops the tablets with market-restructuring noise at a fixed "
        "rate. Strategy development and calibration.",
    ),
    (
        "Nuclear",
        "nuclear",
        "Load oscillation, swarm injection and high-traffic smart "
        "wire. Measures performance, stability and reliability – not "
        "trade validity.",
    ),
)

```

The table under it is the same class the Trading tab uses, with the same ten columns and a privacy dot on each header.

Column 2 is now Current Position Value, priced from the exchange, and the state colour it used to carry sits on the Bot ID cell. The Trading tab page describes both — see The bot tables.

src/gui/widgets/bot_status_table.py — BotStatusTable.SCRUMMING_COLUMNS

```

SCRUMMING_COLUMNS = ColumnSpec(
    labels=(
        "Bot ID",
        "Symbol",
        "Mode",
        "Trades",
        "Target",
        "Target BTC",
        "Target ETH",
        "Ammo",
        "Fire",
        "",
    ),
)

```

Fleet Replay fills the left column. Load live fleet builds the bots, Fetch YTD pulls the year of live trades the run is compared against, and Reset clears both. Full evaluation widens the run. The line beside the buttons counts the trades and symbols the History tab front-loaded. Loaded fleet carries three columns: Symbol, Target USD and Sim Trades, and the last of those increments during a run. The progress label reads `Replay idle` until Start Replay runs, and Stop drains the current tick and exits.

The panel says so out loud when a run cannot be compared to live, rather than letting a synthetic run look like a parity run.

src/gui/simulator_tab/fleet/fleet_replay_panel.py — _note_parity_state The Simulator rebuild removed this file; it is not in the tree.

```
def _note_parity_state(self) -> None:
    """Say so when the run cannot be compared to live.
```

The right column stacks two panels, each with its own Expand button. The first charts historical price against position VWAP for the bot its picker names, drawn from that bot's Stone Tablet candles. The second is the same Indicator Voting Panel the Trading tab carries: a bot picker, the vote count badge, the TF Lock, the currency rate line, then the first six voters — BB, VTX, MACD, SRsi, Ichi and Vol — with Net, Comp and Conf, and the confidence bars under them. The remaining six, Sling, ADX, STrd, ZSc, KER and RSI, sit below the area the figure shows.

Simulator Log and Gate Status close the tab. Gate Status draws the same nineteen lights the History table draws, and No fleet loaded. stands in its place until a fleet loads.

What is on the tab now

The rebuild's first tab is on the bar. It is a clone of the Trading tab reading Stone Tablets: the bot list with the live table's own ten columns, the Indicator Voting Panel, and a second layer holding the VWAP window over the tablet playback window, with one button between them. The crypto news ticker and the data pool line are not copied, and the rows they held stay empty for Import Live Fleet and Generate From YTD.

src/gui/main_tabs/simulator_tab_surface.py — the candle window

```
#: The candle window live reads. ``ScrummingBot`` asks ``get_ohlcv`` for 100,
#: and the Simulator reads the same count off the tablet.
WINDOW_CANDLES = 100
```

The Simulator receives and asks; it never sends. Its one data path reads tablet files, holds no venue, and refuses by name anything that is not a read.

Validation Mode

Those two rows now carry the buttons that start a run. Validation snaps each YTD trade to the historical candle its timestamp falls in, reruns the gates on that candle, and puts the gate row it latched beside the gate row the log recorded. It passes when the gates latch identically, and it is never judged on profit or on a trade count.

```
Import Live Fleet      38 bots from the bot_state load, matched by bot id
Generate From YTD     39 new bots, one per traded pair, matched by pair
```

A run always reports what it could not verify. The tablets end on 1 August 2026 and the trade export runs to 7 September, so about five weeks of trades have no candle to snap to. The pane names that period and counts the entries on both sides of it.

```
3942 of 4904 YTD entries snapped to a candle; 962 could not be.
uncovered span 2026-08-01T18:52:58Z to 2026-09-07T21:46:48Z
```

Detail: 08-tabs/simulator.md.

Paper Trader Tab (To Be Built)

Real-time, API-fed trades against a fake budget. This is designed as the second tier of strategy validation within the platform.

The tab is now called Paper. It sits second on the bar, on a white ground with black text, and it is the only white tab.

The screen was empty until issue #19 built it. The tab row carried a Paper Trader placeholder, which drew its name, one sentence saying it was not built, and the issue that owned it. The section under this one holds what the tab is now.

`src/gui/main_tabs/paper_trader_tab_surface.py` — the empty state it carried

```
HEADING = "Paper Trader"
ISSUE = 19
BUILT = False
STATE_TEXT = "This tab is not built."
ISSUE_TEXT = f"Issue #{ISSUE} carries the build-out."
```

The Qt tab and the `src/gui/web/paper_trader_tab.js` panel both draw that one view model, so neither can drift from the other.

Live, Paper and the Simulator differ in one thing only, where the data comes from. The trading logic stays one body of pure code all three call, and only the stateful shells fork. Real time is Paper's defining property, and its budget is twice the dollar target.

`src/trading/container/config.py` — `BotConfig`, the field that budget doubles

```
target_balance: float = 200.0 # Balance the bot trades relative to
```

Two live surfaces once offered the step. The Swarm tab's Paper Swarm sub-tab is chrome: its Start button flips a flag and relabels itself, and no bot is constructed. Its caption is now an empty string, so it promises nothing. The wing toggle no longer names a Paper Trader in the Qt window, while the React mirror still does. Issues #422 and #426 folded into issue #19.

The tab as it stands now

The screen is built. It is the Trading tab's panes over the live exchange feed: the Privacy Mode row, the market selector, the two button rows, the bot list, the Indicator Voting Panel, the Live Feed pane, the Fake Balance table and the Paper Run table. The Qt window and the React page draw one view model.

`src/gui/main_tabs/paper_trader_tab_surface.py` — what the tab answers

```
METHOD = "paper_trader_tab.state"

HEADING = "Paper"
ISSUE = 19
BUILT = True
```

The data path is the venue's public market feed, and it goes one way. The reader names the six things it answers, and every other name raises. An order, a cancellation or a venue write cannot be written through it.

src/paper/live_feed_source.py — the read set and the refusal

```
READ_NAMES = (
    "venue",
    "product_id",
    "granularity",
    "candles",
    "ticker",
    "asked_at",
)

class SendRefused(AttributeError):
    """Raised when a name outside ``READ_NAMES`` is asked of ``LiveFeedSource``."""
```

Live candles enter the same gate chain a live bot runs. Nothing on the paper side defines a gate of its own.

src/paper/paper_run.py — the tick

```
context = tape_context(bot, balance, window, reading, summary)
armed = latch(context)
```

The money is fake and lives in memory alone. A run writes no file, and no figure it produces reaches the stored bot record or the header strip.

src/paper/fake_balance.py — the budget

```
BUDGET_MULTIPLE = 2.0
```

The two strips the Trading tab carries are not copied, and neither is their supporting code. Their rows carry Import Live Fleet and the Start or Stop press.

The fake balance is a fleet figure. Five paper bots with a \$200 target open Paper Spendable at \$1000 and Paper Locked at \$1000, each holding the full fleet sum. Adding or removing a bot moves both openings on the next press of Start.

src/paper/fake_balance.py — the fleet total both openings take

```
def fleet_target_usd(bots: Sequence[Any]) -> float:
    """The sum of every bot's ``target_usd``, the figure both openings take."""
```

Two more figures are tracked beside them. Paper Realized Profits moves when a fold buy closes the tranche a scrum sell opened, and Paper Mature Profits is the profit on positions past 200 percent growth, which is the figure the live wing already reads. No paper figure is written into the stored bot record or the header strip.

src/paper/fake_balance.py — the strip's four figures

```

FIGURE_LABELS = (
    ("spendable_usd", SPENDABLE_LABEL),
    ("locked_usd", LOCKED_LABEL),
    ("realized_profit_usd", REALIZED_LABEL),
    ("mature_profit_usd", MATURE_LABEL),
)

```

Every paper action is recorded in one file of its own. Each line carries the gate decision and, when one filled, the pretend trade in the shape a Coinbase year-to-date row takes. The file sits beside the live folders and never inside one.

src/paper/paper_paths.py — where the log lives

```
PAPER_ROOT: Path = Path.home() / ".acervator_paper"
```

Detail: 08-tabs/paper-trader.md.

Proof of Accumulation (Anonymized Trading Tournaments Via Blockchain)

This is currently proposed as a concept but will likely require the building of a supporting blockchain team for proper / full implementation. This system is designed to enable users of Acervator to compete against each anonymously via our own Proof of Accumulation blockchain. The idea is to convert trades executed into videogame metrics such as damage to a coliseum style monster or a fellow trader in a 1v1 face off. This further positions the platform as a surgical tool that can be finely tuned and customized to produce intense competition scenarios between entire groups of traders. This, of course, opens the door for actual tokenized Trading Guilds who may require their members to have a certain number of PoA tokens under their belt to join. There will be much more to follow on this as I do intend to scaffold it out for internal testing.

The tab is now called Accumulation. It sits seventh on the bar, on the gold ground with red text.

The screen is not built. The window builds neither the Competition tab nor the Local Testnet tab. The tab row now carries a Proof of Accumulation skeleton, which draws its name, one sentence saying it is not built, and the issue that owns it. Issue #147 carries the build-out. The engine behind it runs today, and the rest of this section is that engine.

src/gui/main_tabs/proof_of_accumulation_tab_surface.py — the whole empty state

```

HEADING = "Accumulation"
ISSUE = 147
BUILT = False
STATE_TEXT = "This tab is not built."
ISSUE_TEXT = f"Issue #{ISSUE} carries the build-out."

```

src/competition/ is the Proof of Accumulation package. Each bot signs every trade with an Ed25519 key and signs no strategy parameter, so authorship is provable while the method stays private.

src/competition/bot_identity.py — BotIdentity

```
class BotIdentity:
    """
    Manages a bot's Ed25519 keypair. The private key never leaves this object
    unencrypted. The public key is the bot's network-visible identity.
```

The signed trades commit to a Merkle root a third party can verify one trade against without receiving the log. The competition itself runs four phases.

src/competition/competition_engine.py — the module's own summary

1. REGISTRATION — bots register with capital commitment + config hash
2. ACTIVE — bots trade; each trade appended to their Merkle log
3. SUBMISSION — trading closes; bots submit Merkle root + performance claim
4. ADJUDICATION — arbiter verifies submissions, ranks bots, awards tokens

The token ledger is append-only and idempotent, with five rarity tiers by rank. Its hard cap is ten million ACRV, and each season awards less than the one before it.

src/competition/season_schedule.py — the supply constants

```
TOTAL_SUPPLY_CAP = 10_000_000 # Hard cap — immutable
GENESIS_SEASON = 1
INITIAL_REWARD = 500_000 # season 1 pool, in ACRV tokens
DECAY_FACTOR = 0.85
MIN_SEASON_REWARD = 100
```

TournamentEngine in src/trading/poa_tournament.py builds the duel, the melee and the gauntlet. A local testnet module beside it simulates the whole Base environment in memory, with no wallet and no network.

Detail: 08-tabs/proof-of-accumulation.md.

Market Inspector Tab

The concept with the Market Inspector Tab is evaluate markets from a higher point of view and provide strategy proposals in three forms: Oppositional Trading Pairs (Trading Pairs w/ Opposing Trends), Bot Swarm Topologies (Bot Swarm Network Proposals), and Exchange Comparison Arbitrage.

The tab is now called Inspector. It sits fifth on the bar, on the gold ground with red text.

The tab splits in two. The left half holds the HTF Signals table and the Opposing Pairs table, both scored by one analyzer. One method drives the whole pipeline and keeps the results on that analyzer.

src/trading/market_inspector.py — MarketInspector.scan_universe

```
def scan_universe(
    self,
    candles_by_symbol_by_tf: dict,
    active_symbols: set,
    closes_by_symbol: dict,
) -> None:
```

The right half holds proposal cards from four detectors: momentum funnel, mean-reversion pair, sector cluster and distance to band. The engine takes a plain dictionary, unions the detectors, drops overlapping proposals by asset and caps the result.

src/trading/topology_proposals.py — detect_all_topologies

```
def detect_all_topologies(
    context: dict[str, Any],
    cap: int = PROPOSAL_CAP,
) -> list[dict[str, Any]]:
```

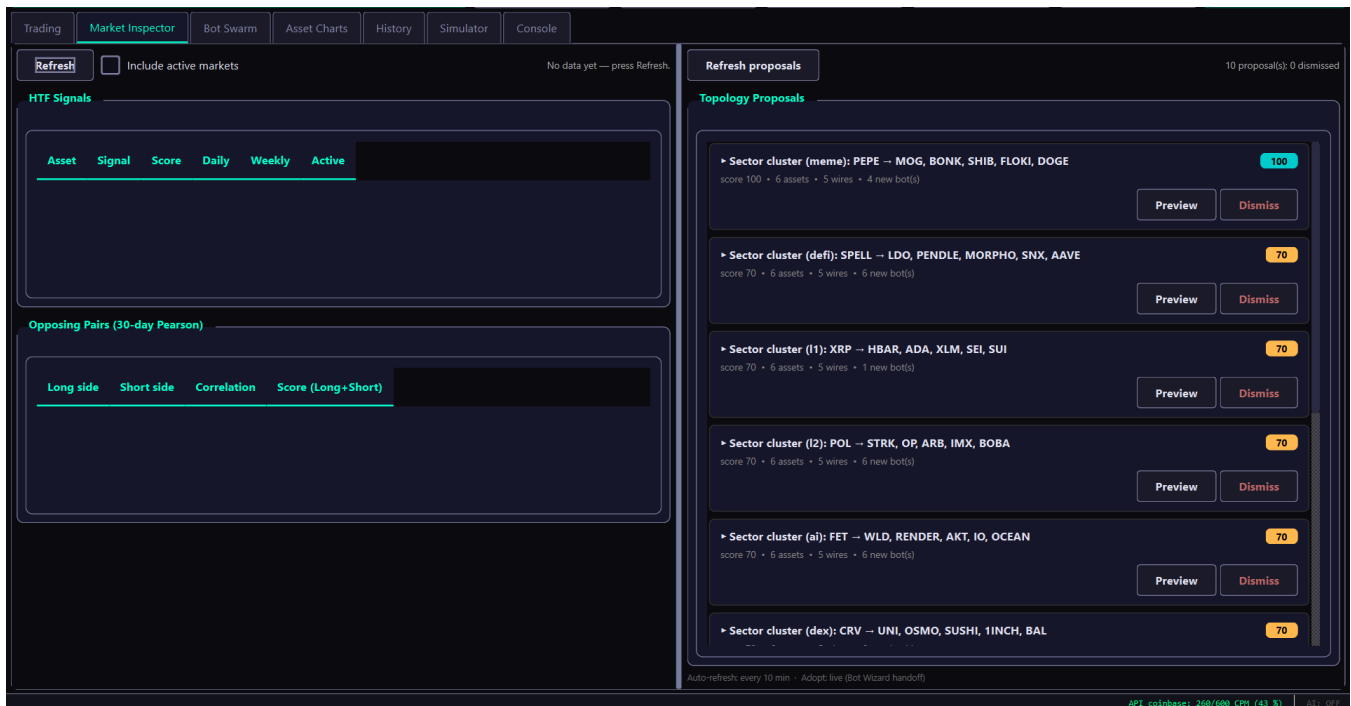
Adopting a card names the new bots, their combined budget and every existing wire the adopt would change before it creates anything.

src/gui/main_window.py — _adopt_topology_proposal

```
new_bots = [
    b for b in proposal.get("bots", []) if not b.get("existing_bot_id")
]
wires = list(proposal.get("wires", []))
new_count = len(new_bots)
```

Two of the three proposal forms reach the screen. Exchange comparison arbitrage has a module, src/trading/arbitrage.py, and no importer under src/; the tab draws no arbitrage panel.

In development.



The Market Inspector tab, before the first scan.

Refresh and the Include active markets checkbox sit above the two tables, and the status line reads No data yet — press Refresh. until a scan lands, which is the state the figure captures. Leaving the checkbox clear

hides the markets a bot already holds.

HTF Signals carries six columns: Asset, Signal, Score, Daily, Weekly and Active. Opposing Pairs carries four: Long side, Short side, Correlation and Score. The pairing enumerates every long against every short and keeps the ones whose thirty-day return correlation sits in the configured negative window.

src/trading/market_inspector.py — MarketInspector._find_opposing_pairs

```
def _find_opposing_pairs(self, signals: list, closes_by_symbol: dict) -> list:
    """Enumerate long × short candidates; keep pairs whose 30-day
    return correlation sits in the configured negative window."""
    longs = [s for s in signals if s.direction == "long" and s.score >= 0.3]
    shorts = [s for s in signals if s.direction == "short" and s.score >= 0.3]
```

Refresh proposals runs the detectors, and the line beside it counts the proposals held and the cards dismissed. Each card names its archetype and the assets in it, then a line reading the score, the asset count, the wire count and the number of bots an adopt would create. The badge takes its colour from the score: teal at the high threshold and above, amber at the middle one, grey below. Every card in the figure comes from the sector-cluster detector. Preview lists the wires as Source, Target, Pct and Rationale.

Dismiss hides a card for a day, and the dismissal does not survive a restart. The write-through never raises, so the pane logs the failure and carries on, which leaves the dismissed count at zero on every launch.

src/gui/main_tabs/market_inspector_topologies_surface.py — persist_dismissed

```
def persist_dismissed(self) -> None:
    """Best-effort write-through. Never raises."""
    if self.dismiss_store is None:
        return
    try:
        self.dismiss_store.set(DISMISS_SETTINGS_KEY, dict(self.dismissed))
```

The key it writes is not one the settings schema declares, so the write fails every time. Issue #424 carries it.

The footer names the auto-refresh period and the adopt route. The status bar under it carries the API load pill, drawn green below half load, amber above it, and red past the monitor's safety percentage, then the AI : state label.

React draws this screen. Both halves are one web page inside a single view, and the build decides which class the tab builder makes. src/gui/main_tabs/market_inspector_tab.py — _build_market_inspector_tab

```
from ..variant_surface import MARKET_INSPECTOR, surface_class

self._market_inspector = surface_class(MARKET_INSPECTOR)()
```

MarketInspectorReactTab subclasses the Qt tab, so the scan cycle, the filtering and the analyzer writes are the same code on both sides. Five accessors are what the two sides answer differently: _set_status, _set_refresh_enabled, _shown_signals, _fill_signal_rows and _fill_pair_rows. The Qt tab writes them into its widgets and the React tab writes them into the page.

src/gui/react_market_inspector_tab.py — MarketInspectorReactTab._build_ui

```
self._web = QWebEngineView(self)
self._web_page = MarketInspectorPage(self)
self._web.setPage(self._web_page)
self._web.loadFinished.connect(self._on_load_finished)
self._web.setHtml(tab_html())
```

Refresh and the Include active markets checkbox report their press back to Python, and the right pane reports Refresh proposals, Preview, Dismiss, Cancel and Adopt the same way. Setting ACERVATOR_VARIANT to qt builds the Qt widgets instead, unchanged.

Detail: 08-tabs/market-inspector.md.

Bot Swarm Tab

The Bot Swarm is a proprietary capital reinforcement network system that allows profit to be dynamically and strategically routed between positions with the primary intention of this being to accelerate accumulation curves. Smart Wires are dragged between active bots or the quick connection matrix can be used to route multiple streams to different destinations. Each wire can carry a different percentage amount of profit. Profit sent over Smart Wires are registered at the destination as Wire Credits and these are then distributed into standing Fold Tranches which allow them to have a Surplus that will be spent to increase the Target Balance up to the Maximum Growth Per Cycle. Yes, that is probably a mouth full but hopefully the settings and names are, for the most part, self-explanatory.

The tab is now called Swarm. It sits sixth on the bar, on the gold ground with red text.

Each bot draws as one locust card, and the tab switches between the list and the grid. Wires drag between nodes on the grid.

src/gui/visualizer/bot_node.py — BotNodeWidget

```
class BotNodeWidget(QWidget):
    """Paint one bot as a locust, fed by set_bot_data and animate.

    paintEvent draws the wings, abdomen, thorax and head, then the
    symbol, the P/L and the bot_id as text.
    """
```

SmartWireManager in src/trading/smart_wire.py owns the topology and one ledger per bot. One function bounds what a bot may export, reading the target balance, the band edges and the ammunition the next fold needs, which stops a bot sending away the capital it holds for its own dip.

src/trading/smart_wire.py — compute_safe_outflow_pct

```
def compute_safe_outflow_pct(
    scrum_profit_usd: float,
    target_balance_usd: float,
    current_price: float,
    band_lower: float,
    band_upper: float,
    next_fold_amm0_usd: float,
    current_cash_usd: float,
    compound_growth_pct: float,
    retained_this_cycle_usd: float,
) -> float:
```

At the far end, arriving USD spreads across the open fold tranches, or parks as a pending wire credit while no tranche stands.

src/trading/scrumming/wire_routing.py — apply_wire_income

```
def apply_wire_income(self, usd: float, source: str, ref: str = "") -> dict:
    """Apply incoming Smart Wire USD to this bot's fold queue.
```

Three functions collapse or remove a tranche and no others: merge, despawn and clear. Despawn removes an aged record outright rather than holding it in a delisted state, and the count lands in the discarded ledger rather than the closed one.

src/trading/scrumming/fold_tranches.py — _despawn_aged_tranches

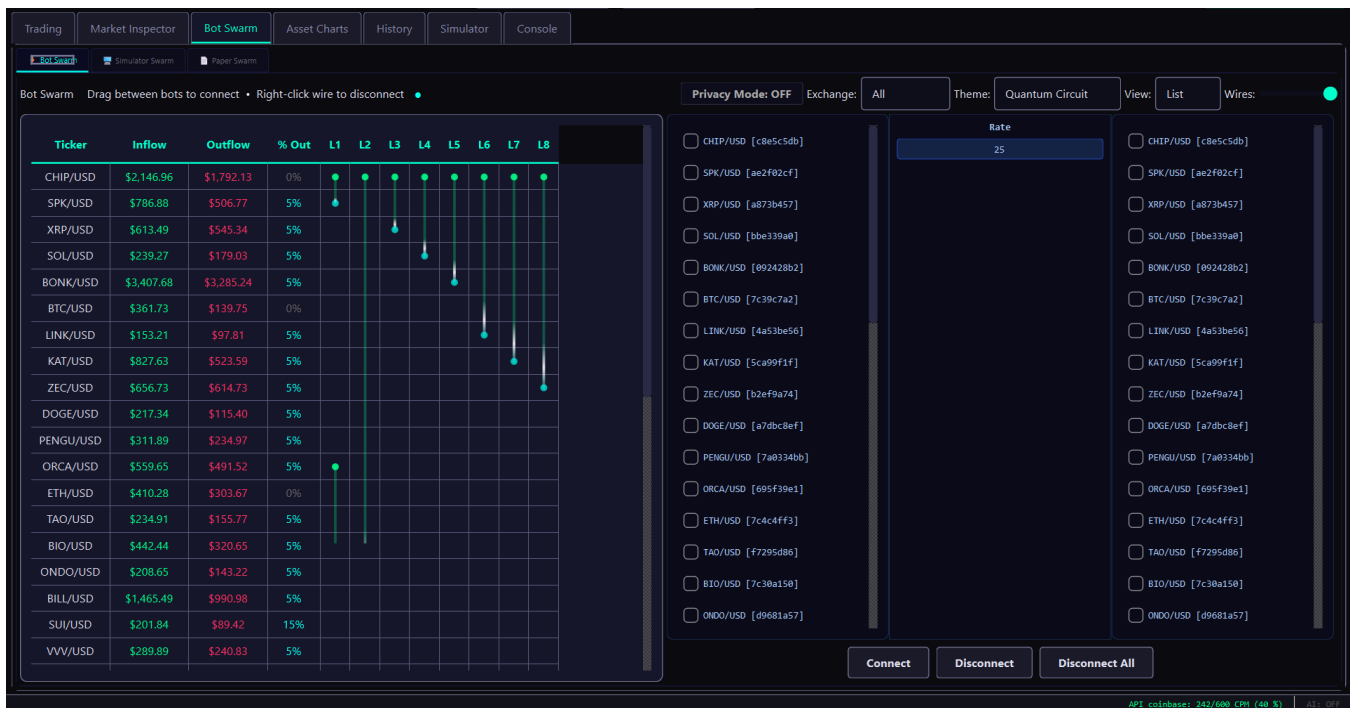
```
_days = self._despawn_threshold_days()
report = {
    "threshold_days": _days,
    "fold_delisted": 0,
    "stack_delisted": 0,
    "stack_kept_live_order": 0,
    "ageless_kept": 0,
    "usd_delisted": 0.0,
}
```

The fleet, and its wires, ledgers and tranches with it, comes from one state file alone. A restored bot connects to nothing and places no order until the operator starts it.

src/trading/container/restore.py — StateRestoreMixin.restore_bots_from_state

```
def restore_bots_from_state(self, state: dict) -> list[str]:
    """Recreate every persisted bot in IDLE state; return the ids restored.
```

(List View)



Three sub-tabs open the screen: Bot Swarm, Simulator Swarm and Paper Swarm. The header row holds the drag hint, an identifier privacy dot that masks the bot hashes and the symbol labels together, the Privacy Mode button, and four controls. Exchange filters both the swarm and the quick routing scope. Theme lists the four palettes. View chooses List or Grid. The Wires slider sets wire opacity from 0 to 100 %.

The table carries four named columns and eight wire lanes.

src/gui/bot_swarm_list.py — COLUMN_HEADERS

```
COLUMN_HEADERS = ["Ticker", "Inflow", "Outflow", "% Out"] + [
    f"L{i + 1}" for i in range(LANE_COUNT)
]
```

Inflow draws green and Outflow red. % Out sums a bot's outbound wire percentages, and a bot with no outbound wire reads 0 %. Two wires share a lane only when their row spans do not overlap, and the dot at each end of a lane marks the source row and the target row.

The quick routing matrix fills the right half in three zones: the source list, the Rate field between them, and the destination list. Every checked source wires to every checked destination at that rate, and four conditions are refused out loud before anything is created.

src/gui/visualizer/quick_routing.py — QuickRoutingMatrix._on_connect_clicked

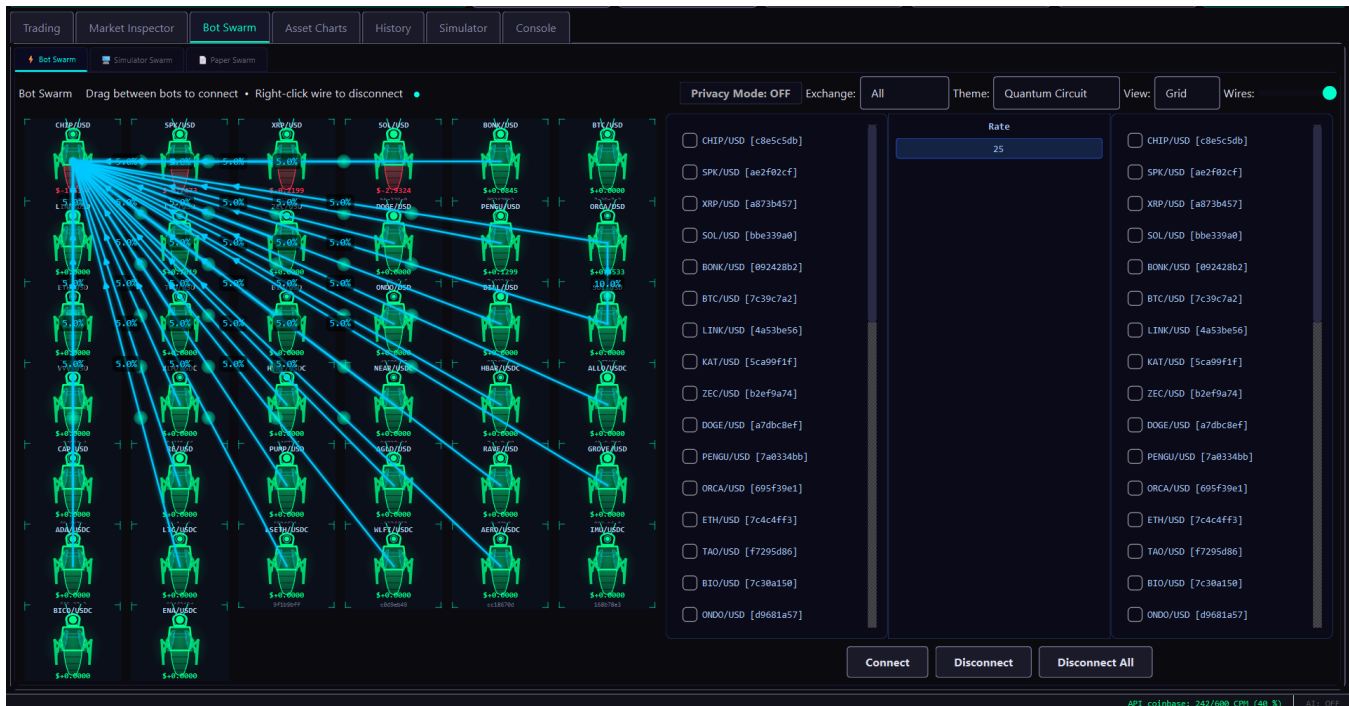
```

if not (0.0 <= pct <= 100.0):
    self._reject(f"Rate {pct} is outside 0-100%.")
    return
if not sources:
    self._reject("No SOURCE bots are checked.")
    return
if not dests:
    self._reject("No DESTINATION bots are checked.")
    return
if pct <= 0:
    self._reject(
        "Rate is 0% – that would create wires that route " "nothing."
    )
    return

```

A confirmation puts the count in front of the operator before Connect, Disconnect or Disconnect All runs.

(Grid View)



The view switch swaps the table for the node canvas. Each card carries the symbol above, the realised profit below it, and the first eight characters of the bot id at the foot. The profit colour is the theme's success colour at zero and above and the error colour below, and the abdomen gradient takes that same colour with its alpha scaled by the size of the figure.

Privacy Mode is off in the figure, so the symbol and the id both draw plain. One field id covers all three readings on a card, the symbol, the short bot id and the tooltip, so a card is never half masked.

src/gui/visualizer/bot_node.py — the symbol, drawn through the one mask

```

p.drawText(
    QRectF(0, 2, w, 11),
    Qt.AlignCenter,
    _mask_or(symbol, "bot_swarm.identifiers"),
)

```

Theme names the palette the canvas paints in. The module declares four, and the figure sits on Quantum Circuit. The palette decides the locust body, the success and error colours the profit line takes, and the four corner brackets around each card, which draw in the second accent at low alpha.

src/gui/visualizer/themes.py — the palette the figure is on

```

"quantum": {
    "name": "Quantum Circuit",
    "bg": QColor(10, 15, 25),
    "accent": QColor(0, 200, 255),
    "accent2": QColor(0, 255, 200),
    "success": QColor(0, 255, 136),
    "warning": QColor(255, 180, 0),
    "error": QColor(255, 50, 80),
}

```

The wires cover the grid alone, because the grid is the only view whose coordinates a wire can be drawn against. Each wire carries its own percentage, drawn beside it. Drag between two nodes to create a wire, and right-click a wire to remove it. The quick routing matrix on the right is the same widget the List view shows.

A wire's percentage sits in a rounded dark badge at the midpoint of the curve, so a wire crossing a card still reads. The Wires slider at the top right sets the opacity of the whole overlay rather than of any one wire.

src/gui/visualizer/wire_canvas.py — the badge under the label

```

label = f"{pct}%"
font = QFont("Consolas", 8, QFont.Bold)
p.setFont(font)
fm = p.fontMetrics()
tw = fm.horizontalAdvance(label) + 8
badge = QRectF(label_pt.x() - tw / 2, label_pt.y() - 9, tw, 18)

```

React draws this tab. The choice is made in one place, under the screen name BOT_SWARM. A build stamped for Qt gets the old widget. Every other build gets the web one, which holds the tab's page in a single view and answers the calls the main window already made.

src/gui/main_tabs/bot_swarm_tab.py — BotSwarmTabMixin._build_bot_swarm_tab

```

def _build_bot_swarm_tab(self) -> None:
    """Build the Bot Swarm tab and add it to the main tab widget."""
    from ..variant_surface import BOT_SWARM, surface_class

    try:
        built = surface_class(BOT_SWARM)()
    except:
        pass

```

Every header control reports back to Python. The privacy dot, the Privacy Mode button, the Exchange, Theme and View pickers and the Wires slider each send one action. The screen then redraws from the answer.

src/gui/react_bot_swarm_tab.py — BotSwarmReactTab.take

```
def take(self, params: dict) -> dict:
    """Apply one action to the model, push the payload and return it."""
    answer = surface.apply_action(self._state, params)
    self._last_model = answer
    if self._page_ready:
        self._run(push_script(answer))
    return answer
```

A drag between two bots opens the rate box. A drag between two bots already wired opens the disconnect confirmation. A drag released on empty space offers the source bot's outgoing wires. One method decides which of the three it is, so both screens ask the same question in the same words.

src/gui/main_tabs/bot_visualizer_surface.py — BotVisualizerModel.finish_drag

```
def finish_drag(self, source_id: str, target_id: str) -> Optional[dict]:
    """Set ``panel`` to what a wire drag from ``source_id`` asks for next.

    ``WireBoard.finish_drag`` decides the outcome; this turns it into
    the rate box, the disconnect confirmation or the wire picker the
    screen shows.
    """
```

The List view draws the same twelve-column table in the React tab. The tab surface builds the list payload once, from the rows the last fleet load sent and the wires the board holds, and hands it to the list module inside the page.

src/gui/main_tabs/bot_visualizer_surface.py — swarm_list

```
def swarm_list(model: BotVisualizerModel) -> dict:
    """The dense list payload, from the rows ``update_bots`` last sent.

    ``bot_swarm_list_surface`` builds it, so the list draws the table and
    the lane wires the Qt ``BotListView`` and ``LaneWireCanvas`` paint.
    """
```

The list module draws the table and the see-through lane sheet over it in one box, so a wire runs down a lane column between the two rows it joins.

src/gui/web/bot_swarm_list.js — SwarmList

```
return element(
  DIV_TAG,
  swarmProps,
  element(ListTable, { key: LIST_PART, model: model }),
  element(LaneSheet, { key: SHEET_PART, model: model })
);
```

Detail: 08-tabs/bot-swarm.md.

The window a bot row opens

Clicking a bot opens its own settings window. Which window opens depends on the build. The React build draws the header, the tab strip and every tab inside one embedded browser; the Qt build draws the widget tree those tabs describe. The Simulator opens the same window from its own bot rows.

src/gui/main_window.py — `_on_bot_clicked`

```
from .variant_surface import BOT_LIVE_SETTINGS, surface_class

_cls = surface_class(BOT_LIVE_SETTINGS)
dlg = _cls(current_bot, self._bot_manager, self)
```

The React side is a subclass, not a rewrite. It replaces only the method that builds the widgets, so an edit still travels the same route into the bot. `_mark_changed` records the field against what the bot holds now, and `_apply_changes` writes it. The window carries no second copy of that logic.

src/gui/react_bot_live_settings.py — `BotLiveSettingsReactDialog.apply_edit`

```
field = field_for(str(asked.get("name") or ""))
if not field:
    return
self._mark_changed(field, asked.get("value"))
```

Each tab is drawn by its own renderer module into a named empty space the window leaves for it. `TAB_PLAN` pairs the tab with that space, its module and the surface that builds its payload from the bot. A tab whose surface refuses the bot draws the reason in that space rather than nothing.

src/gui/react_bot_live_settings.py — `TAB_PLAN`, the first two rows

```
("Status", "status-page", "acervatorLiveStatus", "live_status_tab_surface"),
(
    "Settings",
    "settings-page",
    "acervatorLiveSettingsTab",
    "live_settings_tab_surface",
),
```

Asset Charts

Under the Asset Charts Tab, you will find our active bot (position) chart display. This will be upgraded to display only one chart at a time and will be able to display all indicators found in the Indicator Voting Panel.

The tab is now called Charts. It sits fourth on the bar, straight after Live, on the gold ground with red text.

The tab scrolls one panel per active bot, each painted with `QPainter` and no browser, carrying trade markers, position markers, grid lines, the standing tranche floors and the target balance line.

src/gui/main_tabs/charts_tab.py — ChartsTabMixin._build_charts_tab

```
def _build_charts_tab(self) -> None:
    """Build the Asset Charts tab and add it to the main tab widget."""
    # --- Tab 2: Charts ---
    self._charts_tab = TradeChartsTab()
    self._main_tabs.addTab(self._charts_tab, "Asset Charts")
```

The chart reads its overlays from the engine rather than computing any of them.

src/gui/native_chart.py — CandlestickChart._compute_indicators

```
candles = self._candles
self._bb_data = BollingerBands(20, 2.0).bands(candles)
vi_plus, vi_minus = VortexIndicator(14).lines(candles)
```

Candles arrive through a fetcher with three sources in order: exchange OHLCV first, a public API second, the last good fetch third.

src/exchange/chart_data.py — ChartDataFetcher.fetch

```
# --- Source 1: Exchange OHLCV via CCXT ---
if exchange is not None:
    try:
        candles, source = await self._fetch_exchange(
            exchange, symbol, timeframe, limit
        )
        if candles:
            self._set_cache(symbol, timeframe, candles)
            return candles, source
```



The Asset Charts tab, one panel per running bot.

The panel set rebuilds from the current bot roster, and the tab scrolls them.

The header line of a panel names the symbol, the last price, the bot state, the timeframe, the feed and the candle count. The line under it carries the open, high, low and close of the current candle, the change in price and in percent, and the volume. Rising candles draw teal and falling candles red.

The dashed lines across the plot are the standing fold-tranche floors, each labelled with its price. TB-Ceiling is the dollar-target line. The price axis runs down the right edge with the last price boxed on it, and the volume bars run along the foot.

The toolbar under each panel holds the TF picker and eight indicator toggles.

src/gui/main_tabs/native_chart_surface.py — INDICATOR_TOGGLES

```
INDICATOR_TOGGLES = (  
    ("bb", "BB", "#50a0f0"),  
    ("vortex", "Vortex", "#00ff88"),  
    ("macd", "MACD", "#ffcc00"),  
    ("stochrsi", "SRsi", "#ff9060"),  
    ("ichimoku", "Ichi", "#c080ff"),  
    ("volume", "Vol", "#80ffcc"),  
    ("slingshot", "Sling", "#ff4488"),  
    ("bbullseye", "BBull", "#ff00aa"),  
)
```

Volume starts on and the other seven start off, which is the state the figure shows. Sling and BBull paint a placeholder shape rather than the indicator, and each says as much in its own tooltip. Issue #430 carries a tooltip on this toolbar that names a colour the chart does not paint.

The legend at the right names the two position markers, Invisible and On Book, and the feed label closes the row.

Detail: 08-tabs/asset-charts.md.

History Tab

The History Tab is able to pull trade history from all active exchanges via their respective APIs. It also pairs each imported trade with its in-platform trading logic and applies our trade grading system.

The tab keeps the name History. It sits eighth on the bar, on the gold ground with red text.

The tab fetches from every active venue and draws the rows with React. Every cell value comes from one read contract; the table renders those fields and derives none, which stops a second History growing behind the render.

src/exchange/history_read_contract.py — build_page

```
def build_page(
    filtered: list[dict],
    page: int = 0,
    bot_manager: Any = None,
    gate_index: Optional[dict] = None,
    voting_index: Optional[dict] = None,
) -> HistoryPage:
```

The grade comes from the trade grader and the gate lights come from the same vocabulary the Simulator's gate cell reads. A refresh also hands the Simulator its year of live trades, so the tab's signal front-loads the parity run.

Trading

Market Inspector

Bot Swarm

Asset Charts

History

Simulator

Console

Filters

From: 2026-04-01 00:00

To: 2026-08-25 09:57

Exchange: (all)

Symbol: (all)

Side: (all)

Apply

Reset

Refresh

5026 of 5026 trades shown · BUYS: 2842 (\$18,367.92) · SELLS: 2184 (\$14,507.19) · fetched 0s ago

	Timestamp (UTC)	Exchange	Symbol	Bot	Side	Amount	Price	Cost USD	Fee	Trade ID	Grade	Gates
1	2026-08-25 16:25:18	coinbase	ZEC/USD	ZEC/9a74	BUY	0.00007474	\$798.03310917	\$0.0596	0.000716 USD	1aed91c2-cbd9-44...	A+	So de as s Fw A conf below floor (c=EARL) conf=0.04 (0.2)
2	2026-08-25 16:25:18	coinbase	ZEC/USD	ZEC/9a74	BUY	0.01390169	\$798.74560865	\$11.1039	0.133247 USD	2d5c4011-4d0b-47...	D	So de as s Fw A conf below floor (c=EARL) conf=0.04 (0.2)
3	2026-08-25 14:57:03	coinbase	VVV/USD	VVV/5fed	SELL	0.27300000	\$18.23710000	\$4.9787	0.059745 USD	d5cb6f31-8d6b-45...	D	So de as s Fw A conf below floor (c=EARL) conf=0.04 (0.2)
4	2026-08-25 14:06:48	coinbase	IMU/USDC	IMU/78e3	BUY	2.698.00000000	\$0.00263593	\$7.1118	0.085341 USDC	41489e17-eeca-4d...	A+	So de as s Fw A conf below floor (c=EARL) conf=0.04 (0.2)
5	2026-08-25 13:51:14	coinbase	DOGE/USD	DOGE/c8ef	BUY	82.20000000	\$0.08814000	\$7.2451	0.086941 USD	ae82e6c4-33df-49...	D	So de as s Fw A conf below floor (c=EARL) conf=0.04 (0.2)
6	2026-08-25 13:32:36	coinbase	GROVE/USD	GROVE/b6a8	BUY	1.085.00000000	\$0.01050000	\$11.3925	0.136710 USD	82feb290-6c8e-48...	A+	So de as s Fw A conf below floor (c=EARL) conf=0.04 (0.2)
7	2026-08-25 13:24:36	coinbase	IMU/USDC	IMU/78e3	SELL	2.233.00000000	\$0.00294107	\$6.5674	0.078809 USDC	1225822d-35af-42...	A+	So de as s Fw A conf below floor (c=EARL) conf=0.04 (0.2)
8	2026-08-25 13:18:51	coinbase	IMU/USDC	IMU/78e3	SELL	2.417.00000000	\$0.00266100	\$6.4316	0.077180 USDC	6ba7f991-edf2-43...	F	So de as s Fw A conf below floor (c=EARL) conf=0.04 (0.2)
9	2026-08-25 13:12:43	coinbase	CHIP/USD	CHIP/c5db	BUY	596.00000000	\$0.03191000	\$19.0184	0.228220 USD	6e085ade-9961-47...	A+	So de as s Fw A conf below floor (c=EARL) conf=0.04 (0.2)
10	2026-08-25 12:37:20	coinbase	GROVE/USD	GROVE/b6a8	SELL	976.00000000	\$0.01128028	\$11.0096	0.132115 USD	0ff592ad-d1bf-40...	A+	So de as s Fw A conf below floor (c=EARL) conf=0.04 (0.2)
11	2026-08-25 10:48:01	coinbase	GROVE/USD	GROVE/b6a8	BUY	1.136.00000000	\$0.01051000	\$11.9394	0.143272 USD	5fea6017-79ec-4b...	D	So de as s Fw A conf below floor (c=EARL) conf=0.04 (0.2)
12	2026-08-25 10:35:18	coinbase	GROVE/USD	GROVE/b6a8	SELL	1.111.00000000	\$0.01158029	\$12.8657	0.154388 USD	b61f117a-9a64-44...	A+	So de as s Fw A conf below floor (c=EARL) conf=0.04 (0.2)
13	2026-08-25 09:52:37	coinbase	GROVE/USD	GROVE/b6a8	SELL	1.124.00000000	\$0.00936000	\$10.5206	0.126248 USD	d8c2521f-d125-49...	F	So de as s Fw A conf below floor (c=EARL) conf=0.04 (0.2)
14	2026-08-25 09:52:30	coinbase	GROVE/USD	GROVE/b6a8	SELL	1.135.00000000	\$0.00936000	\$10.6236	0.127483 USD	410604ab-1f97-44...	F	So de as s Fw A conf below floor (c=EARL) conf=0.04 (0.2)
15	2026-08-25 09:11:02	coinbase	ALLO/USDC	ALLO/e720	SELL	0.51000000	\$0.26080000	\$0.1330	0.001596 USDC	a64df963-1988-4b...	A	So de as s Fw A conf below floor (c=EARL) conf=0.04 (0.2)
16	2026-08-25 08:51:43	coinbase	ALLO/USDC	ALLO/e720	BUY	4.75000000	\$0.25950000	\$1.2326	0.014792 USDC	b6382954-a90b-47...	A+	So de as s Fw A conf below floor (c=EARL) conf=0.04 (0.2)

◀ Prev

Page 1 / 51 (5026 trades)

Next ▶

Export CSV...

API columns: 235/500 (CW: 37.3%)

The History tab: filters, graded rows and the gate lights.

The Filters group carries From and To date pickers, then Exchange, Symbol and Side, each filled from the rows in hand. Apply narrows the set, Reset restores it, and Refresh goes back to the venues. The line under the group counts the retained rows against the loaded rows, both sides with their dollar totals, and the age of the last fetch.

Thirteen columns carry a row, newest first, and nothing re-sorts. BUY draws green and SELL red.

src/exchange/history_read_contract.py — COLUMNS, the first ten


```
COLUMNS: tuple[HistoryColumn, ...] = (
    HistoryColumn(0, "timestamp", "Timestamp (UTC)"),
    HistoryColumn(1, "exchange", "Exchange"),
    HistoryColumn(2, "symbol", "Symbol"),
    HistoryColumn(3, "bot", "Bot"),
    HistoryColumn(4, "side", "Side"),
    HistoryColumn(5, "amount", "Amount"),
    HistoryColumn(6, "price", "Price"),
    HistoryColumn(7, "cost", "Cost USD"),
    HistoryColumn(8, "fee", "Fee"),
    HistoryColumn(9, "trade_id", "Trade ID"),
```

Grade, Gates and Voting close the set.

The Gates cell opens with the blocker text, then the lights. Nineteen of them: the ten scrum gates TGT, INT, BB, FIRE, TA, LS, TRND, HTF, CB and OTD, then the nine fold gates BB, MID, TA, LS, TRNQ, CEIL, HTF, CB and OTD. Each light draws under its own label, and each bank ends with an S or an F marker. Five states, five colours.

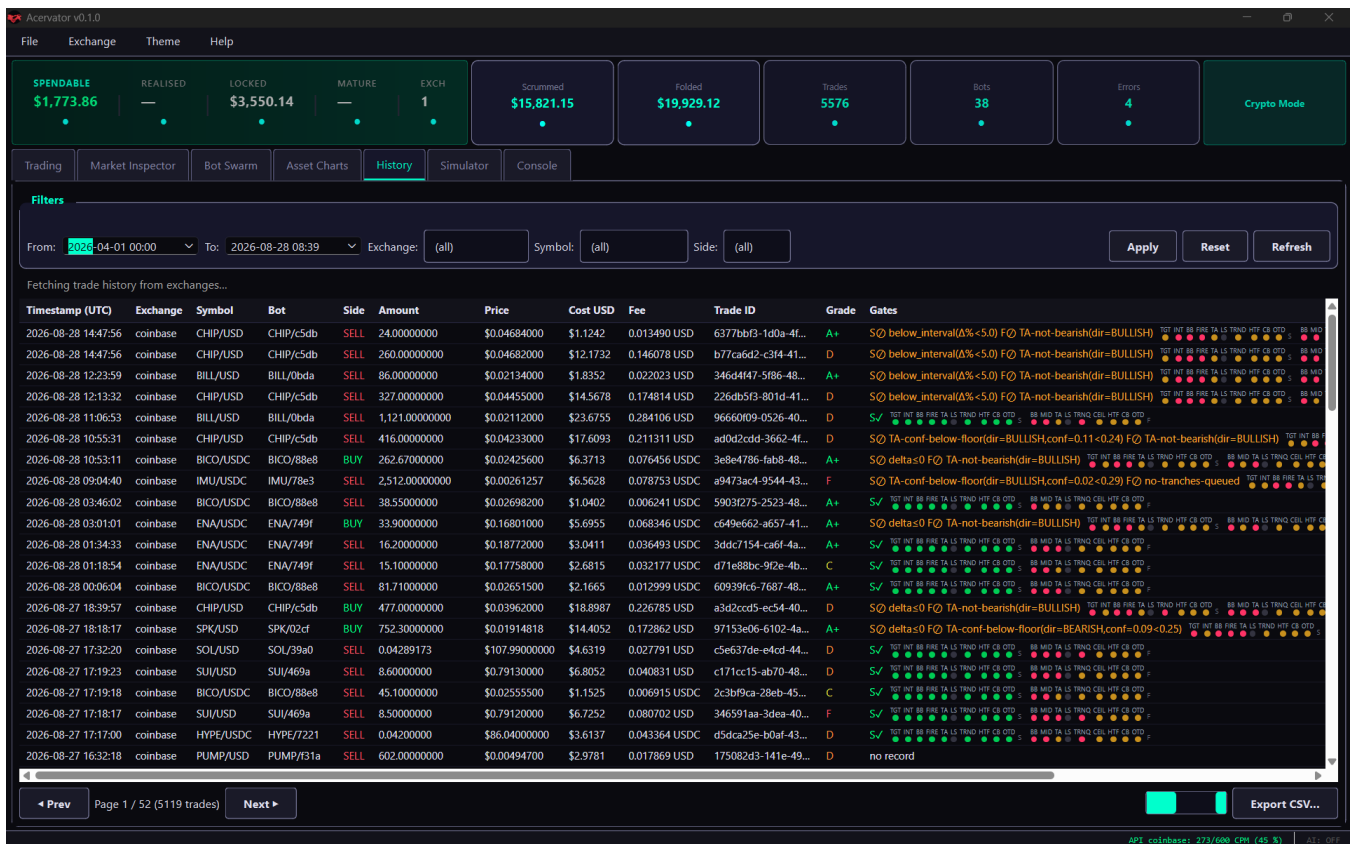
src/trading/gate_vocabulary.py — LIGHT_COLORS

```
LIGHT_COLORS: dict[str, str] = {
    "override": "#22d3ee",
    "not_evaluated": "#333340",
    "blocked": "#ff3366",
    "passed": "#00cc55",
    "not_the_blocker": "#c8901e",
}
```

Green passed, red the gate that blocked, amber not armed for another reason, grey not evaluated at this candle, and cyan a landing-strip override. The blocker text and the light banks overlap in the figure; both occupy the one cell.

Prev, the page counter and Next page the result at the foot, and Export CSV writes the current selection out. The tab builds the summary line and the page counter itself, and the read contract already declares both, so two implementations of the same two strings stand in the tree. Issue #425 carries it.

History Tab (React):



The same tab, captured while the asynchronous fetch runs. Five fixed status strings cover the states before any row exists, and the one on screen is the fetching entry.

src/exchange/history_read_contract.py — STATUS_TEXT

```
STATUS_TEXT = {
    "idle": "No history loaded yet – click Refresh.",
    "no_bot_manager": "Bot manager unavailable – cannot fetch history.",
    "no_async_loop": "Async loop not ready – try again after platform starts.",
    "fetching": "Fetching trade history from exchanges...",
    "timeout": "Fetch timeout (60s). Exchange may be rate-limited; try again.",
}
```

Two constants bound the fetch, in the same module: the poll runs on a 400 ms timer, so an observed latency is the true latency plus up to one interval, and the fetch is abandoned after sixty seconds.

src/exchange/history_read_contract.py — the fetch bounds

```
FETCH_POLL_INTERVAL_S = 0.4
```

The header strip is visible here, which marks the screen as a live-trading tab. REALISED and MATURE draw the exchange figures the Live tab draws, which issue #418 fed. A row whose gate log holds no entry inside the join window reads no record.

Both columns draw an exchange figure here too, because one strip serves every tab that shows it. Where the venue has answered for no bot they stay empty, and Privacy Mode leaves an empty column empty.

Detail: 08-tabs/history.md.

Trade grading

The grader scores a trade that has already happened. It never feeds a trading decision, it never tunes a parameter, and no gate consults it. The same inputs always give the same grade.

Four axes are scored. An axis with no input is skipped: it neither credits nor penalises, and the overall number is the unweighted mean of the rest.

src/trading/trade_grader.py — grade_trade

```
exec_score, exec_bps = _score_execution(record, ctx)
timing_score, mfe, mae = _score_timing(record, ctx)
strategic_score, sb_delta = _score_strategic(record, ctx)
outcome_score, realized = _score_outcome(record, ctx)
```

Axis	What it reads	What a 1.0 means
Execution	fill price against the reference price at decision time	no slippage
Timing	the favourable excursion against the adverse one over the following candles	the trade went far more right than wrong before it closed
Strategic	the asset's discipline ratio before the trade against after it	the trade moved the ratio the operator's way
Outcome	realised profit per unit	a clear gain

The regime tag reaches the grade and the rationale string and gets no score of its own. Grading the same record with a regime tag and without one returns the same number, which is the control that proves it.

The scale is 0.0 to 1.0, not 0 to 100, and the mean becomes a letter at fixed boundaries.

src/trading/trade_grader.py — _letter_from_numeric

```
if num >= 0.93:
    return "A+"
elif num >= 0.85:
    return "A"
elif num >= 0.70:
    return "B"
elif num >= 0.55:
    return "C"
elif num >= 0.40:
    return "D"
else:
    return "F"
```

Driving the mapper across each boundary and just under it flips the letter exactly where the code says, in both directions.

One trap worth knowing. A trade with no gradeable input at all scores 0.5 and therefore reads as a **D**, not as ungraded. A fully specified trade scores 1.0 and reads A+. A column of Ds may mean the trades were poor, or it may mean the surrounding price context never arrived.

Console

This tab is focused on displaying Python activity and errors. The lower half, which is displaying the Emitter Network activity, will be migrated to the System Status Tab (under the Watchdog) which is to be built in the near future.

The tab keeps the name Console. It sits last on the bar, on the gold ground with red text.

The tab builds both panes into a vertical splitter. The upper one is a raw log tail: a handler formats each record and a relay paints it on the GUI thread, which lets a log call from any thread reach the pane safely.

src/gui/qt_console_tab.py — ConsoleQtTab.__init__

```
self.log_pane = QPlainTextEdit()
self.log_pane.setReadOnly(True)
self.log_pane.setFont(
    QFont(surface.PANE_FONT_FAMILY, surface.PANE_FONT_POINT_SIZE)
)
```

The tab sets its own handler to debug and changes no logger's level; an earlier build raised the root logger and never restored it.

src/gui/qt_console_tab.py — ConsoleQtTab.__init__

```
# The tab sets its own handler's level and leaves every logger level alone.
self.log_handler.setLevel(surface.HANDLER_LEVEL)
```

Pause holds lines in a bounded buffer and announces anything it had to drop.

src/gui/main_tabs/console_log_handler.py — QtLogHandler.set_paused

```
self._paused = bool(paused)
if not self._paused and self._buffer:
    for msg, r, g, b in self._buffer:
        self._append_signal.emit(msg, r, g, b)
    self._buffer.clear()
```

The lower pane is the Emitter Network. It reads the sink twice a second and renders the newest 200 records with their expected and actual values, keeping seven counters, which lets a quiet sink and a stopped timer read differently.

src/gui/main_window.py — _drain_signals

```
sink = get_sink()
if sink is None:
    return
new = sink.since(getattr(self, "_signal_seq", 0))
```


A line reaches the page as markup the surface writes, with the message escaped, so a log message holding a tag is shown rather than read as one.

src/gui/main_tabs/console_tab_surface.py — line_html

```
def line_html(text: Any, red: int, green: int, blue: int) -> str:
    """One console line as the markup a pane paints it with.

    The text is escaped, so a message holding a tag is shown rather than
    read as markup.
    """
```

Logger	Writes
acervator.scrumming	The bot warnings
acervator.api	The venue calls
acervator.gui	The panel feed

Five levels each carry a colour, and the figure shows three of them: warning, info and debug, with debug the most muted of the three. A sixth colour follows the text of the message rather than its level, and it marks the indicator panel feed.

src/gui/main_tabs/console_log_handler.py — the level colours

```
COLORS = {
    "DEBUG": QColor(ds.TEXT_MUTED),
    "INFO": QColor(ds.TEXT_INACTIVE),
    "WARNING": QColor(ds.WARNING),
    "ERROR": QColor(ds.ERROR),
    "CRITICAL": QColor(ds.MAIN_LOG_CRITICAL),
}
```

Two of those lines are worth reading against the code. The repeated warning naming a capital-reservation over-commit comes from src/trading/capital_reservation.py, where a claim whose total would pass the holdings is refused and the existing claims, the request and the holdings are named. The FETCH_OHLCV line names a seven-indicator engine and lists seven indicators; the voting engine builds twelve and the Indicator Voting Panel shows twelve, so the count in the log line and the count in the engine disagree.

The lower pane opens with the header SIGNALS — name · expected · actual. Each record draws a pass marker, a fail marker or a neutral one, then the signal name, the site that emitted it, the actual value and the expected one. A satisfied expectation is recorded the same as a violated one, which keeps a call site that never ran distinct from one that always passed.

src/gui/main_window.py — _drain_signals, the three markers

```

if r.ok is True:
    mark, colour = "OK ", ds.SUCCESS
elif r.ok is False:
    mark, colour = "FAIL", ds.ERROR
else:
    mark, colour = "-- ", ds.STATUS_NEUTRAL

```

The site is the file and the line of the frame that made the call, read off the stack at the moment of emission. It moves whenever the code moves. The numbers in the figure are the numbers those call sites held on the day of the capture, not the numbers they hold now.

src/core/signal_contract.py — `_caller_site`

```

def _caller_site(depth: int = 2) -> str:
    """Return `file:line` of the calling frame at the given stack `depth`."""
    try:
        f = sys._getframe(depth)
        return f"{Path(f.f_code.co_filename).name}:{f.f_lineno}"

```

Detail: 08-tabs/console.md.

The two snapshot emitters

Two symmetric records go to the bot log, and the Console tab is where an operator reads them. A blocked trade and a fired trade write the same shape, so the log holds both halves of the decision and not only the half that acted.

src/trading/scrumping/snapshots.py — `SnapshotEmitterMixin._emit_risk_gate_snapshot`

```

f"RISK GATE SNAPSHOT [{side.upper()}] "
f"risk_blockers={risk_blockers_sorted} "
f"ticker_last={ticker_last:.6g} "
f"panel={snapshot}"

```

The fired-side companion carries the same prefix shape.

src/trading/scrumping/snapshots.py —
`SnapshotEmitterMixin._emit_trade_fire_snapshot`

```

f"TRADE FIRED SNAPSHOT [{side.upper()}] "
f"ticker_last={ticker_last:.6g} "
f"panel={snapshot}{extra_str}"

```

Method	Line prefix	Written when
<code>_emit_risk_gate_snapshot</code>	RISK GATE SNAPSHOT [SIDE]	a risk gate blocks a trade
<code>_emit_trade_fire_snapshot</code>	TRADE FIRED SNAPSHOT [SIDE]	a scrum or a fold actually fires

Both lines carry the last ticker price and a panel dictionary holding every voter's direction, confidence, weight and detail at that moment. The fire record adds the list of overrides that engaged, so a reader can tell an override-driven fire from a consensus-driven one without opening anything else.

The two code blocks above and the prefixes in the table are the shape these lines had before the tick-message rewrite. The current prefixes are RISK GATE [SIDE] and TRADE FIRED [SIDE], and both write the same panel through one renderer.

Method	Line prefix	Written when
<code>_emit_risk_gate_snapshot</code>	RISK GATE [SIDE]	a risk gate blocks a trade
<code>_emit_trade_fire_snapshot</code>	TRADE FIRED [SIDE]	a scrum or a fold actually fires

The panel prints as a count and three direction groups rather than as a dictionary. A voter's weight is a configuration value that never moves between ticks, and a neutral voter's confidence is fixed at zero in the indicator code, so neither reaches the line. Direction, confidence and an indicator's own raw detail all remain.

`src/trading/scrumping/snapshots.py` — the grouped panel

```
counts = ", ".join(f"{len(grouped[name])} {name.lower()}" for name in _PANEL_GROUPS)
rows = [f"Panel {counts}."]
for direction, voters in grouped.items():
    voters.sort(key=lambda one: (-one[0], one[1]))
```

The measurement behind the change is in `tests/debug_reports/2026-09-07_activity_log_format.md`: the dictionary form measured 1,005 to 1,017 characters across 1,143 recorded snapshots, and the grouped form of the same panel measures 322.

Three further emitters sit in the same mixin: a trade notification carrying its own text prefix, a voting-panel snapshot at fire time, and a gate decision at fire time. The last two emit events rather than text lines, and the Console tab's signal pane reads them.

System Status Tab (To Be Built)

This tab consists of two distinct but closely related parts. The Emitter Network is an embedded system of data activity detectors intended to allow for detailed subsystem performance monitoring. The Watchdog is the raw signal capture for the Emitter Network's output.

The tab is now called Status. It sits ninth on the bar, on the gold ground with red text.

The screen is not built. The tab row now carries a System Status skeleton, which draws its name, one sentence saying it is not built, and the issue that owns it. Issue #34 carries the build-out. Both halves run today, and the Console tab shows the first of them.

`src/gui/main_tabs/system_status_tab_surface.py` — the whole empty state

```
HEADING = "Status"
ISSUE = 34
BUILT = False
STATE_TEXT = "This tab is not built."
ISSUE_TEXT = f"Issue #{ISSUE} carries the build-out."
```

The Emitter Network is one plain function and one sink. A call site says what it expected and what it actually saw, and a satisfied expectation is recorded the same as a violated one, which keeps a call site that never ran distinct from one that always passed.

src/core/signal_contract.py — emit

```
def emit(
    name: str,
    actual: Any,
    expected: Any = None,
    ok: Optional[bool] = None,
    context: Optional[dict] = None,
    every: float = 0.0,
    instance: Optional[str] = None,
    module: Optional[str] = None,
    duration: Optional[float] = None,
) -> Optional[Signal]:
```

A second module declares every emitter. A contract names a topic's required fields and its vocabulary, and the observer reports a topic that never fired, a missing field or a value outside that vocabulary.

src/core/emit_contracts.py — the gate-decision contract

```
EmitContract(
    topic="bot.gate_decision",
    required=("symbol",),
    optional=(
        "scrum_armed",
        "fold_armed",
        "scrum_blockers",
        "fold_blockers",
        "scrum_fixture",
        "fold_fixture",
    ),
    description="Gate evaluation snapshot at fire time.",
),
```

The Watchdog is `acervator_watchdog.py` at the repository root. It launches the application as a child process, tees both output streams to its own log, polls the heartbeat file, and writes a post-mortem after the child dies.

`acervator_watchdog.py` — the stall threshold and the poll

```
DEFAULT_STALL_SECONDS = 60 # must match or exceed Acervator's longest sync call (CCXT: ~15s typical, 30s timeout)
HEARTBEAT_POLL_INTERVAL = 2.0
```

Detail: [08-tabs/system-status.md](#).

Post-mortem bundle rotation

The Watchdog writes a post-mortem bundle every time the application dies. Each bundle copies the runner's console log, the crash log, the fault-handler log and a thread dump, which runs to hundreds of megabytes. Per-run log rotation capped the size of one run's logs and capped nothing about the number of bundles, so weeks of restarts grew without limit until the log directory took the host down.

`acervator_watchdog.py` — `prune_postmortem_bundles`

```
def prune_postmortem_bundles(
    keep_latest: int = POSTMORTEM_KEEP_LATEST,
    max_age_days: int = POSTMORTEM_MAX_AGE_DAYS,
    log_dir: Path | None = None,
) -> tuple[int, int]:
```

It keeps the most recent bundles, drops anything past a maximum age whatever its position, and runs at two sites: once at Watchdog startup, which clears what earlier runs left behind, and once after each post-mortem is written, which holds the ceiling during a long session.

acervator_watchdog.py — the three constants

```
POSTMORTEM_KEEP_LATEST: int = 20 # most-recent N bundles preserved
POSTMORTEM_MAX_AGE_DAYS: int = 30 # anything older is dropped
POSTMORTEM_SIZE_WARN_BYTES: int = 5 * 1024 * 1024 * 1024 # 5 GB warning threshold
```

The third is a warning threshold read by `report_log_dir_footprint` and it deletes nothing, so two constants govern the rotation and one reports on it. The only other inputs are the log directory itself, derived from the home directory, and the literal bundle-name prefix inside the function. Neither is a rotation constant.

No test in this repository references any of the three names. The cap runs and nothing holds it in place.

Settings

The File menu opens this dialog, and which one it opens depends on the build. The React build draws every page below inside one embedded browser. The Qt build draws the widget tree the pages describe. One line in the running window picks the side.

src/gui/main_window.py — `_open_settings`

```
from .variant_surface import SETTINGS_DIALOG, surface_class

_cls = surface_class(SETTINGS_DIALOG)
dlg = _cls(self._settings, self._status_log, self, wing=_wing)
```

The React side is a subclass, not a rewrite. It replaces only the method that builds the widgets, so the loading and the saving below are the same code on both sides. Each named control gets a small holder that answers the same calls the widget answered, and the page reads and writes those holders.

src/gui/react_settings_dialog.py — `SettingsDialogReact._setup_ui`

```
self._model = surface.SettingsDialogModel(wing=self._wing)
self._model.build()
self._build_holders()
self._web = QWebEngineView(self)
self._web.setPage(SettingsDialogPage(self))
self._web.setHtml(dialog_html(self._theme_name()))
```

An edit on the page arrives as one line the host reads, and the holder takes it the way the control takes a stored value. A radio inside a group box clears the others, which is what `QGroupBox` does for the widget side.

src/gui/react_settings_dialog.py — SettingsDialogReact.apply_edit

```
self._holders[name].admit(asked.get("value"))
```

Settings > User Tab

This needs to be built out to accept and contain individual end user credentials. This will also be where a license is entered or a keyfile is imported depending on how we design the authentication piece.

The dialog holds the eleven pages below, added in one order.

src/gui/settings_dialog.py — SettingsDialog._setup_ui

```
tabs.addTab(self._create_user_tab(), "User")
tabs.addTab(self._create_exchange_tab(), "Exchanges")
tabs.addTab(self._create_trading_tab(), "Trading")
tabs.addTab(self._create_folding_tab(), "Profit Folding")
tabs.addTab(self._create_ta_tab(), "TA Indicators")
tabs.addTab(self._create_phantom_tab(), "Phantom Bots")
tabs.addTab(self._create_theme_tab(), "Theme")
tabs.addTab(self._create_logging_tab(), "Logging")
tabs.addTab(self._create_sound_tab(), "Sound")
tabs.addTab(self._create_sms_tab(), "SMS")
tabs.addTab(self._create_ai_monitor_tab(), "AI Monitor")
```

_save and _load_current decide which of them persists, and each page says which under its own heading. Issue #423 carries the four pages whose controls Save never reads.

The User page itself is one form row, and the settings manager persists it. No credential field, no licence field and no key import.

src/gui/settings_dialog.py — _create_user_tab

```
def _create_user_tab(self) -> QWidget:
    w = QWidget()
    form = QFormLayout(w)
    self._username = QLineEdit()
    form.addRow("Username:", self._username)
    return w
```

Both halves a credential store would rest on already exist, and the page reads neither.

src/core/encryption.py encrypts a secret with AES-256-GCM under a key derived from a passphrase and a random salt.

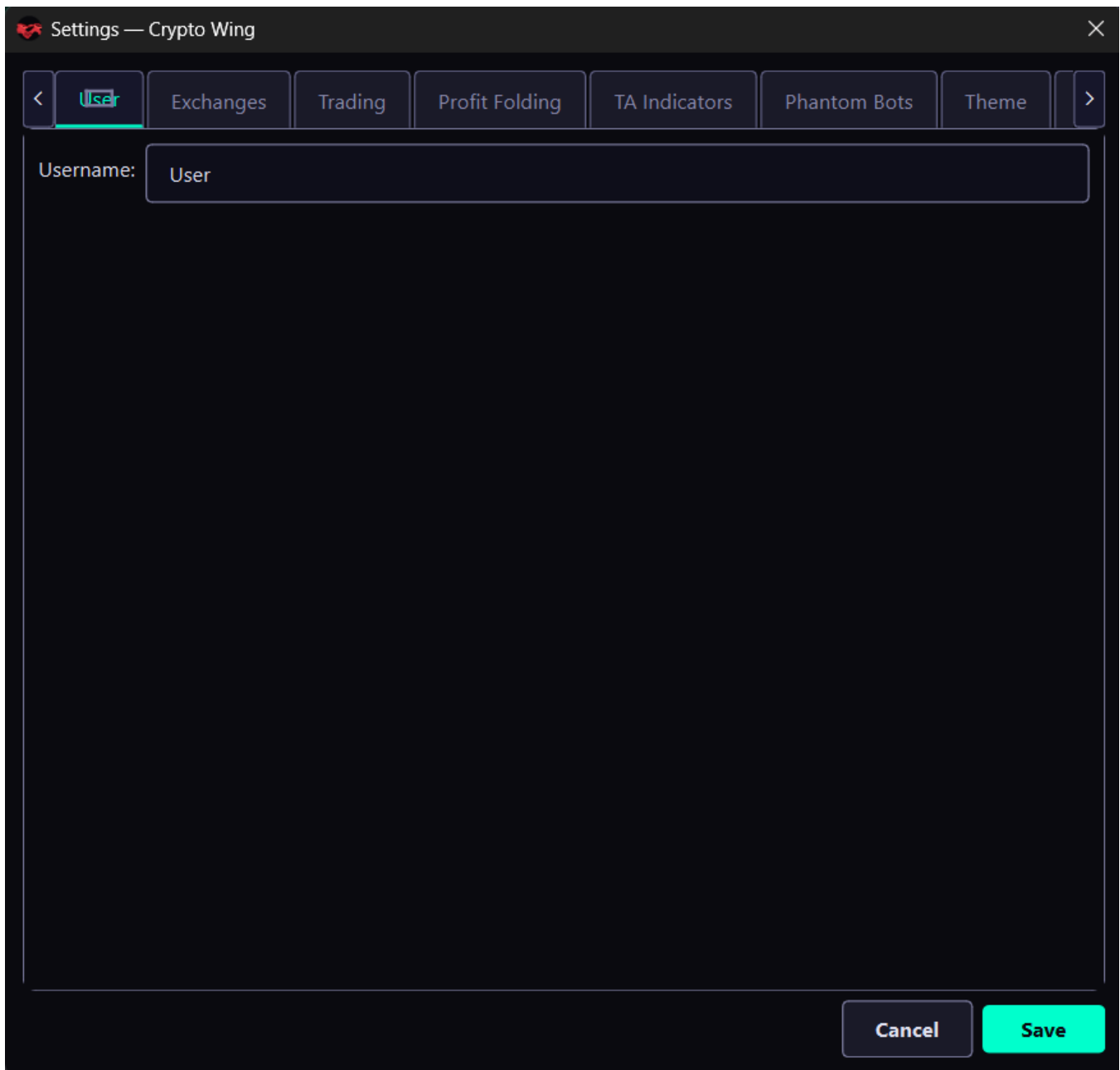
src/core/encryption.py — derive_key

```
def derive_key(passphrase: str, salt: bytes) -> bytes:
    """Derive a 256-bit AES key from *passphrase* + *salt* via PBKDF2."""
```

src/core/usb_auth.py writes an encrypted credential file keyed to a USB volume's serial, which decrypts on that stick and nowhere else.

src/core/usb_auth.py — write_auth_file

```
def write_auth_file(
    usb_path: Path,
    volume_serial: str,
    credentials: list[dict],
) -> Path:
```



Settings, the User page.

The title bar names the wing the dialog was opened for. The arrows at each end of the page row scroll it, because the eleven do not fit the dialog's minimum width. Cancel and Save close the dialog, and Save always closes it, reporting any group that failed to persist. The page itself carries one row, Username, and nothing else.

That one row is not cosmetic. The stored username keys the credential vault. Adding a venue on the next page builds a master phrase from that name, then encrypts the API key and the API secret with the phrase.

src/gui/settings_dialog.py — `_add_exchange`, the phrase and the encryption

```
master = f"qat_{self._sm.get('username', 'user')}_vault"
config.api_key_enc = encrypt(key, master)
config.api_secret_enc = encrypt(secret, master)
```

The figure shows a stored value, User. The declared default is an empty string, and the settings object always declares the field, so the fallback written into the phrase never fires. An install where the row was never filled encrypts under a phrase with an empty middle. Change the name on a machine that already holds credentials and the stored secrets stop opening, because the connect path rebuilds the same phrase from the new name. Issue #436 carries it.

src/gui/main_window.py — the decrypt side, rebuilding the same phrase

```
master = f"qat_{self._settings.get('username', 'user')}_vault"
api_key = decrypt(exch_config["api_key_enc"], master)
api_secret = decrypt(exch_config["api_secret_enc"], master)
```

Three sites write that phrase out as a literal, one to encrypt and two to decrypt, and a fourth declares the format none of them reads. The same issue carries that half.

src/gui/main_tabs/settings_dialog_surface.py — the declaration

```
MASTER_FORMAT = "qat_{username}_vault"
```

Detail: 08-tabs/settings.md.

Settings > Exchanges

The page lists the configured exchanges and adds one: the exchange picker, an API key, an API secret, and a passphrase field that appears only when the venue needs it. The picker fills from the connector layer, so it lists what that layer can reach.

src/exchange/ccxt_connector.py — `SUPPORTED_EXCHANGES`, the first six

```
SUPPORTED_EXCHANGES: dict[str, str] = {
    "binance": "binance",
    "coinbase": "coinbase",
    "kraken": "kraken",
    "kucoin": "kucoin",
    "bybit": "bybit",
    "okx": "okx",
```

Three venues need the passphrase field, and it appears for those alone.

src/exchange/ccxt_connector.py — `PASSPHRASE_EXCHANGES`

```
PASSPHRASE_EXCHANGES: set[str] = {
    "kucoin",
    "okx",
    "bitget",
}
```

The key field accepts a Coinbase CDP key string and the secret field a PEM elliptic-curve private key. The connection test runs before anything is stored, and adding one routes the new tab to the crypto or the stock layer.

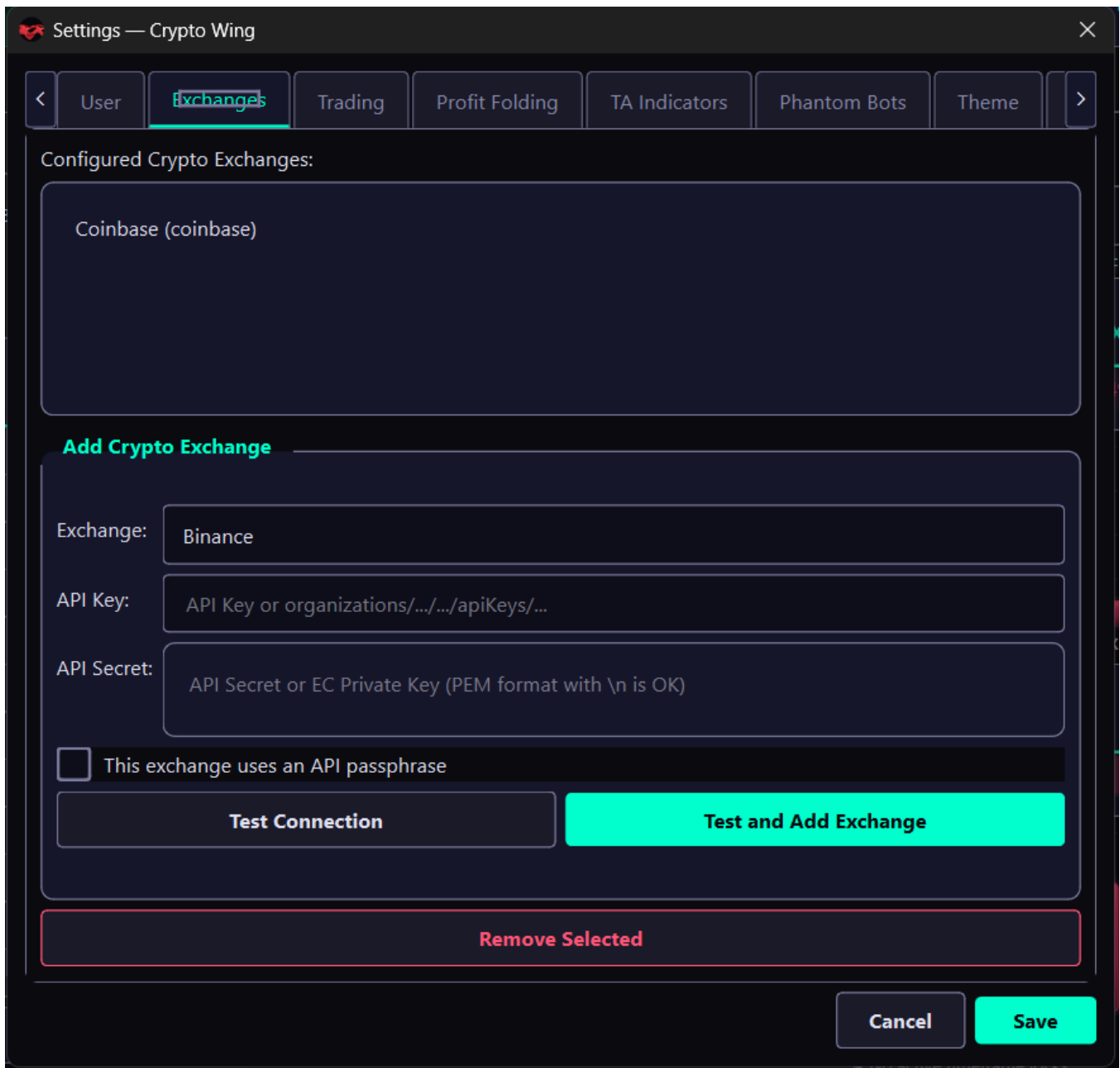
src/gui/main_window.py — add_exchange_tab

```
def add_exchange_tab(self, exchange_id: str, display_name: str) -> None:
    """Add exchange tab to the correct layer (crypto or stock)."""
    is_equity = self._is_equity_exchange(exchange_id)
```

The same page in the stock wing shows a banner and disables Add and Test, because no broker connector ships yet.

src/gui/settings_dialog.py — _create_exchange_tab

```
_banner = QLabel(
    "<b>Stock Wing:</b> equity-broker integration is "
    "queued – no live brokers are wired up yet. The "
    "list below shows the planned brokers; Add / Test "
    "are disabled until the broker connectors ship. "
    "Use the Crypto Wing for active trading today."
)
```



Settings, the Exchanges page.

Configured Crypto Exchanges lists what the manager returns for this wing, each entry naming its display name and its exchange id. The stock wing lists the equity ids instead and disables both buttons.

The Add Crypto Exchange group holds four rows. Exchange fills from the supported list. API Key takes a plain key or a Coinbase CDP key string, and its placeholder shows the CDP shape. API Secret takes a plain secret or a PEM elliptic-curve private key, and escaped newlines inside a pasted PEM convert on the way in. The passphrase checkbox appears for the three venues above.

The picker in the figure reads Binance, the first supported id in alphabetical order, and the passphrase box under it stays clear. Changing the picker sets that box for the three venues that need one and clears it for the rest. It also wipes the feedback line, so an answer about the last venue cannot stand as an answer about this one.

src/gui/settings_dialog.py — _on_exchange_changed

```
def _on_exchange_changed(self) -> None:
    eid = self._new_exchange.currentData()
    needs_pp = eid in self._passphrase_exchanges
    self._pp_check.setChecked(needs_pp)
    self._api_feedback.setText("")
```

The page hides the passphrase field until you tick the box, which is why the figure shows the box and no field under it. The box carries the state; the field only collects the value.

src/gui/settings_dialog.py — the field the box reveals

```
self._new_passphrase = QLineEdit()
self._new_passphrase.setEchoMode(QLineEdit.Password)
self._new_passphrase.setPlaceholderText(
    "Passphrase set when creating API key"
)
self._new_passphrase.setVisible(False)
```

Test Connection runs the check and reports the answer. Test and Add Exchange runs the same check first and stores nothing when it fails. Remove Selected drops the highlighted entry.

Both buttons go dead for the length of a call, so a second click cannot start a second test. Test and Add returns on a failed check, which is what keeps a venue that did not answer out of the store.

src/gui/settings_dialog.py — _add_exchange, the guard before the store

```
if key and secret:
    result = self._test_api_connection()
    if result is None or not result.success:
        return
```

Detail: 08-tabs/settings.md.

Settings > Trading

Settings — Crypto Wing

< User Exchanges **Trading** Profit Folding TA Indicators Phantom Bots Theme >

Position Distance: 2.0%

Increment Style: linear

Default Positions: 10

Default Target Balance: \$200.00

Bot Visibility: orderbook

☐ Enable aggressive trading mode

Cancel Save

Settings, the Trading page.

Six rows carry the defaults a new bot starts from, not a running bot's settings.

Position Distance - Sets the spacing a new bot puts between its positions. Key `position_distance_pct`, 1.0 % to 50.0 %, at 2.0 %.

The dialog stores the figure and reads it back the next time the page opens. No bot ever sees it. The bot factory strips the name before it builds a config, so the value cannot reach a bot by any route.

`src/gui/settings_dialog.py` — the Position Distance row

```

self._pos_distance = QDoubleSpinBox()
self._pos_distance.setRange(1.0, 50.0)
self._pos_distance.setSuffix("%")
self._pos_distance.setDecimals(1)
form.addRow("Position Distance:", self._pos_distance)

```

The strip list holds eleven names. This page contributes one of them and the Profit Folding page below contributes four more.

src/trading/container/config.py — `_DEPRECATED_KWARGS`

```

_DEPRECATED_KWARGS: frozenset = frozenset(
    {
        "bulk_trading", # renamed to stack_mode
        "bulk_partial_on_return",
        "market_check_interval",
        "position_count",
        "position_distance_pct",
        "fold_mode",
        "fold_target",
        "fold_target_count",
        "profit_fold_pct",
        "distribute_target",
        "distribute_target_count",
    }
)

```

Increment Style - Chooses whether that spacing stays even or widens on a curve. Key `increment_style`, linear or logarithmic, at linear.

The dialog stores the choice and restores it on the next open. A bot takes its own increment style from the wizard rather than from this page.

src/gui/settings_dialog.py — the Increment Style row

```

self._increment_style = QComboBox()
self._increment_style.addItem("linear")
self._increment_style.addItem("logarithmic")
form.addRow("Increment Style:", self._increment_style)

```

Default Positions - Sets how many positions a new bot opens with. Key `default_position_count`, 1 to 100, at 10.

The dialog stores the count and restores it. Nothing outside the dialog reads the name, so no bot opens with it.

src/gui/settings_dialog.py — the Default Positions row

```

self._default_positions = QSpinBox()
self._default_positions.setRange(1, 100)
form.addRow("Default Positions:", self._default_positions)

```

Default Target Balance - Sets the dollar figure the wizard's own Target Balance field opens at. Key `default_target_balance`, \$1.00 to \$1,000,000.00, at \$200.00.

This is the one row on the page that reaches a new bot. The wizard's parameter page opens its Target Balance field at the stored figure.

`src/gui/bot_wizard.py` — the wizard field that reads the stored default

```
self._target_balance = QDoubleSpinBox()
self._target_balance.setRange(1.0, 1000000.0)
self._target_balance.setDecimals(2)
self._target_balance.setPrefix("$ ")
self._target_balance.setValue(defaults.get("default_target_balance", 200.0))
```

Bot Visibility - Chooses whether a new bot rests its orders on the book or tracks them inside the platform. Key `bot_visibility`, `orderbook` or `internal`, at `orderbook`.

Save writes the name and the load path never reads it. The box opens on `orderbook` whatever was stored, and a Save with no edit writes that first entry back over the stored value. A bot takes its visibility from the wizard, under a different name.

`src/gui/settings_dialog.py` — the Bot Visibility row

```
self._visibility = QComboBox()
self._visibility.addItem("orderbook")
self._visibility.addItem("internal")
form.addRow("Bot Visibility:", self._visibility)
```

Enable aggressive trading mode - Starts a new bot pricing for an immediate fill. Key `aggressive_trading`, a checkbox, off.

The same gap as the row above. Save writes it, the load path skips it, and the box opens clear. The wizard carries its own aggressive trading checkbox, and that is the one a new bot reads.

`src/gui/settings_dialog.py` — the aggressive trading row

```
self._aggressive = QCheckBox("Enable aggressive trading mode")
form.addRow(self._aggressive)
```

Save writes all six keys. The load path restores the first four, which is why the last two rows on the page open at their built-in defaults.

`src/gui/settings_dialog.py` — `_load_current`, the four it restores

```

self._pos_distance.setValue(self._sm.get("position_distance_pct", 2.0))
self._default_positions.setValue(self._sm.get("default_position_count", 10))
self._default_balance.setValue(
    self._sm.get("default_target_balance", 200.0)
)
style = self._sm.get("increment_style", "linear")
idx = self._increment_style.findText(style)
if idx >= 0:
    self._increment_style.setCurrentIndex(idx)

```

Settings > Profit Folding

Settings — Crypto Wing

< User Exchanges Trading **Profit Folding** TA Indicators Phantom Bots Theme >

☒ Profit Folding / Upward Distribution Active

Distribution Mode

☒ Equal distribution

☐ Logarithmic distribution

Profit Folding Target

☒ Fold to ALL buy positions

☐ Fold to X# of buy positions: 5

☐ Fold to most recent buy positions

Upward Distribution Target

☒ Distribute to ALL sell positions

☐ Distribute to X# of sell positions: 5

☐ Distribute to most recent sell positions

Cancel Save

Settings, the Profit Folding page.

One master checkbox and three groups of radio buttons, eleven controls in all. Every key below sits inside one stored `profit_folding` group.

Profit Folding / Upward Distribution Active - Turns the whole page on. Key `active`, a checkbox, clear at build and set to on when the stored group loads.

This is the one control on the page the load path restores, so it is the only one of the six that reopens on what was stored.

`src/gui/settings_dialog.py` — the master switch

```
self._folding_active = QCheckBox(  
    "Profit Folding / Upward Distribution Active"  
)  
layout.addWidget(self._folding_active)
```

Distribution Mode - Chooses whether a fold spreads evenly across the chosen positions or on a curve. Key `mode`, equal or logarithmic, at equal.

Two radio buttons in one group. Equal is checked at build, and Save reads the logarithmic button to decide which of the two words it stores.

`src/gui/settings_dialog.py` — the Distribution Mode group

```
mode_group = QGroupBox("Distribution Mode")  
mode_layout = QVBoxLayout(mode_group)  
self._fold_equal = QRadioButton("Equal distribution")  
self._fold_log = QRadioButton("Logarithmic distribution")  
self._fold_equal.setChecked(True)
```

Profit Folding Target - Chooses which buy positions a fold reaches: all of them, a counted number of them, or the most recent. Key `fold_target`, at all buy positions.

Three radio buttons and the count box the middle one gates. The all-buy button is checked at build.

`src/gui/settings_dialog.py` — the Profit Folding Target group

```
self._fold_all = QRadioButton("Fold to ALL buy positions")  
self._fold_x = QRadioButton("Fold to X# of buy positions:")  
self._fold_recent = QRadioButton("Fold to most recent buy positions")  
self._fold_x_count = QSpinBox()  
self._fold_x_count.setRange(1, 100)  
self._fold_x_count.setValue(5)  
self._fold_all.setChecked(True)
```

Fold to X# of buy positions - Sets that counted number. Key `fold_target_count`, 1 to 100, at 5.

Upward Distribution Target - The same three choices on the sell side. Key `distribute_target`, at all sell positions.

The sell-side group is built the same way as the buy-side group above, with its own count box at 5 and its own all-sell button checked at build.

src/gui/settings_dialog.py — the Upward Distribution Target group

```
self._dist_all = QRadioButton("Distribute to ALL sell positions")
self._dist_x = QRadioButton("Distribute to X# of sell positions:")
self._dist_recent = QRadioButton("Distribute to most recent sell positions")
self._dist_x_count = QSpinBox()
self._dist_x_count.setRange(1, 100)
self._dist_x_count.setValue(5)
self._dist_all.setChecked(True)
```

Distribute to X# of sell positions - Sets the sell-side count. Key `distribute_target_count`, 1 to 100, at 5.

Save collapses the two target groups into one dictionary. Each group reduces to a single word, taken from whichever of its three buttons is checked.

src/gui/settings_dialog.py — `_save`

```
fold_target = "all_buy"
if self._fold_x.isChecked():
    fold_target = "x_buy"
elif self._fold_recent.isChecked():
    fold_target = "most_recent_buy"
dist_target = "all_sell"
if self._dist_x.isChecked():
    dist_target = "x_sell"
elif self._dist_recent.isChecked():
    dist_target = "most_recent_sell"
```

The load path restores the master switch alone, so the mode and both targets open at the checked defaults the figure shows.

src/gui/settings_dialog.py — `_load_current`

```
pf = self._sm.get("profit_folding", {})
self._folding_active.setChecked(pf.get("active", True))
```

Issue #423 carries that half of it. Four of the six keys then fail a second time further down: the two target names and their two counts are four of the eleven the bot factory strips, listed under Settings > Trading above. A group that did round-trip still could not carry those four to a bot.

In development. Which of the six the load path should restore has not been settled against the rest of the dialog, so nothing is proposed here.

Settings > TA Indicators

Indicator	Weight
Bollinger Bands:	1.00
Vortex:	0.90
Macd:	1.20
Stochastic Rsi:	1.00
Ichimoku:	1.10
Volume:	0.80
Slingshot:	1.00
Adx:	1.00
Kaufman Er:	1.00
Supertrend:	1.00
Zscore:	0.90
Rsi:	0.80

Settings, the TA Indicators page.

One slider per weight, twelve in all, each labelled from its key and each running 0.00 to 2.00. The value beside a slider follows it as it moves, and the starting values are the weights themselves.

Indicator weight - Sets how far one indicator's vote counts against the other eleven when the engine totals a direction. The key behind each slider, and the value it starts at, are the twelve below.

src/trading/ta_engine.py — DEFAULT_WEIGHTS

```

DEFAULT_WEIGHTS = {
    "bollinger_bands": 1.0,
    "vortex": 0.9,
    "macd": 1.2,
    "stochastic_rsi": 1.0,
    "ichimoku": 1.1,
    "volume": 0.8,
    "slingshot": 1.0,
    "adx": 1.0,
    "kaufman_er": 1.0, # Perry Kaufman Efficiency Ratio (regime classifier)
    "supertrend": 1.0, # Olivier Seban ATR-trailing trend (reactive flip)
    "zscore": 0.9, # Statistical extremity over a longer window than BB
    # Both are built on Wilder's RSI, so this vote overlaps stochastic_rsi.
    "rsi": 0.8,
}

```

A weight moved here reaches nothing. Save reads no widget on this page, the load path restores none, and the settings schema declares no field for indicator weights. The voting engine takes a weights argument and falls back to the defaults above, and no construction site under `src/` supplies one from the settings store. The page label promises an adjustment the engine never sees. Issue #423 carries it.

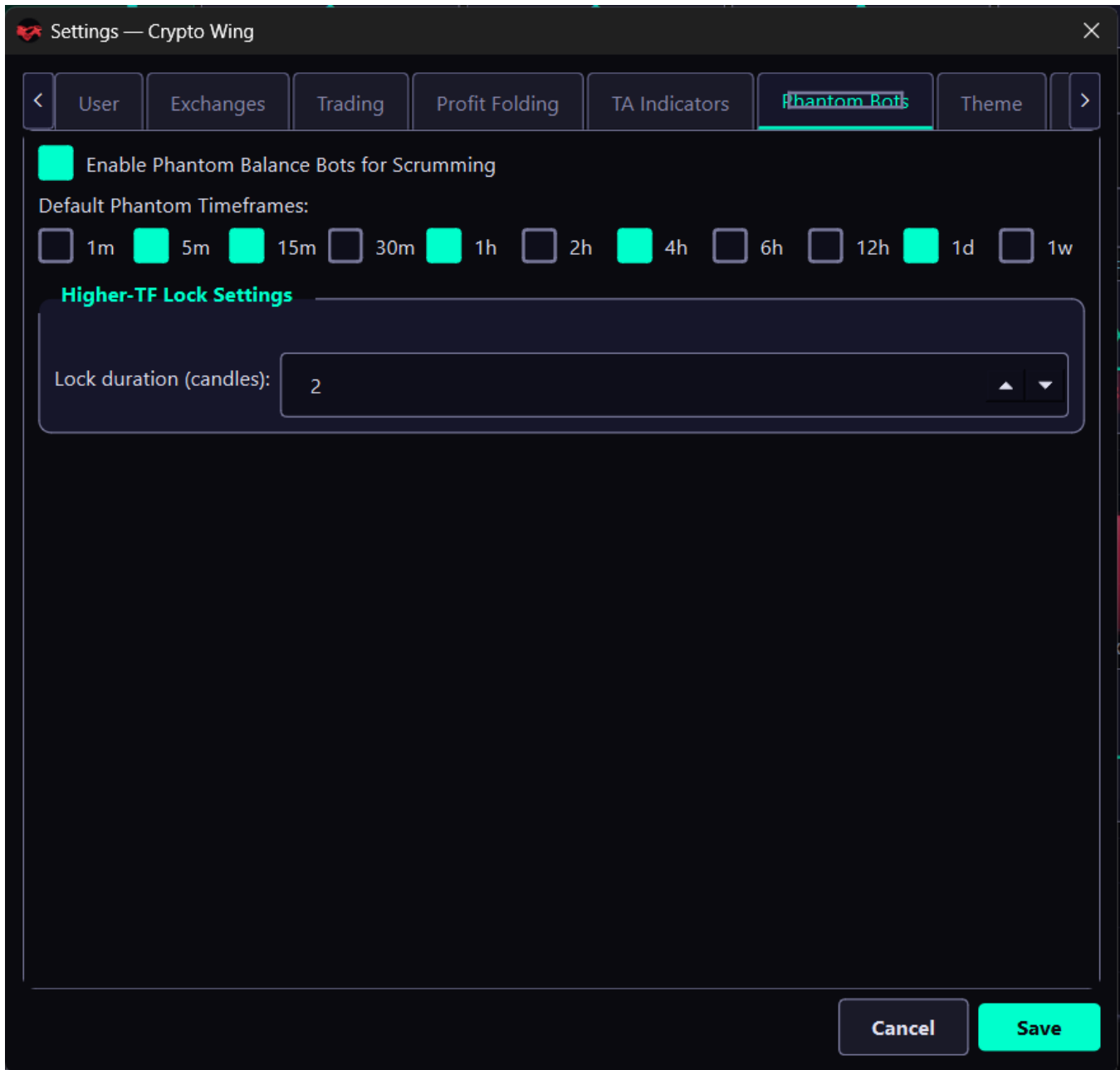
`src/trading/ta_engine.py` — `VotingEngine.__init__`, the fallback every construction site takes

```

self.weights = weights or DEFAULT_WEIGHTS.copy()

```


Settings > Phantom Bots



Settings — Crypto Wing

< User Exchanges Trading Profit Folding TA Indicators **Phantom Bots** Theme >

☒ Enable Phantom Balance Bots for Scrumming

Default Phantom Timeframes:

☐ 1m ☒ 5m ☒ 15m ☐ 30m ☒ 1h ☐ 2h ☒ 4h ☐ 6h ☐ 12h ☒ 1d ☐ 1w

Higher-TF Lock Settings

Lock duration (candles):

Cancel Save

Settings, the Phantom Bots page.

The master checkbox, eleven timeframe boxes — 1m, 5m, 15m, 30m, 1h, 2h, 4h, 6h, 12h, 1d and 1w — and one Higher-TF Lock Settings group holding Lock duration in candles, 1 to 10. The build checks 5m, 15m, 1h, 4h and 1d, which is the state the figure shows.

Enable Phantom Balance Bots for Scrumming - Turns the phantom overrides on for a new bot. Per-bot key `enable_phantoms`, a checkbox, checked at build.

This is the only control on the page that starts on. Save never reads it, so the page stores nothing under the name it shows.

`src/gui/settings_dialog.py` — the phantom master switch

```

self._phantoms_enabled = QCheckBox(
    "Enable Phantom Balance Bots for Scrumping"
)
self._phantoms_enabled.setChecked(True)
layout.addWidget(self._phantoms_enabled)

```

Default Phantom Timeframes - Chooses which charts a phantom watches. Per-bot key `phantom_timeframes`, eleven boxes, five of them checked at build.

One box per timeframe in a single row. The build decides which five open checked, and those five are the ones the figure shows.

`src/gui/settings_dialog.py` — the box built for each timeframe

```

cb = QCheckBox(tf)
cb.setChecked(tf in ["5m", "15m", "1h", "4h", "1d"])
self._phantom_tf_checks[tf] = cb
tf_grid.addWidget(cb)

```

Lock duration (candles) - Sets how many candles a higher-timeframe lock holds an opposing trade back for. Per-bot key `lock_candle_count`, 1 to 10, at 2.

The one spin box on the page, inside its own group. It opens at 2 and Save reads it no more than it reads the other two.

`src/gui/settings_dialog.py` — the lock duration box

```

lock_group = QGroupBox("Higher-TF Lock Settings")
lock_form = QFormLayout(lock_group)
self._lock_candles = QSpinBox()
self._lock_candles.setRange(1, 10)
self._lock_candles.setValue(2)
lock_form.addRow("Lock duration (candles):", self._lock_candles)

```

Those eleven boxes are the eleven timeframes the voting engine already weights, from 1m at 0.3 up to 1w at 1.6, so a slower chart counts for more when several are combined.

These three controls carry the same gap the TA Indicators page carries. Save reads none of them, the load path restores none, and the schema refuses a key it does not declare rather than storing it somewhere unread.

`src/core/settings.py` — `SettingsManager.set`

```

with self._lock:
    if not hasattr(self._settings, key):
        raise KeyError(f"Unknown setting: {key}")
    setattr(self._settings, key, value)
    self._save()

```

Issue #423 carries this page. The per-bot equivalents do persist: the wizard's phantom page emits the same three names into the bot's own config, and the Live Bot Settings dialog edits them there.

src/gui/bot_wizard.py — PhantomConfigPage.get_config, the three keys that do reach a bot

```
return {
    "enable_phantoms": self._enable.isChecked(),
    "phantom_timeframes": checked,
    "lock_candle_count": self._lock_candles.value(),
}
```

Settings > Theme

Settings — Crypto Wing

< User Exchanges Trading Profit Folding TA Indicators Phantom Bots **Theme** >

Visual Theme:

Cyberpunk Dark

Accent Color:

#00ffcc

Font Settings

Font Family: Segoe UI

Base Font Size: 11 pt

Heading Font Size: 14 pt

Log Font Size: 10 pt

Preview: The quick brown fox jumps over the lazy dog

Cancel Save

Settings, the Theme page.

The theme picker, the accent colour field and a Font Settings group. Six controls and one preview.

Visual Theme - Chooses the palette the whole window paints in. Key `theme`, five display names — Cyberpunk Dark, Neon Light, Classic Terminal, Minimal Modern and Glass & Metal — at Cyberpunk Dark.

This is the one control on the page that reaches the running window. Save emits a signal, the main window reads the stored theme back, and the window repaints without a restart.

`src/gui/main_window.py` — `_on_settings_changed`

```
theme = self._settings.get("theme", "cyberpunk_dark")
self._switch_theme(theme)
```

Accent Color - Sets the highlight colour. Key `accent_color`, a free-text field, at `#00ffcc`.

A plain text box. The default is placeholder text, not a value, so the field opens empty until something is typed or a stored value loads. Save stores what is there and the load path puts it back, but nothing paints with it.

`src/gui/settings_dialog.py` — the Accent Color field

```
layout.addWidget(QLabel("Accent Color:"))
self._accent_color = QLineEdit()
self._accent_color.setPlaceholderText("#00ffcc")
layout.addWidget(self._accent_color)
```

Font Family - Sets the typeface for application text. Key `font_family`, an editable combo over twelve named families, at Segoe UI.

The box is editable, so a family outside the twelve can be typed in. Save stores this value and the three sizes below it. Nothing reads any of the four, and the load path does not restore them either, so the whole Font Settings group reopens at its built-in figures.

`src/gui/settings_dialog.py` — the Font Family row

```
self._font_family.addItem(fonts)
self._font_family.setCurrentText("Segoe UI")
self._font_family.setToolTip("Font family for all application text")
font_form.addRow("Font Family:", self._font_family)
```

Base Font Size - Sets the size of ordinary interface text. Key `font_size`, 8 pt to 24 pt, at 11 pt.

The preview line below the group follows this box and the family box as either changes. That preview is the only place either value has an effect.

`src/gui/settings_dialog.py` — the Base Font Size row

```
self._font_size = QSpinBox()
self._font_size.setRange(8, 24)
self._font_size.setValue(11)
self._font_size.setSuffix(" pt")
self._font_size.setToolTip("Base font size for all UI text")
font_form.addRow("Base Font Size:", self._font_size)
```

Heading Font Size - Sets the size of headings and stat-card values. Key `heading_font_size`, 10 pt to 32 pt, at 14 pt.

The widest of the three ranges. The preview does not follow this box.

`src/gui/settings_dialog.py` — the Heading Font Size row

```
self._heading_size = QSpinBox()
self._heading_size.setRange(10, 32)
self._heading_size.setValue(14)
self._heading_size.setSuffix(" pt")
self._heading_size.setToolTip("Font size for headings and stat card values")
font_form.addRow("Heading Font Size:", self._heading_size)
```

Log Font Size - Sets the size of the Activity Log and API Log text. Key `log_font_size`, 8 pt to 18 pt, at 10 pt.

The narrowest of the three ranges, and the last row before the preview.

`src/gui/settings_dialog.py` — the Log Font Size row

```
self._log_font_size = QSpinBox()
self._log_font_size.setRange(8, 18)
self._log_font_size.setValue(10)
self._log_font_size.setSuffix(" pt")
```

Preview redraws in the chosen family and base size as either changes. It is a sample line, not a setting, and nothing stores it.

The five theme names come from one mapping built off the five theme objects.

`src/gui/theme_engine.py` — THEMES

```
THEMES: dict[str, ThemeTokens] = {
    t.name: t
    for t in [CYBERPUNK_DARK, NEON_LIGHT, CLASSIC_TERMINAL, MINIMAL_MODERN, GLASS_METAL]
}
```

Save writes the theme, the Accent Color field and all four font values.

`src/gui/settings_dialog.py` — `_save`, the theme and font keys

```
"theme": lambda: self._theme_combo.currentData(),
"accent_color": lambda: self._accent_color.text().strip(),
"font_family": lambda: self._font_family.currentText(),
"font_size": lambda: self._font_size.value(),
"heading_font_size": lambda: self._heading_size.value(),
"log_font_size": lambda: self._log_font_size.value(),
```

The load path reads back the theme and the accent field only, so the four font rows open at Segoe UI, 11, 14 and 10 whatever was stored.

Of the six, the theme alone reaches the window. `main.py` reads it at startup, and the window reads it again in `_on_settings_changed` when the dialog saves.

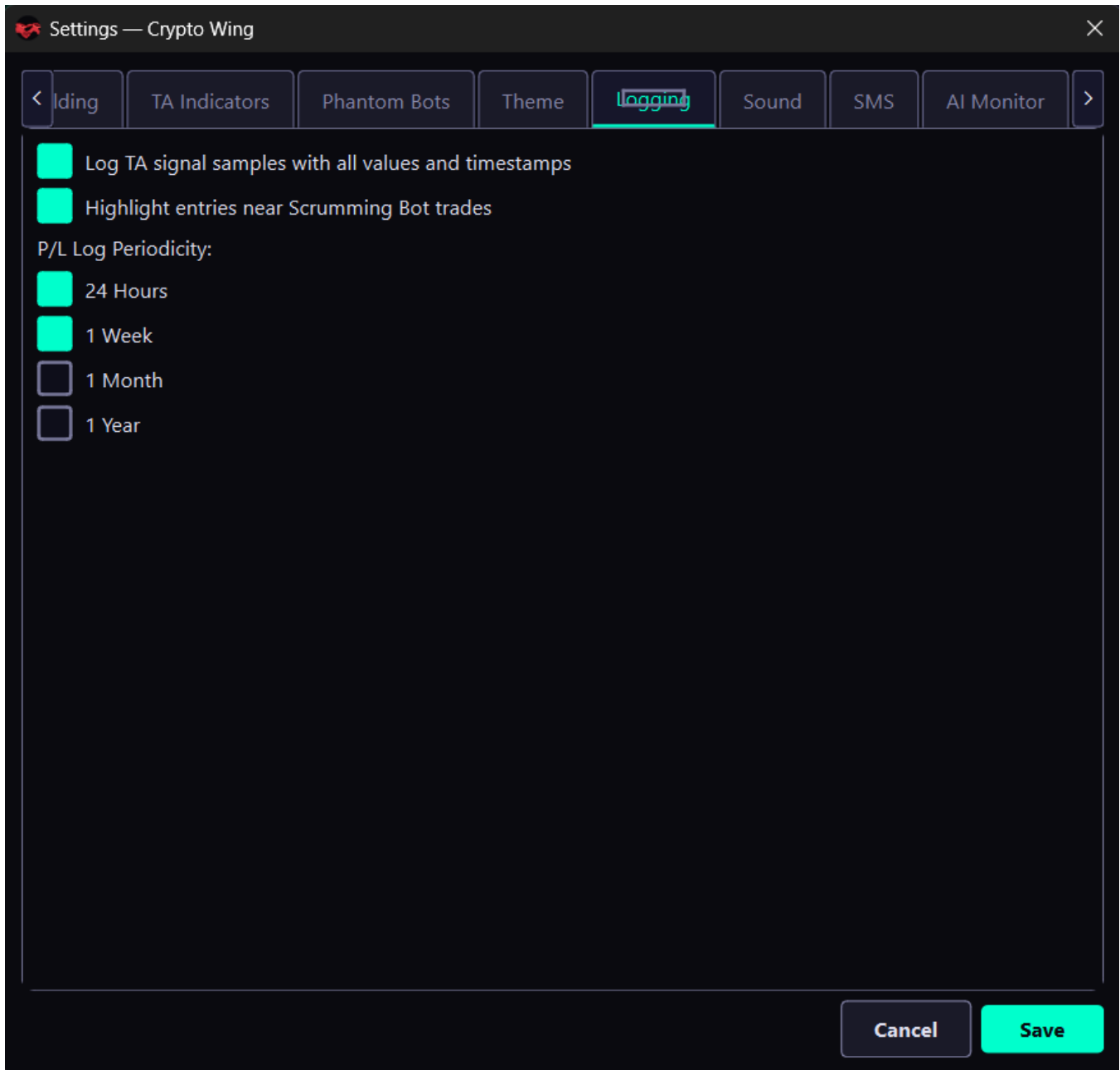
The other five are stored and never read again. Each palette carries its own typeface in the theme object it declares, so the theme decides the font and the four font rows decide nothing. The accent field has the same shape: declared, written, read back into its own line edit, and painted nowhere.

`src/core/settings.py` — `AppSettings`, the five that are stored and unread

```
accent_color: str = "#00ffcc"

# These four mirror the font widgets in settings_dialog._create_theme_tab.
font_family: str = "Segoe UI"
font_size: int = 11
heading_font_size: int = 14
log_font_size: int = 10
```

Settings > Logging



Settings — Crypto Wing

< Logging TA Indicators Phantom Bots Theme **Logging** Sound SMS AI Monitor >

☒ Log TA signal samples with all values and timestamps

☒ Highlight entries near Scrumming Bot trades

P/L Log Periodicity:

☒ 24 Hours

☒ 1 Week

☐ 1 Month

☐ 1 Year

Cancel Save

Settings, the Logging page.

Two checkboxes and four periodicity boxes. Every key below sits inside one stored `data_logging` group.

Log TA signal samples with all values and timestamps - Writes each indicator reading out with its value and its time. Key `ta_signal_logging`, a checkbox, checked at build.

The first of the two flags on the page, and the one that decides whether the indicator record is written at all.

`src/gui/settings_dialog.py` — the TA logging checkbox

```

self._ta_logging = QCheckBox(
    "Log TA signal samples with all values and timestamps"
)
self._ta_logging.setChecked(True)
layout.addWidget(self._ta_logging)

```

Highlight entries near Scrumming Bot trades - Marks the log lines that sit close to a fire, so a decision can be read in its own context. Key `highlight_trade_proximity`, a checkbox, checked at build.

The second flag, built the same way and checked at build like the first.

`src/gui/settings_dialog.py` — the highlight checkbox

```

self._highlight_trades = QCheckBox(
    "Highlight entries near Scrumming Bot trades"
)
self._highlight_trades.setChecked(True)
layout.addWidget(self._highlight_trades)

```

P/L Log Periodicity - Chooses the windows a profit and loss figure is summarised over: 24 Hours, 1 Week, 1 Month and 1 Year. Key `active_periodicities`, four boxes written as a list, with the first two checked at build.

Four separate boxes rather than one control. Save walks them in page order and writes the checked ones into a list.

`src/gui/settings_dialog.py` — the four periodicity boxes

```

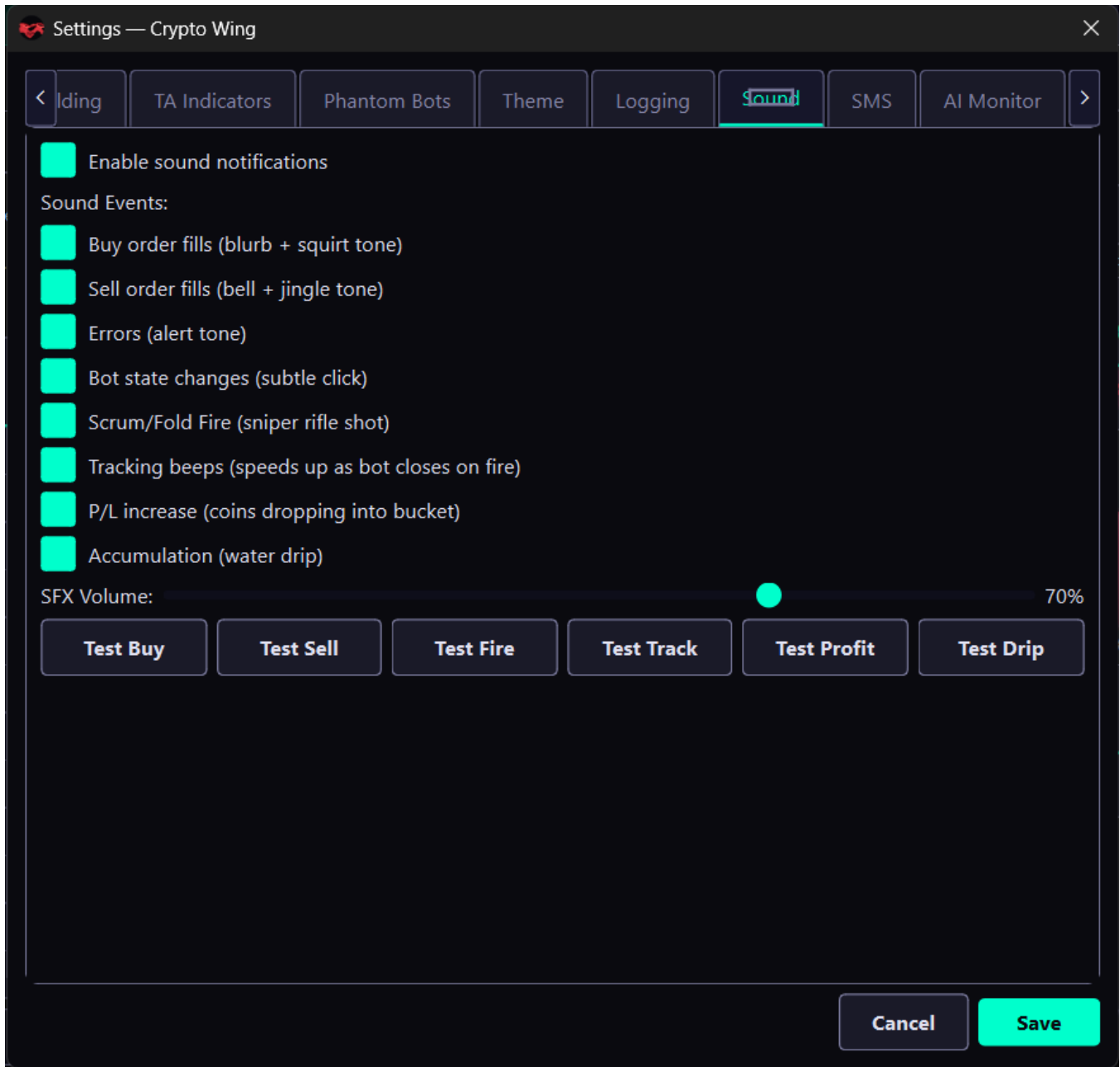
self._log_24h = QCheckBox("24 Hours")
self._log_24h.setChecked(True)
self._log_1w = QCheckBox("1 Week")
self._log_1w.setChecked(True)
self._log_1m = QCheckBox("1 Month")
self._log_1y = QCheckBox("1 Year")

```

Save collapses all six into one dictionary holding the two flags and the list of active periodicities. The load path reads none of them back, so every open shows the build state rather than the stored one. No reader outside the dialog reads the group either, which leaves the six controls writing to a key nothing consults. Issue #423 carries this page.

In development. What the load path should restore for this page has not been settled against the rest of the dialog, so nothing is proposed here.

Settings > Sound



Settings, the Sound page.

The master switch, eight event checkboxes, a volume slider and six test buttons. Every checkbox is checked at build.

Each checkbox names its sound in its own label, and the key behind each control is a field of the sound configuration the block below builds.

Enable sound notifications - Turns every sound on or off at once. Key enabled, on.

The engine tests this flag before it tests any other, so a clear box silences the page whatever the eight below are set to.

src/gui/settings_dialog.py — the master switch

```
self._sound_enabled = QCheckBox("Enable sound notifications")
self._sound_enabled.setChecked(True)
self._sound_enabled.setToolTip("Master switch for all audio notifications")
```

Buy order fills - Plays a low rising tone when a buy fills. Key buy_sound, on.

The label carries the sound's own name in brackets, as every box on this page does. The engine plays this one under the name buy.

src/gui/settings_dialog.py — the buy fill box

```
self._sound_buy = QCheckBox("Buy order fills (blurb + squirt tone)")
self._sound_buy.setChecked(True)
self._sound_buy.setToolTip(
    "Plays a low bubbly rising tone when a buy order is filled"
)
```

Sell order fills - Plays a bright bell when a sell fills. Key sell_sound, on.

The counterpart to the box above, played under the name sell.

src/gui/settings_dialog.py — the sell fill box

```
self._sound_sell = QCheckBox("Sell order fills (bell + jingle tone)")
self._sound_sell.setChecked(True)
self._sound_sell.setToolTip(
    "Plays a high bright bell tone when a sell order is filled"
)
```

Errors - Plays an alert tone on an error. Key error_sound, on.

One of the two boxes on the page with no tooltip of its own.

src/gui/settings_dialog.py — the error box

```
self._sound_error = QCheckBox("Errors (alert tone)")
self._sound_error.setChecked(True)
layout.addWidget(self._sound_error)
```

Bot state changes - Plays a click when a bot starts, pauses or stops. Key bot_state_sound, on.

The other box with no tooltip. The engine plays it under the name state, not under the key.

src/gui/settings_dialog.py — the bot state box

```
self._sound_state = QCheckBox("Bot state changes (subtle click)")
self._sound_state.setChecked(True)
layout.addWidget(self._sound_state)
```

Scrum/Fold Fire - Plays a rifle shot when a scrum or a fold executes, and on a Manual Fire. Key `fire_sound`, on.

This is the first of the four the engine reads with a default rather than straight off the configuration. A stored configuration written before these four existed leaves all four on.

`src/gui/settings_dialog.py` — the fire box

```
self._sound_fire = QCheckBox("Scrum/Fold Fire (sniper rifle shot)")
self._sound_fire.setChecked(True)
self._sound_fire.setToolTip(
    "Synthesized rifle shot plays when a scrum or fold\n"
    "actually executes. Also plays on Manual Fire."
)
```

Tracking beeps - Paces a beep by the scrum phase: silent in SEARCH, slow in TRACK, fast in FIRE. Key `track_sound`, on.

The tooltip gives the two paces in milliseconds: 800 in TRACK and 200 in FIRE.

`src/gui/settings_dialog.py` — the tracking box

```
self._sound_track = QCheckBox(
    "Tracking beeps (speeds up as bot closes on fire)"
)
self._sound_track.setChecked(True)
```

P/L increase - Plays coins dropping into a bucket on any trade that realised a profit. Key `profit_sound`, on.

The tooltip is the only place on the page that names the condition exactly: realised profit above zero, on grid and scrumming bots alike.

`src/gui/settings_dialog.py` — the profit box

```
self._sound_profit = QCheckBox("P/L increase (coins dropping into bucket)")
self._sound_profit.setChecked(True)
```

Accumulation - Plays a water drip on a FOLD, and on nothing else. Key `drip_sound`, on.

The last of the nine. Its tooltip calls a fold the canonical accumulation moment and states that a scrum or a distribution does not fire it.

`src/gui/settings_dialog.py` — the accumulation box

```
self._sound_drip = QCheckBox("Accumulation (water drip)")
self._sound_drip.setChecked(True)
```

SFX Volume - Sets how loud every sample plays. Key `volume`, 0 % to 100 %, at 70 %.

Test Buy, Test Sell, Test Fire, Test Track, Test Profit and Test Drip play one sample each. They are buttons, not settings.

One handler is the only path from this page to the engine. It builds a configuration from every checkbox and the slider, pushes it in and clears the sample cache, because each sample bakes its volume at synthesis time.

src/gui/settings_dialog.py — `_on_sfx_volume_changed`

```
se = get_sound_engine()
new_cfg = SoundConfig(
    enabled=self._sound_enabled.isChecked(),
    buy_sound=self._sound_buy.isChecked(),
    sell_sound=self._sound_sell.isChecked(),
    error_sound=self._sound_error.isChecked(),
    bot_state_sound=self._sound_state.isChecked(),
    fire_sound=self._sound_fire.isChecked(),
    track_sound=self._sound_track.isChecked(),
    profit_sound=self._sound_profit.isChecked(),
    drip_sound=self._sound_drip.isChecked(),
    volume=v / 100.0,
)
```

That path runs when the slider moves or a test button is pressed, and at no other time. Save reads no widget on this page and the schema declares no sound field, so nothing here survives the dialog closing. A checkbox cleared and the dialog saved comes back checked. Issue #423 carries it.

src/core/sound_engine.py — `SoundConfig`, the ten fields and their defaults

```
enabled: bool = True
buy_sound: bool = True
sell_sound: bool = True
error_sound: bool = True
bot_state_sound: bool = True
fire_sound: bool = True
track_sound: bool = True
profit_sound: bool = True
drip_sound: bool = True
volume: float = 0.7
```

Settings > SMS

Settings, the SMS page.

A scrolling page holding the master switch and three groups.

Seventeen controls, and the key behind each is a field of the message configuration the engine reads.

Enable SMS notifications - Turns every message on or off at once. Key enabled, clear at build.

The only control on the page outside the three groups, and the only checkbox here that opens clear rather than checked.

src/gui/settings_dialog.py — the SMS master switch

```

self._sms_enabled = QCheckBox("Enable SMS notifications")
self._sms_enabled.setToolTip(
    "Send text messages to your phone for trading events"
)
layout.addWidget(self._sms_enabled)

```

Provider - Chooses how a message leaves the machine: a carrier's email gateway, or the Twilio web interface. Key provider, at the email gateway. Nothing loads a stored choice into the picker, so it opens on the first entry whatever the figure shows.

Two entries, both written as display labels. The engine tests a short code rather than either label, so neither entry matches what the send path looks for.

src/gui/settings_dialog.py — the Provider row

```

self._sms_provider = QComboBox()
self._sms_provider.setMinimumHeight(28)
self._sms_provider.addItem("Email-to-SMS Gateway", "Twilio API")
pf.addRow("Provider:", self._sms_provider)

```

Phone Number - Sets the number every message goes to. Key phone_number, empty, with a placeholder showing the international shape.

The placeholder is the only guidance on the row. It is grey sample text, not a value, so the field starts genuinely empty.

src/gui/settings_dialog.py — the Phone Number row

```

self._sms_phone = QLineEdit()
self._sms_phone.setMinimumHeight(28)
self._sms_phone.setPlaceholderText("+15551234567")
pf.addRow("Phone Number:", self._sms_phone)

```

Carrier - Names the carrier whose gateway address a message would be built for. Ten entries, opening on the first. It has no stored field of its own, and nothing on the page reads the choice.

The ten names come from one mapping in the engine. The last entry is a manual option whose address is deliberately empty.

src/gui/settings_dialog.py — the Carrier row

```

self._sms_carrier = QComboBox()
self._sms_carrier.setMinimumHeight(28)
from src.core.sms_engine import CARRIER_GATEWAYS

for carrier in CARRIER_GATEWAYS:
    self._sms_carrier.addItem(carrier)
pf.addRow("Carrier:", self._sms_carrier)

```

Gateway Email - Sets the full gateway address a message is sent to. Key gateway_email, empty.

The row a carrier choice would otherwise fill in. Nothing builds the address from the picker above, so this field is typed by hand or left blank.

src/gui/settings_dialog.py — the Gateway Email row

```
self._sms_gateway = QLineEdit()
self._sms_gateway.setMinimumHeight(28)
self._sms_gateway.setPlaceholderText("5551234567@vtext.com")
self._sms_gateway.setToolTip("Full email address for carrier SMS gateway")
pf.addRow("Gateway Email:", self._sms_gateway)
```

SMTP Username - Sets the mailbox a message is sent from. Key smtp_username, empty.

The first of the two mailbox rows. Its placeholder shows an ordinary address.

src/gui/settings_dialog.py — the SMTP Username row

```
self._sms_smtp_user = QLineEdit()
self._sms_smtp_user.setMinimumHeight(28)
self._sms_smtp_user.setPlaceholderText("your.email@gmail.com")
pf.addRow("SMTP Username:", self._sms_smtp_user)
```

SMTP Password - Sets that mailbox's application password. Key smtp_password, masked, empty.

The only masked field on the page. Its placeholder says an application password rather than the account password.

src/gui/settings_dialog.py — the SMTP Password row

```
self._sms_smtp_pass = QLineEdit()
self._sms_smtp_pass.setMinimumHeight(28)
self._sms_smtp_pass.setEchoMode(QLineEdit.Password)
self._sms_smtp_pass.setPlaceholderText(
    "App password (not regular password)"
)
pf.addRow("SMTP Password:", self._sms_smtp_pass)
```

Buy fills - Sends a message when a buy fills. Key notify_buy_fills, checked at build.

The first of the eight event rows, and one of the four that open checked.

src/gui/settings_dialog.py — the buy fills row

```
self._sms_buy = QCheckBox("Buy fills")
self._sms_buy.setChecked(True)
ef.addRow(self._sms_buy)
```

Sell fills - Sends a message when a sell fills. Key notify_sell_fills, checked at build.

The counterpart to the row above, built and checked the same way.

src/gui/settings_dialog.py — the sell fills row

```
self._sms_sell = QCheckBox("Sell fills")
self._sms_sell.setChecked(True)
ef.addRow(self._sms_sell)
```

Bot state changes - Sends a message when a bot starts, stops or errors. Key `notify_bot_state_changes`, checked at build.

The label names the three states in brackets, which the page's own text does not repeat anywhere else.

`src/gui/settings_dialog.py` — the bot state row

```
self._sms_state = QCheckBox("Bot state changes (start/stop/error)")
self._sms_state.setChecked(True)
ef.addRow(self._sms_state)
```

API errors and failures - Sends a message when a venue call fails. Key `notify_errors`, checked at build.

The last of the four rows that open checked.

`src/gui/settings_dialog.py` — the API errors row

```
self._sms_errors = QCheckBox("API errors and failures")
self._sms_errors.setChecked(True)
ef.addRow(self._sms_errors)
```

P/L threshold alerts - Sends a message when profit or loss passes a dollar figure. Key `notify_pl_threshold`, clear at build.

The first row in the group that opens clear. It gates the dollar figure on the row below it.

`src/gui/settings_dialog.py` — the P/L alert row

```
self._sms_pl = QCheckBox("P/L threshold alerts")
ef.addRow(self._sms_pl)
```

P/L threshold - Sets that dollar figure. Key `pl_threshold_amount`, \$1 to \$100,000, at \$100.

The one spin box in the events group, and the only row there that is not a checkbox.

`src/gui/settings_dialog.py` — the P/L threshold row

```
self._sms_pl_amount = QDoubleSpinBox()
self._sms_pl_amount.setMinimumHeight(28)
self._sms_pl_amount.setRange(1, 100000)
self._sms_pl_amount.setValue(100)
self._sms_pl_amount.setPrefix("$")
ef.addRow("P/L threshold:", self._sms_pl_amount)
```

Low balance warnings - Sends a message when a balance runs low. Key `notify_balance_warning`, clear at build.

No figure on the page sets what counts as low, so the row carries the switch alone.

src/gui/settings_dialog.py — the low balance row

```
self._sms_balance = QCheckBox("Low balance warnings")
ef.addRow(self._sms_balance)
```

Exchange connection status - Sends a message when a venue connects or drops. Key notify_connection_status, clear at build.

The last event row, and the third of the three that open clear.

src/gui/settings_dialog.py — the connection status row

```
self._sms_connection = QCheckBox("Exchange connection status")
ef.addRow(self._sms_connection)
```

Max messages per hour - Caps how many messages one hour may carry. Key max_messages_per_hour, 1 to 100, at 20. Below the area the figure shows.

The first of the two rows in the rate limiting group.

src/gui/settings_dialog.py — the hourly cap row

```
self._sms_max_hour = QSpinBox()
self._sms_max_hour.setMinimumHeight(28)
self._sms_max_hour.setRange(1, 100)
self._sms_max_hour.setValue(20)
rf.addRow("Max messages per hour:", self._sms_max_hour)
```

Min time between messages - Sets the wait between two messages. Key cooldown_seconds, 5 to 300 seconds, at 30. Below the area the figure shows.

The second rate limiting row, and the last control on the page. It is the one spin box here that carries a unit in its own suffix.

src/gui/settings_dialog.py — the cooldown row

```
self._sms_cooldown = QSpinBox()
self._sms_cooldown.setMinimumHeight(28)
self._sms_cooldown.setRange(5, 300)
self._sms_cooldown.setValue(30)
self._sms_cooldown.setSuffix(" sec")
rf.addRow("Min time between messages:", self._sms_cooldown)
```

The Carrier list fills from one mapping of carrier name to gateway address.

src/core/sms_engine.py — CARRIER_GATEWAYS, the first four

```
CARRIER_GATEWAYS = {
    "AT&T": "{number}@txt.att.net",
    "T-Mobile": "{number}@tmomail.net",
    "Verizon": "{number}@vtext.com",
    "Sprint": "{number}@messaging.sprintpcs.com",
```

None of it reaches the SMS engine. Save reads no widget on this page and the schema declares no SMS field. Neither provider label the combo offers matches the value the engine tests for, and the page carries no field for the three Twilio credentials the send path reads.

src/core/sms_engine.py — SMSConfig.provider, the value the send path tests

```
provider: str = "email_gateway"
```

Issue #423 carries the page.

src/core/sms_engine.py — SMSConfig, the three credentials no row fills

```
twilio_sid: str = ""
twilio_auth_token: str = ""
twilio_from_number: str = ""
```

Settings > AI Monitor

Settings — Crypto Wing

< Iding TA Indicators Phantom Bots Theme Logging Sound SMS **AI Monitor** >

Claude API Connection

Anthropic API Key: sk-ant-api03-...

Check interval: 4.0 hours

Handshake Authentication

The connect phrase is embedded in the system prompt sent to Claude.
The confirm phrase is what Claude must respond with to prove identity.
Change both phrases together. Keep them secret.

Connect phrase: acervator-heapbuilder-live

Confirm phrase: the-heap-grows-by-accumulation

Monitor Behavior

☐ Enable AI Monitor feedback loop

☒ Auto-handshake on first analysis

☒ Log AI feedback to trade journal

Cancel Save

Settings, the AI Monitor page.

A scrolling page holding four groups.

Seven settings, and every key below sits inside one stored `ai_monitor` group.

Anthropic API Key - Holds the key the monitor authenticates with. Key `api_key`, masked, empty.

The first row of the Claude API Connection group. It is masked like the SMS password, and its placeholder shows the shape of a key rather than a key.

`src/gui/settings_dialog.py` — the API key row

```

self._ai_api_key = QLineEdit()
self._ai_api_key.setMinimumHeight(28)
self._ai_api_key.setEchoMode(QLineEdit.Password)
self._ai_api_key.setPlaceholderText("sk-ant-api03-...")
af.addRow("Anthropic API Key:", self._ai_api_key)

```

Check interval - Sets how long the monitor waits between two reviews. Key `interval_hours`, 0.5 to 24.0 hours, at 4.0.

The one figure on the page. It carries a decimal place, so half an hour is the shortest wait it accepts.

`src/gui/settings_dialog.py` — the check interval row

```

self._ai_interval = QDoubleSpinBox()
self._ai_interval.setMinimumHeight(28)
self._ai_interval.setRange(0.5, 24.0)
self._ai_interval.setValue(4.0)
self._ai_interval.setSuffix(" hours")
self._ai_interval.setDecimals(1)
af.addRow("Check interval:", self._ai_interval)

```

Connect phrase - Sets the phrase the platform puts into the prompt it sends. Key `connect_phrase`, empty.

The first of the two handshake rows. A grey note above the pair states the rule the two follow, and the placeholder shows the shape of a phrase.

`src/gui/settings_dialog.py` — the connect phrase row

```

self._ai_connect_phrase = QLineEdit()
self._ai_connect_phrase.setMinimumHeight(28)
self._ai_connect_phrase.setPlaceholderText("acervator-heapbuilder-live")
hf.addRow("Connect phrase:", self._ai_connect_phrase)

```

Confirm phrase - Sets the phrase the answer must carry back to prove it came from the same conversation. Key `confirm_phrase`, empty. Change both phrases together and keep both secret.

The second handshake row, built like the first. Neither field is masked.

`src/gui/settings_dialog.py` — the confirm phrase row

```

self._ai_confirm_phrase = QLineEdit()
self._ai_confirm_phrase.setMinimumHeight(28)
self._ai_confirm_phrase.setPlaceholderText("the-heap-grows-by-accumulation")
hf.addRow("Confirm phrase:", self._ai_confirm_phrase)

```

Enable AI Monitor feedback loop - Turns the monitor on. Key `enabled`, clear at build.

The one control on the page that opens clear. Save emits a signal and the main window reads the whole group back, so this box and the key beside it decide whether the header reads READY or OFF.

`src/gui/main_window.py` — `_on_settings_changed`, the monitor branch

```

ai_cfg = self._settings.get("ai_monitor", {})
if self._bot_manager:
    self._bot_manager.configure_live_monitor(ai_cfg)
    if ai_cfg.get("enabled") and ai_cfg.get("api_key"):
        self._ai_monitor_label.setText("AI: READY")

```

Auto-handshake on first analysis - Runs the phrase exchange on the first review rather than waiting for one to be asked for. Key `auto_handshake`, checked at build.

The first of the two Monitor Behavior rows that open checked.

`src/gui/settings_dialog.py` — the auto-handshake row

```

self._ai_auto_handshake = QCheckBox("Auto-handshake on first analysis")
self._ai_auto_handshake.setChecked(True)
bf.addRow(self._ai_auto_handshake)

```

Log AI feedback to trade journal - Writes each answer into the trade journal. Key `log_feedback`, checked at build.

The last control before the status group, and the second of the two that open checked.

`src/gui/settings_dialog.py` — the journal logging row

```

self._ai_log_feedback = QCheckBox("Log AI feedback to trade journal")
self._ai_log_feedback.setChecked(True)
bf.addRow(self._ai_log_feedback)

```

Connection Status closes the page, below the area the figure shows: a status line, a journal hash, a completed-check count and a Test Handshake button. The three readings report and the button acts. None of the four stores anything.

Save writes all seven controls into one dictionary and the load path reads all seven back, which makes this the one page besides User whose whole state round-trips.

`src/gui/settings_dialog.py` — `_load_current`, the seven it restores

```

ai = self._sm.get("ai_monitor", {})
self._ai_api_key.setText(ai.get("api_key", ""))
self._ai_interval.setValue(ai.get("interval_hours", 4.0))
self._ai_connect_phrase.setText(ai.get("connect_phrase", ""))
self._ai_confirm_phrase.setText(ai.get("confirm_phrase", ""))
self._ai_enabled.setChecked(ai.get("enabled", False))
self._ai_auto_handshake.setChecked(ai.get("auto_handshake", True))
self._ai_log_feedback.setChecked(ai.get("log_feedback", True))

```

Test Handshake runs no handshake. It checks that the key and both phrases hold text, then writes either an error line or the message saying the handshake runs on the next bot cycle.

`src/gui/settings_dialog.py` — `_test_ai_handshake`

```
self._ai_status.setText("Settings saved – handshake runs on next bot cycle")
self._ai_status.setStyleSheet("color: #00ddff; font-weight: bold;")
```

The button reports the fields it read, never a venue answer. Issue #327 carries that.

Six of the seven do reach the engine. `configure_live_monitor` in `src/trading/bot_container.py` reads five of them and builds or clears the monitor, and the main window reads the journal switch when an answer arrives. The auto-handshake box is the seventh: written, read back, and read by nothing else.

`src/trading/bot_container.py` — `configure_live_monitor`, the five keys it reads

```
self._live_monitor = LiveMonitor(
    api_key=settings["api_key"],
    journal=journal,
    interval_hours=settings.get("interval_hours", 4.0),
    connect_phrase=settings.get("connect_phrase", ""),
    confirm_phrase=settings.get("confirm_phrase", ""),
)
```

Converting the Interface to React

Every screen in this application is drawn by Qt. Issue #128 replaces them with React and keeps the backend in Python. This section is the running record of that work. One row per screen, one column per step, and a mark only where the step is finished.

One screen draws React in the application today: the History table. It runs inside the Chromium that the Qt toolkit already ships, not inside Electron.

`src/gui/react_history_panel.py` — `HistoryWebTable._setup_ui`

```
self._web = QWebEngineView()
self._web.loadFinished.connect(self._on_load_finished)
self._web.setHtml(panel_html(theme))
layout.addWidget(self._web, 1)
```

What the tree holds

Measured on 5 September 2026 over the tracked files.

React modules	70 files, 80,292 lines, 0 stub markers
	65 of them call <code>React.createElement</code>
	React and ReactDOM are vendored on disk
View-model modules	74 files, 74,207 lines
Parity tests	72 files, 170,244 lines
Electron shell	8 code files, 791 lines, plus a package manifest
Renderer manifest	70 names
Qt modules under <code>src/gui</code>	69, every one importing <code>PySide6</code>
Tabs drawn by React	0

No file in this repository has ever carried a JavaScript extension other than plain .js. A history search over every branch returns no commit that added, deleted or renamed one, and the same search for the plain extension returns 81 paths, so the search does find a file that existed.

```
git log --all --diff-filter=ADR --name-only -- "*.jsx" "*.tsx" "*.ts"    no commits
git log --all --diff-filter=ADR --name-only -- "*.js"                  81 paths
```

What the Electron shell drew

The shell has been started once, and what it drew is measured. All 70 renderer modules loaded and none failed. Two of them register a panel for the page to draw, so the page had nothing to ask the other 68 for. The window showed an empty History table, with no tab bar and no navigation.

```
React modules the page list names    70
modules that failed to load          0
modules that register a panel        2
panels the page asked to draw        2
panels that failed to draw            0
```

The two that register are the Console tab panel and the header strip. The full per-file record is in docs/audits/2026-09-05_128_file_verification.md.

The tab bar the shell draws now

The shell builds the same ten tabs the application builds, with the same labels and in the same order. It holds no list of its own. main_window.state reports tab_labels and tab_methods; the bar keeps one tab per label, filled by the registered panel that calls that label's bridge method.

```
Sim Paper Live Charts Inspector
Swarm Accumulation History Status Console
```

A module that draws a screen inside a dialog registers no bridge method and takes no tab. market_inspector_tab.js and bot_swarm_tab.js are the two: both belong to the Live Bot Settings dialog.

Measured in one Electron launch, with the backend serving a two-bot fleet:

```
tabs on the bar                10
panels that drew text          10
panels that failed to draw      0
child panels drawn inside a tab 7
```

The header strip stays above the bar for every tab.

How to read the table

Each row is a Qt module the running window builds. The columns are the steps of the conversion, in the order the work happens.

Column	What the mark means
React module	A renderer module in <code>src/gui/web</code> serves this screen
Surface	A Python view model of the screen exists, with no toolkit in it
Bridge	The view model is registered as a method the frontend may call
Manifest	The renderer module is named in the shell's module list
Ships in the build	The built React application carries the module inside it
RENDERS	What the operator sees when he opens that screen
Scope	Whether the row is work this item must do

The last column is the item. A row is finished only when it reads yes. Every other column can be marked while the operator sees no change, and that is the difference between wiring written and a screen replaced.

The RENDERS column takes three values, and none of them is derived from the bridge or the manifest. yes means React draws where Qt used to. shell means the module draws in the Electron shell and nowhere else. A dash means the Qt widget is still the screen.

The Scope column names the module that builds a screen the item has still to convert. Header strip and window chrome are the parts of the Main Window that sit outside the tab row.

The table carries nine columns and no tab column. The screen a module draws is named in that screen's own section above. Header strip and window chrome are the parts of the Main Window that sit outside the tab row.

The Scope column takes five values.

Value	What it means
in scope	The running window builds this screen, and the cell names what builds it
React side	The file exists because of this conversion, so a React module for it would be a module for a module
not a screen	The module defines no view
builds the shell	The module fills the tab bar rather than sitting in it
shelved	Nothing the window builds reaches it

The walk that answers the column starts at the main window. It follows an imported name only where the module calls that name, subclasses it or reads an attribute of it, so a re-export block reaches nothing. A row is in scope when some module on that walk builds one of the classes the row defines.

`tools/conversion_scope.py` writes the column, so the answers are re-measured rather than typed.

The conversion table

Qt file	React module	Uses React	Bridge	Manifest	Registers in Electron	Ships in the build	REN DER S	Scope
splash_screen.py	no	-	yes	no	no	-	no	shelved
src/gui/alerts_tab.py	alerts_tab.js	yes	yes	yes	no	yes	yes	in scope
src/gui/analytics_tab.py	analytics_tab.js	yes	yes	yes	no	yes	no	shelved
src/gui/audio_suite.py	audio_suite.js	yes	yes	yes	no	yes	no	shelved
src/gui/bot_live_settings.py	bot_live_settings.js	yes	yes	yes	no	yes	yes	in scope
src/gui/bot_swarm_list.py	bot_swarm_list.js	yes	yes	yes	yes	yes	yes	in scope
src/gui/bot_visualizer.py	bot_visualizer.js	yes	yes	yes	yes	yes	yes	in scope
src/gui/bot_wizard.py	bot_wizard.js	yes	yes	yes	no	yes	yes	in scope
src/gui/buy_confirmation_dialog.py	buy_confirmation.js	yes	yes	yes	no	yes	yes	in scope
src/gui/competition_tab.py	competition_tab.js	yes	yes	yes	no	yes	no	shelved
src/gui/crypto_news_ticker.py	crypto_news_ticker.js	yes	yes	yes	no	yes	yes	in scope
src/gui/history_qt_table.py	no	-	no	no	no	-	no	React side
src/gui/history_tab.py	history_tab.js	yes	yes	yes	yes	yes	yes	in scope
src/gui/indicator_panel.py	indicator_panel.js	yes	yes	yes	no	yes	yes	in scope
src/gui/init_wizard.py	init_wizard.js	yes	yes	yes	no	yes	no	shelved
src/gui/instance_consent_dialog.py	instance_consent_dialog.js	yes	no	yes	no	yes	no	shelved
src/gui/journal_tab.py	journal_tab.js	yes	yes	yes	no	yes	no	shelved
src/gui/launcher.py	launcher.js	yes	yes	yes	no	yes	no	shelved
src/gui/live_bot_window.py	no	-	yes	no	no	-	no	shelved
src/gui/live_settings/bot_swarm_tab.py	bot_swarm_tab.js	yes	yes	yes	yes	yes	yes	in scope

Qt file	React module	Uses React	Bridge	Manifest	Registers in Electron	Ships in the build	RENDER S	Scope
src/gui/live_settings/fold_chrome.py	fold_chrome.js	yes	yes	yes	no	yes	yes	in scope
src/gui/live_settings/fold_tokens.py	fold_tokens.js	yes	yes	yes	no	yes	no	not a screen
src/gui/live_settings/fold_tranches_tab.py	fold_tranches_tab.js	yes	yes	yes	no	yes	yes	in scope
src/gui/live_settings/market_inspector_tab.py	market_inspector_tab.js	yes	yes	yes	yes	yes	yes	in scope
src/gui/live_settings/phantom_bots_tab.py	phantom_bots_tab.js	yes	yes	yes	no	yes	yes	in scope
src/gui/live_settings/positions_held_tab.py	no	-	no	no	no	-	yes	in scope
src/gui/live_settings/settings_tab.py	live_settings_tab.js	yes	yes	yes	no	yes	yes	in scope
src/gui/live_settings/stack_tranches_tab.py	stack_tranches_tab.js	yes	yes	yes	no	yes	yes	in scope
src/gui/live_settings/status_tab.py	no	-	yes	no	no	-	yes	in scope
src/gui/main_tabs/audio_suite_surface.py	no	-	yes	no	no	-	no	React side
src/gui/main_tabs/buy_confirmation_surface.py	no	-	yes	no	no	-	no	React side
src/gui/main_tabs/console_log_handler.py	no	-	no	no	yes	-	yes	in scope
src/gui/main_tabs/console_tab.py	console_tab.js	yes	yes	yes	yes	yes	yes	in scope
src/gui/main_tabs/empty_tabs.py	system_status_tab.js, proof_of_accumulation_tabs.js	yes	yes	yes	yes	no	yes	in scope
src/gui/main_tabs/header_strip.py	header_strip.js	yes	yes	yes	yes	yes	yes	in scope
src/gui/main_tabs/stock_main_window_surface.py	no	-	yes	no	no	-	no	React side
src/gui/main_tabs/trading_tab.py	trading_tab.js	yes	yes	yes	yes	yes	yes	in scope
src/gui/main_tabs/tradingview_chart_surface.py	no	-	yes	no	no	-	no	React side
src/gui/main_window.py	main_window.js	yes	yes	yes	no	no	yes	in scope
src/gui/market_inspector.py	market_inspector.js	yes	yes	yes	no	yes	yes	in scope

Qt file	React module	Uses React	Bridge	Manifest	Registers in Electron	Ships in the build	RENDER S	Scope
src/gui/market_inspector_topologies.py	market_inspector_topologies.js	yes	yes	yes	no	yes	yes	in scope
src/gui/native_chart.py	native_chart.js	yes	yes	yes	no	yes	yes	in scope
src/gui/paper_trader_tab.py	paper_trader_tab.js	yes	yes	yes	yes	no	yes	in scope
src/gui/qt_safe_events.py	no	-	yes	no	no	-	no	not a screen
src/gui/react_history_panel.py	no	-	yes	no	no	-	no	React side
src/gui/react_history_tab.py	no	-	no	no	no	-	no	React side
src/gui/react_paper_trader_tab.py	no	-	no	no	no	-	no	React side
src/gui/react_simulator_tab.py	no	-	no	no	no	-	no	React side
src/gui/risk_tab.py	risk_tab.js	yes	yes	yes	no	yes	no	shelved
src/gui/settings_dialog.py	settings_dialog.js	yes	yes	yes	no	yes	yes	in scope
src/gui/shared_testnet.py	shared_testnet.js	no	yes	yes	no	yes	no	shelved
src/gui/simulator_tab.py	simulator_tab.js	yes	yes	yes	yes	yes	yes	in scope
src/gui/simulator_tab/fleet/fleet_replay_panel.py	fleet_replay_panel.js	yes	yes	yes	no	yes	yes	shelved
src/gui/simulator_tab/fleet/sim_visuals.py	sim_visuals.js	yes	yes	yes	no	yes	yes	shelved
src/gui/simulator_tab/nuclear_mode_panel.py	nuclear_mode_panel.js	yes	yes	yes	no	yes	yes	shelved
src/gui/simulator_tab/sim_stat_strip.py	sim_stat_strip.js	yes	yes	yes	no	yes	yes	shelved
src/gui/simulator_tab/simulator_tab.py	simulator_tab.js	yes	yes	yes	yes	yes	yes	shelved
src/gui/start_all_progress_dialog.py	start_all_progress.js	yes	yes	yes	no	yes	yes	in scope
src/gui/stock_main_window.py	no	-	yes	no	no	-	no	shelved
src/gui/testnet_tab.py	testnet_tab.js	yes	yes	yes	no	yes	no	shelved

Qt file	React module	Uses React	Bridge	Manifest	Registers in Electron	Ships in the build	RENDER S	Scope
src/gui/tradingview_chart.py	tradingview_chart.js	yes	yes	yes	no	yes	no	shelved
src/gui/usb_auth_widget.py	usb_auth_widget.js	yes	yes	yes	no	yes	no	shelved
src/gui/visualizer/bot_node.py	bot_node.js	yes	yes	yes	yes	yes	yes	in scope
src/gui/visualizer/quick_routing.py	quick_routing.js	yes	yes	yes	yes	yes	yes	in scope
src/gui/visualizer/themes.py	visualizer_themes.js	-	yes	yes	yes	yes	yes	in scope
src/gui/visualizer/wire_canvas.py	wire_canvas.js	yes	yes	yes	yes	yes	yes	in scope
src/gui/widgets/__init__.py	no	-	yes	no	no	-	no	not a screen
src/gui/widgets/api_tester_tab.py	api_tester_tab.js	yes	yes	yes	no	yes	no	shelved
src/gui/widgets/bot_selection.py	bot_selection.js	yes	yes	yes	no	yes	no	not a screen
src/gui/widgets/bot_status_table.py	bot_status_table.js	yes	yes	yes	no	yes	yes	in scope
src/gui/widgets/capital_registry_panel.py	no	-	no	no	no	-	no	shelved
src/gui/widgets/dashboard_stat_card.py	dashboard_stat_card.js	yes	yes	yes	no	yes	yes	in scope
src/gui/widgets/exchange_tab.py	exchange_tab.js	yes	yes	yes	no	yes	yes	in scope
src/gui/widgets/extractor_bot_table.py	extractor_bot_table.js	yes	yes	yes	no	yes	yes	in scope
src/gui/widgets/notification_spool.py	notification_spool.js	yes	yes	yes	no	yes	no	shelved
src/gui/widgets/privacy_dot.py	privacy_dot.js	yes	yes	yes	no	yes	yes	in scope
src/gui/widgets/pulse_manager.py	pulse_manager.js	yes	yes	yes	no	yes	no	shelved
src/gui/widgets/spendable_profits.py	spendable_profits.js	yes	yes	yes	no	yes	yes	in scope
src/gui/widgets/status_log.py	status_log.js	yes	yes	yes	no	yes	yes	in scope
src/gui/widgets/trade_charts_tab.py	trade_charts_tab.js	yes	yes	yes	yes	yes	yes	in scope

The Simulator rebuild removed the files above; they are not in the tree.

Totals across the 76 rows above, measured on 5 September 2026:

React module	62
Uses React	60
Bridge	72
Manifest	62
Registers in Electron	18
Ships in the build	59
RENDERS	48
RENDERS, in scope	43 of 43
out of scope	37

Four columns are all but complete. RENDERS, the column that is the item, is not.

Re-measured on 6 September 2026, counted off the 76 rows above:

React module	55
Uses React	53
Bridge	67
Manifest	55
Registers in Electron	15
Ships in the build	55
RENDERS	29

Only the last column moved. The Settings row already carried a React module, a bridge method and a manifest entry; what it gained is the screen.

Re-measured later on 6 September 2026, counted off the same 76 rows:

React module	55
Uses React	53
Bridge	67
Manifest	55
Registers in Electron	15
Ships in the build	55
RENDERS	30
RENDERS, in scope	30 of 43
out of scope	33

The Create Auto Trader wizard is the row that moved. `bot_wizard.js` draws it into a space the React exchange screen keeps once + New Bot is pressed, so the wizard is a screen and no longer a module with nowhere to draw.

Counted once more the same day, after the Live Bot Settings window went over:

React module	55
Uses React	53
Bridge	67
Manifest	55
Registers in Electron	15
Ships in the build	55
RENDERS	38
RENDERS, in scope	38 of 43
out of scope	33

Eight rows moved together: the window a bot row opens, six of the files its tabs are built from, and the delegate that draws the fold-tranche row borders. `fold_tokens.py` keeps its dash. Its colours are on the drawn rows, and the file names no screen of its own.

When that count was taken the Paper Trader, Proof of Accumulation and System Status tabs had no Qt module to replace and no React module to replace it with, so the table carried no row for them. `empty_tabs.py` drew all three empty, and its own row read builds the shell. Issue #19 has since built the Paper tab, which now has a Qt module and a React module of its own.

In development.

That is no longer the state. Each of the three has a renderer module in `src/gui/web`, a bridge method and a manifest entry, and `empty_tabs.py` carries the row for all three. Its Scope cell reads `in scope` and its RENDERS cell reads yes. The section below says what each tab draws.

Counted once more on 6 September 2026, after the header strip and the Start All dialog went over:

React module	58
Uses React	56
Bridge	69
Manifest	58
Registers in Electron	15
Ships in the build	58
RENDERS	42
RENDERS, in scope	42 of 43
out of scope	33

Six rows moved. Three are the header strip parts. One is the Start All progress dialog. One is the Live Bot Settings screen tab, where the React module cell was the wrong one and now names the module that draws it. The last is the buy confirmation dialog, which gained a module, a bridge method and a build entry and still reads no under RENDERS.

Counted again on 6 September 2026, after the buy confirmation dialog went over:

React module	58
Uses React	56
Bridge	69
Manifest	58
Registers in Electron	15
Ships in the build	58
RENDERS	43
RENDERS, in scope	43 of 43
out of scope	33

One row moved. It is the buy confirmation dialog, and it was the last in-scope row reading no. The count comes from `tools/conversion_table.py`, which reads the cells of the table above.

Counted again on 6 September 2026, after the window's own tab bar went over:

React module	59
Uses React	57
Bridge	69
Manifest	59
Registers in Electron	15
Ships in the build	58
RENDERS	44
RENDERS, in scope	44 of 44
out of scope	32

One row moved, and it is `main_window.py`. The row read `builds the shell` because the module filled the tab bar rather than sitting in it; the bar it fills is now React, so the row is in scope and it renders. `Ships in the build` stays no: the newest bundle under `dist` carries 70 renderer modules and `main_window.js` is not one of them, because the module is newer than that build.

Counted again on 6 September 2026, after the three unbuilt tabs went over:

React module	60
Uses React	58
Bridge	70
Manifest	60
Registers in Electron	16
Ships in the build	58
RENDERS	45
RENDERS, in scope	45 of 45
out of scope	31

One row moved, and it is `empty_tabs.py`. It is the second row that read `builds the shell`, and it is the only row naming three React modules: the Paper Trader, System Status and Proof of Accumulation skeletons each have a renderer module, a bridge method and a manifest entry, and all three register a panel with the Electron shell. `Ships in the build` reads no for the same reason as the row above: the newest bundle under `dist` carries 70 renderer modules and these three are not among them.

No row now reads `builds the shell`.

Counted again on 6 September 2026, after the Notifications and Alerts tab went over:

React module	60
Uses React	58
Bridge	70
Manifest	60
Registers in Electron	16
Ships in the build	58
RENDERS	46
RENDERS, in scope	45 of 45
out of scope	31

One row moved, and it is a `alerts_tab.py`. Its Scope cell still reads shelved, because nothing the running window builds reaches the screen. `variant_surface` now holds the row under the name `ALERTS`, so the React build makes `AlertsReactTab` and the Qt build makes `AlertsTab` unchanged. The React tab draws the whole screen in one web view from `src/gui/web/alerts_tab.js`, and `AlertsTab.refresh`, `AlertsTab._save_config`, `AlertsTab._test_telegram` and `AlertsTab._acknowledge_all` are the same methods on both sides.

The window's own tab bar

The row of tab names across the top of the window is drawn by React under the React build. `MainWindow._setup_ui` no longer builds a `QTabWidget` by name. It asks the variant seam for the tab book, so one builder gives the operator the Qt bar he runs today or the React one.

```
self._main_tabs = _main_tab_book_class>()
self._main_tabs.setMovable(True)
```

Under the Qt build the seam answers with the widget the operator runs today and nothing changes. Under the React build it answers with a book that keeps that widget as the stack of pages, hides its own bar, and puts a web view above it. Every call the window makes is answered over the widget behind the bar, so the tab order and the pages are the same objects on both sides.

```
def addTab(self, widget, label: str) -> int:
    """Add ``widget`` under ``label`` and redraw the bar."""
    index = self._book.addTab(widget, label)
    self._push_tabs()
    return index
```

The bar itself is the Electron shell's navigation, run inside the window. The page is built from the shell's own scripts, read off disk, and the buttons are drawn by one React module both sides reach. Neither host holds a bar of its own, so the two cannot draw a different one.

```
#: The shell's own navigation, read from ``desktop/renderer`` and inlined.
SHELL_SCRIPTS: tuple[str, ...] = ("panel_host.js", "tab_bar.js")
```

The shell's navigation asks that React module for the buttons and passes it the tab names, the selected one, and what to do with a press and a drop. It holds no button of its own.


```
drawn.renderTabBar(bar, {
    names: found,
    selected: current,
    onSelect: select,
    onMove: onMove
});
```

A press on a button is reported back to Python, which moves the real tab book to that tab. Dragging a tab onto another reports the two slots the same way, and the same handler hands them to the hidden bar, so a drag reorders the pages as it did before.

```
move = request.get("move") or {}
if "from" in move and "to" in move:
    self._book.tabBar().moveTab(int(move["from"]), int(move["to"]))
```

The menu bar, the window title, the window icon and the status bar are still Qt. `MainWindow._setup_menu` builds the four menus and `_setup_status_bar` builds the status line; neither was changed, and every menu action the window offered before it is on it now.

The three tabs with nothing behind them yet

Paper Trader, System Status and Proof of Accumulation were named on the tab bar and were not built when that count was taken. Each one drew a heading, a sentence saying the tab is not built, and the issue that carries the build-out. Every word of it came from that tab's own Python surface, so the page invented nothing. Two of the three are still empty; issue #19 built the Paper tab.

```
return {
    "accessible_name": HEADING,
    "built": BUILT,
    "heading": HEADING,
    "issue": ISSUE,
    "issue_text": ISSUE_TEXT,
    "method": METHOD,
    "state_text": STATE_TEXT,
}
```

Under the Qt build the three sentences are Qt labels, as before. Under the React build the same view model goes to the tab's renderer module, which draws it in a web view. The builder asks the variant seam which of the two to make, and it hands both the same values.

```
model = surface.view_model({})
panel = _empty_tab_class()(model)
```

Neither side holds a colour or a size of its own. `SKIN` names the ground, the two text colours, the two text sizes, the padding and the gap; the Qt style sheets are built from it and the page reads it as custom properties.

```
SKIN = {
    "--empty-tab-ground": ds.SURFACE_0,
    "--empty-tab-heading-colour": ds.TEXT_MAX,
    "--empty-tab-body-colour": ds.TEXT_EMPTY_STATE,
```

The page runs the Electron shell's `panel_host.js`, so the module is drawn the same way the shell draws it: by name, into one host element, with the reason written onto the page when it does not draw.

The header strip

The strip and the five counters above the tab row are no longer built by name.

`HeaderStripMixin._build_header_strip` asks the variant seam for each class, so one builder draws the Qt widgets or the React pages.

```
self._spendable_widget = _spendable_profits_class()()
top_row.addWidget(self._spendable_widget, stretch=3)

card_class = _stat_card_class()
self._stat_scrummed = card_class("Scrummed", "$0.00")
```

The spendable strip is drawn by `SpendableProfitsReact`, which hosts the strip module in one web view. It answers the two calls the main window makes, and writes both into the same view model the Qt strip is described by.

```
def update_profits(self, data: dict) -> None:
    self._model.update_profits(data)
    self._push()
```

Each column carries a privacy dot drawn by the dot module. A press on one runs the model in `privacy_dot_surface`, which flips the field in the process privacy registry and repaints the amount as four stars.

```
def clicked(self) -> None:
    self.calls.append(CLICKED)
    try:
        registry = get_privacy_mask_registry()
        registry.set_masked(self._field_id, not registry.is_masked(self._field_id))
    except Exception:
        self.calls.append(TOGGLE_FAILED)
    self.refresh()
```

One card is drawn by `StatCardReact`. A press on the card emits the card's one signal, and only while the header strip has armed it.

The glyph, the tooltip and the skin of a dot now have one home. Both header surfaces return what the dot surface paints, so the card and the strip cannot drift apart.

```
def privacy_dot(field_id: str, masked: Optional[bool] = None) -> dict:
    painted = privacy_dot_surface.dot_view(field_id, masked)
    return {name: painted[name] for name in _DOT_FIELDS}
```

The Start All progress dialog

The dialog the Start All button opens is drawn by `StartAllProgressReactDialog`. It replaces only the method that builds the controls. The progress handling, the Cancel press and the unsubscribe on close are the same code the Qt dialog runs.

```
def store(self) -> surface.StartAllProgressModel:
    return self._model
```

Every control it needs is a small holder over that store. The list answers `addItem` and `count`; the buttons answer `setEnabled`. Both build sites, in the main window and in the launcher, now ask the variant seam for the class.

The buy confirmation dialog

This is the one in-scope row that still reads no, and the reason is reachability rather than missing work.

The main window builds the broker in this file. Nothing calls the broker's request method, which is the only sender of the signal that opens the dialog, so the running window has no path to the screen.

```
async def request_confirmation(
    self,
    *,
    bot_id: str,
    symbol: str,
    reason: str,
```

Measured by walking every call in `src` and `main.py`: zero call sites, and no import of that name under another. The module, the bridge method and the build entry are all present, so the screen is one caller away.

Re-measured on 6 September 2026, by the same walk over 354 modules: still zero call sites. That count decides what the operator can reach, and it did not move. The row's `RENDERS` cell moved for a different measurement — the class the broker raises.

The dialog the broker raises is now `BuyConfirmationReactDialog`. The broker asks the variant seam for that class, so the React build opens the page and `ACERVATOR_VARIANT=qt` opens the same Qt widgets as before.

```
def dialog_class() -> type:
    """The confirmation dialog class the running build variant draws."""
    from .variant_surface import BUY_CONFIRMATION, surface_class

    return surface_class(BUY_CONFIRMATION)
```

Every figure the page shows is written in Python. `details_text` in `buy_confirmation_surface` writes the seven detail rows. The Qt label is set from it, and the browser draws the same seven lines sliced into their

values. No arithmetic runs in the page.

```
def after_buy_usd(holdings_before: float, price: float, cost_usd: float) -> float:
    """Dollar value the position reaches once this buy fills."""
    return holdings_usd(holdings_before, price) + cost_usd
```

A press on the page names a button. Python turns that name into an answer and accepts the dialog through the same `_answer` the Qt buttons call. The page places no order and decides nothing.

```
def run_action(self, name: str) -> bool:
    answer = surface.BUTTON_ANSWERS.get(name)
    if answer is None:
        return False
    self._answer(answer)
    return True
```

A window closed without a press is still no. A request nobody answers inside sixty seconds is refused too.

The gap

Issue #128 says this item replaces the Qt screens with React. Measured against that sentence:

screens the running window builds	52
screens that draw React	1
screens that draw in the shell only	2
tabs drawn by React	0

The renderer manifest names 70 modules. The application the operator runs hosts no manifest, and loads exactly one of those modules: the History table's, which one Qt widget puts on a page of its own. The shell that does host the manifest draws two panels and offers no way to reach a third.

One call site carries the whole difference between the React build and the Qt build.

`src/gui/history_table_variant.py` holds the only call to the variant resolver in the tree, and every other tab builds the same Qt widget either way.

`src/gui/history_table_variant.py` — `history_table_class`

```
chosen = resolve_variant() if variant is None else variant
if chosen == QT:
    from .history_qt_table import HistoryQtTable

    return HistoryQtTable
from .react_history_panel import HistoryWebTable

return HistoryWebTable
```

Why the count read as near-complete

Two instruments report on this work, and both answer a narrower question than the item asks.

The progress tool marks a screen paired when a bridge method reaches a renderer module the manifest names. A manifest entry declares that the shell would load the file. It does not put a screen in front of the operator.

tools/conversion_state.py — served_methods, the pairing rule

```
def served_methods(root: pathlib.Path) -> dict[str, str]:
    """Bridge method to the renderer module that speaks it, loaded ones only.

    A method absent from `registered_surfaces` is left out of the result.
    """
    loaded = renderer_modules(root)
```

The parity tests compare a Qt view model against the React view model of the same screen. Both sides are plain Python data and neither has to be displayed. A passing test proves the two descriptions agree with each other. It does not prove either one is on screen.

parity_pins reads only a parity test that imports exactly one surface module and exactly one Qt module. It collects the names on both sides and pairs them.

tools/conversion_state.py — parity_pins, what a pin reads

```
def parity_pins(root: pathlib.Path) -> dict[str, set[str]]:
    qt_by_name = {dotted_name(root, path) for path in qt_modules(root)}
    surfaces = surface_methods(root)
    pins: dict[str, set[str]] = {}
```

Both readings were true. Neither measured the item, and the RENDERS column is the one that does.

Not reached from a live tab

These Qt modules are not built by the live window or by any of its tab builders. They are outside this conversion. One row is a package marker rather than a screen, and it stays where it is.

Qt module	Why it is out of scope
alerts_tab.py	Not reachable from a live tab
analytics_tab.py	Not reachable from a live tab
audio_suite.py	Not reachable from a live tab
competition_tab.py	Not reachable from a live tab
init_wizard.py	Not reachable from a live tab
journal_tab.py	Not reachable from a live tab
launcher.py	Not reachable from a live tab
live_bot_window.py	Not reachable from a live tab
risk_tab.py	Not reachable from a live tab
stock_main_window.py	Not reachable from a live tab
testnet_tab.py	Not reachable from a live tab
usb_auth_widget.py	Not reachable from a live tab

Qt module	Why it is out of scope
widgets/__init__.py	Package marker, no screen
widgets/api_tester_tab.py	Not reachable from a live tab
widgets/capital_registry_panel.py	Not reachable from a live tab
widgets/notification_spool.py	Not reachable from a live tab
widgets/pulse_manager.py	Not reachable from a live tab

The rows in that table are inside this item. The operator's 6 September 2026 directive puts the whole interface in scope, so a screen the window does not build is converted like any other. A converted row keeps shelved in the Scope cell of the table above, because that cell answers reachability. Its RENDERS cell reads yes once variant_surface holds the row and the React class draws the screen.

The reachability walk is rooted at the window and its tab builders, and an edge is a use rather than an import: a call, a base class, or a returned class. An import alone is not an edge, so the re-export block in the main window leaves a widget unreached. Ten screens known to be built and eleven known to be dead were run through the walk as a control, and all twenty-one came back on the expected side.

Detail on each live screen sits one file down, in 08-tabs/README.md.

Subsystem Detail

Reference. One file per subsystem tab, written from the source. Each file names the module that implements the screen, the symbols inside it, the bridge method that serves its renderer, and the screen's current state.

The parent section is 08-tabs.md, which carries the manual's own text for each of these screens. The rows below run in the order the tab list in 04-manual-parts.md sets, which is the order 08-tabs.md runs its sections in.

Contents

File	Covers	State
portfolio-panels.md	The header strip: spendable columns, five counter cards, privacy dots	Live
simulator.md	Validation, Back Test, Portfolio Battery, the gate-latch criterion	Live, rebuilt under issue #117
paper-trader.md	The live feed, the fake balance, the paper log	Live — issue #19
proof-of-accumulation.md	Identity, Merkle log, competitions, ACRV, the local chain	Skeleton tab, engine only — issue #147
market-inspector.md	Higher-timeframe scanner, opposing pairs, topology proposals, adopt	Live
bot-swarm.md	Nodes, Smart Wires, wire credits, the fold-tranche book	Live
asset-charts.md	One candlestick panel per traded symbol	Live
history.md	Venue trade history, grading, gate analysis	Live
console.md	Python log tail and the emitter signal stream	Live
system-status.md	Emitter Network and Watchdog, the two halves that run today	Skeleton tab — issue #34
settings.md	The User page, the Exchanges page, and what each of the eleven pages persists	Live
promotion-pipeline.md	How a strategy earns its way to real money	Four of four steps built

The live tab set

One list names the ten tabs the window builds, and each one is moved into that position after the builders run. The last two are empty.

src/gui/main_tabs/main_window_surface.py — CANONICAL_TAB_ORDER

```
CANONICAL_TAB_ORDER = (  
    SIM_TAB,  
    PAPER_TAB,  
    LIVE_TAB,  
    CHARTS_TAB,  
    INSPECTOR_TAB,  
    SWARM_TAB,  
    ACCUMULATION_TAB,  
    HISTORY_TAB,  
    STATUS_TAB,  
    CONSOLE_TAB,  
)
```

The Live tab has its own section: 06-trading-tab.md. The Indicator Voting Panel has 07-indicators.md.

Every screen the window does not build gets a sentinel instead of a widget, so a legacy reader gets that sentinel rather than an error. The list covers the Competition and Local Testnet tabs, the analytics, risk, journal,

alerts and audio pages, and the retired Paper Trader widget, which is not the Paper tab issue #19 built.

src/gui/main_tabs/retired_tabs.py —
RetiredTabsMixin._install_retired_tab_sentinels

```
def _install_retired_tab_sentinels(self) -> None:
    """Assign None to ``_paper_trader``, ``_competition_tab`` and the rest.

    ``main_window`` tests ``_analytics_tab``, ``_risk_tab``,
    ``_journal_tab`` and ``_alerts_tab`` before refreshing them.
    """
```

The underlying engines behind several of those keep running; only the surface is gone.

How a screen reaches its renderer

One registry maps a method name to a view-model function. A surface is a plain function that takes the request's parameters and returns a serialisable result, and a screen is reachable exactly when its method appears in that registry.

src/core/desktop_bridge.py — build_registry

```
def build_registry() -> Dict[str, Handler]:
    """Return the methods the frontend may call, keyed by method name.

    This is the wiring point for the whole frontend: a surface is
    reachable exactly when it appears here. Surfaces are imported inside
    the function so that the transport above carries no dependency on any
    one domain package.
    """
```

The transport is one JSON object per line over the pipe the parent process already owns: no socket and no port.

Each surface module holds the screen as data. Nothing in one imports Qt, which is why the same view model serves the Qt tab and the renderer without a second description of the screen existing.

Back to the manual index.

Portfolio Information Panels

Reference. The strip above the tab bar, present on every tab except the two isolated ones.

What builds it

One method builds the strip. It makes the window's central widget, lays a single row across the top of it, and hands back the layout the tab row goes into. That row holds the five spendable columns on the left and the five counter cards on the right, and it does not move as you change tabs.

src/gui/main_tabs/header_strip.py — HeaderStripMixin._build_header_strip


```
self._spendable_widget = SpendableProfitsWidget()
top_row.addWidget(self._spendable_widget, stretch=3)
```

Left: the spendable columns

Five labelled columns fill the left half, divided by thin vertical rules.

Column	Meaning
SPENDABLE	Cash the exchange reports, across the bots sharing one wallet
REALISED	Realised profit and loss
LOCKED	Value the open positions hold
MATURE	Wire profit old enough to move
EXCH	Connected exchange count

The widget builds four of the five from one list, and each entry carries the label the operator reads and the payload key that fills it.

src/gui/widgets/spendable_profits.py — `SpendableProfitsWidget.__init__`

```
for label_text, key in [
    ("REALISED", "total_realised"),
    ("LOCKED", "locked"),
    ("MATURE", "mature"),
    ("EXCH", "exchanges"),
]:
```

One payload draws all five. A value the payload leaves out draws an em dash rather than a zero, which keeps "no reading" and "a reading of nothing" apart. That is the difference between a column with nothing behind it and a column reporting a genuine zero, and on this strip it matters: two of the five have nothing behind them today.

src/gui/widgets/spendable_profits.py — `SpendableProfitsWidget._money_text`

```
@staticmethod
def _money_text(value) -> str:
    """Render one amount as money text, or as the empty marker."""
    amount = SpendableProfitsWidget._amount_of(value)
    if amount is None:
        return "-"
    return f"${amount:,.2f}"
```

REALISED and MATURE are the two. Both are handed a literal absence at the one call site that fills the strip, so both draw the marker on every tick. The Trading tab section carries the proposal for the first of them: 06-trading-tab.md.

Both now carry a figure, and the figure comes from the exchange. Realised is the venue's own matched profit and loss across the fleet. Mature is the profit held in positions worth more than three times what they cost, which is the growth threshold the platform applies everywhere it decides maturity.

src/gui/main_tabs/header_strip_surface.py — profits_payload

```
"total_realised": exchange_amount(data, "total_realized_exchange"),
"mature": exchange_amount(data, "total_mature_exchange"),
```

Absence is still a reading. When the venue has answered for no bot, both columns take nothing and draw the marker, and the mask leaves that marker alone. A hidden figure shows four asterisks; an empty column stays empty under Privacy Mode.

src/core/privacy_mask_registry.py — mask_or

```
text = str(value)
if field_id not in ALL_FIELD_IDS or text == ABSENT_TEXT:
    return text
```

Right: the five counter cards

Five cards close the row, and each one counts a single thing.

Card	What it counts
Scrummed	Cumulative USD sold across every bot since the process started
Folded	Cumulative USD bought across every bot since the process started
Trades	Executed buys and sells across the active bots
Bots	Bots in the RUNNING state
Errors	Errors across every bot since the last reset

Errors is the one card you can click. Arming it takes one call, and the click opens the rolling error buffer with its Reset button.

src/gui/main_tabs/header_strip.py — HeaderStripMixin._build_header_strip

```
self._stat_errors.set_clickable(True, "Click to open the error log.")
self._stat_errors.clicked.connect(self._show_error_log_dialog)
```

Issue #428 carries a disagreement between what the tooltips on these cards promise and what the cards draw.

A sixth card, P/L, is constructed and then hidden. Code that still writes to it keeps working and nothing appears on screen.

src/gui/main_tabs/header_strip.py — HeaderStripMixin._build_header_strip

```
self._stat_pnl = StatCard("P/L", "$0.00")
self._stat_pnl.setVisible(False)
```

The mode button on the right ends the row. `_toggle_trading_mode` swaps the window between the crypto and stock layers.

Privacy

Every field on the strip carries its own privacy dot. Attaching one registers the field, and from then on the card draws its value through the mask.

src/gui/main_tabs/header_strip.py — HeaderStripMixin._build_header_strip

```
self._stat_scrummed.attach_privacy_dot("counter.scrummed")
self._stat_folded.attach_privacy_dot("counter.folded")
self._stat_trades.attach_privacy_dot("counter.trades")
self._stat_bots.attach_privacy_dot("counter.bots")
self._stat_errors.attach_privacy_dot("counter.errors")
```

The registry behind those field ids is shared. Hide a field here and the same field hides on the Bot Swarm tab, because both surfaces ask the one registry.

src/core/privacy_mask_registry.py — mask_or

```
def mask_or(value, field_id: str, mask: str = "****") -> str:
    """Return ``mask`` when ``field_id`` is masked, else ``str(value)``.

    A ``field_id`` outside ``ALL_FIELD_IDS`` never masks, which covers every
    ``TA_FIELD_IDS_EXCLUDED`` entry.
    """
```

Where the numbers come from

One aggregate call fills the whole strip, once a tick. Every card and every column is written from that single answer, so two cards cannot disagree about the same fleet.

src/gui/main_window.py — _refresh_dashboard

```
agg = self._bot_manager.get_aggregate_stats()
_scr = float(agg.get("total_scrummed_usd", 0.0) or 0.0)
_fld = float(agg.get("total_folded_usd", 0.0) or 0.0)
self._stat_scrummed.set_value(f"${_scr:,.2f}")
self._stat_folded.set_value(f"${_fld:,.2f}")
```

The aggregate itself is FleetAggregationMixin.get_aggregate_stats in src/trading/container/aggregation.py.

Hiding

Two screens are isolated from live trading, and on those two the strip hides itself. The absent strip is the signal: no live numbers are on this screen.

src/gui/main_window.py — _on_main_tab_changed

```

isolated_tabs = set(ISOLATED_TABS)
container = getattr(self, "_header_strip_container", None)
if container is not None:
    container.setVisible(tab_name not in isolated_tabs)

```

The window surface carries the same pair as a constant, and issue #450 renamed both labels.

src/gui/main_tabs/main_window_surface.py — ISOLATED_TABS

```

ISOLATED_TABS = (SIM_TAB, PAPER_TAB)

```

The Simulator draws its own strip in place of this one. SimStatStrip in src/gui/simulator_tab/sim_stat_strip.py mirrors these ten fields against sim balances. The Simulator rebuild removed this file; it is not in the tree.

Bridge

The strip answers one bridge method, and one renderer module draws it.

src/gui/main_tabs/header_strip_surface.py — METHOD

```

METHOD = "header.strip"

```

The renderer module is header_strip.js.

Back to the subsystem index.

Simulator Tab

Reference. The second step of the promotion pipeline: the live fleet, replayed against stored history. Issue #117 rebuilt the screen, and it carries three modes: Validation, Back Test and Portfolio Battery.

The clone the tab draws now

The Sim tab is a clone of the Trading tab, and its data source is the Stone Tablets on disk. It carries the bot list, the Indicator Voting Panel, and a second layer holding the VWAP window over the tablet playback window. One button flips the panel area between those two layers.

src/gui/main_tabs/simulator_tab_surface.py — the two layers and the button

```

LAYER_INDICATORS = "indicators"
LAYER_PLAYBACK = "playback"
LAYERS = (LAYER_INDICATORS, LAYER_PLAYBACK)

FLIP_BUTTON_TEXT = {
    LAYER_INDICATORS: "Show Playback",
    LAYER_PLAYBACK: "Show Indicators",
}

```

One model is built from the tablet, and both builds draw it.

Diagram: flowchart, left to right

- RA-StoneTablet files -> **TabletSource**
- TabletSource -> **simulator_tab_surface**
- simulator_tab_surface -> **SimulatorTabQt**
- simulator_tab_surface -> **simulator_tab.js**

It receives and asks. It never sends.

The Simulator's whole data path is one class that reads tablet files. It holds no venue and defines no write. It answers five names and refuses every other name itself, so a send cannot be expressed through it.

src/simulator/tablet_source.py — the refusal

```
def __getattr__(self, name: str):
    """Refuse every name outside ``READ_NAMES``."""
    raise SendRefused(
        f"TabletSource answers {READ_NAMES} and cannot {name!r}. "
        "The Simulator receives and asks; it sends nothing."
    )
```

Asked for nine venue calls — an order, a market buy, a cancellation, an edit, a withdrawal, a leverage change, a transfer, a tablet write and a save — it refused all nine and answered every read.

The columns are the Trading tab's own

The bot list draws the ten columns the live table draws, and the same two fixed widths, because the surface imports them rather than restating them. A column added to the live table appears here with no second edit.

```
from .bot_status_table_surface import COLUMN_LABELS, FIXED_WIDTHS
```

No simulated fleet exists yet, so the list holds no rows and says why. Import Live Fleet and Generate From YTD are what fill it, and both are later units.

One candle window, on both sides

The live engine asks the venue for one hundred candles. The Simulator reads the last hundred rows off the tablet, so the indicator window is one number on both sides rather than two.

```
#: The candle window live reads. ``ScrummingBot`` asks ``get_ohlc`` for 100,
#: and the Simulator reads the same count off the tablet.
WINDOW_CANDLES = 100
```

The live engine now receives that hundred, and the newest row in it is the bar that just closed. The connector sends the count in the exchange library's count slot. Until this change it sent the count in the start-time slot and left the count slot empty, so the venue answered with a page of its own and the library kept the oldest 300 rows of it. Every indicator then read a window ending 50 five-minute bars behind the price the gate compared it against. That is four hours and ten minutes.

src/exchange/ccxt_connector.py — four arguments, each in its own slot

```
data = await self._call_sync(
    self._ex.fetch_ohlcv,
    symbol,
    timeframe,
    None if since is None else int(since),
    int(limit),
)
```

The library's own slice was driven over 1,215 recorded five-minute pages. The window ended 50 bars early on every page before the change and on none after it. On 162 rows of the recorded gate log, the band latch differs between the stale window and the current one on 88.

The VWAP window

VWAP is the published cumulative figure: typical price times volume, running, divided by running volume, where typical price is the average of the high, the low and the close. A point with no volume behind it yet carries nothing rather than a number.

```
def vwap_series(candles: Sequence[Sequence[float]]) -> list[Optional[float]]:
    """Cumulative ``sum(typical_price * volume) / sum(volume)`` per row.

    A row whose cumulative volume is still zero carries None.
    """
```

The two strips that are not copied

The crypto news ticker and the data pool line are not on this tab, and their code is not carried into it. A live news feed and a live cache-health line describe nothing a tablet reader does. The rows they held keep their height and hold nothing, and that space is where Import Live Fleet and Generate From YTD go.

```
#: The rows the two strips held on the Trading tab, and the height each keeps.
RESERVED_ROWS: tuple[dict[str, Any], ...] = (
    {"name": "news_ticker_row", "height_px": 24},
    {"name": "data_pool_row", "height_px": 18},
)
```

With no tablet on disk

The tab draws an empty state and names what is missing. The panel reads No TA data — No Stone Tablet on disk., the selector holds nothing, and both windows hold zero points. Nothing is invented in place of the missing tape.

The YTD trade files

The Simulator's second data source is the operator's own trade record, kept on disk under the exchange history bucket. One file holds one exchange, one symbol and one year. Each file names its exchange inside itself, not only in its filename, so a file that is moved or renamed still says where its trades came from.

src/exchange/ytd_trade_store.py — what one row holds

```
@dataclass
class YtdTrade:
    """One fill: ``id``, ``ts_ms``, ``side``, ``amount``, ``price``, ``cost``
    and ``fee``."""

    id: str
    ts_ms: int
    side: str
    amount: float
    price: float
    cost: float
    fee: float
    fee_currency: str
```

Beside the rows, each file carries a schema version, the time of the import, and one entry per export that contributed rows. That entry names the export file and its digest, so a file that grew over two imports records both. A MANIFEST beside the files indexes every one of them with its row count and its date span.

src/exchange/ytd_trade_store.py — the provenance on each file

```
@dataclass
class ImportSource:
    """One import that contributed rows: ``file``, ``sha256`` and ``rows_added``."""

    file: str
    sha256: str
    imported_at: str
    rows_added: int
```

The columns the import requires

The record arrives as a transactions CSV exported from the exchange. The import reads nine columns and refuses a file missing any one of them, naming which. Notes, sender address and recipient address are never carried in.

src/exchange/ytd_csv_import.py — the required columns

```
REQUIRED_COLUMNS: tuple[str, ...] = (
    COL_ID,
    COL_TIMESTAMP,
    COL_TYPE,
    COL_ASSET,
    COL_QUANTITY,
    COL_PRICE_CURRENCY,
    COL_PRICE,
    COL_SUBTOTAL,
    COL_FEES,
)
```

The transaction type decides the side. Buys and sells are kept; reward income, deposits and withdrawals are counted and dropped, because they are not trades and must never reach a validation run as though they were.

The asset and the price currency together give the pair. The quantity is negative on a sell in the export, so its sign is checked against the type and then the magnitude is stored.

A second import of an overlapping export adds only the rows whose id is new. A file that gains nothing is not rewritten at all.

A period with no rows is a gap

Where the export carries no trade for a symbol in a period the export covers, that period is written to a gap record. Nothing is invented to fill it.

src/exchange/ytd_trade_store.py — the gap record

```
@dataclass
class TradeGap:
    """One period between ``since_ms`` and ``until_ms`` that no row covers."""

    exchange_id: str
    symbol: str
    year: int
    since_ms: int
    until_ms: int
    reason: str
    checked_at: str
```

Reading the trade files

The Simulator reads these files through one class, on the same rule as the tablet reader. It answers six names and refuses every other name itself.

src/simulator/ytd_trade_source.py — the refusal

```
def __getattr__(self, name: str):
    """Refuse every name outside ``READ_NAMES``."""
    raise SendRefused(
        f"YtdTradeSource answers {READ_NAMES} and cannot {name!r}. "
        "The Simulator receives and asks; it sends nothing."
    )
```

What the import run measured

The operator's own export was read into a throwaway directory. The figures below come from that run.

export read	5,798 rows, 13 columns
kept	5,766 trades - 3,229 buys, 2,537 sells
dropped	32 - reward income 16, deposits 15, withdrawals 1
written	39 files, one per symbol, all of them 2026
range	2026-04-12 to 2026-09-07
gaps recorded	76
second import	0 rows added, 39 files byte-identical

What one run measured

Both builds were opened and read off the drawn window and the drawn page. The figures below come from that run.

```
tablet          XRP_1d_2026_coinbase, 250 candles, 2026-01-01 to 2026-09-07
window          100 candles
tablets listed  411
votes           5 bullish, 2 bearish, 5 neutral
Qt to React     32 of 32 values matched, with a tablet and again with none
```

What the tab holds today

The Simulator is removed. The tab is named Sim, it opens first on the bar, and it draws an empty panel with a heading and two sentences. Nothing behind it runs: no fleet, no replay, no practice venue and no Nuclear Mode. Every section below this one describes the screen that was removed, and is kept as the record of what the rebuild replaces.

The panel is the one the Paper, Status and Accumulation tabs already draw. Its words come from a view model, so the Qt build and the React build say the same thing.

```
METHOD = "simulator_tab.state"

HEADING = "Sim"
ISSUE = 117
BUILT = False
STATE_TEXT = "This tab is not built."
ISSUE_TEXT = f"Issue #{ISSUE} carries the build-out."
```

What builds it

The window constructs the tab and inserts it beside Trading. Three of the four wiring calls take a lambda, so the exchange connectors, the swarm and the Market Inspector proposals resolve when they are asked for rather than when the tab is built. The bot manager is handed over directly.

src/gui/main_tabs/simulator_tab.py — SimulatorTabMixin._build_simulator_tab The Simulator rebuild removed this file; it is not in the tree.

```
self._simulator = SimulatorTab()
# _reorder_main_tabs runs after every builder, so index 1 is not the final slot.
self._main_tabs.insertTab(1, self._simulator, "Simulator")
if hasattr(self._simulator, "set_connectors_getter"):
    self._simulator.set_connectors_getter(
        lambda: getattr(self, "_exchange_connectors", {})
    )
if hasattr(self._simulator, "set_bot_manager"):
    self._simulator.set_bot_manager(self._bot_manager)
```

The tab is isolated. The window's header strip hides while the Simulator is in front, and the tab draws its own strip of the same ten fields against sim balances.

src/gui/simulator_tab/sim_stat_strip.py — SimStatStrip.FIELDS The Simulator rebuild removed this file; it is not in the tree.

```
FIELDS = (  
    "Spendable",  
    "Realised",  
    "Locked",  
    "Mature",  
    "Exch",  
    "Scrummed",  
    "Folded",  
    "Trades",  
    "Bots",  
    "Errors",  
)
```

The Mode picker names three modes and drives two panels. Nuclear raises its own page. Validation and Looping Back Test both raise Fleet Replay.

src/gui/simulator_tab/simulator_tab.py — SimulatorTab._on_sim_mode_changed The Simulator rebuild removed this file; it is not in the tree.

```
key = self.sim_mode()  
stack = getattr(self, "_stack", None)  
if stack is not None:  
    # Validation and Looping both drive the fleet page.  
    stack.setCurrentIndex(1 if key == "nuclear" else 0)
```

The mode goes no further than the picker. The handler passes the key on only to a panel that carries a matching setter, and Fleet Replay carries none, so the two fleet modes run the same replay and the choice changes the hint line alone.

src/gui/simulator_tab/simulator_tab.py — the mode hand-off The Simulator rebuild removed this file; it is not in the tree.

```
panel = getattr(self, "fleet_replay", None)  
if panel is not None and hasattr(panel, "set_sim_mode"):  
    try:  
        panel.set_sim_mode(key)
```

Fleet Replay

FleetReplayPanel in src/gui/simulator_tab/fleet/fleet_replay_panel.py is the panel. Four steps run it. The Simulator rebuild removed this file; it is not in the tree.

Load. The panel builds one real bot for every saved config whose symbol has a Stone Tablet, and names the symbols it had to skip. The class body is live's own, so the sim runs live code against a stored tape rather than

a second implementation; the single sim-only branch gives each bot a private event bus, which is why a sim fill never reaches the live sound engine or the live logs.

src/gui/simulator_tab/fleet/fleet_replay_panel.py — _spawn_sim_fleet The Simulator rebuild removed this file; it is not in the tree.

```
def _spawn_sim_fleet(self) -> int:
    """Construct a real sim bot for every loaded config.

    Returns the number spawned. Reads the Stone Tablet registry
    through the same call Start Replay uses, rather than adding a
    second candle path.

    The controller is built but NOT started: the bots exist, hold
    their imported state and can be inspected, and the tape only
    advances when the operator presses Start Replay.
    """
```

Configs. The loader reads the operator's state file read-only and returns each bot's saved config.

load_smart_wires_from_state does the same for the topology, and summarize_loaded_configs reports what loaded. The fleet is never fabricated; it is whatever that file holds.

src/simulator/fleet/bot_state_loader.py — load_bot_configs_from_state The Simulator rebuild removed this file; it is not in the tree.

```
def load_bot_configs_from_state(
    path: Optional[Path] = None,
    mode_filter: Optional[str] = "scrumming",
) -> list[dict[str, Any]]:
```

Fetch. Fetch YTD calls the same history function the History tab's Refresh calls, so the two return the same trades. A replay still runs without it, and the panel then says on screen that the run is synthetic.

src/gui/simulator_tab/fleet/fleet_replay_panel.py — _on_fetch_ytd_clicked The Simulator rebuild removed this file; it is not in the tree.

```
def _on_fetch_ytd_clicked(self) -> None:
    """Call the same function the History tab calls.

    The History tab's Refresh dispatches
    ``fetch_all_history_chunked(bot_manager, since_ts)``. No custom
    enumeration, no exchange matching, no connector picking: if the
    History tab returns N trades, Fleet Replay returns the same N.
    """
```

Run. FleetReplayController in src/simulator/fleet/fleet_replay_controller.py drives it. The Simulator rebuild removed this file; it is not in the tree.

Piece	What it does
_build_sim	Assembles the bots and the tape

Piece	What it does
sim_bot_id, live_bot_id	Keep the two id spaces apart
_on_sim_trade	Records each fill

One assertion runs once, at the point where the bot set is final and before any of them can trade.

src/simulator/fleet/fleet_replay_controller.py — `_assert_capital_isolation` The Simulator rebuild removed this file; it is not in the tree.

```
def _assert_capital_isolation(self) -> None:
    """Every constructed sim bot must be off the live registry.

    The guards upstream -- the factory raising, `_instantiate_bot`
    refusing, `_crr()` returning None in sim mode -- each protect
    one path. This checks the outcome they exist to produce, once,
    at the point where the bot set is final and before any of them
    can trade. Aborts the run rather than let a replay write sim
    reservations into the operator's live capital state.
    """
```

The bots trade against a tape, not against a venue. TabletBackend implements the ccxt surface beneath a real connector, holds the seeded balances, sweeps resting orders, and owns the one clock every symbol advances on.

src/simulator/fleet/fleet_replay_controller.py — the tape the bots are handed The Simulator rebuild removed this file; it is not in the tree.

```
# TabletBackend implements the ccxt surface beneath a real CCXTConnector.
self._tape = TabletBackend(
    {
        sym: [list(r) for r in rows]
        for sym, rows in self._candles_by_symbol.items()
    },
    balances=dict(seed_by_quote),
    fee_rate_by_symbol=_fees,
)
_conn = CCXTConnector("coinbase")
_conn.attach_backend(self._tape)
self._exchange = _conn
```

Three more modules sit beside the controller and none of them runs a replay. The older fake exchange is built only by tests, the union clock is constructed only inside that exchange, and the candle series a replay builds is thrown away once its symbol names have been read.

src/simulator/fleet/sim_exchange.py — `FleetSimExchange` The Simulator rebuild removed this file; it is not in the tree.

```

"""Candle-driven fake exchange for Fleet Replay, over real symbols.

Serves per-symbol ``CandleSeries`` through the ``ExchangeInterface`` API,
so a bot trades BTC/USD or ETH/USD against stored candles instead of a
venue. ``NuclearSimExchange`` (``src/simulator/nuclear_sim_exchange.py``)
is the other simulated venue and serves synthetic TAPEA/TAPEB symbols.
The Simulator rebuild removed this file; it is not in the tree.

Reachability: nothing under ``src/`` constructs ``FleetSimExchange``.
``FleetReplayController`` imports only ``make_symbol_series_map`` from
this module and runs against ``TabletBackend``
(``src/exchange/tablet_backend.py``). The class itself is built by tests.

```

ReplayProgress carries candles played, trades fired and tick failures back to the panel's progress timer, and a finished run is written under the log root in the live schema.

src/trading/sim_run_log.py — SimRunLog The Simulator rebuild removed this file; it is not in the tree.

```

"""Persist a Fleet Replay run under ~/.acervator_logs/sim/ in the live schema."""

```

Two bot tables draw at once. The Trading tab's own table is mounted into the Simulator, and the fleet panel's three-column table is still built and still filled. Collapsing the two is the first of the seven pieces of issue #117.

src/gui/simulator_tab/fleet/fleet_replay_panel.py — FleetReplayPanel.COLUMNS The Simulator rebuild removed this file; it is not in the tree.

```

# Three columns. The gate row lives in GateStatusPanel, not here.
COLUMNS = ("Symbol", "Target USD", "Sim Trades")

```

The candles

Stone Tablets are the history and a replay only reads them. The registry stores one timeframe and derives every other from it.

src/trading/stone_tablets/registry.py — NATIVE_TIMEFRAME

```

NATIVE_TIMEFRAME: str = "5m"
"""Timeframe every tablet stores; ``_rollup`` derives all the others."""

```

Three functions in src/trading/stone_tablets/storage.py move tablets between disk and memory.

Function	What it does
read_tablet	Loads one tablet from disk
write_tablet	Writes one back
read_manifest	Carries the index of what exists

The tape a replay hands the bots does not roll up. Fleet Replay asks the registry for the native timeframe only, and the tape refuses any other unless the caller supplied that series when the tape was built, which Fleet

Replay does not do.

src/exchange/tablet_backend.py — TabletBackend.fetch_ohlcv

```
key = (str(symbol), tf)
rows = self._tf_rows.get(key)
if rows is None:
    raise ValueError(
        f"no {tf} series for {symbol!r}. Serving the native "
        f"{NATIVE_TIMEFRAME} series instead would make every "
        f"timeframe agree with itself; supply tf_rows[{key!r}].")
)
```

Portfolio Battery history

The thirty-five portfolios from the historical archive are now named in code, and their sixty-three symbols are what the new mode runs over. Each one carries its symbols, an equal weight for every symbol, and the archive it was read out of.

src/simulator/portfolios.py — one portfolio

```
"CRYPTO_BLUE": Portfolio(
    name="CRYPTO_BLUE",
    symbols=("BTC", "ETH", "BNB"),
    description="Large-cap crypto – institutional grade",
),
```

Their price history is real, and it is kept apart from the live fleet's. The tablets a replay reads sit under one root; the battery's sit under a second one, and a battery build never opens the first.

src/trading/stone_tablets/ra_paths.py — the second root

```
_RA_ROOT: Path = Path.home() / ".acervator_ra_tablets"
```

Two sources fill it and neither needs a key. Crypto arrives through the adapter the fleet already uses, driven by the venue's own public candle endpoint. Everything else arrives through a second adapter beside it.

src/trading/stone_tablets/ra_fetcher.py — the non-crypto adapter

```
class YahooChartAdapter(ExchangeAdapter):
    exchange_id = "yahoo"
    chunk_limit = RA_CHUNK_DAYS
    BASE_URL: str = "https://query1.finance.yahoo.com/v8/finance/chart"
    SOURCE: str = "yahoo_chart_v8_ONE_DAY_SPLIT_ADJUSTED"
```

Every tablet records where its numbers came from and when they were fetched, and the builder refuses to write one without both. A price with no source is what made the archive's own figures worthless.

src/trading/stone_tablets/ra_fetcher.py — the refusal

```

    resolved = source or str(getattr(adapter, "SOURCE", ""))
    if not resolved:
        raise ValueError(
            f"{type(adapter).__name__} carries no SOURCE; pass source= naming "
            f"the endpoint the candles come from. {adapter.exchange_id!r} is "
            f"an exchange id, not provenance."
        )

```

Where a source has nothing, nothing is written in its place. The missing days are recorded as missing, in their own file beside the tablets.

src/trading/stone_tablets/ra_fetcher.py — what a missing period records

```

@dataclass
class TabletGap:
    """One requested period a source returned no rows for."""

    asset: str
    exchange_id: str
    timeframe: str
    year: int
    since_ms: int
    until_ms: int
    reason: str
    checked_at: str

```

Nothing runs the bot logic over these tablets yet, and no screen shows them. That is the next unit.

In development.

Portfolio Battery coverage

Every symbol in every portfolio has real daily prices on disk. No portfolio names a period of its own, so all six archive periods apply to all thirty-five of them, and the span asked for is 2020 to 2026.

src/trading/stone_tablets/ra_import.py — the years asked for

```

def _archive_years() -> tuple[int, ...]:
    """Return every calendar year ``PERIODS`` touches, ascending."""
    years: set[int] = set()
    for start, end in PERIODS.values():
        years.update(range(int(start[:4]), int(end[:4]) + 1))
    return tuple(sorted(years))

```

A symbol reaches one source or the other by what it is. A coin the crypto venue never listed falls to the second source instead of being left empty, and every tablet records which one served it.

src/trading/stone_tablets/ra_import.py — the routing

```
def route_for(symbol: str) -> SymbolRoute:
    """Return ``symbol``'s route, crypto to Coinbase and everything else to Yahoo."""
    upper = symbol.upper()
    if is_crypto(upper):
        return SymbolRoute(upper, COINBASE_SOURCE, YAHOO_CRYPTO_SOURCE)
    return SymbolRoute(upper, YAHOO_SOURCE)
```

The import reads its own result back off disk and says what it holds. A count on its own would not be an answer, so the statement carries every source with its fetch times and every period no source served.

`python -m src.trading.stone_tablets.ra_import coverage` — the headline

```
SYMBOLS
  asked for ..... 63
  returned data ..... 60
  tablets ..... 411
  candles ..... 105336

SOURCES
  coinbase_exchange_candles_ONE_DAY: 71 tablets
  yahoo_chart_v8_ONE_DAY_SPLIT_ADJUSTED: 340 tablets
```

A share year is not a calendar year. The market shuts at weekends and on public holidays, and those days are recorded as missing rather than filled, so a share year holds about 250 days where a coin year holds every one.

`python -m src.trading.stone_tablets.ra_import coverage` — a share and a coin

```
SPY
  2022    251 rows  2022-01-03..2022-12-30
  2023    250 rows  2023-01-03..2023-12-29
BTC
  2022    365 rows  2022-01-01..2022-12-31
  2023    365 rows  2023-01-01..2023-12-31
```

A ticker that no longer reaches the company the archive meant is recorded as a finding, not as a failure. Three of the sixty-three return nothing at all, and what the endpoint answered is written down in place of a price.

`~/acervator_ra_tablets/GAPS.json` — a ticker that no longer trades

```
CCIV    2021 yahoo  [2021-01-01..2021-12-31] HTTPError: HTTP Error 404: Not Found
EXPR    2021 yahoo  [2021-01-01..2021-12-31] HTTPError: HTTP Error 404: Not Found
IPOF    2021 yahoo  [2021-01-01..2021-12-31] HTTPError: HTTP Error 404: Not Found
```

Three more symbols are short at one end. BBBY reaches a company first traded in July 2026 rather than the one the archive ran, and MATIC stops on 14 October 2025 because the coin was renamed.

`python -m src.trading.stone_tablets.ra_import coverage` — a symbol cut short


```
MATIC missing: 2026
2024      366 rows  2024-01-01..2024-12-31
2025      287 rows  2025-01-01..2025-10-14
```

Nothing runs the bot logic over these prices yet, and no screen shows them.

In development.

The validation criterion

The Simulator is validated when the gates latch identically on the same data. Not profit and loss, and not the trade count.

No module measures that. Nothing in the tree reads a sim gate latch and a live gate latch and compares the two.

In development. Validation Mode measures it now; the section at the end of this page states how.

What the screen does show is each bot's arm state while the tape runs. On every refresh tick the panel reads each bot's last gate state and paints a double-stacked row of lights, ten scrum then nine fold.

src/gui/simulator_tab/fleet/sim_visuals.py — GateLightsCell The Simulator rebuild removed this file; it is not in the tree.

```
class GateLightsCell(QWidget):
    """One linear labelled row of trading gates.

    ``_draw_bank`` paints the ten ``_GATE_ORDER_SCRUM`` lights then the
    nine ``_GATE_ORDER_FOLD`` lights, each below its own label.
    ``gate_light_color`` picks every colour, and ``update_gates`` sets the
    arm and blocker state ``paintEvent`` reads.
    """
```

That cell and the History table's gate cell read one vocabulary, so the two surfaces cannot drift apart on what a gate is called or on what blocked it.

src/trading/gate_vocabulary.py — the shared labels

```
"""gate_vocabulary.py -- the bot's gate labels, blocker map and light states.

Qt-free. Two renderers read it: the Simulator's ``GateLightsCell``
(``src/gui/simulator_tab/fleet/sim_visuals.py``) and the History table's
gate cell (``src/exchange/history_read_contract.py``). One vocabulary,
so a gate added on one surface cannot be missing from the other.
```

The Simulator rebuild removed this file; it is not in the tree.

After a run the panel matches each live trade to the nearest sim fill on the same symbol and side, and prints the result into the Performance log. That is a report on trade counts and timing. It is not the criterion above.

src/trading/stone_tablets/parity_harness.py — compare_trades

```
def compare_trades(
    live_trades: list[dict],
    sim_trades: list[Any],
    tolerance_s: float = DEFAULT_TOLERANCE_S,
    window_since_ts: float = 0.0,
    window_until_ts: float = 0.0,
) -> ParityReport:
```

The panel puts the honest label on screen. A run with no live trades loaded is comparable to nothing, and the panel says so rather than looking like a parity run.

src/gui/simulator_tab/fleet/fleet_replay_panel.py — `_note_parity_state` The Simulator rebuild removed this file; it is not in the tree.

```
def _note_parity_state(self) -> None:
    """Say so when the run cannot be compared to live.

    ``_ytd_trades`` being empty is the NORMAL condition, and the panel
    said nothing about it, so a synthetic run looked exactly like a
    parity run on screen.
    """
```

SimValidationGuard names five conditions under which a run cannot be trusted. The last two halt immediately whatever their count. The guard is defined and covered by tests, and no module under src/ calls it.

src/trading/sim_validation_guard.py — `ValidationIssueType` The Simulator rebuild removed this file; it is not in the tree.

```
class ValidationIssueType:
    NO_TABLET = "no_tablet"
    PRE_LISTING = "pre_listing"
    UNRESOLVABLE_TS = "unresolvable_timestamp"
    SCHEMA_DRIFT = "schema_drift"
    ADDRESS_MISMATCH = "address_mismatch"
```

The voting readout beside the fleet can fill itself with invented numbers. Three seconds after the tab is built, with no bot yet in its selector, it runs the real voting engine over a seeded random walk and draws the vote. On the Simulator that is the normal state until a fleet loads, so the panel can read as a result when it is a placeholder.

src/gui/indicator_panel.py — `IndicatorVotingPanel._generate_demo_ta`

```
def _generate_demo_ta(self):
    """Generate demo TA from synthetic candles.

    Gated by ``_may_fabricate()``. Produces a deterministic
    random walk (``seed = md5(bot_id)``) and runs the real
    VotingEngine over it, labelled with the real bot's symbol
    so the panel stays stable across refreshes.
    """
```

Validation Mode

Validation takes each YTD trade, finds the historical candle its timestamp falls in, and reruns the gates on that candle. It then puts the gate row it latched beside the gate row the log recorded, one light at a time, over the nineteen lights the shared vocabulary names. A run passes when every light reads the same. It is never judged on profit and never on a trade count.

src/simulator/validation.py — the criterion

```
@property
def latches_identically(self) -> bool:
    """True while every light and both armed flags read the same."""
    return (
        not self.disagreed
        and self.recorded_scrum_armed == self.rerun_scrum_armed
        and self.recorded_fold_armed == self.rerun_fold_armed
    )
```

The two ways in

Import Live Fleet clones the live running fleet from the bot_state load and keeps each bot's own id, so a recorded gate row belongs to a bot by that id. Generate From YTD scans the trade files and creates one new bot per pair that was traded. Those bots are new, they carry new ids, and they never wrote a gate row, so a recorded row belongs to one of them by exchange, symbol and the time window instead.

Import Live Fleet	38 bots, matched on bot id and time window
Generate From YTD	39 bots, matched on exchange, symbol and time window

The two counts differ, and that is a fact about the two sources rather than a fault. A pair traded earlier in the year can have no live bot now, and a live bot can have traded nothing yet. Neither path invents a bot to close the gap.

When more than one exchange is active the tab asks which one to use before it runs anything. One chooser serves both buttons.

src/simulator/fleet_source.py — the chooser

```
def exchange_choice(exchanges: list[str], chosen: str = "") -> dict:
    """Whether the operator must pick an exchange, and which one is in force.

    ``prompt`` is True while more than one exchange is active and ``chosen``
    names none of them.
    """
```

It reports what it could not verify

The tablets end on 1 August 2026 and the trade export runs to 7 September, so about five weeks of trades have no candle to snap to. Validation names that period and counts the entries on both sides of it. A count of verified trades with no denominator is exactly what this replaces.

```
3942 of 4904 YTD entries snapped to a candle; 962 could not be.  
after_last_candle: 905  
inside_gap: 57  
uncovered span 2026-08-01T18:52:58Z to 2026-09-07T21:46:48Z; the newest  
candle is 2026-08-01T18:35:00Z
```

Which half of the rerun moves

A candle supplies the price and the Bollinger reading. It cannot supply the bot's own state, so the delta, the tranche counts, the circuit breaker and every flag come from the row the log recorded. Each light says which half drove it, so a disagreement can be read back to its cause.

S/BB	F/BB	F/MID	the tablet candle
S/LS	F/LS		the landing-strip override
every other light			the recorded row

What the first full run measured

Every recorded reading the run checked was reproduced from the tablet exactly, but from a window sitting four hours behind the trade's own candle. The tablet prices are right: measured over one asset, 9,869 of 10,038 recorded prices sit inside the tablet candle covering their own moment. The tape is therefore sound, and the live reading it is compared against was taken from older candles than the ones that moment held.

```
38 bots, matched to a recorded gate row on bot id and time window.  
62 of 227 reruns latched every gate identically.  
3572 of 4313 gate lights agreed.  
60 of 60 recorded readings were reproduced exactly from the tablet.  
45 of them came from a window 50 candles (250 minutes) behind the trade's  
own candle.
```

Back Test Mode

Back Test runs the bot logic over a recorded tape. It walks each bot along its own Stone Tablet, feeds every window into the gates that run live, and records where a scrum and a fold latch. A latch that clears every gate then trades: the scrum sells the excess above the dollar target and the fold buys it back with the cash that scrum put aside.

`src/simulator/back_test.py` — the one gate chain, shared with Validation

```

from .validation import (
    BB_MIDLINE,
    MIN_RERUN_CANDLES,
    RERUN_WINDOW_CANDLES,
    bb_reading,
    candle_interval_ms,
    iso_stamp,
    latch,
    tablet_for,
)

```

The mode selector sits above the two button rows, and the result pane below the Replay Log changes with it.

Mode: Validation	Import Live Fleet	Generate From YTD
Mode: Back Test	Import Live Fleet	Create New Bots

The two ways into a back test

Import Live Fleet clones the live running fleet from the bot_state load, the same reader Validation uses. Create New Bots makes one simulated bot on the Stone Tablet the selector is showing, taking its target balance and its scrumming interval from the Bot Wizard's own defaults.

src/gui/main_tabs/simulator_tab_surface.py — the new bot the selector defines

```

def new_bot_specs(entry) -> list[dict]:
    """One new-bot spec for ``entry``'s asset, on the Bot Wizard's own
    defaults."""

```

One clock

Every bot ticks on one cadence. The step is sized once from the longest tape in the run, so a short tablet and a long one are read on the same clock rather than each on its own.

```

def shared_step(entries: Sequence[Any], ticks_per_bot: int, max_candles: int) -> int:
    """One step in candles, sized off the longest tape so every bot ticks
    together.

    A ``ticks_per_bot`` of zero, or no entry, answers one.
    """

```

What one back test measured

The imported fleet, read on 8 September 2026 from the live Stone Tablets.

```

38 of 38 bots ran over 1610472 tablet candles.
2026-01-01T00:00:00Z to 2026-08-01T18:35:00Z, 5272 gate-chain evaluations.
58 scrum latches and 55 fold latches.
58 scrum sells and 55 fold buys filled, $38.02 in fees.

```

Each row names the bot, its tablet, how much tape it read, how often the gates latched, how many trades filled, the coin it gained or gave up and the cash it now holds. Units alone do not answer whether a cycle accumulated, because a bot that sold into a rise holds fewer coins and more cash.

```
Bot ID | Symbol | Tablet | Candles | Ticks | Scrum / Fold latched |  
Scrum / Fold filled | Units gained | Cash held
```

A missing tablet is named, not fetched silently

A bot whose asset has no tablet is listed with no tape and counted in the run's own lines. Filling one is a call for information and goes through the same gap filler the tablet build uses, into whichever tablet root the caller names.

src/simulator/back_test.py — the fetch, and the answer with no connector

```
async def download_missing(  
    pairs: Sequence[tuple[str, str]],  
    connector: Any,  
    since_ms: int,  
    until_ms: Optional[int] = None,  
    quote_currency: str = "USD",  
    registry: Any = None,  
    ) -> list[Any]:
```

Driven into a throwaway root with a recorded tape, it wrote one tablet of 701 candles and a manifest, and the live tablet root kept its 407 files.

```
connector calls: 3  
report: ZZZTEST coinbase chunks_ok 3 errors 0 candles 701  
entry: ZZZTEST coinbase 5m 701 candles read back: 701  
live root unchanged: True 407
```

Portfolio Battery Mode

The third mode runs the thirty-five archive portfolios over their own price history. Every symbol streams its daily bars through the same gate chain a live bot ticks, at three timeframes, and each result is set against what holding that symbol untraded would have paid over the same span.

src/simulator/portfolio_battery.py — the three timeframes

```
#: The timeframes a daily tablet builds, spelled as ``HTF_TIMEFRAMES`` spells  
#: them.  
TIMEFRAMES = ("1d", "1w", "1M")
```

The baseline comes off the same walk

Buy and hold is not a figure carried over from the archive. It is the capital the symbol opened with, carried from the opening price to the closing price of the same walk, so the two sides cannot read different prices.

src/simulator/portfolio_battery.py — the baseline

```
@property
def baseline_usd(self) -> float:
    """`capital_usd` carried from `start_price` to `end_price`,
    untraded."""
    if self.start_price <= 0.0:
        return 0.0
    return self.capital_usd * self.end_price / self.start_price
```

One gate chain, fed a different source

The battery defines no gate of its own. It hands its bars to the Back Test walker, which builds the shipped scrum and fold chains and reads their answer.

src/simulator/portfolio_battery.py — the walk

```
bot = battery_bot(asset, exchange_id, timeframe, capital_usd)
result = walk(bot, candles_from_raw(bars), walk_step(len(bars), ticks))
```

A weekly and a monthly bar are folded, never invented

The tablets hold one bar a day. A coarser bar is the calendar bucket its days fall in: the first open, the highest high, the lowest low, the last close and the summed volume. A trailing bucket is kept as it stands rather than padded out.

src/simulator/portfolio_battery.py — one folded bar

```
def fold_bucket(rows: Sequence[Sequence[float]]) -> list[float]:
    """One bar from `rows`: first open, highest high, lowest low, last close,
    summed volume."""
    return [
        float(rows[0][0]),
        float(rows[0][1]),
        max(float(one[2]) for one in rows),
        min(float(one[3]) for one in rows),
        float(rows[-1][4]),
        sum(float(one[5]) for one in rows),
    ]
```

Three symbols hold no tablet

CCIV, IPOF and EXPR stopped trading and no source served them. A portfolio holding one runs on the symbols that exist and reports the share of its capital that reached no tape. That share is a column on every row, beside the count of symbols that ran.

src/simulator/portfolio_battery.py — the missing share

```

@property
def missing_weight(self) -> float:
    """`missing_usd` as a share of every symbol's `capital_usd`."""
    whole = sum(one.capital_usd for one in self.runs)
    return self.missing_usd / whole if whole > 0.0 else 0.0

```

The two ways into a battery

The two rows the removed strips left carry one portfolio and all of them. A portfolio selector and a span selector sit under the mode selector, and the span list is the archive's six periods followed by the whole tape.

src/gui/main_tabs/simulator_tab_surface.py — the two ways in

```

BATTERY_ROWS: tuple[dict[str, Any], ...] = (
    {
        "name": NEWS_TICKER_ROW,
        "height_px": 24,
        "action": RUN_PORTFOLIO_ACTION,
        "text": RUN_PORTFOLIO_TEXT,
        "button_name": button_name(RUN_PORTFOLIO_ACTION),
    },
    {
        "name": DATA_POOL_ROW,
        "height_px": 18,
        "action": RUN_EVERY_PORTFOLIO_ACTION,
        "text": RUN_EVERY_PORTFOLIO_TEXT,
        "button_name": button_name(RUN_EVERY_PORTFOLIO_ACTION),
    },
)

```

What one battery measured

SPAC_BUST over the archive's Apr24-Apr25 period, driven through the Qt tab. Three of its five symbols have a tablet. The accumulation logic ended ahead of buy and hold at the daily bar and at the weekly bar. The monthly bar holds twelve bars, under the thirty the Bollinger window and the voting engine need, so it reports no run rather than a result.

```

SPAC_BUST  1d  3 of 5  756 bars  669 ticks  35 trades
            buy and hold $390.75  accumulation $415.60  +24.85 (+6.36%)
SPAC_BUST  1w  3 of 5  159 bars  72 ticks   6 trades
            buy and hold $458.26  accumulation $478.74  +20.48 (+4.47%)
SPAC_BUST  1M  0 of 5    0 bars    0 ticks   0 trades  missing weight 100%

missing: IPOF (no_tablet), CCIV (no_tablet)
7 recorded gaps fall inside the span

```

Every portfolio over the whole tape reads 189 symbol runs in 26 seconds. Twelve of the thirty-five ended ahead of their own buy and hold at one timeframe or more.

It is not a Monte Carlo

A Monte Carlo samples many paths and reports the spread of what they pay. This mode reads one recorded path per symbol, so it reports one outcome per timeframe and no distribution. Nothing on the screen carries a percentile.

src/simulator/portfolio_battery.py — the spans, one recorded path each

```
#: Every span a run may be asked for: the archive's six, and the whole tape.
SPANS = (FULL_SPAN,) + tuple(PERIODS)
```

Nuclear Mode

NuclearModePanel drove the controller. The controller looped the same state-file fleet over the tablet window until stopped, recording each cycle so a late failure traced back to the cycle that produced it. Start needed the window's async loop; without it the panel said so and no soak began. The operator cancelled Nuclear Mode and the rebuild deleted its code, so none of the files this section names is in the tree.

NuclearFleetController, in the Simulator package the rebuild deleted

```
class NuclearFleetController:
    """Loops the bot_state fleet over Stone Tablet history until stopped.

    GUI-agnostic: all operator-visible output goes through the callbacks,
    so this is testable without Qt.
    """
```

The market structure varied per loop, and nothing on disk was touched.

noised_series, in the candle source the rebuild deleted

```
def noised_series(
    rows: list,
    seed: int,
) -> tuple[list, float]:
    """Return ``(noised_rows, noise_pct)`` for one pass over *rows*.

    A ``_Tape`` seeded from *seed* perturbs a copy through ``_at``; *rows* is
    left as it was and nothing here touches disk.
    """
```

A second oscillator supplied the load pulse, and it was capped while its cooling regime held, because the pulse shared a machine with the live trading engine. That module is still in the tree and nothing constructs it.

src/core/system_load_oscillator.py — SystemLoadOscillator

```

class SystemLoadOscillator:
    """Dual oscillator for Nuclear Mode tick rate and per-tick workload.

    ``start`` takes the monotonic reference and spawns the COOLING
    daemon; ``stop`` joins it and is idempotent. ``current_multiplier``
    returns ``BASE_MULTIPLIER`` before ``start``.
    """

```

set_swarm_hooks connected the controller to three methods on the Swarm tab, which is how a nuclear run drew its rows in the sim layer of that swarm. set_topologies replayed a Market Inspector proposal shape across the sim bots. Neither name is in the tree now.

Hook	Fires when
register_sim_run	A run starts
update_sim_run	A cycle reports
stop_sim_run	The run ends

The panel's own visuals stayed empty for a whole run. It fed the shared price chart and the shared voting readout from four fields — the symbol, the last price, the last volume and the voting summary — and the controller's snapshot carried none of them.

NuclearFleetController.snapshot, in the controller the rebuild deleted

```

def snapshot(self) -> dict:
    """Return the GUI panel's polled view of this soak."""
    s = self.state
    return {
        "running": s.running,
        "uptime_seconds": s.uptime_s,
        "fleet_size": s.fleet_size,
        "symbols": s.symbols,
        "cycles_completed": s.cycles_completed,
        "current_cycle": s.current_cycle,
        "noise_pct": s.noise_pct,
        "wires_loaded": len(self._smart_wires),
        "total_candles": s.total_candles,
        "total_trades": s.total_trades,
        "total_exceptions": s.total_exceptions,
        "load_multiplier": s.load_multiplier,
        "cooling": s.cooling,
        "load_sensed": self._sensed,
        "last_error": s.last_error,
        "failed_cycles": sum(1 for c in s.cycles if not c.ok),
    }

```

Nuclear was not a validator. It was to run after trade-logic alignment was earned, its noised tape was deliberately not history, and its criteria were coverage and survival. The operator cancelled it before that point was reached.

NuclearModePanel, in the panel the rebuild deleted — what it was for

NOT A VALIDATOR. Nuclear runs AFTER trade-logic alignment is proven on the real tablets, and its noised tape is deliberately not history, so nothing here compares its output to YTD or live. Its criteria are coverage and survival.

NuclearController was the earlier single-tape prototype: one scout bot walking one tape. Nothing under the source tree constructed it, and it stayed because both controllers shared the noise source.

NuclearModePanel, in the panel the rebuild deleted — why the prototype stayed

`nuclear\_controller.py` and `nuclear\_candle\_source.py` are deliberately NOT deleted: the latter owns `noised\_series`, which v2 depends on for exactly the market-structure noise above.

That comment recorded the state before the cancellation. Every file named in this section was deleted with the rest of the old Simulator, and no Nuclear Mode runs.

Bridge

Five methods are registered for this screen, and the renderer modules carry the matching names. They serve the Electron frontend; the tab the operator runs today is drawn in Qt.

Bridge method	Serves
simulator_tab.state	The tab, its mode switcher and its table
fleet_replay_panel.state	Fleet Replay
nuclear_mode_panel.state	Nuclear Mode
sim_stat_strip.state	The tab's own ten-field strip
sim_visuals.state	The gate lights and the sim charts

src/core/desktop_bridge.py — build_registry

```
def build_registry() -> Dict[str, Handler]:
    """Return the methods the frontend may call, keyed by method name.

    This is the wiring point for the whole frontend: a surface is
    reachable exactly when it appears here. Surfaces are imported inside
    the function so that the transport above carries no dependency on any
    one domain package.
    """
```

Back to the subsystem index.

Paper Trader Tab

Reference. The third step of the promotion pipeline. Issue #19 built the screen. The tab row carried an empty tab until that build landed, and The screen as it stands now describes what draws there today.

The empty tab it replaced

The tab drew three lines: its name, one sentence saying it was not built, and the issue that owned it. It read no bot, no price and no trade.

src/gui/main_tabs/paper_trader_tab_surface.py — the empty state it carried

```
HEADING = "Paper Trader"
ISSUE = 19
BUILT = False
STATE_TEXT = "This tab is not built."
ISSUE_TEXT = f"Issue #{ISSUE} carries the build-out."
```

Two frontends draw that one view model. EmptyTabsMixin in src/gui/main_tabs/empty_tabs.py builds the Qt tab, and src/gui/web/paper_trader_tab.js registers a panel with the Electron shell's panel host. src.core.desktop_bridge serves the model under paper_trader_tab.state.

What the step is

Real time is its defining property. The Simulator replays stored candles as fast as the machine allows. Paper takes the live feed at the speed the market delivers it, and that is the whole difference between them.

Live, Paper and the Simulator differ in one thing only: where the data comes from. The trading logic stays one body of pure code that all three call. Only the stateful shells fork, which is what keeps a paper run from holding a live object.

The paper budget is twice the dollar target, so a bot has room to fold without the exercise ending on the first dip.

src/trading/container/config.py — BotConfig, the field it doubles

```
target_balance: float = 200.0 # Balance the bot trades relative to
```

Paper sat behind the Simulator, which had to reproduce the gate latches first. Nuclear Mode was to be a second gate; the operator cancelled it, so one gate remains and issue #19 built the tab after the Simulator rebuild landed.

Where the platform once offered it

Two live surfaces named the step. The first is the Swarm tab's third sub-tab, labelled Paper Swarm. Its Start button flips a flag and relabels itself. No bot is constructed, no feed is attached and no order is recorded. Its caption is an empty string now, so the sub-tab promises nothing.

src/gui/bot_visualizer.py — the Start handler inside _create_paper_bot_row

```
def _toggle():
    if bot_data["running"]:
        bot_data["running"] = False
        start_btn.setText("■ Start")
```

The second is the wing toggle on the header strip. Pressing it swaps the whole wing. The Qt window no longer names a Paper Trader in the line it writes, and the React mirror still does, so the two hosts print different text.

src/gui/main_tabs/main_window_surface.py — the line the React mirror writes

```
"→ STOCK WING: equity exchanges + equity Paper Trader. (Crypto wing paused.)"
```

Issues #422 and #426 both folded into issue #19, which built the tab.

The screen as it stands now

The tab is built. It sits second on the bar and carries the Trading tab's panes over the live exchange feed.

src/gui/paper_trader_tab.py draws them in Qt and src/gui/web/paper_trader_tab.js draws them in React, from one view model.

src/gui/main_tabs/paper_trader_tab_surface.py — the fields both hosts read

```
DECLARED_FIELDS = (  
    "accessible_name",  
    "balance",  
    "built",  
    "feed",  
    "fleet",  
    "heading",  
    "indicators",  
    "issue",  
    "method",  
    "panes",  
    "privacy_button",  
    "reserved_rows",  
    "run",  
    "skin",  
    "symbol",  
    "symbols",  
)
```

The left pane holds the Privacy Mode button, the market selector, the two button rows and the bot list. The bot list uses the Trading tab's own columns. The right pane holds the Indicator Voting Panel, computed from the live window. An empty selector draws the empty state and invents no reading.

src/gui/main_tabs/paper_trader_tab_surface.py — the empty state

```
def indicator_payload(
    candles: Sequence[Any], timeframe: str, symbol: str, refusal: str
) -> dict:
    model = ivp.IndicatorPanelModel()
    if refusal:
        model.show_no_data(refusal)
        return ivp.build_payload(model)
    summary = multi_tf_summary(candles, timeframe, symbol)
    if not summary:
        model.show_no_data(NO_SYMBOL_TEXT)
        return ivp.build_payload(model)
    model.set_summary(summary, symbol)
    return ivp.build_payload(model)
```

The two presses

Import Live Fleet reads the stored bot record and lists what it finds. Start Paper Run opens the run and begins ticking. The same button then reads Stop Paper Run and ends it, leaving the balances and the trades on screen.

src/gui/main_tabs/paper_trader_tab_surface.py — the second row's two faces

```
{
    "name": DATA_POOL_ROW,
    "height_px": 18,
    "action": STOP_RUN_ACTION if running else START_RUN_ACTION,
    "text": STOP_RUN_TEXT if running else START_RUN_TEXT,
    "button_name": button_name(
        STOP_RUN_ACTION if running else START_RUN_ACTION
    ),
}
```

Downstream only

The feed reader answers six names and refuses every other one, so a send has no spelling. It calls the venue's public market endpoints, which carry no key and no signature.

src/paper/live_feed_source.py — the refusal

```
def __getattr__(self, name: str):
    """Refuse every name outside ``READ_NAMES``."""
    raise SendRefused(
        f"LiveFeedSource answers {READ_NAMES} and cannot {name!r}. "
        "The Paper Trader receives and asks; it sends nothing."
    )
```

One gate chain

A tick builds the same GateContext a live bot builds and evaluates the shipped scrum and fold chains on it. The paper side adds no gate and changes none.

src/paper/paper_run.py — what a tick evaluates

```
window = list(candles[-WINDOW_CANDLES:])
reading = bb_reading(window, bot)
summary = VotingEngine().compute_all(window, bot.ta_timeframe, symbol=bot.symbol)
context = tape_context(bot, balance, window, reading, summary)
armed = latch(context)
```

The fake balance

Each bot opens a budget of twice its dollar target: units worth the target, and the rest in cash to fund a fold. The balances live in memory and are dropped when the tab goes away.

src/paper/fake_balance.py — the opening

```
def opening_balance(target_usd: float, price: float) -> FakeBalance:
    whole = budget_usd(target_usd)
    held_usd = min(whole / BUDGET_MULTIPLE, whole)
    units = held_usd / float(price) if float(price) > 0.0 else 0.0
    return FakeBalance(
        units=units,
        cash_usd=whole - units * float(price),
        tranches=0,
        budget_usd=whole,
    )
```

The fleet ledger

The fake balance is one figure for the whole fleet, not one figure per bot. Five paper bots with a \$200 target open Paper Spendable at \$1000 and Paper Locked at \$1000. Each variable holds the full fleet sum, which is twice the fleet's dollar target in total.

src/paper/fake_balance.py — the opening

```
def opening_ledger(bots: Sequence[Any]) -> PaperLedger:
    total = fleet_target_usd(bots)
    return PaperLedger(
        fleet_target_usd=total,
        opening_spendable_usd=total,
        opening_locked_usd=total,
    )
```

The fleet changes when a bot is added or removed. Every press of Start Paper Run rebuilds the ledger from the fleet it is handed, so both opening figures move on the next press and never inside a run. Six \$200 bots open \$1200 in each figure and four open \$800.

Four figures are tracked. Paper Spendable is the cash the fleet holds. Paper Locked is what its units are worth at the last price the feed answered. Paper Realized Profits moves when a fold buy closes the tranche a scrum sell opened. Paper Mature Profits is the profit on positions past 200 percent growth, the figure the live wing already reads.

src/trading/smart_wire.py — the maturity rule both wings call

```
MATURE_GROWTH_PCT: float = 200.0
"""A position is mature once its value exceeds its cost basis by this percent."""
```

The four figures are held apart from the live ones. They live in memory on the run, they are never written into the stored bot record, and no paper figure reaches the header strip's real-money columns.

src/paper/fake_balance.py — the four labels the strip prints

```
FIGURE_LABELS = (
    ("spendable_usd", SPENDABLE_LABEL),
    ("locked_usd", LOCKED_LABEL),
    ("realized_profit_usd", REALIZED_LABEL),
    ("mature_profit_usd", MATURE_LABEL),
)
```

The Qt window and the React page print one line from one view model, so the two hosts cannot report different money.

src/gui/main_tabs/paper_trader_tab_surface.py — the one line

```
"text": LEDGER_SEPARATOR.join(f"{one['label']} {one['text']}" for one in cells),
```

The Paper Trader log

Every paper action is written to one file. Each line carries two halves: the gate decision that produced the action and, when one filled, the pretend trade in the shape a Coinbase year-to-date row takes. Validation reads both of those shapes already, so a paper run is checked by the reader that checks the live wing.

src/paper/paper_log.py — the row

```
return {
    "timestamp": iso_stamp(tick.wall_ms),
    "category": CATEGORY,
    "exchange": str(exchange_id),
    "bot_id": str(tick.bot_id),
    "data": gate_half(tick),
    "trade": trade_half(tick.filled) if tick.filled is not None else None,
    "ledger": dict(figures or {}),
}
```

The log has a home of its own. It is a sibling of the live folders and never sits inside one, so nothing a paper run writes can land in a live tree.

src/paper/paper_paths.py — the root

```
PAPER_ROOT: Path = Path.home() / ".acervator_paper"
```

Gate names come from one place. The row records the blocker phrases the chain produced and the labels those phrases map to, and it spells no gate name of its own.

src/trading/gate_vocabulary.py — the mapping the log calls

```
def gate_for_blocker(blocker: str) -> str:
    """Map one blocker string to its gate label, or "" if unknown."""
```

A trade is never invented. A line carries a trade half only because the gate chain latched and the fill was applied to a fake balance.

src/paper/paper_run.py — the only writer

```
paper_log.append_row(
    paper_log.paper_row(seen, bot.exchange_id, run.figures())
)
```

Real time

A run ticks on the wall clock and asks the feed for one bot per fire, so a fleet of many bots never blocks the window on a single pass. One bar is shared across the open bots.

src/gui/paper_trader_tab.py — the round robin

```
def advance_once(self) -> list:
    """Tick the next open bot against its newest live window."""
    if self._run is None or not self._run.running or not self._run.bots:
        return []
    chosen = self._run.bots[self._cursor % len(self._run.bots)].bot_id
    self._cursor += 1
    made = surface.advance_run(self._run, self._feed, chosen)
    self.refresh()
    return made
```

Paper Trade History

The manual's part list names a second History tab reading a paper trade log. No such module exists. HistoryTab in src/gui/history_tab.py reads live venue history alone.

The ledger such a tab would read now exists. Every paper action lands in the file the Paper Trader log names, and no screen reads that file yet.

In development.

Back to the subsystem index.

Proof of Accumulation

Reference. **Not built.** The window builds neither the Competition tab nor the Local Testnet tab. The tab row carries an empty tab labelled Accumulation in their place, and issue #147 carries the build-out.

`src/competition/` is the Proof of Accumulation package, and its engine runs today with no screen in front of it. The rest of this file describes that engine and the contract design behind it, as a design, not as a shipped feature.

The design is an on-chain competition layer where bots compete publicly and the winners are awarded ACRV tokens on Base, which is Coinbase's L2. It evolved from a bot identity and an append-only trade log into Elo ratings, tournament brackets and an in-platform chain. Every Acervator bot has a cryptographic identity. During a competition each trade is signed and appended to a Merkle tree, and at the end the bot submits only the Merkle root — a 32-byte hash that commits to the whole trade history without revealing one trade of it. The strategy stays private, the proof is public, and the winner takes ACRV and an NFT trophy.

The skeleton

The tab exists and draws three lines: its name, one sentence saying it is not built, and the issue that owns it. It reads no competition, no token balance and no trophy.

`src/gui/main_tabs/proof_of_accumulation_tab_surface.py` — the whole empty state

```
HEADING = "Accumulation"
ISSUE = 147
BUILT = False
STATE_TEXT = "This tab is not built."
ISSUE_TEXT = f"Issue #{ISSUE} carries the build-out."
```

Two frontends draw that one view model. `EmptyTabsMixin` in `src/gui/main_tabs/empty_tabs.py` builds the Qt tab, and `src/gui/web/proof_of_accumulation_tab.js` registers a panel with the Electron shell's panel host. `src.core.desktop_bridge` serves the model under `proof_of_accumulation_tab.state`.

Identity

Each instance gets an Ed25519 keypair. The private key signs every trade in the log. The public key is the identity another party verifies against. Strategy parameters are never signed and never published, so authorship is provable while the method stays private.

`src/competition/bot_identity.py` — `BotIdentity`

```
class BotIdentity:
    """
    Manages a bot's Ed25519 keypair. The private key never leaves this object
    unencrypted. The public key is the bot's network-visible identity.
```

The trade log

Each signed trade appends to a Merkle tree. Leaves are the SHA-256 of a trade's canonical bytes, the root is the commitment, and a proof lets a third party confirm one trade sits in the log without receiving the log.

src/competition/merkle_log.py — verify_proof

```
def verify_proof(leaf_hash: str, proof: List[dict], root: str) -> bool:
    """Verify a Merkle inclusion proof."""
    current = leaf_hash
    for step in proof:
        if step["position"] == "left":
            current = _node_hash(step["hash"], current)
        else:
            current = _node_hash(current, step["hash"])
    return current == root
```

merkle_root returns the commitment and merkle_proof builds the inclusion proof one trade at a time.

The competition lifecycle

Four phases run in order.

src/competition/competition_engine.py — the module's own summary

1. REGISTRATION — bots register with capital commitment + config hash
2. ACTIVE — bots trade; each trade appended to their Merkle log
3. SUBMISSION — trading closes; bots submit Merkle root + performance claim
4. ADJUDICATION — arbiter verifies submissions, ranks bots, awards tokens

A competition runs either locally across instances on one machine and one price feed, or between two machines exchanging signed submissions.

Tournaments

TournamentEngine in src/trading/poa_tournament.py builds the game shapes. Each builder returns a tournament from a configuration, and one run method plays it over a candle provider to an outcome.

Builder	Shape
build_duel	Two participants, head to head
build_melee	A whole field at once
build_gauntlet	One challenger against a sequence

src/trading/poa_tournament.py — TournamentEngine.build_duel

```
def build_duel(
    self,
    a: Participant,
    b: Participant,
    season: Season,
    acrv_purse: int = 10,
    seed: Optional[int] = None,
```

DynamicEventScheduler places market shocks, puzzle events and regime flips from the configuration's seed, so the same seed replays the same tournament. LocalACRVAdapter settles the award and each tournament persists as JSON.

The token

The ledger is append-only, and four methods are its whole surface.

src/competition/token_ledger.py — TokenLedger

```
class TokenLedger:
    """
    Append-only ACRV token ledger.

    award() — mint tokens for a competition result (idempotent)
    balance() — current balance for a bot
    total_minted() — total ACRV in existence
    remaining_supply() — tokens still mintable this season
    """
```

The hard cap is ten million, and the ledger refuses an award that would pass it. Each season awards less than the one before, so later tokens are harder to earn.

src/competition/season_schedule.py — the supply constants

```
TOTAL_SUPPLY_CAP = 10_000_000 # Hard cap — immutable
GENESIS_SEASON = 1
INITIAL_REWARD = 500_000 # Season 1 reward pool
DECAY_FACTOR = 0.85 # Each season awards 85% of the prior season
MIN_SEASON_REWARD = 100 # Floor — never less than this per season
```

Awards are idempotent: settling the same result twice writes one record. Balances are replayed from the log, and no operation edits a balance.

Five tiers are awarded on rank within the field, and the rarest three carry a lifetime cap on how many can ever exist. Each tier is named for a stage of the Corpus Hermeticum alchemical path, and pays a fixed number of tokens.

Tier	Stage	Rank	ACRV paid	Ever minted, at most
Harvest	NIGREDO	top 50%	10	no cap
Gold Fold	ALBEDO	top 10%	50	no cap
Bear Slayer	CITRINITAS	top 25% in a verified bear market	100	10,000

Tier	Stage	Rank	ACRV paid	Ever minted, at most
Grand Accumulator	RUBEDO	top 1% across three consecutive seasons	500	1,000
Ekthelius	UNIO MYSTICA	perfect score across every metric	10,000	21

The token contract

The ledger above is the platform's own record. On the chain the token is an ERC-20 called Acervator Token. Three of its properties are fixed when the contract is deployed and cannot be changed afterwards: the supply cap, the one address allowed to mint, and the absence of any way to burn.

contracts/ACRV.sol — the header, and the guard on minting

```
// • MAX_SUPPLY = 10,000,000 ACRV (10_000_000 * 10^18 wei)
// • Tokens can never be burned – supply monotonically increases
// • The registry address is set once at construction and cannot change

modifier onlyRegistry() {
    require(msg.sender == registry, "ACRV: caller is not the registry");
```

Property	Value
Standard	ERC-20, named Acervator Token
Chain	Base, chain id 8453
Hard cap	10,000,000 ACRV, held on the chain
Minting	the CompetitionRegistry contract only
Burning	never

The contracts are not deployed. Two npm packages supply the libraries they build on, and the deploy script takes the network and the signing key. Its own instructions name the Base test network first.

contracts/deploy.py — how it is called

```
npm install @openzeppelin/contracts @chainlink/contracts
export ACERVATOR_PRIVATE_KEY=0x...
python deploy.py --network sepolia
```

src/competition/season_schedule.py — the last tier

```
RarityTier(
    name="Ekthelius",
    emoji="∞",
    description="Perfect score across all metrics, any season",
    rank_pct_max=0.001,
    condition="100% win rate + top Sharpe + max capital efficiency",
    max_ever=21,
    base_value=10_000,
),
```

Head to head

challenge_protocol.py carries the Elo ladder. A challenger sends a signed challenge, the target accepts or declines, both trade the agreed asset for the agreed duration, and the shared engine adjudicates.

src/competition/challenge_protocol.py — the ladder constants

```
ELO_K_FACTOR = 32
MIN_ELO = 100
```

The stake flows from loser to winner and both ratings move.

Trophies

One SVG per tier, picked by name. An unknown tier raises rather than returning an empty drawing, and generate_preview_html lays the whole set out on one page.

src/competition/trophy_generator.py — generate_trophy

```
def generate_trophy(tier: str, data: TrophyData) -> str:
    fn = GENERATORS.get(tier)
    if not fn:
        raise ValueError(f"Unknown tier: {tier!r}")
    return fn(data)
```

Inside harvest_svg, a text path letters the epigraph's own instruction around the trophy ring, in its usual form: SOLVE ET COAGULA. Each drawing also letters its own alchemical stage across the face.

src/competition/trophy_generator.py — the stage each tier is lettered with

```
"Harvest": "NIGREDO",
"Gold Fold": "ALBEDO",
"Bear Slayer": "CITRINITAS",
"Grand Accumulator": "RUBEDO",
```

Where a trophy's artwork is kept

The artwork and the metadata are held on the chain itself. The token's own metadata call returns the JSON inline, and the picture inside that JSON is the tier's drawing, also inline. Nothing points at IPFS, the file-sharing network most NFT projects park their pictures on, and nothing points at any other host, so a trophy lasts as long as the chain does.

contracts/AcervatorTrophy.sol — the metadata call

```
'", "image": "data:image/svg+xml;base64, ', svgB64,
" data:application/json;base64, ",
```

The chain

`local_testnet.py` simulates the whole Base environment in memory, with no wallet, no ETH and no network. Four classes make it up.

Class	Holds
LocalChain	The blocks
LocalACRV	The ERC-20 balances and the mint history
LocalRegistry	Competitions, submissions and adjudications
LocalTestnet	The three above, plus a mock oracle and transaction receipts

One call runs a whole competition on that chain and returns the result table. It registers the entrants, trades them, takes their submissions, adjudicates, awards the tokens and mints the trophy, with nothing written outside a temporary directory.

`src/competition/local_testnet.py` — a demo run

```
from src.competition.local_testnet import LocalTestnet

testnet = LocalTestnet()
result = testnet.run_demo_competition(n_bots=3, season=1)
print(result["results_table"])
```

The real targets sit ready for the day it deploys.

`src/competition/base_config.py` — the two chains

```
Base Mainnet: chain_id=8453  - production
Base Sepolia: chain_id=84532 - testnet (deploy here first)
```

The contract addresses and ABIs sit beside them.

The two shelved tabs

Both tab modules exist and the window builds neither. Their surfaces stay registered in the bridge, so the view models answer with no Qt tab in front of them. The Testnet tab is where a competition would be run and its blocks read in a block explorer; today the demo run above is the only way to reach either.

`src/gui/main_tabs/retired_tabs.py` —
`RetiredTabsMixin._install_retired_tab_sentinels`

```
self._competition_tab = None

self._testnet_tab = None
```

Issue #147 carries the initial build-out.

Back to the subsystem index.

Market Inspector Tab

Reference. The higher-timeframe scanner and the topology proposal pane. First step of the promotion pipeline.

What builds it

MarketInspectorTabMixin._build_market_inspector_tab in src/gui/main_tabs/market_inspector_tab.py constructs the tab and wires its three injection points before adding it to the row.

The tab splits horizontally. The left half holds the scanner, the right half holds the proposal cards.

Left half: the scanner

A filter row carries a Refresh button and an "Include active markets" checkbox. The default hides the markets already under a bot, which keeps the table on entry opportunities.

HTF Signals lists one row per market: Asset, Signal, Score, Daily, Weekly, Active.

Opposing Pairs (30-day Pearson) lists Long side, Short side, Correlation and the combined score.

Where the analysis happens

MarketInspector in src/trading/market_inspector.py owns the maths. One method drives the whole pipeline and keeps the results on the analyzer.

src/trading/market_inspector.py — MarketInspector.scan_universe

```
def scan_universe(
    self,
    candles_by_symbol_by_tf: dict,
    active_symbols: set,
    closes_by_symbol: dict,
) -> None:
```

Two steps run under it.

Step	Produces
_analyze_tf	One TimeframeAnalysis per timeframe
_score_market	One MarketSignal per market

The pairing is a separate step. It enumerates every long against every short and keeps the ones whose thirty-day return correlation sits in the configured negative window.

src/trading/market_inspector.py — MarketInspector._find_opposing_pairs

```
def _find_opposing_pairs(self, signals: list, closes_by_symbol: dict) -> list:
    """Enumerate long × short candidates; keep pairs whose 30-day
    return correlation sits in the configured negative window."""
    longs = [s for s in signals if s.direction == "long" and s.score >= 0.3]
    shorts = [s for s in signals if s.direction == "short" and s.score >= 0.3]
```


One analyzer instance serves two readers. `get_shared_inspector` returns it, the tab owns the fetch cycle and writes into it, and the per-bot Market Inspector page in the Bot Details dialog reads back out of it through `build_per_bot_view`. One analyzer, two readers, no second copy of the score.

Fetching

`set_exchange_source` binds a connectors getter and the application scheduler, so the tab always sees the current connector dict rather than a snapshot taken at build time.

The fetch serves its last network result while that result stays young enough, and the Refresh button passes a flag that goes to the network regardless.

`src/exchange/market_inspector_fetcher.py` — `fetch_hft_universe`

```
async def fetch_hft_universe(
    exchange_connectors: dict,
    active_symbols: Optional[set] = None,
    top_n: int = DEFAULT_TOP_N,
    progress_cb=None,
    force_network: bool = False,
    min_refresh_s: float = DEFAULT_MIN_REFRESH_S,
) -> FetchResult:
```

Three helpers do the work inside it.

Helper	What it does
<code>_pick_universe</code>	Ranks each connector's bulk tickers by 24-hour quote volume
<code>_fetch_one_symbol</code>	Pulls per-symbol OHLCV on the connector's single-worker executor
<code>_resample_daily_to_weekly</code>	Derives the weekly series on the client when the venue lists no weekly timeframe

What a scan reports

An empty table can mean three different things. The note under it says which:

- no scan has run yet
- a scan is running
- a scan finished and found nothing

Before this, all three looked the same. An empty table gave no sign of which one it was, so a tab that was working looked exactly like a tab that was broken.

`src/gui/market_inspector.py` — `_empty_table_text`

```
def _empty_table_text(scan_state: str, noun: str) -> str:
    """The sentence an empty table carries for one scan state.

    ``SCAN_NOT_ASKED``, ``SCAN_RUNNING`` and ``SCAN_FINISHED`` each get
    their own wording, so the three never read alike.
    """
    if scan_state == SCAN_RUNNING:
        return f"Scanning for {noun}..."
    if scan_state == SCAN_FINISHED:
        return f"Scan finished. No {noun} found."
    return f"No scan yet. Press Refresh to look for {noun}."
```

scan_state reports the same three states to the renderer. The React side draws the note under the table, the same way the tab does.

The scan also writes to the log. It writes one line when it starts and one line when it finishes.

The start line names two things: whether Refresh forced the scan, and how many exchange connectors it reached.

The finish line names four things: how many markets the scan covered, how many signals and pairs it scored, how long it took, and where the data came from.

The finish line is always written. A scan that scored nothing still writes one. That is what makes a quiet market different from a scan that never ran.

Each line also goes on the event bus as a record other parts of the platform can read.

Topic	Carries
market_inspector.scan_started	forced, connector count, active symbols
market_inspector.scan_finished	market count, duration, signals, pairs, source, error

Both topics are declared in src/core/emit_contracts.py. A topic declared there is a tracked emitter, and a run that never fires it is reported.

Right half: topology proposals

MarketInspectorTopologies in src/gui/market_inspector_topologies.py renders one card per proposal and opens a preview dialog on Preview.

The engine behind the cards takes a plain dictionary, which keeps it free of any exchange or bot-manager coupling. It unions the detectors, drops overlapping proposals by asset and caps the result.

src/trading/topology_proposals.py — detect_all_topologies

```
def detect_all_topologies(
    context: dict[str, Any],
    cap: int = PROPOSAL_CAP,
) -> list[dict[str, Any]]:
    """Union every detector's proposals, dedupe them and cap at ``cap``.
```

Four detectors feed it.

Detector	Shape
detect_momentum_funnel	Correlated cluster, leader into lagggers
detect_mean_reversion_pair	Anti-correlated pair, wired both ways
detect_sector_cluster	Same-sector star, hub into spokes
detect_distance_to_band	One asset, scrum-deep into fold-deep

Two helpers read from disk under `src/trading/`. `suggested_target_usd` sizes each proposed bot from the asset target defaults, and the sector map names each asset's sector.

Dismissal

Dismissing a card hides it for a day. The write-through never raises, because losing a dismissal is a nuisance and taking down the pane is not.

`src/gui/market_inspector_topologies.py` — `_persist_dismissed`

```
def _persist_dismissed(self) -> None:
    """Best-effort write-through. Never raises: losing a
    dismissal is a nuisance, taking down the pane is not."""
    if self._dismiss_store is None:
        return
    try:
        self._dismiss_store.set(DISMISS_SETTINGS_KEY, dict(self._dismissed))
```

That write fails on every call today, because the key it writes is not one the settings schema declares. The pane logs the failure and carries on, so the dismissed count reads zero on every launch. Issue #424 carries it.

`set_dismiss_store` hands the pane the settings manager it persists through. The pane never resolves settings itself.

Adopt

The pane emits an adopt request and the window handles it. The handler counts the new bots and their combined budget, asks which of the proposed wires already exist and would change, and shows all of it before anything is created.

`src/gui/main_window.py` — `_adopt_topology_proposal`

```
def _adopt_topology_proposal(self, proposal: dict) -> None:
    """Confirm, open the wizard for each new bot, then emit `wire.created`."""
```

An adopt that aborts part way calls `_report_adopt_orphans`, which names the bot ids it created and left unwired.

Exchange comparison arbitrage

`src/trading/arbitrage.py` holds the cross-exchange price monitor and its spread tracking. No module under `src/` imports it, and the tab draws no arbitrage panel. The first two proposal forms are on screen; the third is not.

In development.

Bridge

Three methods serve this screen, and the renderer modules carry the matching names.

Bridge method	Serves
<code>market_inspector.state</code>	The scanner and its two tables
<code>market_inspector_tab.state</code>	The per-bot page in the Bot Details dialog
<code>market_inspector_topologies.state</code>	The proposal cards

[Back to the subsystem index.](#)

Bot Swarm Tab

Reference. The capital reinforcement network: the nodes, the wires between them, and the accounting that lands routed profit.

What builds it

`BotSwarmTabMixin._build_bot_swarm_tab` in `src/gui/main_tabs/bot_swarm_tab.py` constructs the visualizer and adds it to the tab row. The tab holds three sub-tabs.

Sub-tab	State
Bot Swarm	The live fleet
Simulator Swarm	Rows the Simulator registers while a run plays
Paper Swarm	Chrome only — see <code>paper-trader.md</code>

The builder asks `variant_surface` for the tab class under the screen name `BOT_SWARM`. A build stamped for Qt gets the old widget. Every other build gets the React tab, which holds the whole screen in one web view. React is the running choice.

`src/gui/main_tabs/bot_swarm_tab.py` — `BotSwarmTabMixin._build_bot_swarm_tab`

```
from ..variant_surface import BOT_SWARM, surface_class

built = surface_class(BOT_SWARM)()
```

The nodes

Each bot draws as one locust card, painted with `QPainter`. A theme module supplies the palette and a particle module the drifting motes.

src/gui/visualizer/bot_node.py — BotNodeWidget

```
class BotNodeWidget(QWidget):
    """Paint one bot as a locust, fed by set_bot_data and animate.

    paintEvent draws the wings, abdomen, thorax and head, then the
    symbol, the P/L and the bot_id as text.
    """
```

In the React build the card is drawn by `bot_node.js` from the payload `bot_node_surface` publishes. The tab page carries that module beside the wire canvas, the quick routing matrix, the list and the theme palettes.

src/gui/react_bot_swarm_tab.py — TAB_SCRIPT_ASSETS

```
TAB_SCRIPT_ASSETS: tuple[str, ...] = (
    "vendor/react.production.min.js",
    "vendor/react-dom.production.min.js",
    "design_tokens.js",
    "shared_widgets.js",
    "header_strip.js",
    "visualizer_themes.js",
    "bot_node.js",
    "wire_canvas.js",
    "quick_routing.js",
    "bot_swarm_list.js",
    "bot_visualizer.js",
)
```

One handler switches between the list and the grid. The wire overlay covers the grid alone, because the grid is the only view whose coordinates a wire can be drawn against.

Drawing a wire

Drag from one node to another. Wires that arrive on the bus draw through the same handler, which is the path the startup restore and the topology adopt both take, so one subscription covers every source.

src/gui/bot_visualizer.py — `_on_external_wire_created`

```
def _on_external_wire_created(self, event) -> None:
    """Add the ``wire.created`` event's wire to ``_wires`` when new.

    A repeated pair updates its pct and adds no second entry.
    """
```

The quick routing matrix is the bulk alternative: tick source bots, tick destination bots, and every pair lands at once. Before that runs, the handler refuses four things and says which.

src/gui/visualizer/quick_routing.py — `QuickRoutingMatrix._on_connect_clicked`

```

if not (0.0 <= pct <= 100.0):
    self._reject(f"Rate {pct} is outside 0-100%.")
    return
if not sources:
    self._reject("No SOURCE bots are checked.")
    return
if not dests:
    self._reject("No DESTINATION bots are checked.")
    return
if pct <= 0:
    self._reject(
        "Rate is 0% – that would create wires that route " "nothing."
    )
    return

```

`_confirm_mass` puts a count and a description in front of the operator before a bulk connect or disconnect runs.

Routing the profit

`SmartWireManager` in `src/trading/smart_wire.py` owns the topology and one ledger per bot.

Method	What it does
<code>register_wire</code>	Adds an edge carrying its own percentage
<code>unregister_wire</code>	Drops one edge
<code>distribute_fold_profit</code>	Moves that percentage of realised profit to each wired target
<code>export_wires, import_wires</code>	Carry the topology through a save and a restart
<code>export_ledgers, import_ledgers</code>	Carry the provenance through the same

Three readings on the per-bot ledger answer where a bot's capital came from and how much of it has aged enough to move on.

Reading on BotLedger	Answers
<code>predominant_source</code>	Which bot sent the most of what this one holds
<code>mature_profit_total</code>	Wire profit past its maturity age
<code>mature_profit_available</code>	How much of that is free to move now

Maturity is a growth threshold, not an age and not a share. A bot's capital is mature once it is worth more than three times what it started with, and the mature figure is then the whole profit above that starting capital. Below the threshold the figure is zero, so a bot cannot fund a child on a small gain.

`src/trading/smart_wire.py` — `mature_profit_usd`

```

MATURE_GROWTH_PCT: float = 200.0

if value < basis * (1.0 + MATURE_GROWTH_PCT / 100.0):
    return 0.0
return value - basis

```

One function decides maturity everywhere. The per-bot ledger reads it against the seed capital, and the header strip reads it against the exchange cost basis of each bot's holdings, so the spawn gate and the screen cannot disagree about what mature means.

src/trading/smart_wire.py — BotLedger.mature_profit_total

```
return mature_profit_usd(
    self.starting_balance, self.starting_balance + self.total_profit
)
```

The tab's own row label carries the threshold and reads the same constant the maths reads, so the words on screen and the rule applied to the money move together.

src/gui/main_tabs/bot_swarm_tab_surface.py — the row label

```
MATURE_TOTAL_ROW_FORMAT = "Mature profit total (position grown past {pct}%):"

def mature_growth_pct() -> int:
    return int(round(smart_wire.MATURE_GROWTH_PCT))
```

One function bounds what a bot may send away. It reads the target balance, the band edges, the ammunition the next fold needs and the cash on hand, and returns the exportable share as a percentage. A bot never exports the capital it is holding to buy the dip it is waiting for.

src/trading/smart_wire.py — compute_safe_outflow_pct

```
def compute_safe_outflow_pct(
    scrum_profit_usd: float,
    target_balance_usd: float,
    current_price: float,
    band_lower: float,
    band_upper: float,
    next_fold_ammun_usd: float,
    current_cash_usd: float,
    compound_growth_pct: float,
    retained_this_cycle_usd: float,
) -> float:
```

Landing the profit

The USD arrives at the destination and spreads evenly across the open fold tranches. Where no tranche stands, it parks instead, and the parked pool drains into the next tranche that opens. Parked credits are routed income, never a tranche.

src/trading/scrumping/wire_routing.py — apply_wire_income

```
def apply_wire_income(self, usd: float, source: str, ref: str = "") -> dict:
    """Apply incoming Smart Wire USD to this bot's fold queue.
```

Each tranche keeps its wire-credit provenance, held by three methods.

Method	What it does
<code>_add_wire_credits</code>	Appends one detail entry per arrival
<code>_compact_wire_credits</code>	Bounds how many entries a tranche holds
<code>_roll_wire_credit_overflow</code>	Folds everything past the cap into a rolled total that keeps the sums exact

The tranche book

`FoldTrancheAccountingMixin` in `src/trading/scrumping/fold_tranches.py` owns the queued fold tranches, the discharge order the buy path reads, and the lifetime counters. It places no orders.

Four methods carry the fold through, in this order.

Method	Step
<code>_fold_eligible_tranches</code>	Selects what may fold at the current price
<code>_fold_discharge_order</code>	Fixes the order they discharge in
<code>_plan_fold_consumption</code>	Builds the plan
<code>_settle_fold_plan</code>	Books it

Merge, despawn, clear

Three functions collapse or remove a tranche. No other code path does.

Merge. `_top_up_remnant_fold_tranches` folds a new sell's USD into part-spent tranches instead of opening more records, and `_bound_new_fold_tranches` merges one sell's fresh slice back to a single record when the fill arrived as many lots. Both reduce the record count and move USD; neither loses any.

Despawn. Aged tranches are removed outright. The record leaves the book, the queue total recomputes from what is left, and the count lands in the discarded ledger rather than the closed one, because a closed tranche is one that folded. That split holds one invariant true.

`src/trading/scrumping/fold_tranches.py` — `_despawn_aged_tranches`

```
def _despawn_aged_tranches(self, now: Optional[float] = None) -> dict:
    """Remove tranches older than ``config.tranche_despawn_days``.

    Removing a fold tranche recomputes ``_fold_queue_usd``, the
    derived aggregate behind the tick's ``_fold_queue_usd == 0``
    short-circuit and the panel's "Parked USD", and counts the
    record in ``_tranches_discarded_lifetime``, never in
    ``_tranches_closed_lifetime``, because a closed tranche is one
    that FOLDED. That split keeps
    ``created - closed - discarded == standing`` true.
```

An aged stack tranche still holding a resting limit order is kept and counted separately, since removing it would leave that order on the book with nothing tracking it. The threshold reads the same shared helper the settings spinbox reads, so the sweep and the control cannot disagree about what a stored value means.

Clear. `clear_fold_tranches` discards every queued tranche and trades nothing. Holdings, the main lots, the target balance and the parked wire credits all survive it, and the report warns when parked credits remain.

Where the fleet comes from

One file is the only source of the bots, and of their wires, ledgers and tranches with them. `StateManager` in `src/core/state_manager.py` reads and writes it. Two restore methods rebuild the fleet and its topology from it, and one save method writes it back. A restored bot connects to nothing and places no order until the operator starts it.

`src/trading/container/restore.py` — `StateRestoreMixin.restore_bots_from_state`

```
def restore_bots_from_state(self, state: dict) -> list[str]:
    """Recreate every persisted bot in IDLE state; return the ids restored.

    Connects to no exchange and places no order. Each restored bot carries
    a ``_restored`` flag and stays IDLE until the operator starts it, which
    is what triggers the exchange sync before any trading.
    """
```

`restore_smart_wires_from_state` in the same mixin rebuilds the topology. `delete_bot` is the only path that removes a record.

The list view

The table carries four named columns and eight wire lanes.

`src/gui/bot_swarm_list.py` — `COLUMN_HEADERS`

```
COLUMN_HEADERS = ["Ticker", "Inflow", "Outflow", "% Out"] + [
    f"L{i + 1}" for i in range(LANE_COUNT)
]
```

Two wires share a lane only when their row spans do not overlap, and the allocator takes the first free lane in order.

`src/gui/bot_swarm_list.py` — `BotSwarmLaneAllocator.assign`

```
def assign(self, wires: list[tuple[str, int, int]]) -> dict[str, Optional[int]]:
    """Place every ``(wire_id, row_a, row_b)`` triple in ``wires``.
```

Capital claims

One reservation per bot stands against a wallet. A total above the wallet figure is refused whole, and the free figure reports what is left. The claims settle against the venue, because the exchange is the authority.

`src/trading/capital_registry.py` — `CapitalRegistry.request_reservation`

```

"""Create or replace ``bot_id``'s ``Reservation`` and return
``(granted, reason, granted_usd)``.

A total above the ``_wallet_usd`` figure is refused whole with
``granted_usd`` 0, and a ``_wallet_usd`` of None grants unchecked.
"""

```

The reservation table reaches the frontend through its own bridge method, named in the table at the foot of this file. The Qt panel beside it, `CapitalRegistryPanel`, is declared and no screen builds it.

Bridge

Eight methods serve this screen, and the renderer modules carry the matching names.

Bridge method	Serves
<code>bot_visualizer.state</code>	The whole tab
<code>bot_swarm_tab.state</code>	The three sub-tabs and the header row
<code>bot_swarm_list.state</code>	The list view and its lanes
<code>bot_node.state</code>	One locust card
<code>wire_canvas.state</code>	The wire overlay on the grid
<code>quick_routing.state</code>	The quick routing matrix
<code>capital_registry.rows</code>	The reservation table
<code>fold_tranches_tab.state</code>	The tranche book

Back to the subsystem index.

Asset Charts

Reference. One candlestick panel per traded symbol.

What builds it

The builder mixin makes the tab and adds it to the row. The tab is a scroll area holding one panel per active bot.

`src/gui/main_tabs/charts_tab.py` — `ChartsTabMixin._build_charts_tab`

```

def _build_charts_tab(self) -> None:
    """Build the Charts tab and add it to the main tab widget."""
    self._charts_tab = TradeChartsTab()
    self._main_tabs.addTab(self._charts_tab, CHARTS_TAB)

```

Issue #450 renamed the tab from Asset Charts to Charts, and `CHARTS_TAB` holds that one word.

One method rebuilds that panel set from the current roster.

`src/gui/widgets/trade_charts_tab.py` — `TradeChartsTab.update_charts`

```
def update_charts(
    self,
    bot_statuses: list[dict],
    bot_manager=None,
    exchange_connectors: dict = None,
) -> None:
    """Create or update chart panels for each bot.
```

Three more methods keep a panel current.

Method	What it does
<code>_on_tf_changed</code>	Changes one panel's timeframe, leaving the others alone
<code>log_trade</code>	Adds a fill marker to the panel that owns the symbol
<code>push_synthetic_candles</code>	Feeds a panel whose venue returned no history

The chart widget

`CandlestickChart` in `src/gui/native_chart.py` paints each panel with `QPainter`. No browser and no `WebEngine` dependency.

Four record types carry everything drawn over the candles.

Record	Draws
<code>Candle</code>	One open, high, low, close and volume bar
<code>TradeMarker</code>	An executed buy or sell
<code>PositionMarker</code>	A holding, with a distinct icon for invisible and visible
<code>GridLine</code>	A buy or sell level

The position record is the one that carries the visibility mode, which is what tells an invisible holding from one resting on the book.

`src/gui/native_chart.py` — `PositionMarker`

```
@dataclass
class PositionMarker:
    """Active position shown on chart with visibility mode icon."""

    price: float
    side: str # "buy" or "sell"
    visibility: str # "internal" (invisible) or "orderbook" (visible)
    level: int = 0 # Grid level index
    filled: bool = False
    asset_held: float = 0.0
```

The setters name what they place.

Setter	Places
<code>set_candles</code>	The candle series

Setter	Places
add_marker	One executed fill
set_trade_history_markers	The fills already on record
set_positions	The holdings, invisible and on book
set_grid_lines	The buy and sell levels
set_tranche_floors	The standing fold-tranche floors
set_target_balance_lines	The dollar target
set_fire_armed_state	An armed manual fire
set_source_label	The feed the candles arrived on
set_error	A failure message in place of the panel body

The mouse handlers give the panel a crosshair with a price and time readout, drag panning and a wheel zoom bounded by `_effective_visible_start` and `_effective_visible_count`.

Indicators on the chart

The chart calls the engine rather than computing anything of its own. Five series arrive that way, and an entry with no value is skipped when the panel paints.

`src/gui/native_chart.py` — `CandlestickChart._compute_indicators`

```
def _compute_indicators(self):
    """Fill ``_bb_data``, ``_vortex_data``, ``_macd_data``,
    ``_stochrsi_data`` and ``_ichimoku_data`` from the engine
    classes. A ``None`` entry marks a candle with no value, and
    ``paintEvent`` skips it.
    """
```

The toolbar under each panel holds the timeframe picker and eight toggles.

`src/gui/main_tabs/native_chart_surface.py` — `INDICATOR_TOGGLES`

```
INDICATOR_TOGGLES = (
    ("bb", "BB", "#50a0f0"),
    ("vortex", "Vortex", "#00ff88"),
    ("macd", "MACD", "#ffcc00"),
    ("stochrsi", "SRsi", "#ff9060"),
    ("ichimoku", "Ichi", "#c080ff"),
    ("volume", "Vol", "#80ffcc"),
    ("slingshot", "Sling", "#ff4488"),
    ("bbullseye", "BBull", "#ff00aa"),
)
```

Volume starts on and the other seven start off.

`src/gui/main_tabs/native_chart_surface.py` — `INDICATOR_DEFAULTS`

```

INDICATOR_DEFAULTS = {
    "bb": False,
    "vortex": False,
    "macd": False,
    "stochrsi": False,
    "ichimoku": False,
    "volume": True,
    "slingshot": False,
    "bullseye": False,
}

```

Sling and BBull paint a placeholder shape rather than the indicator, and each says as much in its own tooltip. Issue #430 carries one further disagreement between a tooltip on this toolbar and what the chart paints.

The complete voter set and the maths behind each indicator belong to the Indicator Voting Panel.

Where the candles come from

Three sources in order: the connected exchange first, a public API second, and the last successful fetch third. Each source that fails is recorded and the next one runs.

src/exchange/chart_data.py — `ChartDataFetcher.fetch`

```

# --- Source 1: Exchange OHLCV via CCXT ---
if exchange is not None:
    try:
        candles, source = await self._fetch_exchange(
            exchange, symbol, timeframe, limit
        )
        if candles:
            self._set_cache(symbol, timeframe, candles)
            return candles, source

```

With every source exhausted the fetcher returns an empty list and a label naming what failed, and the panel draws that label rather than an empty grid.

src/exchange/chart_data.py — `ChartDataFetcher.fetch`

```

# All sources failed
error_detail = " | ".join(errors) if errors else "No sources available"

```

Each row arrives as an `OHLCVCandle`.

The other chart

src/gui/tradingview_chart.py holds the `QWebEngineView` chart, which runs HTML and JavaScript inside the Chromium that PySide6 already ships. The Asset Charts tab does not use it; the panels above are painted natively. Its one construction site is the stock window, and the application builds no stock window, so no screen draws it today.

src/gui/stock_main_window.py — the one construction site

```
from .tradingview_chart import TradingViewChart

self._chart = TradingViewChart(symbol="AAPL", theme="dark")
```

Bridge

Three methods serve this screen.

Bridge method	Renderer module
native_chart.state	native_chart.js
trade_charts_tab.state	trade_charts_tab.js
tradingview.chart	tradingview_chart.js

[Back to the subsystem index.](#)

History Tab

Reference. Trade history pulled from the venues, paired with the platform's own trading logic and graded.

What builds it

One builder makes the tab, adds it to the row, and hands it the bot manager.

[src/gui/main_tabs/history_tab.py](#) — `HistoryTabMixin._build_history_tab`

```
def _build_history_tab(self) -> None:
    """Build the History tab and add it to the main tab widget."""
```

Fetching

Refresh starts an asynchronous fetch that asks every active exchange for the operator's own fills. The helper walks each exchange and symbol pair the bot manager exposes, and turns each venue trade into one row.

[src/exchange/history_helpers.py](#) — `fetch_all_history_chunked`

```
async def fetch_all_history_chunked(
```

The exchange is the authority here. The rows on screen are the venue's answer, and the platform's own log joins to them rather than replacing them.

The tab refreshes itself on activation when the last fetch is older than five minutes and no fetch is already in flight.

[src/gui/main_window.py](#) — `_on_main_tab_changed`

```
is_stale = last_ts == 0 or _t.time() - last_ts > 300
if is_stale and not in_flight:
    hist.refresh()
```

Where the trade record is kept

Refresh reads the venue live and keeps nothing. The year of trades the Simulator needs is kept separately, on disk, under the exchange history bucket. One file holds one exchange, one symbol and one year, and every file names its exchange inside itself.

src/core/log_paths.py — the bucket

```
def get_exchange_history_dir() -> Path:
    """`exchange_history/` bucket — the YTD trade files.
```

The files are filled from a transactions CSV exported from the exchange. The import keeps buys and sells, counts and drops everything that is not a trade, and refuses a file that is missing a column it needs. Re-importing an overlapping export adds only the rows whose id is new.

src/exchange/ytd_csv_import.py — the import

```
def import_ytd_csv(
    csv_path: Path,
    exchange_id: str,
    root: Optional[Path] = None,
) -> ImportResult:
    """Read `csv_path` and write `exchange_id`'s trade files under
    `get_ytd_root(root)`."""
```

Where the export covers a period and carries no trade for a symbol in it, that period is written to a gap record rather than filled. See the Simulator tab for the file format and the reader.

Filtering, paging, export

Eight methods carry the controls under the table.

Method	What it does
_populate_filter_options	Fills the exchange, symbol and side pickers from the rows in hand
_apply_filters	Narrows the set
_reset_filters	Restores the full set
_current_filters	Reports the active state
_remember_to_bound, _to_bound_ts	Hold the upper date bound across a refresh
_render_page, _prev_page, _next_page	Page the result
_export_csv	Writes the current selection out

Where the cell values come from

One function returns the rendered page: the rows plus the pager state. It is the only disk read on that path.

src/exchange/history_read_contract.py — build_page

```
def build_page(
    filtered: list[dict],
    page: int = 0,
    bot_manager: Any = None,
    gate_index: Optional[dict] = None,
    voting_index: Optional[dict] = None,
) -> HistoryPage:
    """One rendered page: the ``HistoryRow`` list plus the pager state.

    ``build_join_indexes`` runs when neither index is supplied, and it is the
    only disk read on this path.
    """
```

Thirteen columns each yield one cell carrying a value, its text, its colour and its tooltip.

src/exchange/history_read_contract.py — COLUMNS, the first ten

```
COLUMNS: tuple[HistoryColumn, ...] = (
    HistoryColumn(0, "timestamp", "Timestamp (UTC)"),
    HistoryColumn(1, "exchange", "Exchange"),
    HistoryColumn(2, "symbol", "Symbol"),
    HistoryColumn(3, "bot", "Bot"),
    HistoryColumn(4, "side", "Side"),
    HistoryColumn(5, "amount", "Amount"),
    HistoryColumn(6, "price", "Price"),
    HistoryColumn(7, "cost", "Cost USD"),
    HistoryColumn(8, "fee", "Fee"),
    HistoryColumn(9, "trade_id", "Trade ID"),
```

Grade, Gates and Voting close the set. The ordering is fixed newest-first and nothing re-sorts.

src/exchange/history_read_contract.py — ROW_ORDER

```
ROW_ORDER = "timestamp_desc"
"""Newest first. ``fetch_all_history_chunked`` sorts ``reverse=True``,
``HistoryTab`` calls no ``setSortingEnabled``, and nothing re-sorts."""
```

The table renders those fields and derives none of them. A cost, a grade, a colour or a gate light on screen is always the contract's answer, which is what keeps a second implementation of History from growing behind the renderer.

Two strings depart from that pattern. The contract already declares the summary line and the page counter, and the tab builds both itself, so two implementations of the same two strings stand in the tree. Issue #425 carries it.

src/exchange/history_read_contract.py — page_label, the declaration the tab does not call


```
def page_label(page: int, total: int) -> str:
    """The footer's page counter, or "No matches" when ``total`` is zero."""
    if total == 0:
        return "No matches"
    page = clamp_page(page, total)
    return f"Page {page + 1} / {page_count(total)} ({total} trades)"
```

Grading

One record and its price context go in, and an unweighted mean comes out. An axis with no input is skipped, so it neither credits nor penalises.

src/trading/trade_grader.py — grade_trade

```
def grade_trade(record: TradeRecord, ctx: PriceContext) -> TradeGrade:
    """Grade one ``record`` against its ``ctx``.

    ``TradeGrade.overall_numeric`` is the unweighted mean of the sub-scores
    whose inputs were present, or 0.5 when no axis could be scored.
    """
    exec_score, exec_bps = _score_execution(record, ctx)
    timing_score, mfe, mae = _score_timing(record, ctx)
    strategic_score, sb_delta = _score_strategic(record, ctx)
    outcome_score, realized = _score_outcome(record, ctx)
```

The mean becomes a letter at fixed boundaries.

src/trading/trade_grader.py — _letter_from_numeric

```
def _letter_from_numeric(num: float) -> str:
    """Map ``num`` in [0.0, 1.0] to ``A+``, ``A``, ``B``, ``C``, ``D`` or ``F``."""
    if num >= 0.93:
        return "A+"
    elif num >= 0.85:
        return "A"
    elif num >= 0.70:
        return "B"
    elif num >= 0.55:
        return "C"
    elif num >= 0.40:
        return "D"
    else:
        return "F"
```

grade_trades runs the batch. The regime tag reaches the grade's regime and its rationale without a sub-score of its own.

Gate analysis

Four helpers in src/exchange/history_helpers.py join the gate log to a trade.

Helper	What it does
build_page_gate_index	Buckets the gate log entries by bot id and minute
lookup_gate_entry	Picks the closest entry to a trade
gate_cell_text	Renders the cell text
gate_cell_tooltip	Renders its tooltip

The lights themselves come from one vocabulary. Nineteen of them, the scrum bank then the fold, each carrying its bank marker, its label, its state and its colour. No Qt type crosses that boundary.

src/trading/gate_vocabulary.py — gate_light_row

```
def gate_light_row(
    scrum_armed: bool,
    fold_armed: bool,
    scrum_blockers: Optional[list] = None,
    fold_blockers: Optional[list] = None,
    landing_strip_side: str = "",
    evaluated: bool = True,
) -> list[dict]:
    """The nineteen lights, in draw order: the scrum bank then the fold.

    Each entry carries ``bank`` ("S" or "F"), ``label``, ``state`` and
    ``color``. Serialisable: no Qt type crosses this boundary.
    """
```

Five states, five colours.

src/trading/gate_vocabulary.py — LIGHT_COLORS

```
LIGHT_COLORS: dict[str, str] = {
    "override": "#22d3ee",
    "not_evaluated": "#333340",
    "blocked": "#ff3366",
    "passed": "#00cc55",
    "not_the_blocker": "#c8901e",
}
```

That vocabulary is shared on purpose. The Simulator's gate cell and this table read the same labels and the same blocker prefixes, so a gate added on one surface cannot be missing from the other. That shared seam is what makes the Simulator's gate comparison meaningful.

Coverage across a window is `classify_trades` in `src/trading/gate_coverage.py`. Every trade gets one status and the report counts only the trades that found a gate. Two thresholds decide what counts as a match and what counts as a hole in the log.

src/trading/gate_coverage.py — the two thresholds

```

DEFAULT_TOLERANCE_S: float = 300.0
"""One 5m candle either side of a trade, the window ``_nearest`` searches."""

LOG_GAP_THRESHOLD_S: float = 1800.0
"""Six 5m candles of silence, above which ``_in_log_gap`` returns True."""

```

compute_validation_window turns coverage into the window the run can be judged over.

build_page_voting_index and lookup_voting_entry do the same for the indicator voting snapshot behind each trade.

The renderer

HistoryWebTable in src/gui/react_history_panel.py draws the table with React inside the Chromium PySide6 ships, and sits where the Qt table used to. One History tab, one renderer, one set of rows.

The Simulator front-load

Refreshing History hands the Simulator the year of live trades it needs before a parity run. The tab's signal connects to the Simulator's panel, and the builder makes that connection once both tabs exist.

src/gui/history_tab.py — HistoryTab.history_refreshed

```

history_refreshed = Signal(list)

```

Paper History

The manual's part list names a Paper History Tab beside this one. No such module exists. Issue #19 built the Paper tab and its log, so the ledger such a tab would read is on disk now, and no screen reads it. See paper-trader.md.

Bridge

Three methods serve this screen.

Bridge method	Serves
history.view_model	The whole screen, assembled from the read contract
history_tab.chrome	The surrounding controls
react_history_panel.state	The table

Three renderer files draw it: history_tab.js for the chrome, then history_panel.js and its stylesheet for the table.

Back to the subsystem index.

Console

Reference. A raw log tail in the upper pane and the emitter signal stream in the lower one.

What builds it

ConsoleTabMixin._build_console_tab in src/gui/main_tabs/console_tab.py builds both panes into a vertical splitter, weighted three to two, and starts three timers.

The builder does not hold the panes. It asks variant_surface for the tab class, takes the parts off it, and adds it to the tab row.

src/gui/main_tabs/console_tab.py — ConsoleTabMixin._build_console_tab

```
tab = surface_class(CONSOLE)()
self._console_tab = tab
self._console = tab.log_pane
self._signal_view = tab.signal_view
self._console_pause_btn = tab.pause_button
self._console_pause_indicator = tab.pause_indicator
```

Two classes answer that call. ConsoleQtTab builds the Qt panes, the control bar and the vertical splitter. ConsoleReactTab holds the same six parts and draws all of them in one web view. React is the running choice.

src/gui/variant_surface.py — the two loaders for the Console

```
def _qt_console() -> type:
    """Import and return the Qt Console tab."""
    from .qt_console_tab import ConsoleQtTab

    return ConsoleQtTab

def _react_console() -> type:
    """Import and return the React Console tab."""
    from .react_console_tab import ConsoleReactTab

    return ConsoleReactTab
```

_start_console_timers starts two timers, the signal drain and the console health. The pause refresh timer is built with the tab and started by the pause action.

src/gui/main_tabs/console_tab.py — ConsoleTabMixin._start_console_timers

```
self._signal_timer = QTimer(self)
self._signal_timer.setInterval(surface.DRAIN_INTERVAL_MS)
self._signal_timer.timeout.connect(self._drain_signals)
self._signal_timer.start()
```

Upper pane: the Python log

The upper pane is a read-only text view capped at two thousand blocks, which is what bounds its memory.

src/gui/main_tabs/console_tab.py — ConsoleTabMixin._build_console_tab

```
self._console = QPlainTextEdit()
self._console.setReadOnly(True)
self._console.setFont(QFont("Consolas", 9))
```

ConsoleQtTab builds that view. Under React the same calls land on PagePane, which holds the blocks and pushes them to the page.

src/gui/qt_console_tab.py — ConsoleQtTab.__init__

```
self.log_pane = QPlainTextEdit()
self.log_pane.setReadOnly(True)
self.log_pane.setFont(
    QFont(surface.PANE_FONT_FAMILY, surface.PANE_FONT_POINT_SIZE)
)
self.log_pane.setMaximumBlockCount(surface.PANE_MAX_BLOCKS)
```

A log handler formats each record and hands the line to a relay. The relay carries it over a Qt signal to the thread that constructed it and paints it there, so a log call from any thread reaches the pane without touching the widget from that thread. Colour follows the level, and a message holding the phrase INDICATOR PANEL paints in the accent colour.

The handler attaches at two points, and both are needed because the logging engine clears propagation on the application logger.

src/gui/main_tabs/console_tab.py — ConsoleTabMixin._build_console_tab

```
logging.getLogger().addHandler(qt_handler)
logging.getLogger("acervator").addHandler(qt_handler)
```

The builder reads those two names from the surface and adds the tab's own handler to each.

src/gui/main_tabs/console_tab.py — ConsoleTabMixin._build_console_tab

```
for name in surface.HANDLER_LOGGERS:
    logging.getLogger(name).addHandler(tab.log_handler)
```

The handler level

The tab sets its own handler to debug and changes no logger's level. An earlier build raised the root logger and never restored it, which left the whole application logging at debug volume for the life of the process. Setting the handler instead keeps the verbosity local to this pane.

src/gui/main_tabs/console_tab.py — ConsoleTabMixin._build_console_tab

```
# The tab sets its own handler's level and leaves every logger level alone.
qt_handler.setLevel(logging.DEBUG)
```

Each tab class sets that level on its own handler, from HANDLER_LEVEL in the surface.

src/gui/qt_console_tab.py — ConsoleQtTab.__init__

```

self.log_handler.setFormatter(
    logging.Formatter(surface.LOG_FORMAT, datefmt=surface.LOG_DATEFMT)
)
self.log_handler.setLevel(surface.HANDLER_LEVEL)

```

Pausing

Pause holds new lines in a bounded buffer. On resume the buffer replays and one notice names anything the cap dropped, so a gap is never silent.

src/gui/main_tabs/console_log_handler.py — `_QtLogHandler.set_paused`

```

def set_paused(self, paused: bool) -> None:
    """Hold new lines in ``_buffer`` while ``paused``, and drain them on resume.

    A drain that followed a ``_buffer_max`` overflow appends one notice line.
    """

```

buffered_count feeds the pause indicator, and Clear empties the pane.

Lower pane: the Emitter Network

The signal tail drains twice a second and renders the newest two hundred records. Each line carries a pass marker, a fail marker or a neutral one, then the signal name, the site, the actual value and the expected one. Records the slice drops are counted and announced with a gap marker drawn above the slice.

src/gui/main_window.py — `_drain_signals`

```

def _drain_signals(self) -> None:
    """Render sink records past `_signal_seq`, keeping the newest 200."""

```

Seven counters ride along, published every five seconds: the sequence watermark, drain ticks, records read, records rendered, records the slice dropped, gap markers drawn, and health ticks seen. Ticks count invocations rather than records, which is what separates a quiet sink from a stopped timer.

src/gui/main_window.py — `_drain_signals`

```

# Counts invocations, not records: a quiet sink is not a stopped timer.
self._signal_drain_ticks = getattr(self, "_signal_drain_ticks", 0) + 1

```

Where the lower pane goes

The signal pane belongs to the Status tab once that screen exists. See system-status.md.

Bridge

Two methods serve this screen.

Bridge method	Serves	Renderer module
<code>console.tab</code>	Both panes	<code>console_tab.js</code>
<code>console.log_lines</code>	The log tail	<code>console_log.js</code>

The surface holds the two behaviours the tab owns rather than describes, so neither is written twice.

In the surface	Holds
<code>ConsolePane</code>	The same capped block buffer, at the same two-thousand-block limit
<code>SignalLedger</code>	The same seven counters
<code>format_record</code>	The same formatter the tab attaches to its handler

[Back to the subsystem index.](#)

System Status Tab

Reference. The screen is not built. The tab row carries an empty tab labelled Status, and issue #34 carries the build-out. Both halves the operator's text describes run today, and this file is those two halves.

The skeleton

The tab exists and draws three lines: its name, one sentence saying it is not built, and the issue that owns it. It reads no emitter, no heartbeat and no post-mortem.

`src/gui/main_tabs/system_status_tab_surface.py` — the whole empty state

```
HEADING = "Status"
ISSUE = 34
BUILT = False
STATE_TEXT = "This tab is not built."
ISSUE_TEXT = f"Issue #{ISSUE} carries the build-out."
```

Two frontends draw that one view model. `EmptyTabsMixin` in `src/gui/main_tabs/empty_tabs.py` builds the Qt tab, and `src/gui/web/system_status_tab.js` registers a panel with the Electron shell's panel host. `src.core.desktop_bridge` serves the model under `system_status_tab.state`.

Half one: the Emitter Network

One plain function is the whole connection point. A call site says what it expected and what it actually saw, and the record goes to the sink. It is a function rather than a bus subscription, so a call site needs no reference to anything, and with no sink installed the call is a dictionary lookup and a return.

`src/core/signal_contract.py` — emit

```
def emit(
    name: str,
    actual: Any,
    expected: Any = None,
    ok: Optional[bool] = None,
    context: Optional[dict] = None,
    every: float = 0.0,
    instance: Optional[str] = None,
    module: Optional[str] = None,
    duration: Optional[float] = None,
) -> Optional[Signal]:
```

A satisfied expectation records the same as a violated one. That is the design point. In a log that only records failures, a call site that never ran and a call site that always passed look identical. Here they do not.

Three constants bound what the sink holds: how many rows buffer before a write, how many stay in memory, and the size at which the file rotates.

`src/core/signal_contract.py` — the sink's bounds

```
DEFAULT_FLUSH EVERY = 200
"""Rows buffered in memory before `flush()` writes them to disk."""

RETAIN_ROWS = 350_000
"""Signal records kept in memory at once; older ones are evicted from memory but remain
in the file on disk.
"""

MAX_FILE_BYTES = 50 * 1024 * 1024
"""Byte size at which the sink's file rotates to `.1`."""
```

Tracked emitters

A second module declares every emitter. A contract names a topic, the fields a payload must carry, the fields it may carry, and the vocabulary a field is allowed to draw from.

`src/core/emit_contracts.py` — the gate-decision contract

```
EmitContract(
    topic="bot.gate_decision",
    required=("symbol",),
    optional=(
        "scrum_armed",
        "fold_armed",
        "scrum_blockers",
        "fold_blockers",
        "scrum_fixture",
        "fold_fixture",
    ),
    description="Gate evaluation snapshot at fire time.",
),
```


EmitObserver records what actually arrived and reports each violation in one of three classes: a topic that never fired, a required field that was missing, and a value outside its vocabulary.

Topic	Required	Notes
trade.filled	side, amount, price	The trade kind is type here; the persisted log row calls it action
bot.gate_decision	symbol	Scrum and fold arm state, blockers and fixture at fire time
pnl.event	none	Realised, unrealised, symbol
ta.voting	none	Voting-engine snapshot

The note on the first row is the reason the module exists. A consumer read one key from a payload that carried another. Both sides succeeded, the consumer got nothing, and a coverage report read zero while hundreds of trades flowed past it. A key-name mismatch is invisible to the producer and to the consumer; only a declared contract makes it visible.

The observer already has a formatter that turns its counts into operator-facing lines. Its one caller was the Nuclear fleet controller writing a run report; the Simulator rebuild deleted that file, so nothing calls it and no screen reads it.

src/core/emit_contracts.py — format_observer_lines, what it returns

```
d = obs.to_dict()
out = [
    f"Emit contracts: {d['topics_seen']}/{d['topics_declared']} "
    "topic(s) observed",
]
```

Half two: the Watchdog

The Watchdog runs outside the process that can crash. It launches the application as a subprocess, tees both output streams to the console and to its own log, and polls a heartbeat file. A heartbeat whose modification time stops advancing past the stall threshold counts as a hang. When the child dies the Watchdog writes a post-mortem, which is the record an in-process hook cannot produce for a hard abort or an operating-system kill.

acervator_watchdog.py — the stall threshold and the poll

```
DEFAULT_STALL_SECONDS = 60 # must match or exceed Acervator's longest sync call (CCXT: ~15s typical, 30s timeout)
HEARTBEAT_POLL_INTERVAL = 2.0
```

Every post-mortem is a bundle of logs, and bundles once grew without limit. Three constants bound them now. Two govern the rotation and the third only warns.

acervator_watchdog.py — the rotation bounds

```
POSTMORTEM_KEEP_LATEST: int = 20 # most-recent N bundles preserved
POSTMORTEM_MAX_AGE_DAYS: int = 30 # anything older is dropped
POSTMORTEM_SIZE_WARN_BYTES: int = 5 * 1024 * 1024 * 1024 # 5 GB warning threshold
```

The Watchdog writes to files under the log root, and nothing in the application reaches into it. Surviving the application dying is the whole point of it.

Until the tab lands

The Console tab's lower pane is the Emitter Network readout. It renders the same sink records, capped at the newest 200 per drain.

Back to the subsystem index.

Settings

Reference. `SettingsDialog` in `src/gui/settings_dialog.py`. The two pages below are the ones Part 3 names.

The eleven pages

One method adds them, in one order.

`src/gui/settings_dialog.py` — `SettingsDialog._setup_ui`

```
tabs.addTab(self._create_user_tab(), "User")
tabs.addTab(self._create_exchange_tab(), "Exchanges")
tabs.addTab(self._create_trading_tab(), "Trading")
tabs.addTab(self._create_folding_tab(), "Profit Folding")
tabs.addTab(self._create_ta_tab(), "TA Indicators")
tabs.addTab(self._create_phantom_tab(), "Phantom Bots")
tabs.addTab(self._create_theme_tab(), "Theme")
tabs.addTab(self._create_logging_tab(), "Logging")
tabs.addTab(self._create_sound_tab(), "Sound")
tabs.addTab(self._create_sms_tab(), "SMS")
tabs.addTab(self._create_ai_monitor_tab(), "AI Monitor")
```

What each page persists

`_save` writes and `_load_current` reads back. A control neither method touches keeps its build-time default for the life of the process and reaches no engine.

Page	Controls	<code>_save</code> writes	<code>_load_current</code> restores
User	1	the row	the row
Exchanges	6 plus 3 buttons	on Add and Remove	the configured list
Trading	6	all six	four of six
Profit Folding	11	the whole group	active alone
TA Indicators	12	nothing	nothing
Phantom Bots	13	nothing	nothing
Theme	6	all six	the theme and the accent
Logging	6	the whole group	nothing

Page	Controls	_save writes	_load_current restores
Sound	10 plus 6 buttons	nothing	nothing
SMS	17	nothing	nothing
AI Monitor	7	all seven	all seven

AppSettings in `src/core/settings.py` is the whole schema, and a key it does not declare is refused outright rather than stored somewhere unread.

`src/core/settings.py` — `SettingsManager.set`

```
def set(self, key: str, value: Any) -> None:
    """Set the ``AppSettings`` attribute *key* and call ``_save``.

    A *key* that ``AppSettings`` does not declare raises ``KeyError``.
    """
    with self._lock:
        if not hasattr(self._settings, key):
            raise KeyError(f"Unknown setting: {key}")
```

Four pages hold controls `_save` never reads, and the schema declares no field any of them could land in: TA Indicators, Phantom Bots, Sound and SMS. Issue #423 carries all four and counts the controls.

The Exchanges page is the exception to the whole pattern. `_add_exchange` and `_remove_exchange` write through the settings manager as the operator uses them, rather than waiting for Save.

The Sound page reaches its engine by one other route. The slider handler builds a fresh configuration from every checkbox, pushes it in, and clears the sample cache, because each sample bakes its volume at synthesis time.

`src/gui/settings_dialog.py` — `_on_sfx_volume_changed`

```
se = get_sound_engine()
new_cfg = SoundConfig(
    enabled=self._sound_enabled.isChecked(),
    buy_sound=self._sound_buy.isChecked(),
    sell_sound=self._sound_sell.isChecked(),
    error_sound=self._sound_error.isChecked(),
    bot_state_sound=self._sound_state.isChecked(),
    fire_sound=self._sound_fire.isChecked(),
    track_sound=self._sound_track.isChecked(),
    profit_sound=self._sound_profit.isChecked(),
    drip_sound=self._sound_drip.isChecked(),
    volume=v / 100.0,
)
```

That path runs when the slider moves or a test button plays a sample, and at no other time.

The parent section, 08-tabs.md, carries the figure for each of the eleven pages and names every control on it.

Settings, User page

The whole page is one form row. No credential field, no licence field and no key import.

src/gui/settings_dialog.py — _create_user_tab

```
def _create_user_tab(self) -> QWidget:
    w = QWidget()
    form = QFormLayout(w)
    self._username = QLineEdit()
    form.addRow("Username:", self._username)
    return w
```

_load_current reads the stored value and _save writes it back through the settings manager, which persists the schema under a home-relative directory as a TOML file, or as JSON when no TOML writer is available. One lock guards every read and write, shared across every manager instance.

What a credential store would rest on

Both halves already exist, and the User page reads neither.

The first is encryption. A passphrase and a random salt derive a 256-bit key, and each token carries its own salt and nonce.

src/core/encryption.py — derive_key

```
def derive_key(passphrase: str, salt: bytes) -> bytes:
    """Derive a 256-bit AES key from *passphrase* + *salt* via PBKDF2."""
```

KeyringManager holds the passphrase in the operating system's credential store. Without the cryptography package a fallback keystream with an HMAC-SHA256 tag takes over.

The second is the hardware-key path. A credential list encrypts against a USB volume's serial and writes at that mount point, so the file decrypts on that stick and nowhere else.

src/core/usb_auth.py — write_auth_file

```
def write_auth_file(
    usb_path: Path,
    volume_serial: str,
    credentials: list[dict],
) -> Path:
    """Encrypt *credentials* under ``_derive_usb_key`` and write ``AUTH_FILENAME``.

    Each dict carries ``exchange_id``, ``api_key``, ``api_secret`` and
    ``passphrase``, and the written path under *usb_path* is returned.
    """
```

find_auth_volume matches the volume serial that read_auth_file needs to derive the same key.

Settings, Exchanges page

`_create_exchange_tab` lists the configured exchanges and offers a form to add one: the exchange picker, an API key, an API secret, and a passphrase field that appears only when the exchange needs one.

The picker lists what the connector layer can actually reach.

`src/exchange/ccxt_connector.py` — `SUPPORTED_EXCHANGES`, the first six

```
SUPPORTED_EXCHANGES: dict[str, str] = {
    "binance": "binance",
    "coinbase": "coinbase",
    "kraken": "kraken",
    "kucoin": "kucoin",
    "bybit": "bybit",
    "okx": "okx",
```

Three venues need a passphrase, and the field appears for those alone.

`src/exchange/ccxt_connector.py` — `PASSPHRASE_EXCHANGES`

```
PASSPHRASE_EXCHANGES: set[str] = {
    "kucoin",
    "okx",
    "bitget",
}
```

The key field accepts a Coinbase CDP key string as well as a plain key, and the secret field accepts a PEM elliptic-curve private key. Escaped newlines inside a pasted PEM convert on the way in.

Three handlers sit behind the buttons.

Handler	What it does
<code>_test_api_connection</code>	Authenticates against the venue and reports through <code>_set_feedback</code>
<code>_add_exchange</code>	Runs that test first and stores nothing when it fails
<code>_remove_exchange</code>	Drops the selected entry

The stock wing

The same page in the stock wing shows a banner and disables Add and Test.

`src/gui/settings_dialog.py` — `_create_exchange_tab`

```
_banner = QLabel(
    "<b>Stock Wing:</b> equity-broker integration is "
    "queued – no live brokers are wired up yet. The "
    "list below shows the planned brokers; Add / Test "
    "are disabled until the broker connectors ship. "
    "Use the Crypto Wing for active trading today."
)
```

EQUITY_EXCHANGE_IDS fills the list with the planned brokers, each labelled as planned. Crypto is the wing that trades today.

What adding one does

The new exchange routes to the crypto layer or the stock layer, and the method removes the placeholder tab if one stood there.

src/gui/main_window.py — add_exchange_tab

```
def add_exchange_tab(self, exchange_id: str, display_name: str) -> None:
    """Add exchange tab to the correct layer (crypto or stock)."""
    is_equity = self._is_equity_exchange(exchange_id)
```

The method emits a signal naming which layer the tab actually landed in, read back from both layer widgets rather than from the argument it was given.

Bridge

Two methods serve this screen.

Bridge method	Serves	Renderer module
settings_dialog.state	The dialog and its eleven pages	settings_dialog.js
exchange_tab.state	One venue's tab	exchange_tab.js

Back to the subsystem index.

The Promotion Pipeline

Reference. A strategy earns its way to real money in four steps. Each step is a gate: it feeds the next one, and the next one refuses what the previous one could not produce.

Diagram: flowchart, left to right

- Market Inspector — scans markets, proposes topologies -> **Simulator — Stone Tablet history**
- Simulator — Stone Tablet history -> **Paper Trader — live feed, fake budget**
- Paper Trader — live feed, fake budget -> **Live — real orders on the venue**
- Market Inspector — scans markets, proposes topologies ..> **Live — real orders on the venue (adopt)**

What each step contributes

Step	Data source	What it must produce
Market Inspector	Higher-timeframe OHLCV from the connected venues	A signal table, an opposing-pair table and topology proposals
Simulator	Stone Tablets, read-only	The same gates latching on the same candles as live
Paper Trader	The live feed, in real time, against a budget that is not real	A trade record the History surface can grade
Live	The venue	Filled orders, and the exchange values every ledger reconciles to

One logic, three data sources

Live, Paper and the Simulator differ in one thing only: where the candles and the balances come from. The trading logic stays one body of pure code that all three call. Only the stateful shells fork, which keeps a simulated run from touching a live object.

The Simulator's fleet was the worked example. It built real bots against a fake exchange rather than a second bot class, and that fake exchange was a subclass of the same interface the live connector implements.

FleetSimExchange, in the Simulator fleet package the rebuild deleted

```
class FleetSimExchange(ExchangeInterface):
    """Real-symbol candle-driven fake exchange for Fleet Replay.
```

The rebuild replaced it. The Sim tab now reads Stone Tablet files through TabletSource, which holds no venue and defines no write.

The dashed edge

One edge skips the two middle steps. The Market Inspector's adopt path creates real bots and draws real wires on the live fleet, which is why the handler opens a confirmation naming the new bot count, the new budget and every wire the adopt would change.

src/gui/main_window.py — _topology_wire_collisions

```
def _topology_wire_collisions(
    self, wires: list, asset_to_bot: dict
) -> list[dict]:
    """Return one dict per proposal wire whose bot pair is already wired."""
```

The read-only edge into the Simulator was a different hook, and it said so in its own words. It let the sim replay a proposal's shape across sim bots and created nothing. The rebuild removed that hook, so no proposal reaches the Sim tab today.

set_topology_getter, in the Simulator tab the rebuild deleted

```
READ ONLY. This is not the adopt path – adopting a proposal
creates real bots and wires on the live fleet, while this only
lets the simulator replay a proposal's shape across sim bots.
"""
```

Where the chain breaks today

The Paper Trader step has a module now, and issue #19 built it. No step hands its result to the next one: the Simulator hands nothing forward, Paper loads its own fleet from the stored bot record, and Live receives from the Market Inspector's adopt path instead. See [paper-trader.md](#).

[Back to the subsystem index.](#)

Live and Supporting Evidence

- YTD Exchange Data Covering the Entire Development Arc and Exchange Test Coverage

The Year-to-Date Record and Exchange Test Coverage

Reference. What reads the venue's own trade record, and how far the tests around the exchange integration reach, measured rather than asserted.

Part 9 holds the fill record itself: its row counts, its fills per asset, the VWAP charts, the trade grading and the gate logs. This part covers the readers of that record and the exchange code beneath them.

The record is the operator's own venue export, and this repository holds no copy of it. The tree tracks no spreadsheet file at all, and of every path any commit has ever added, deleted or renamed, two carry a spreadsheet suffix and both are function-complexity inventories.

```
git ls-files "*.csv" "*.xlsx"          nothing tracked

git log --all --diff-filter=ADR --name-only  every path the history touched
docs/audits/2026-08-04_function_complexity_inventory.csv
docs/engineering-notes/2026-08-04_function_complexity_inventory.csv
```

The venue holds the authority

The reconciliation walk refills a bot's counters from the venue rather than from the bot's own ledger. It walks the venue's own trade call for the bot's symbol in 30-day windows from a fixed anchor, pages a fixed number of rows at a time, and de-duplicates by trade id across the windows. Each unique sell adds quantity times price to a year-to-date scrummed sum; each unique buy adds to a folded sum.

```
YTD_TRADE_ANCHOR_UTC = 1_775_001_600.0      # src/trading/scrumming/reconciliation.py
YTD_TRADE_PAGE_LIMIT = 500
YTD_TRADE_MAX_PAGES = 40

async def sync_ytd_trade_count(self) -> Optional[int]:
    """Walk ``get_my_trades`` in 30-day windows from the YTD anchor."""
```

Field the walk writes	Value written
stats.total_trades	max of the persisted count, the previous exchange count, and the count just walked
stats.exchange_trade_count	the same value
stats.ytd_scrummed_usd	max of the persisted sum and the walked sell sum
stats.ytd_folded_usd	max of the persisted sum and the walked buy sum
stats.exchange_data_fresh_ts	the clock reading at the end of the walk

Every counter write takes a max, so a walk raises a counter toward the venue and never lowers one. Three paths leave the persisted counters alone and answer nothing: a raise inside the walk, an absent exchange, and an exchange that does not serve the trade call. Eight checks drive it against a stubbed exchange — one pins the anchor, three drive those three paths, two pin that the count never falls, and two walk a paged, duplicated window set.

```
def test_anchor_is_2026_04_01_utc(self): ... # tests/test_ytd_trade_sync.py
def test_single_page_under_limit(self): ...
def test_chunked_window_walk_dedupes_by_id(self): ...
def test_never_lowers_persisted_count(self): ...
def test_does_not_toggle_downward_when_prev_exchange_higher(self): ...
def test_returns_none_when_no_exchange(self): ...
def test_returns_none_when_method_missing(self): ...
def test_returns_none_on_api_exception(self): ...
```

The loop stops after twelve windows. Twelve 30-day windows reach 360 days past the anchor, the furthest forward one sync can carry.

The fleet aggregator sums those per-bot fields across every bot. Its two headline fields carry the year-to-date sum whenever that sum exceeds zero, and fall back to the platform-run accumulator otherwise. Two further fields carry the two figures apart, so a reader who wants one of them can have it unmixed.

```
def get_aggregate_stats(self) -> dict: # src/trading/container/aggregation.py
    return {
        "total_scrummed_usd": ..., # year-to-date, else the run accumulator
        "total_folded_usd": ..., # the same rule
        "total_scrummed_usd_ytd": round(total_scrummed_ytd, 4),
        "total_scrummed_usd_lifetime": round(total_scrummed, 4),
    }
```

Readers of the record

Reader	Input	Output
sync_ytd_trade_count (src/trading/scrumming/reconciliation.py)	get_my_trades pages from the venue	the five stats fields above
fetch_all_history_chunked (src/exchange/history_helpers.py)	venue trades for every exchange and symbol a bot manager exposes	one row dict per fill, for the History tab
compare_trades (src/trading/stone_tables/parity_harness.py)	live fills and Simulator fills	a ParityReport of matched, live-only and sim-only trades, paired inside DEFAULT_TOLERANCE_S of 300 seconds
ytd_compounding_replay (dev_harness/harness/)	the operator's export, read offline	an upper bound on what the surplus-drain formula would have added to each bot's target

The Fleet Replay panel imports the parity comparison and records every call under a feature name. The telemetry behind that name counts calls, skips and exceptions per feature, and flags by name any feature that was never called.

```
TELEMETRY_PARITY = "sim.parity.compare_trades"      # src/gui/main_tabs/fleet_replay_panel_surface.py
The Simulator rebuild removed this file; it is not in the tree.

class FeatureCounter: ...      # src/core/feature_telemetry.py
class FeatureTelemetry: ...
def get_telemetry() -> FeatureTelemetry: ...
```

The connectors

Three classes subclass `ExchangeInterface`. The interface declares seventeen members, sixteen of them abstract. The seventeenth is the venue trade call, and it carries a body that raises until a subclass overrides it.

```
class ExchangeInterface(ABC):      # src/exchange/base.py
    async def get_my_trades(...):   # the one non-abstract member, raises NotImplementedError
```

Implementation	Module	Reaches
CCXTConnector	src/exchange/ccxt_connector.py	any venue id in <code>SUPPORTED_EXCHANGES</code> , through ccxt; or a backend passed to <code>attach_backend</code>
FleetSimExchange	src/simulator/fleet/sim_exchange.py	stored <code>CandleSeries</code> rows over real symbols; no venue
NuclearSimExchange	src/simulator/nuclear_sim_exchange.py	synthetic TAPEA/TAPEB tapes; no venue

The Simulator rebuild removed the files above; they are not in the tree.

One connector method names the fifteen ccxt members a backend must serve, marks the connector connected and drops the rate-limit interval to zero. The Stone Tablet backend is such a backend: it answers those members from tablet rows instead of the network, and the replay controller attaches it to a real connector, so a replay runs the connector's own normalisation and fee code.

```
def attach_backend(self, backend: Any) -> None:      # src/exchange/ccxt_connector.py
    """Serve every ccxt call from *backend* and mark the connector connected.

    *backend* must provide the fifteen ccxt members reached through ``_ex``:
    fetch_ohlcv, fetch_ticker, fetch_tickers, fetch_balance, create_order,
    cancel_order, fetch_order, fetch_open_orders, fetch_my_trades,
    fetch_order_book, markets, market, precisionMode, amount_to_precision
    """

class TabletBackend: ...      # src/exchange/tablet_backend.py
```

The venue registry carries its own status sets, all in `ccxt_connector.py`.

Set	Count
<code>SUPPORTED_EXCHANGES</code>	15
<code>PREFLIGHT_URLS</code>	15
<code>PASSPHRASE_EXCHANGES</code>	3
<code>US_IP_BLOCKED_EXCHANGES</code>	2

Set	Count
US_ACCOUNT_RESTRICTED_EXCHANGES	2
VERIFIED_EXCHANGES	1

One function turns those sets into the picker text a user sees. A blocked venue is labelled as blocked, any other unverified venue is labelled untested, and a venue needing a passphrase says so. Driven over the whole registry, fourteen of the fifteen labels carry a caveat and one does not.

```
def exchange_label(exchange_id: str) -> str:          # src/exchange/ccxt_connector.py
    """Return the picker label for *exchange_id*, carrying its status notes."""
    label = exchange_id.capitalize()
    notes: list[str] = []
    if exchange_id in US_IP_BLOCKED_EXCHANGES:
        notes.append("blocked from US")
    elif exchange_id not in VERIFIED_EXCHANGES:
        notes.append("untested")
    if exchange_id in PASSPHRASE_EXCHANGES:
        notes.append("passphrase required")
    if notes:
        label += f" ({', '.join(notes)})"
    return label
```

Fifteen checks cover those sets. One resolves every registry id to an importable ccxt class through the connector's own resolver. One requires an https pre-flight URL for every supported id. One pins the verified set to a single member. One reads the label of every registry id against the rule above.

```
def resolve_ccxt_class(...): ...                      # src/exchange/ccxt_connector.py
                                                    # tests/test_exchange_registry.py, fifteen checks
```

The exchange package holds 23 modules beside its package initialiser. Twenty sit inside the static import closure of the entry point, which reaches 213 first-party modules and leaves 130 source modules outside it.

```
src/exchange/      23 modules, 20 inside the entry point's import closure
  api_docs         outside
  history_surface  outside
  market_data      outside
```

What the exchange tests exercise

Measurement	Count
test files under tests/	535
of those, naming src.exchange	70
test_* callables in those 70 files	2,080

Sixteen of the seventeen interface members appear in at least one of those 70 files. The seventeenth, the asset logo lookup, appears in none.

member	files naming it
exchange_id	28
connect	22
place_order	9
get_ticker	7
get_asset_logo_url	0

The same 70 files reach some modules far more than others, and leave five of the 23 with no test importing them at all.

module	files importing it
ccxt_connector	20
base	14
currency_rate_monitor	7
history_helpers	7
history_read_contract	7
tablet_backend	7
api_docs	0
chart_data	0
crypto_assets	0
idempotency	0
position_health	0

How a test stands in for a venue

No test opens a socket to an exchange and none carries a credential. Each check measures the shape of a call and the code around it, never a venue's answer. Five stand-ins do that work.

Stand-in	Where	Replaces
a fake ccxt module put into <code>sys.modules</code>	2 of the 70 files	the venue library, sync and async support both
an emptied <code>PREFLIGHT_URLS</code>	<code>tests/test_connect_does_not_block_calling_thread.py</code>	the pre-flight HTTP call inside <code>sync_connect</code>
a <code>socket.socket.connect</code> that raises	19 test files	the wire itself, for the whole file
<code>tests/fixtures/venue_precision_metadata.json</code>	<code>tests/test_venue_precision_is_decimal_places.py</code>	published market metadata for 13 venues, captured from a public <code>load_markets</code> with no credential
<code>TabletBackend</code>	<code>src/exchange/tablet_backend.py</code>	the ccxt surface beneath a real connector

The fixture covers 13 of the 15 registry ids. The two ids missing from it are the pair in US_IP_BLOCKED_EXCHANGES, which refused the address the capture ran from.

The credential file name appears in the test tree three times, across two modules. Each of the three is a check that a tool raises rather than opening such a file, and each builds its target under a temporary path. One of them, the parametrised refusal over the runtime directories, sits beside a positive control that lets a path outside them through.

```
coinbase_credentials      3 occurrences under tests/  
  tests/test_capture_live_baseline.py  
  tests/test_migration_verifier.py
```

The four refusals around an exchange call

Refusal	Where	What it turns away
live-tree guard	tests/conftest.py	a suite run that created a path under ~/.acervator or ~/.acervator_logs, or changed anything under the Stone Tablet archive
safe_urlopen and SafeRequest	src/core/safe_url.py	a URL whose scheme is outside http and https, by policy at construction and at open, and by an opener carrying no file, ftp or data handler
_redact	src/exchange/api_logger.py	a param whose key holds any of nine credential substrings, swapping a mask in before record stores the entry
guarded_place_order	src/trading/bot_container.py	an order whose amount is not a finite positive number, at the single point every engine order passes through

Each of the four carries checks that drive it to the refusal itself, so a green run says the refusal still fires.

- `_live_roots` is injectable, so `tests/test_live_tree_guard.py` drives the guard against temporary roots across 22 checks. Two of them replay the two isolation breaches this project has shipped: a telemetry file created in the live tree, and a reservation-state autosave.
- `tests/test_safe_url_scheme_policy.py` holds 40 checks, splitting the policy half from the transport half because either can fail alone.
- The redaction checks name 13 credential field spellings that must come back masked, five benign fields that must come back unchanged as the positive control, and read the emitted log line for any param value — with a control proving the same handler sees a value placed in a benign field. One substring covers two credential spellings at once.
- Two modules call the venue's order method directly: the bot container, inside the guard itself, and the volume guard the container dispatches to. Ten sites across three trading modules call the guard, and no other route out exists. Five checks cover it: one drives unusable amount shapes into a recorder standing where the exchange stands, one drives a real amount through as the positive control, and three more pin the text of the refusal and which check turns an undersized order back.

The modules on each side of the last two, and the one substring that covers two spellings:

tests/test_api_logger_redaction.py	"sign" masks signature and CB-ACCESS-SIGN the control places a value in reason
exchange.place_order called by	src/trading/bot_container.py
guarded_place_order called by	src/trading/volume_guard.py src/trading/scrumming/execution.py src/trading/scrumming_bot.py src/trading/extractor_bot.py
tests/test_u6_venue_amount_gate.py	ten sites, no other route out five checks

The simulation battery

The venue record above is live money. Beside it sits a second body of evidence that is not live at all: a battery of simulated runs, measured on an earlier release, which is where the platform's headline win rate comes from. It is set down here as the record of that run, with what this repository can and cannot show about it stated beside it.

The battery covered 738 simulations, spanning 27 portfolio configurations, capital from \$400 up to \$100K, six two-year historical periods — including the 2020 COVID crash, the 2021 ATH bubble and the 2022 brutal bear — and the venue's fee tiers from VIP-0 through VIP-3. Parameters were identical throughout, with no optimization per asset.

Measurement	Result
Win rate	715 of 738, 96.9%
Total advantage against buying and holding	+\$78.0B
Canonical 39-sim battery	39 of 39, 100%
Historical 2020 to 2022, 39 sims	39 of 39, 100%, +\$3.16M total advantage
Capital scaling, 156 sims	156 of 156, 100%, superlinear at \$100K
Bear regime stability, 9 sims	\$4,262 to \$4,427 total, 4% CV
Profitable fee ceiling	up to 0.50% roundtrip

The 23 runs that are not wins are concentrated in extreme single-asset bear conditions, with 2022-class drawdowns of 64% to 94%, at the smallest capital levels. Losses in those cases are marginal: \$10 to \$157 on \$400 deployments. The record makes no claim to win in a catastrophic single-asset collapse.

Six competing strategies were run over the same canonical 39 cases.

Strategy compared against	Battery wins
Passive hold	39 of 39, 100%
DCA weekly	39 of 39, 100%
DCA daily	39 of 39, 100%
Grid trading	39 of 39, 100%
SMA 50/200 cross	37 of 39, 94.9%
RSI mean-reversion	35 of 39, 89.7%

This repository holds nothing behind those numbers. No file in the tree carries the per-run rows, and no commit on any branch ever added one. The figures above are a record, not a measurement anything here reproduces.

The Simulator tab did not produce them and could not. It was not built when the table was recorded, and issue #117 has since rebuilt it. Its Portfolio Battery walks the RA-StoneTablets rather than the runs above, so nothing a reader can run in this product re-derives the table.

The queries, and the control that proves they can find a file:

<code>git ls-files</code>	no battery file in the tree
<code>git log --all --diff-filter=ADR --name-only</code>	no commit ever added one
<code>git log --all --diff-filter=ADR --name-only -- generate_essay_ja.py</code>	one commit, so the walk works

The third query is the control. The same walk over every branch does find a file that once existed and is gone, so the empty answer above is a fact about the battery and not about the query.

Where a figure and a run carry less than they read

Two current-state findings bear on how a reader should take the numbers above.

The header counters and their tooltips name different figures. The header cells read the two headline fields out of the fleet snapshot, and that snapshot fills both from the year-to-date sum whenever the sum exceeds zero. The tooltips beside them describe a cumulative total “since the platform run started” that “resets to \$0.00 only on a fresh process start” — the lifetime accumulator, which the same snapshot carries separately and neither cell reads. The card and the tooltip describe two different figures.

<code>src/gui/main_tabs/header_strip_surface.py</code>	counter_cells reads
	total_scrummed_usd
	total_folded_usd
<code>src/gui/main_tabs/header_strip.py</code>	the tooltips describe
	total_scrummed_usd_lifetime

A Simulator run exercises an injected capital registry, not the live one. The bot constructor takes a capital registry parameter. The two Simulator construction sites pass one; the three live sites pass none, and the reservation mixin then falls back to the process-wide registry. The sell path refuses an amount above the effective available quantity, but that pre-check sits inside a try whose handler logs at debug level and continues, so a raise there lets the sell proceed. A Simulator run says nothing about which registry a live bot resolves, and nothing about what follows when that resolution raises.


```
ScrummingBot.__init__(..., capital_registry=None)    src/trading/scrumming_bot.py
```

```
passes one      src/simulator/fleet/fleet_replay_controller.py  
                src/simulator/nuclear_controller.py
```

The Simulator rebuild removed the files above; they are not in the tree.

```
passes none     src/gui/main_window.py  
                src/gui/live_bot_window.py  
                src/trading/container/restore.py
```

```
falls back      _crr in src/trading/scrumming/capital_reservation_mixin.py  
the pre-check   effective_available, inside a try in  
                src/trading/scrumming/execution.py
```

Conceptual Hopscotch: The HOP Protocol, Rules Registry, and SADP v1 and v2

Conceptual Hopscotch

Reference. Three subjects a reader meets around this repository and cannot find in the running code: the handoff file that carries a session across a context boundary, the rule registry no running module reads, and the development protocol, which exists in two versions — one with no source in this repository, one running today.

The handoff file

ACERVATOR_HOP8.md at the repository root is the current handoff and the only one this manual describes. The earlier ACERVATOR_HOP*.md files at the root hold archives of sessions that closed, and nothing in them binds.

A root-file inventory test records why the handoff sits at the root rather than in a directory: the drift check looks for it there, and a fresh clone has to find its orientation without being told where to look. Each root file in that inventory carries a reason naming a mechanism, and a new root file without one fails the check.

```
def test_the_inventory_names_every_tracked_root_file() -> None: ... # tests/test_repo_root_inventory.py
def test_the_inventory_names_no_file_that_has_left() -> None: ...
def test_every_inventory_entry_carries_a_reason() -> None: ...
```

The handoff holds current state and an index, never a history. Its own three standing instructions are to rewrite a section that has drifted rather than append to it, to move anything measurable into a tool rather than prose, and to keep every claim checkable in one command.

What the drift check measures

`python -m tools.hop_check` reads the newest handoff and reports three quantities.

- Commits landed since the handoff was last written, from `git rev-list --count` against the commit that last touched it. STALE_AFTER is 15; at or past that count the report calls the file drifted.
- Cited paths that no longer exist, matched as a backticked path under one of eight tracked directories.
- Cited commits that no longer resolve, matched as a backticked hex string of seven characters or more and checked with `git cat-file -t`.

```
TRACKED = r"``((?:src|tests|tools|docs|desktop|dev_harness|harness_fixtures|\.github)/[w./-]+)``"
```

Any one of the three returns exit 1. Run against the tree, the report reads 107 commits since the handoff was written, three paths declared absent on purpose, no missing path and no unresolved commit, and it exits 1 on the commit count alone.

`declared_absent` reads the paths under the `## CITED AS ABSENT` heading and drops them from the missing-path count. A path under that heading is skipped even when it sits on disk, which makes the heading a

record of a deliberate citation rather than a check on one.

The sort key picks the wrong file at ten

The picker sorts the file names as strings and takes the last one. Every handoff suffix holds one digit today, so the answer is correct. Driven with a two-digit name added, ACERVATOR_HOP10 sorts ahead of the single digits, the last entry stays ACERVATOR_HOP8.md, and the drift report then describes a superseded file. Driven again with only the two newest names present, the answer is the same. The check needs a numeric key before a tenth handoff exists.

```
def newest_hop() -> pathlib.Path | None:          # tools/hop_check.py
    """Returns the highest-numbered ACERVATOR_HOP*.md at the repository root."""
    found = sorted(ROOT.glob("ACERVATOR_HOP*.md"), key=lambda p: p.stem)
    return found[-1] if found else None
```

The rules registry

The registry module declares 35 rule ids, R1 through R35, across ten lettered groups. It holds one entry per id in one of four states — LOCKED, UNLOCKED, SUSPENDED and DEPRECATED — and a parser turns a RULE line into a call on the registry. Five ids are marked CORE: R1, R5, R10, R11 and R12.

```
RULE_META = {...}          # src/core/rule_registry.py, 35 ids, R1 through R35
CORE_RULES = {r for r, m in RULE_META.items() if m["protection"] == "CORE"}
VALID_STATES = {"LOCKED", "UNLOCKED", "SUSPENDED", "DEPRECATED"}

class RuleEntry: ...
class RuleRegistry: ...
def parse_rule_command(...): ...
```

The suspend call refuses a CORE id. Driven both ways against a registry file in a temporary directory, R1 draws a refusal and stays LOCKED while R2 moves to SUSPENDED. The refusal is a real one rather than a declared one.

```
def suspend(self, rule: str, reason: str, expires: str = "") -> str: ...    # src/core/rule_registry.py
```

Nothing in the product reads any of it. The registry path resolves into a top-level directory that no commit in this repository has ever added. An import scan over 957 Python files — parsing each file and reading its import nodes, rather than matching text — finds one importer, tests/test_rule_registry_validation.py, which is also the only caller of any symbol in the module. The same scan against src/core/log_paths.py returns three importers, so the instrument does find an importer where one exists.

```
REGISTRY_PATH = ROOT / "sadb" / "RULE_REGISTRY.json"          # src/core/rule_registry.py
# sadp/ is not in the tree, so REGISTRY_PATH resolves to nothing
```

The module's own docstring says as much. The ids name nothing the running program consults, and they carry no authority.

Version one of the protocol

Version one was never built here. It named a directory, and the history holds no commit that ever added that directory or anything inside it, the battery file and the registry file included. The version sweep still reads three of those paths: the battery file, the registry file, and a simulator module that is not there either. Run the same query against a file that is real and it answers with the commit that added it, which is the control saying the query works.

```
git log --all --diff-filter=ADR --name-only, one path per run
```

```
the query finds no commit that added the first two rows
sadb/, and RAIntSimBat.py and RULE_REGISTRY.json inside it  nothing
src/gui/simulator.py                                         nothing
src/core/log_paths.py                                       the commit that added it
```

The live file behind the second row is `src/gui/simulator_tab/simulator_tab.py`. Version one's R-numbers were never live and must not be cited as though they were. The Simulator rebuild removed this file; it is not in the tree.

What did ship was a set of references to it, and one test module holds the line against their return. Two front-door documents and the build config may not name the protocol or its battery, no shipped module may import it, and no build may carry a path naming it. Fifty-one annotation comments naming those ids survive in the source across 21 files, and they bind nothing.

`tests/test_no_dead_sadb_references.py` — the seven checks

```
def test_sadb_directory_is_absent() -> None:
def test_build_config_names_no_dead_subsystem(name: str) -> None:
def test_front_door_docs_name_no_dead_subsystem(name: str) -> None:
def test_readme_says_the_harness_was_retired() -> None:
def test_no_shipped_python_imports_the_dead_package() -> None:
def test_no_build_datas_pair_ships_a_dead_subsystem() -> None:
def test_every_shipped_tool_imports() -> None:
```

The module records both directions of its own control: reverting the five files to their earlier state failed four of the seven, and restoring them passed all seven.

Version two of the protocol

Version two is the system running now. It has three layers, and only the first is in this repository.

The harness, in the repository

The harness directory holds twelve modules and the shared report contract. Each runs from the command line against one path and reports a passed boolean, exiting non-zero when it is false. That boolean needs five conditions, and the first four concern the run rather than the findings: an unhandled file type, an unscanned target, an analyzer that did not report ok, and a recorded error each hold it false. A guard around every subprocess raises on a tool that wrote to standard error and exited non-zero with empty output, which is how an installed-but-unconfigured linter used to report a clean run.

```
python -m dev_harness.harness.<name> <path>

passed                dev_harness/harness/report.py
refuse_silent_failure dev_harness/harness/report.py
```

Five archetypes, one per domain:

- `coding_archetype.py` — Python source through ruff, mypy, pyright, bandit, vulture and semgrep, each normalised into one Finding schema.
- `docs_archetype.py` — Markdown through proselint and vale, plus a structural check for one H1, no repeated heading text, and a mode signal in the opening 500 characters.
- `gui_archetype.py` — PySide6 widget source through a static AST pass, ruff and bandit. It rejects a colour read back from `styleSheet()`.
- `ta_archetype.py` — indicator source against its published formula, under rule ids TA000 through TA011.
- `watchdog_archetype.py` — the out-of-process crash watchdog, checking that every test pin reaches the one handler.

The gate and the ledger:

- `check_release_readiness.py` — the release gate. It runs the suite, runs each archetype against its good fixture, and reads the claim ledger. It declines to declare a release ready when a step was skipped or when a green pytest run collected nothing. The older copy under `tools/`, and the whole harness directory beside it, are gone from the working tree.
- `claim_ledger.py` — one JSONL row per claim, each row carrying a status. A claim with no measurement behind it stays visible.

A claim holds one of three statuses.

Status	What it records
open	Asserted, no measurement yet. A new claim starts here, and the pre-cascade check returns exit code 1 while one remains
verified	Evidence attached, as a note and a file list. Only an open claim moves here
refuted	Evidence disproved the claim. Only an open claim moves here

Four analysis tools, each answering one question about live behaviour:

- `decision_diff.py` — the live decision diff.
- `manual_fire_frame_check.py` — the Manual Fire target-frame split against live state.
- `reconcile_position_values.py` — displayed position values against their own inputs.
- `ytd_compounding_replay.py` — a venue year-to-date export walked per asset for fold-back opportunities.

Four rule modules run inside the archetypes rather than alone. The first catches a dead path, a dead subsystem name, an import of a module that is not there, a citation past the end of the file it names, and a dotted name split across string literals. The other three carry the numeric, scaffolding and slop families.

```

dev_harness/harness/rules/
  hallucination.py      dead path, dead subsystem, absent import, bad citation, split name
  numeric_guard.py
  scaffolding.py
  slop.py

```

The fixtures directory holds the calibration bodies — a good body and a bad body per archetype. The pair is the control on the gate, and a run of the archetypes that skips the pair proves nothing about the archetypes.

```

harness_fixtures/<archetype>/known_good.<ext>      exits 0
harness_fixtures/<archetype>/known_bad.<ext>        exits 1

```

The skills, outside the repository

Twenty skills sit under `~/ .claude/skills/`, one directory each with a `SKILL.md`. They are machine-local and none is committed, so a reader cannot locate them in this repository. They carry the standing rules that version one put in a numbered registry.

Five carry authority and process.

Skill	What it governs
harness-law	Who may write code, and which archetype clears which domain
found-it-own-it	A defect found is a defect fixed in the same unit
branch-discipline	Work on a branch, never the tree the operator trades from
unit-decomposition	Sizing a task into units one pass can finish
close-package	Building the session close archive

Four carry measurement.

Skill	What it governs
ocir	Observe, calibrate, iterate, repeat, before trusting any zero or pass
two-sided-control	Driving a check to failure before trusting a pass
job-watch	Telling a slow background job from a stuck one
log-pruning	Bounding a log that grows without limit

Seven carry writing.

Skill	What it governs
descriptive-comments-only	A comment states a technical fact or it does not exist
simple-technical-english	Plain words and short sentences in everything the operator reads
variable-naming-precision	Short, functional and exact names
docs-narrative	Narrative shape, citation density, and where the code sits
hyper-refocus	Filtering a report against the item and the directives
prompt-distillation	Decomposing a dense request before the work starts
anti-claudism	The eight recurring failure behaviours

Four carry the domain.

Skill	What it governs
acervator	Project posture and operator identity
ta-canon	An indicator against its published formula
archetype-peer-review	Building or extending an archetype
hop-protocol	The handoff file described above

The hooks, outside the repository

Eight hooks sit under `~/claude/hooks/`, also machine-local and also not committed. They are the blocking layer, and six of them deny a call outright rather than warn about it. - `archetype_gate.py` — in its pre mode, denies a write whose pending content would introduce a high or critical finding the file does not already carry; in its post mode, runs the archetype on what was written and reports the verdict. It blocks a rise, never a level. - `block_heavy_run.py` — denies a test run above the memory ceiling, a run while another is resident, and - `n` auto outright. - `block_heredoc.py` — denies a shell command carrying a heredoc. - `block_unanchored_docstring.py` — denies a Python docstring that argues rather than describes: a justification clause, or a sentence naming no identifier from its own file. - `check_directive_drift.py` — denies a dispatch brief that contradicts a pinned directive. - `verify_release_gate.py` — denies a write to either file that carries the version when the release-ready record is absent or older than an hour, and allows every other path through.

The two files that hook protects, and the record it reads:

```
src/__init__.py      __version__ = resolve_version()
main.py              current_version = _acervator_version
.release_ready.json  written by the release gate, good for one hour
```

Two add context rather than deny, and both are machine-local, not in the tree: `prompt_router.py` names the archetype a task will need, and `session_stop_backstop.py` writes a forensic record of whatever is uncommitted at a session boundary.

The count reads the directory, not a list. Eight names end in `.py` and each of those runs. Six saved copies and one cache directory sit beside them, and the listing marks every entry the count leaves out.

```

~/.claude/hooks/, not committed and not in the tree
archetype_gate.py                counted, denies
block_heavy_run.py               counted, denies
block_heredoc.py                counted, denies
block_unanchored_docstring.py   counted, denies
check_directive_drift.py         counted, denies
verify_release_gate.py           counted, denies
prompt_router.py                 counted, adds context
session_stop_backstop.py         counted, adds context
archetype_gate.py.before_pointer_repair not counted, a saved copy
prompt_router.py.before_catalogue_routing not counted, a saved copy
prompt_router.py.before_env_conflict not counted, a saved copy
prompt_router.py.before_pointer_repair not counted, a saved copy
prompt_router.py.before_skill_pointer_fix not counted, a saved copy
verify_release_gate.py.before_pointer_repair not counted, a saved copy
__pycache__/                     not counted, a directory

```

Where each layer resolves

The harness is in the tree, so every claim above about it resolves to a path a reader can open. The skills and the hooks run, and they are not in the tree — a reader who searches this repository for them finds nothing, and that is the correct result rather than a missing file. The contrast with version one is the point: version one's subject has no source anywhere, in this repository or on disk.

Related parts

- 09-updates-and-versioning.md — the release gate in full, and where the version number comes from.
- 12-adr-index-and-glossary.md — where decisions are recorded, and the vocabulary this manual uses.

ADR Index & Glossary

Decision Records and Glossary

Reference. Where this repository records a decision, and the words the manual uses, each anchored to the module that implements it.

Where a decision is recorded

Four places hold the decisions this repository has made. Each answers a different question.

- `docs/engineering-notes/` — 21 notes, ten with an evidence directory beside them holding the rows and the script that produced them. One note per question, each naming the standard it measures against and the control beside every measurement.
- `.claude/rules/` — three binding standards, on code comments, on documentation and on tests. `CLAUDE.md` at the root holds the repository rules above them: one role per folder, and no machine-specific path anywhere committed.
- Executable guards in the test suite. One holds a reason for every tracked file at the repository root, and a reason has to name a mechanism. A second holds the retirement of the older protocol, and it records driving itself both ways: reverting the five files it guards failed four of its checks, and restoring them passed all seven. A decision written as a guard fails when someone reverses it, which a decision written as prose does not.
- `CHANGELOG.md` and the commit history. See `09-updates-and-versioning.md` for what the changelog holds today.

The two guards:

<code>tests/test_repo_root_inventory.py</code>	a reason per tracked root file
<code>tests/test_no_dead_sadp_references.py</code>	the older protocol's retirement

`docs/engineering-notes/ci_failure_ledger.md` is the narrowest of these: one row per continuous-integration failure, its class, the change that closed it, and the run that shows it closed. A row leaves the file after three consecutive green runs hold its tests, and the file states its own ceiling — past 20 rows, the work is producing failures faster than it closes them.

Glossary

Every term below names something in the code, and carries the module or symbol that implements it. The technical-analysis vocabulary is in `07-indicators.md` with the twelve published formulae, and is not repeated here.

The accumulation cycle

- **Target** — the dollar value a bot aims to hold in an asset. The scrum sells above it and the fold buys back beneath it.

- **Scrum** — a sell of the excess above the target. `ScrummingBot` in `src/trading/scrumming_bot.py` decides and places it.
- **Fold** — the buy back on the dip that follows a scrum, carried by `apply_profit_fold` in `src/trading/profit_fold.py`.
- **Interval** — the smallest price move between two fires, in percent. `scrumming_interval_pct` in `src/trading/container/config.py`, default 1.0.
- **Scrum/Fold cycle** — the pair, run against volatility rather than against a price forecast. A completed cycle ends holding more of the asset than it started with.
- **Price floor** — every unit bought back after a fold costs at or under the lot's original price. `_fold_eligible_tranches` in `src/trading/scrumming/tick_phases.py` enforces it.
- **Fold tranche** — one queued slice of scrummed dollars waiting for its buy back. `src/trading/scrumming/fold_tranches.py` holds the book.
- **Merge** — `_top_up_remnant_fold_tranches` folds a new sell's dollars into part-spent tranches, and `_bound_new_fold_tranches` collapses one sell's fresh slice into a single record when the fill arrived as many lots. Both reduce the record count without losing a dollar.
- **Despawn** — `_despawn_aged_tranches` removes a tranche older than the bot's threshold. It removes the record; it does not delist it, and the count lands in the discarded lifetime counter rather than the closed one.
- **Clear** — `clear_fold_tranches` discards every queued tranche and trades nothing.

Merge, despawn and clear are the only three functions that collapse or remove a tranche.

08-tabs/bot-swarm.md describes each in full.

Bots and what they hold

- **Scrumming Bot** — the accumulation bot, `ScrummingBot` in `src/trading/scrumming_bot.py`. One per traded symbol.
- **Extractor Bot** — `ExtractorBot` in `src/trading/extractor_bot.py`, a chunked state machine rather than a scrum/fold engine.
- **Smart Wire** — a capital reinforcement link between two bots, in `src/trading/smart_wire.py`.
- **Phantom balance** — a bot's view of holdings it may not sell yet, in `src/trading/phantom_balance.py`.
- **Capital claim** — one bot's hold on a quantity of an asset. `effective_available` in `src/trading/capital_reservation.py` answers holdings minus every other bot's claim on the same asset, never the caller's own.
- **Fleet** — the bots a run holds, with their wires, ledgers and tranches. All of it loads from `bot_state.json` through `StateManager`, which is the only source.

The classes behind the wire, the phantom balance and the fleet load:

```

class SmartWireManager: ...      # src/trading/smart_wire.py
class BotLedger: ...
class WireTransaction: ...

class PhantomBalanceManager: ...  # src/trading/phantom_balance.py
class PhantomBalanceBot: ...

class StateManager: ...         # src/core/state_manager.py

```

Deciding a trade

- **Voter** — one indicator's contribution to a direction and a confidence. VotingEngine in src/trading/ta_engine.py runs the twelve.
- **Gate chain** — the ordered checks a candidate trade passes before it fires, in src/trading/gate_chain.py. The sell chain and the buy chain differ, and 07-indicators.md lists both.
- **Heikin-Ashi candle** — a smoothed price bar. detect_landing_strip_v2 in src/trading/indicators/landing_strip.py reads them.
- **Landing Strip** — a tightening detector, detect_landing_strip_v2 in src/trading/indicators/landing_strip.py.
- **Band travel** — the mean-reversion read, MRInspector in src/trading/mr_inspector.py.
- **Trade grade** — a letter per completed trade, from four scored axes in src/trading/trade_grader.py. The market regime is recorded beside them and never scored. See 10-live-trade-history.md.

The gate chain's two classes and the grader's five entry points:

```

class GateChain: ...      # src/trading/gate_chain.py
class GateContext: ...

def grade_trade(record, ctx) -> TradeGrade: ...      # src/trading/trade_grader.py
def _score_execution(...): ...
def _score_timing(...): ...
def _score_strategic(...): ...
def _score_outcome(...): ...

```

Getting a strategy to real money

- **Promotion pipeline** — Market Inspector, then Simulator, then Paper Trader, then Live. Each step is a gate rather than a mode, and 08-tabs/promotion-pipeline.md holds the entry and exit criteria for each.
- **Gate-latch criterion** — the Simulator's own criterion. It passes when the gates latch identically on the same candles as live, not when profit or trade count matches. See 08-tabs/simulator.md.
- **Stone Tablet** — a stored candle history a Simulator run reads. src/simulator/tablet_source.py serves the candles, and src/simulator/back_test.py walks them through the live gate chain.
- **Paper Trader** — the step between Simulator and Live, defined by running the same logic against a real-time feed and a fake budget. Built under issue #19, in src/paper/. The shelved stock window still defaults to paper_trader_available: bool = False. See 08-tabs/paper-trader.md.

- **Proof of Accumulation** — the competition package, `src/competition/`. It holds the bot identity, the Merkle log, the challenge protocol, the token ledger and the local chain. Its two screens are shelved by the operator's own direction. See `08-tabs/proof-of-accumulation.md`. The Simulator rebuild removed the fleet-replay files this glossary once named.

The development harness

- **Archetype** — a domain quality gate under `dev_harness/harness/`, reporting a passed boolean.
- **Handoff** — `ACERVATOR_HOP8.md`, measured for drift by `tools/hop_check.py`.
- **Rule id** — an RN name in `RULE_META`, in `src/core/rule_registry.py`. The registry has no production importer and the ids bind nothing.

`11-hop-protocol-and-rules-registry.md` describes all three.

The title page

- **Solve et coagula** — dissolve and reform, the sense the epigraph *Turbator aequilibrii dissolvendus reformandusque* carries on `01-title.md`. One function letters `SOLVE · ET · COAGULA` around the outer arc of the harvest trophy, and that is the one place the phrase appears in the code.

```
def harvest_svg(d: TrophyData) -> str: ...          # src/competition/trophy_generator.py
```

The operator's own words

These name a posture rather than a component. They appear in the manual because the operator uses them, and no module implements any of them.

- **Accumulation trading** — the product category he places Acervator in: harvesting volatility to end holding more of the asset, rather than predicting a direction.
- **Conceptual hopscotch** — his name for a session hopping across a context boundary, which the handoff file exists to carry.
- **Eckthelius the Accumulator** — his handle.

Recent Updates and Version Tracking

Version Tracking

Reference. Where the running version number comes from, what each part of it says, and the gate a release passes before any surface carries a new one.

Recent updates

CHANGELOG.md at the repository root is the update record. It follows Keep a Changelog and groups entries under six headings. It currently carries an unreleased heading with no entry beneath it, so outside that file the commit history is the only record of what changed.

The instruction that file gives its own writers, and the one heading it carries:

```
Group entries under `Added`, `Changed`, `Deprecated`, `Removed`, `Fixed` or
`Security`.

## [Unreleased]
```

Where the number comes from

One resolver answers the version, and no file in the tree writes it down. It runs `git describe` against a tag pattern that admits only a tag beginning with a `v` and a digit, then turns that output into the reported string. The package initialiser calls the resolver once at import and binds the answer.

```
TAG_GLOB = "v[0-9]*" # src/_version.py
def describe(root: str | Path) -> str: ...
def format_describe(text: str) -> str: ...
def resolve_version(root: str | Path | None = None) -> str: ...

__version__ = resolve_version() # src/__init__.py
```

Three checks hold that shape in place. The first fails the moment the package initialiser gains a written-in number. The second drives a tree whose nearest tag is a local backup name and asserts the resolver skips it. The third drives the same tree without the tag pattern and asserts the backup tag does come back. That last one is the control on the second: without it, a green could mean nothing more than a tree carrying no backup tag.

```
def test_src_init_holds_no_version_literal() -> None: ... # tests/test_version_resolution.py
def test_a_backup_tag_never_becomes_the_version(tmp_path) -> None: ...
def test_the_unguarded_form_would_adapt_a_backup_tag(tmp_path) -> None: ...
```

What the string says

The formatter returns a bare release number for one state only: a clean tree sitting exactly on a version tag. Distance past the tag, a modified working tree, and an unreachable version tag each append a PEP 440 local segment after a +. One check asserts every other shape carries that +.

```
def format_describe(text: str) -> str: ... # src/_version.py

def test_only_an_exact_clean_tag_reports_a_release_number(described: str) -> None: ... # tests/test_version_resolution.py
```

Two things follow. The number on a working checkout is rarely a release number, and the distance term inside it counts commits, not features. And the version tag the operator's own tree describes from is local to that machine: `git ls-remote --tags` lists an older version tag alone, so a fresh clone resolves a lower release number from the same commit.

The six readers

Six modules read the resolved version, and none of them restates it.

- `src/__init__.py` — calls the resolver once at import and binds the answer.
- `main.py` reads the cached `src.__version__` for its startup log line, the application version and the splash paint, and falls back to the resolver and the baked reader when it checks a bundle against the source tree beside it.
- `tools/spec_common.py` bakes it into a build.
- `tools/build_release_zip.py` reads it for the release archive.
- `tools/capture_live_baseline.py` stamps a captured baseline with it.
- `src/core/version_sweep.py` takes it as the canonical value and reports any literal that shadows it.

What a frozen bundle carries

A bundle ships no repository, so the `git` call finds nothing, returns an empty string, and the resolver falls through to the baked file. Two spec helpers resolve the version at build time and write it into a build subdirectory under a fixed file name. The second returns the pair that places that file inside the bundle, where the baked reader reads it back.

```
BAKED_FILENAME = "_baked_version.txt" # src/_version.py
def read_baked_version(root: str | Path) -> str: ...

BAKE_SUBDIR = os.path.join("build", "version") # tools/spec_common.py
def read_acervator_version(project_root: str) -> str: ...
def bake_version_datas(project_root: str) -> list[tuple[str, str]]: ...
```

The frozen check makes a bundle prefer the baked value even when a repository sits around it, so a bundle unpacked inside a checkout reports its own build number rather than the enclosing tree's. Two tests drive it: one with a repository around the bundle, one with `git` taken away.

```
def is_frozen() -> bool: ...          # src/_version.py

def test_a_frozen_bundle_prefers_the_baked_value_over_a_repository() -> None: ... # tests/test_version_resolution.py
def test_a_baked_tree_keeps_its_version_after_git_is_removed() -> None: ...
```

A version that moves while the suite runs

Three test modules compare a freshly resolved version against the cached `src.__version__`. That cached value is computed once, when the package is first imported. The value it is measured against is computed when the assertion runs.

```
tests/test_specs_parity.py
tests/test_version_resolution.py
tests/test_build_variants_and_version_reach.py
```

A commit landing between those two moments changes the distance term, and the two strings disagree. That failure is a property of a git-derived version rather than a defect in the resolver: the derived number moves with every commit and the cached copy does not. Committing to the repository while the suite is running reproduces it.

The release gate

One command is the gate a release passes before any surface carries a new number. It runs the suite, runs each archetype against its good fixture and requires passed true from every one, and reads the claim ledger for open claims. On success it prints a ready line and writes a sidecar record at the repository root. On failure it names the step that failed and exits non-zero.

```
python -m dev_harness.harness.check_release_readiness

[OK] Release-ready (vX.Y.Z, N tests)      printed on success
.release_ready.json                      the sidecar, holding version, tests, timestamp
```

The gate declines to declare a release ready when a check was skipped or when a green pytest run collected nothing. Two tests pin both: one drives the gate with each skip flag and asserts it declines to print the ready line, the other drives a pytest run that collected nothing and asserts the gate calls it a failure.

```
def test_any_skip_flag_refuses_to_declare_ready(...) -> None: ... # tests/test_check_release_readiness.py
def test_green_pytest_with_zero_tests_is_a_failure(...) -> None: ...
```

The gate module is `dev_harness/harness/check_release_readiness.py`, which is the path that test imports. No copy of it lives under `tools/`.

The Development Chronicle As Told By Claude

The Development Chronicle

Live Trade History and Supporting Evidence

- VWAP Charts - YTD Data Analysis and Trade Grading - Deep Indicator Function Investigations Via Trade Logic Gate Logs

VWAP Charts and Trade Grading

Reference. What a year of live fills measures, how the volume-weighted average price is drawn from it, and what the platform's own grading and gate logs add to a fill.

The fill record is the operator's own venue export. It is not committed to this repository and no row of it is reproduced anywhere in this manual. Every figure below is an aggregate over that record.

The record behind the charts

The export holds 5,709 physical rows and 5,705 body rows. 5,661 of the body rows are fills.

Body rows by transaction type: Advanced Trade Buy 3,189; Advanced Trade Sell 2,472; Reward Income 16; Deposit 15; Buy 8; Sell 4; Withdrawal 1.

44 body rows carry no fill, and each one is accounted for by name: 16 Reward Income, 15 Deposit, 12 USDC cash legs of a trade priced elsewhere, and 1 Withdrawal. Nothing is dropped without a reason that says what it was.

40 distinct assets appear in the record. USD and USDC are quote currencies, which leaves 38 charted bases, one for each bot in the fleet, and each of the 38 has fills of its own.

Fills per asset, most to fewest: RAVE 900, CHIP 495, BILL 459, ALLO 322, ZEC 261, KAT 230, BIO 226, SPK 220, ORCA 196, BONK 191, CAP 178, PENGU 159, VVV 147, ETH 141, BTC 124, TAO 124, ONDO 119, XRP 119, IMU 113, BICO 87, LINK 83, DOGE 80, SOL 77, GROVE 75, SUI 74, RE 73, XLM 73, HYPE 66, PUMP 63, NEAR 58, HBAR 25, AERO 24, ENA 23, ADA 23, AGLD 16, LTC 6, WLFI 6, LSETH 5. Those 38 counts sum to 5,661, the fill total above.

Subtotal is the price

A row carries both a Subtotal and a Total, and only one of them belongs in a VWAP. On every row the absolute quantity times the price equals the absolute Subtotal to within 5e-6. Total is that same figure plus the fee.

A fee is a transaction cost, not a price. Fee drag measures between 0.92 and 1.20 percent per asset, so a VWAP built on Total would sit roughly that far above the price the venue actually filled at, on every buy in the record.

How a VWAP is computed here

The running figure is a streaming sum of price times quantity over a streaming sum of quantity, both carried as floats. The price is the export's Price at Transaction column. The generator never reads Subtotal, and it does not need to. The section above measures quantity times price against Subtotal and finds the two equal to within $5e-6$.

Run against the same rows with Python's exact `Fraction`, the two paths agree to $3.63e-15$ across 26 assets, so the float path loses nothing a chart can show.

One control separates a weighted mean from an unweighted one. One unit bought at \$10 and three at \$20 give 17.50 by hand, and both the float path and the `Fraction` path return 17.50. An unweighted mean of the two prices returns 15.00. An implementation that forgot to weight by quantity would open a gap of 2.50 on that case rather than pass unnoticed.

What the combined chart plots

Each asset has a chart of its running buy VWAP against time. The line is the trajectory of the average as accumulation proceeds, and the shape of that trajectory is the reading.

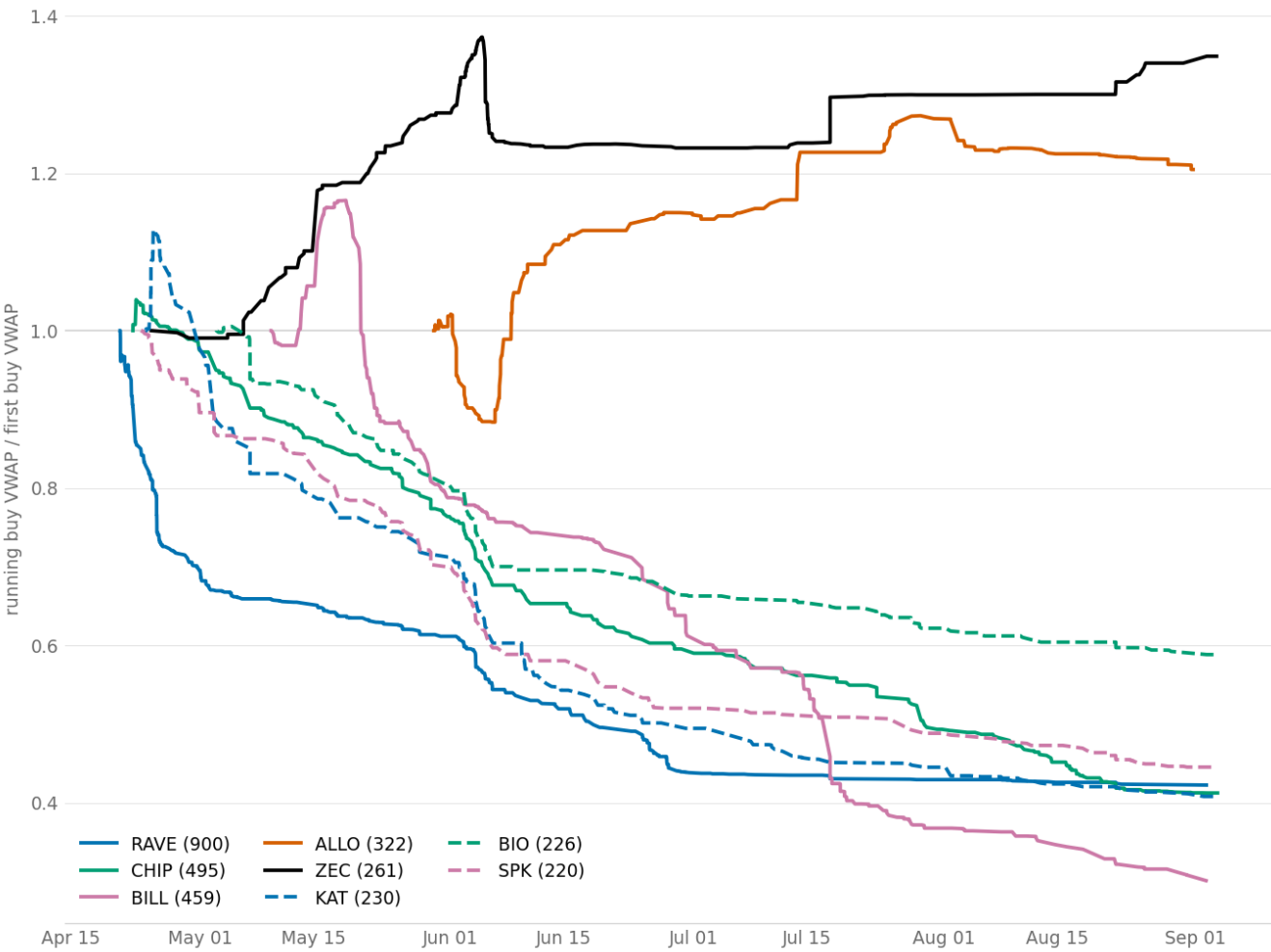
Two metrics run through this part and they answer different questions. The trajectory answers what the units now held cost on average, and it keeps moving while the position is open. The sell-over-buy ratio answers whether the trades already closed sold above what they cost, and it is one settled figure per asset across the whole record. One reads the position, the other reads the realised trades. This part carries both, and neither stands in for the other.

The combined chart indexes every asset to its own first buy VWAP and plots the running buy VWAP divided by that first value, so an asset priced in cents and an asset priced in thousands share one axis and one scale.

Of the eight assets with the most fills, six trend down to between 0.4 and 0.6. ZEC and ALLO trend up.

Read the line as what it is. A falling running buy VWAP says the average cost of the base units now held fell as more were bought. It is not profit. It does not become profit until units are sold, and the index carries no sale in it at all.

Buy-VWAP trajectory, top 8 assets by fill count, each indexed to its own first VWAP



38 charted bases. USD and USDC are excluded as quote currencies.

The running buy VWAP of the eight assets with the most fills, each divided by its own first buy VWAP.

The eight lines are the eight assets with the most fills. Each one is that asset's running buy VWAP divided by its own first buy VWAP, which puts BONK at 5.71e-06 and ZEC at 359.67 on one axis and one scale. The rule at 1.0 marks the first value. The legend names the asset and its fill count in three columns. The note under the axis reads:

38 charted bases. USD and USDC are excluded as quote currencies.

Five pens carry the eight lines. The chart takes the series palette from `src/design_system.py` and keeps only the colours reaching a contrast ratio of 3.0 against the page background. Five of the eight qualify. The five run solid for the first five assets and dashed for the last three.

Series colour	Contrast on the page background	In the chart
#000000	21.00	yes
#0072B2	5.19	yes

Series colour	Contrast on the page background	In the chart
#D55E00	3.87	yes
#009E73	3.42	yes
#CC79A7	3.06	yes
#56B4E9	2.31	no
#E69F00	2.25	no
#F0E442	1.32	no

Measured over the same 5,661 fills, the eight lines end here:

Asset	Fills	First buy VWAP	Final buy VWAP	Final index
RAVE	900	1.5915	0.672605	0.423
CHIP	495	0.10842	0.044733	0.413
BILL	459	0.11868	0.035741	0.301
ALLO	322	0.2478	0.298528	1.205
ZEC	261	359.67	485.048	1.349
KAT	230	0.01581	0.006453	0.408
BIO	226	0.05537	0.032580	0.588
SPK	220	0.0508	0.022611	0.445

Six of the eight end below 1.0 and two end above it. Of the six, five land between 0.408 and 0.588, and BILL lands at 0.301.

The buy and sell ratio

The second metric is one figure per asset: the sell VWAP divided by the buy VWAP, across every fill in the record. Above 1.000 the average sale price beat the average buy price. Below it the average sale came in under the average cost. The figure reads how the closed trades performed, and nothing else.

No chart needs redrawing for it. Every per-asset chart already carries both lines, and the ratio is where the two of them finish: the sell line's last value over the buy line's last value. Both averages come from the same two columns as the charts, the export's Price at Transaction and its Quantity Transacted. Each side keeps its own pair of sums, so neither average pulls on the other.

Of the 38 charted bases, 35 have enough fills on both sides for the figure to mean anything. 23 of those finish above 1.000 and 12 below, and none lands exactly on it. PUMP is the highest at 1.351 and BICO the lowest at 0.879.

Asset	Buys	Sells	Buy VWAP	Sell VWAP	Ratio
RAVE	538	362	0.672605	0.610801	0.908
CHIP	265	230	0.044733	0.0453964	1.015
BILL	275	184	0.0357413	0.040867	1.143
ALLO	143	179	0.298528	0.328486	1.100
ZEC	140	121	485.048	536.675	1.106

Asset	Buys	Sells	Buy VWAP	Sell VWAP	Ratio
KAT	138	92	0.00645325	0.00627872	0.973
BIO	155	71	0.0325805	0.0317443	0.974
SPK	140	80	0.0226113	0.0212018	0.938
ORCA	113	83	1.38181	1.34298	0.972
BONK	104	87	5.98261e-06	6.10717e-06	1.021
CAP	75	103	0.0297901	0.036577	1.228
PENGU	98	61	0.00762374	0.00758363	0.995
VVV	69	78	14.6904	15.4413	1.051
ETH	95	46	1996.6	2047.62	1.026
BTC	95	29	68918	68393.6	0.992
TAO	69	55	242.103	238.855	0.987
ONDO	66	53	0.36133	0.38167	1.056
XRP	82	37	1.39501	1.40129	1.005
IMU	57	56	0.00276124	0.00283892	1.028
BICO	32	55	0.0336835	0.0296231	0.879
LINK	43	40	9.06192	9.27909	1.024
DOGE	43	37	0.0881513	0.0882367	1.001
SOL	43	34	82.9528	83.4024	1.005
GROVE	47	28	0.0116845	0.0116532	0.997
SUI	41	33	0.857863	0.824958	0.962
RE	41	32	0.475448	0.497449	1.046
XLM	45	28	0.198142	0.21519	1.086
HYPE	29	37	61.99	66.8959	1.079
PUMP	23	40	0.00205112	0.00277049	1.351
NEAR	27	31	2.03969	2.09155	1.025
HBAR	16	9	0.0828047	0.0787786	0.951
AERO	8	16	0.431039	0.480832	1.116
ENA	8	15	0.145768	0.164984	1.132
ADA	10	13	0.174843	0.202243	1.157
AGLD	9	7	0.160515	0.164726	1.026
LTC	3	3	46.4508	51.1833	not read
WLF	3	3	0.0589505	0.0591535	not read
LSETH	1	4	2148.63	2576.19	not read

Three rows carry no ratio and the reason is the fill counts beside them. LSETH has one buy, so its buy figure is a single price and not an average of anything. LTC and WLF have three buys and three sells each. A mean over three fills moves with any one of them, so a ratio built on it says more about which trade landed last than about how the asset performed. The table keeps both averages, since the export supports each one, and

leaves the division out, since a number there would read as a result.

Every other row rests on at least seven fills a side, and 30 of the 38 rest on at least 23 a side.

The two metrics leave out different things and both gaps matter. The ratio counts only what completed on both sides, so it says nothing about units still held — and on most of these assets the held position is the larger part of the story. The trajectory counts every buy and no sale at all, so it says nothing about whether a sale ever cleared its cost. A cycle needs both readings. Either one alone overstates what it knows.

Traps in the export

Three shapes in the record break a reader that takes every row the same way.

A plain `Sell` row carries a negative quantity against a positive Subtotal. An `Advanced Trade Sell` row pairs those signs the other way round. Taking the sign from one column and the magnitude from the other turns four trades backwards.

One row carries a negative fee.

Three XLM rows carry a Subtotal that rounds to \$0.00 against a quantity that is real. A filter that drops zero-value rows drops those base units with them.

Where the figures live

The charts are generated output, so they are not tracked. They belong beside the manual's other images in `artifacts/manual-figures/` under the repository root, a path `.gitignore` excludes. `FIGURES.md` inventories the images the manual itself embeds. No chart image is committed to this repository.

The 39 charts of this part live one level down, in a directory of their own beside the manual figures and under the same ignore rule. Keeping the two sets apart keeps the manual's own 38 images and this part's 39 charts from mixing. The combined view above sits in the first file below, and each charted base has one chart of its own.

<code>artifacts/manual-figures/</code>	the manual's own 38 images
<code>artifacts/vwap-charts/</code>	
<code>vwap_combined.png</code>	the eight-line combined chart
<code>vwap_<ASSET>.png</code>	one per charted base, 38 of them

How to read a per-asset chart

Every chart carries two panels over one time axis.

The upper panel draws every fill as a dot at its price, buys in `COLORS["series"][0]` and sells in `COLORS["series"][5]`. Two lines cross it: the running buy VWAP and the running sell VWAP, in those same two colours. Each line steps at a fill of its own side and holds flat across a fill of the other side. The price axis turns logarithmic when the highest fill price exceeds the lowest more than eightfold, which covers RAVE at 10.30 and BILL at 13.25 and no other asset. A chart of 30 fills or fewer marks each step with a dot: HBAR, AERO, ENA, ADA, AGLD, LTC, WLFI and LSETH.

The lower panel draws net units accumulated — every bought quantity minus every sold quantity, running — in `COLORS["accent"]`, filled down to zero. It answers what the price panel cannot: whether the base position grew.

Every chart in this part draws its colours, its type sizes and its page size from `src/design_system.py`, which holds them under five names.

```
COLORS = {"bg": ..., "ink": ..., "accent": ..., "series": [...]}
TYPE = {"h1": ..., "body": ..., "cap": ...}
GRID = {"unit": 8, "page_w_in": 11.0, "page_h_in": 8.5}

def apply_rcparams(): ...

def contrast_ratio(fg: str, bg: str) -> float: ...
```

Read the buy line as a trajectory of average cost. A falling line says the average cost of the units bought so far fell. That fall is not profit, for the same reason the combined chart above carries none: no sale enters that line.

What this repository holds for these charts

The rows behind the charts take the shape `fetch_all_history_chunked` in `src/exchange/history_helpers.py` returns for the History tab, one dict per fill.

The closest committed figure to the buy line is the average entry price `compute_position_health` derives in `src/exchange/position_health.py` from the records a venue returns for its own trade history. The two differ, and the difference matters to a reader holding a chart beside the platform. The average entry follows the open position only: a buy re-weights it, a sell leaves it alone, and a full close resets it to zero. The buy line here counts every buy in the record and never resets.

```
if t.side == OrderSide.BUY:      # t is one get_my_trades record
    new_qty = qty + t.amount
    if new_qty > 0:
        avg_entry = (qty * avg_entry + t.amount * t.price) / new_qty
    qty = new_qty
elif t.side == OrderSide.SELL:
    qty -= t.amount
    if qty < 1e-12:
        qty = 0.0
        avg_entry = 0.0
```

No committed module computes an average sell price, and none computes a ratio of one average over the other. Five names were searched across the source, the harness and the tools, ignoring case, and every one of them returns no file. The control returns files for both of its terms and none for a coined one.

```

searched in src/, dev_harness/ and tools/, ignoring case
  avg_sell          0 files
  average_sell      0 files
  sell_vwap         0 files
  avg_sell_price    0 files
  sb_ratio          0 files
control
  avg_entry         13 files
  vwap              6 files
  a coined term     0 files

```

The chart module computes no ratio either. It draws the two lines and stops there. The ratio table above comes from the export directly, on the same two columns the charts read.

No committed file produces these charts. A walk of every path any commit ever added, renamed or deleted reaches 1,626 distinct paths, and exactly one of them carries the letters vwap: a Simulator test for the price band. A pickaxe over every Python file in every commit returns nothing for the three names a chart builder would carry. The charts and the record behind them belong to the operator. This repository cannot regenerate either.

```

git log --all --diff-filter=ADR --name-only
1,626 distinct paths
tests/test_vwap_band_scales_to_price.py    the one vwap name
                                             deleted; not in the tree

git log -S<name> -- '*.py'
draw_combined          0 commits
vwap_combined          0 commits
buy_vwap              0 commits
control
  avg_entry            16 commits
  sync_ytd_trade_count 7 commits
  a coined term        0 commits

```

The 38 per-asset charts

One chart per charted base, in fill-count order. The table gives what each chart's two lines start and end at, and the window its axis spans.

Asset	Fills	Buy	Sell	First buy VWAP	Final buy VWAP	Index	Window
RAVE	900	538	362	1.5915	0.672605	0.423	21 Apr to 2 Sep
CHIP	495	265	230	0.10842	0.044733	0.413	22 Apr to 3 Sep
BILL	459	275	184	0.11868	0.035741	0.301	9 May to 2 Sep
ALLO	322	143	179	0.2478	0.298528	1.205	29 May to 31 Aug
ZEC	261	140	121	359.67	485.048	1.349	24 Apr to 3 Sep
KAT	230	138	92	0.01581	0.006453	0.408	24 Apr to 3 Sep
BIO	226	155	71	0.05537	0.032580	0.588	3 May to 3 Sep

Asset	Fills	Buy	Sell	First buy VWAP	Final buy VWAP	Index	Window
SPK	220	140	80	0.0508	0.022611	0.445	23 Apr to 3 Sep
ORCA	196	113	83	1.4622	1.381809	0.945	27 Apr to 3 Sep
BONK	191	104	87	5.71e-06	5.98261e-06	1.048	12 Apr to 3 Sep
CAP	178	75	103	0.01722	0.029790	1.730	13 Jul to 3 Sep
PENGU	159	98	61	0.009814	0.007624	0.777	27 Apr to 3 Sep
VVV	147	69	78	15.7253	14.690407	0.934	9 May to 1 Sep
ETH	141	95	46	2308.68	1996.595	0.865	23 Apr to 21 Aug
BTC	124	95	29	77782.1	68917.999	0.886	23 Apr to 3 Sep
TAO	124	69	55	273.4	242.103	0.886	2 May to 1 Sep
ONDO	119	66	53	0.35136	0.361330	1.028	7 May to 3 Sep
XRP	119	82	37	1.4278	1.395015	0.977	23 Apr to 3 Sep
IMU	113	57	56	0.00395	0.002761	0.699	9 Aug to 31 Aug
BICO	87	32	55	0.069776	0.033684	0.483	9 Aug to 2 Sep
LINK	83	43	40	9.296	9.061918	0.975	24 Apr to 3 Sep
DOGE	80	43	37	0.09814	0.088151	0.898	25 Apr to 3 Sep
SOL	77	43	34	85.32	82.952844	0.972	23 Apr to 1 Sep
GROVE	75	47	28	0.02809	0.011685	0.416	8 Jul to 31 Aug
SUI	74	41	33	1.0673	0.857863	0.804	9 May to 3 Sep
RE	73	41	32	0.5153	0.475448	0.923	13 Jul to 30 Aug
XLM	73	45	28	0.205737	0.198142	0.963	29 May to 28 Aug
HYPE	66	29	37	64.42	61.990016	0.962	29 May to 30 Aug
PUMP	63	23	40	0.001603	0.002051	1.280	7 Jul to 30 Aug
NEAR	58	27	31	2.411	2.039694	0.846	29 May to 3 Sep
HBAR	25	16	9	0.09457	0.082805	0.876	29 May to 22 Aug
AERO	24	8	16	0.42307	0.431039	1.019	23 Jul to 3 Sep
ENA	23	8	15	0.14222	0.145768	1.025	22 Aug to 3 Sep
ADA	23	10	13	0.1696	0.174843	1.031	23 Jul to 3 Sep
AGLD	16	9	7	0.1695	0.160515	0.947	13 Jul to 2 Sep
LTC	6	3	3	46.33	46.450791	1.003	23 Jul to 3 Sep
WLFI	6	3	3	0.05997	0.058950	0.983	23 Jul to 16 Aug
LSETH	5	1	4	2148.63	2148.63	1.000	23 Jul to 25 Aug

Those 38 fill counts sum to 5,661, the fill total this part opens with. Eleven of the 38 end at or above 1.000 and 27 end below it.

The 39 charts of this part, the combined view above included, come from one module, `vwap_charts`. It reads the export once, groups the fills by asset, and writes one PNG per charted base.

The line each chart draws is a running mean of price weighted by quantity. The price is the export's Price at Transaction column and the quantity is its Quantity Transacted column. Buys and sells accumulate in two separate pairs of sums, so a fill on one side steps that side's line and leaves the other flat. The lower panel is the same walk of quantities, added on a buy and taken away on a sell.

The module is not committed to this repository. Neither is the export it reads.

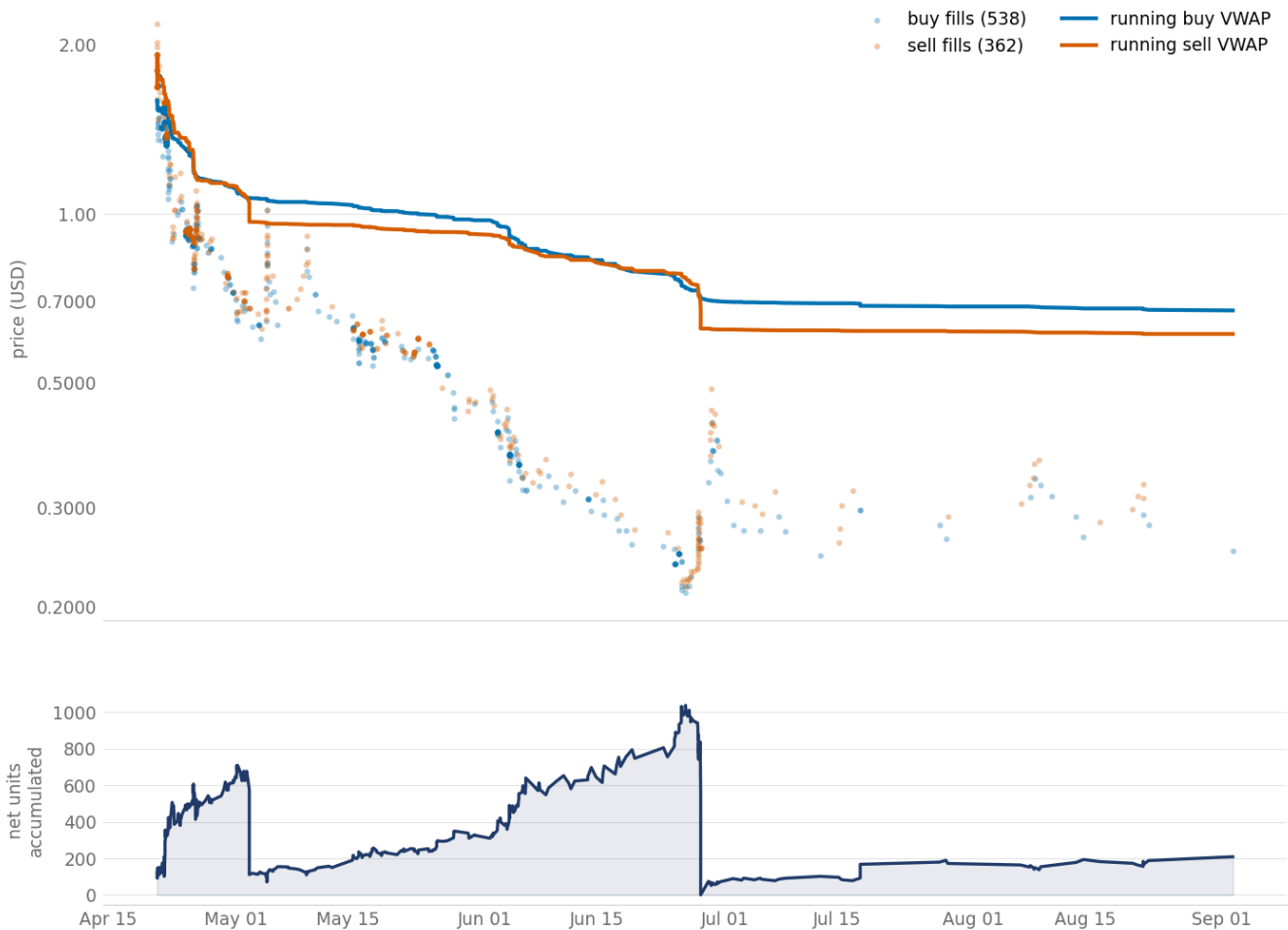
```
def load_fills(src: Path):
    """Return the Fill list and a Counter keyed by drop cause."""
    ...
    price = parse_money(r[idx["Price at Transaction"]])
    qty = abs(parse_money(r[idx["Quantity Transacted"]]))
    ...
    fills.append(Fill(asset, when, side, price, qty))

def trajectory(fills):
    """Return times and the running buy VWAP, sell VWAP and net units."""
    ordered = sorted(fills, key=lambda f: f.when)
    t, buy_v, sell_v, held = [], [], [], []
    bn = bd = sn = sd = pos = 0.0
    for f in ordered:
        if f.side == "buy":
            bn += f.price * f.qty
            bd += f.qty
            pos += f.qty
        else:
            sn += f.price * f.qty
            sd += f.qty
            pos -= f.qty
        t.append(f.when)
        buy_v.append(bn / bd if bd > 0 else float("nan"))
        sell_v.append(sn / sd if sd > 0 else float("nan"))
        held.append(pos)
    return t, buy_v, sell_v, held
```

Four of the 38 charts below carry a block of their own. Each one is a chart where the module takes a turn it does not take for the rest: a logarithmic price axis, the minor ticks that axis needs, a tick label small enough to go scientific, and a fill count low enough to mark every step. The other 34 are drawn by the two functions above and nothing else.

RAVE

RAVE — VWAP trajectory across 900 fills (538 buy, 362 sell)



RAVE, 900 fills, a logarithmic price axis, and a net-units panel that empties twice.

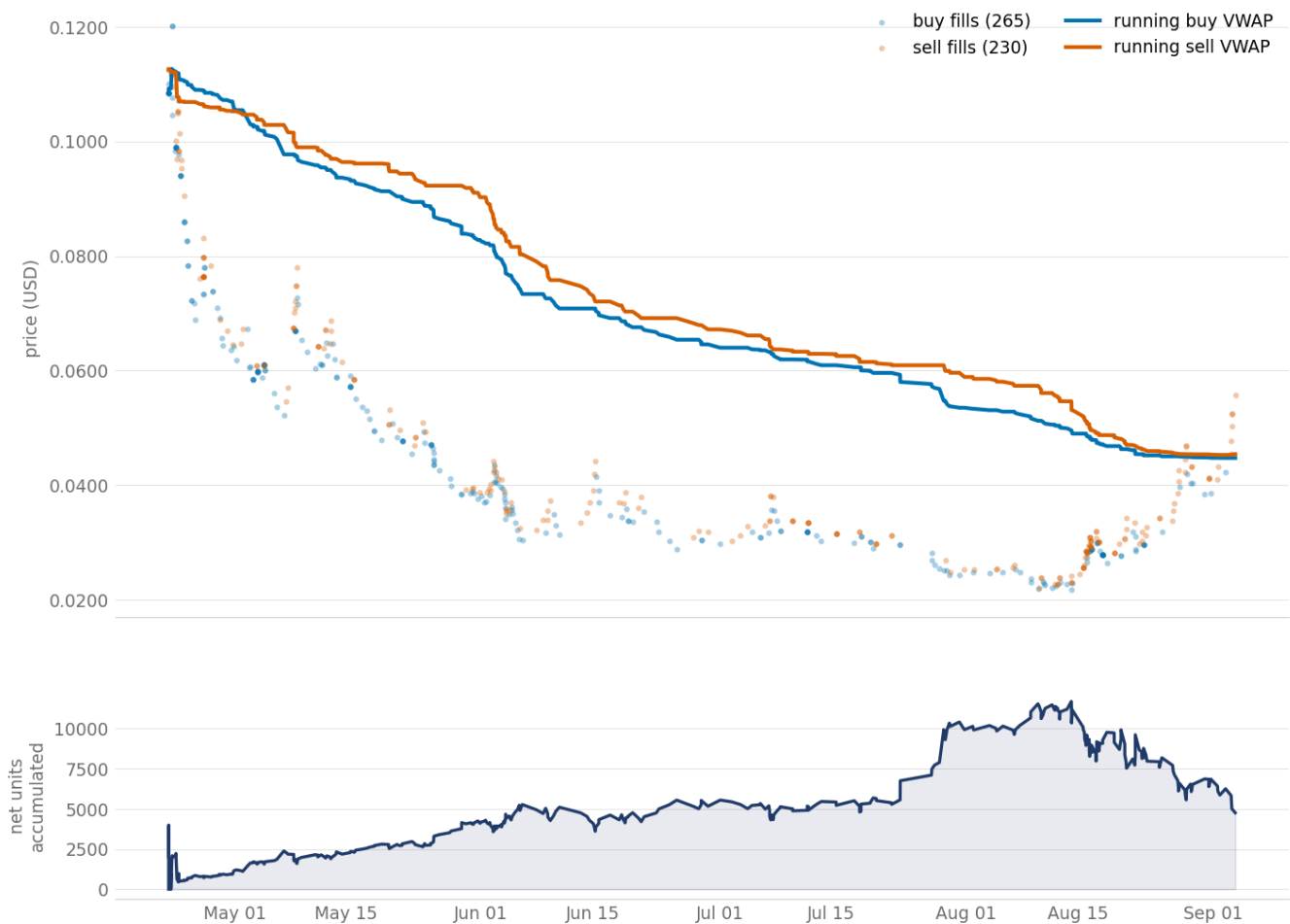
The most-traded base of the 38, and one of the two charts whose price axis runs logarithmic. Both lines step down together to about 1.00 by early May, then to about 0.67 at the end of June, where the sell line settles below the buy line and stays there. The net-units panel peaks at 1,037 units in late June and ends at 209, with two near-vertical drops.

The axis is a measurement, not a choice. RAVE's fills run from 0.2118 to 2.1805, a span of 10.30, and the module turns the axis logarithmic above eight. BILL is the only other chart over that line.

```
lo = min(f.price for f in fills)
hi = max(f.price for f in fills)
if hi / lo > 8:
    ax.set_yscale("log")
```

CHIP

CHIP — VWAP trajectory across 495 fills (265 buy, 230 sell)

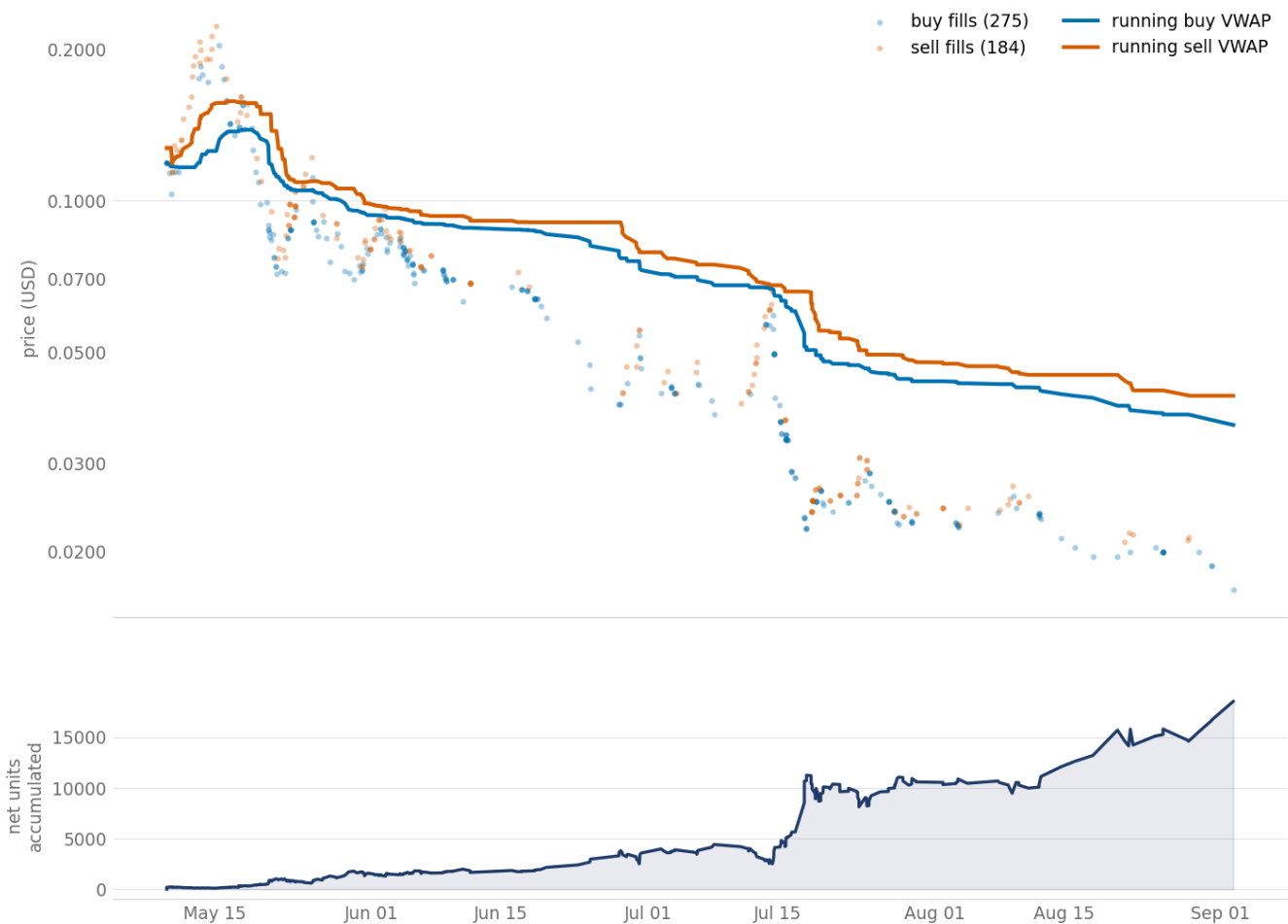


CHIP, 495 fills, with the sell line above the buy line for the whole window.

The sell line runs above the buy line across the whole window and the two meet in September. Net units climb to 11,699 in mid-August and end at 4,765.

BILL

BILL — VWAP trajectory across 459 fills (275 buy, 184 sell)



BILL, 459 fills, ending at the lowest index of the eight most-traded assets.

The second logarithmic axis, at a span of 13.25. Both lines hold near 0.09 through June, then break down in mid-July. Net units end at the highest point of the window, 18,510.

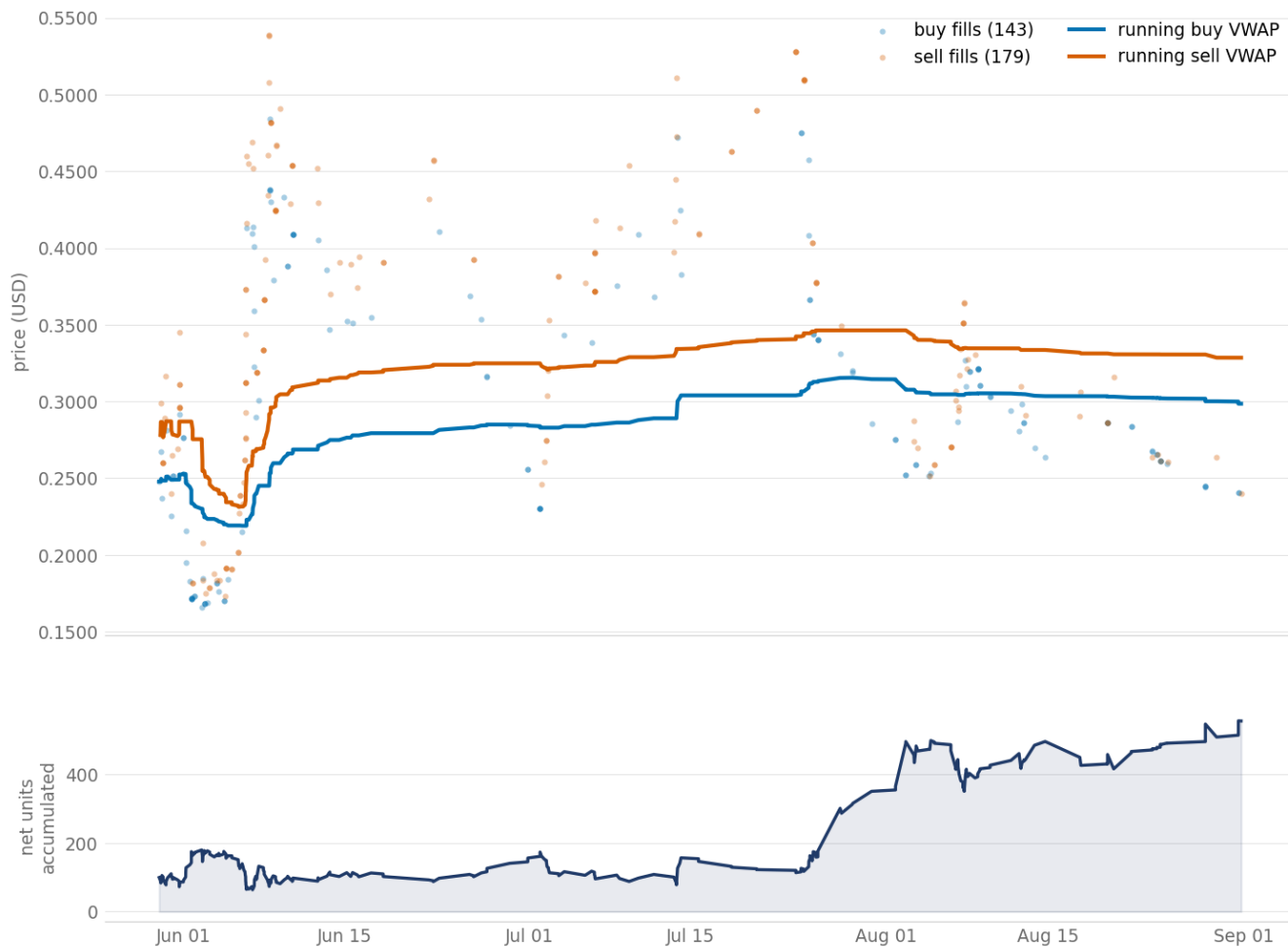
A logarithmic axis labels the powers of ten. BILL's fills run from 0.01682 to 0.22281, a window holding one of them, and RAVE's window holds one as well. The module adds five minor ticks per decade and gives them the same price formatter as the major ones, which is what puts 0.2000 down to 0.0200 on this axis.

```
SUBS = (1.0, 2.0, 3.0, 5.0, 7.0)

ax.yaxis.set_minor_locator(LogLocator(base=10.0, subs=SUBS, numticks=99))
ax.yaxis.set_minor_formatter(FuncFormatter(fmt_price))
ax.tick_params(axis="y", which="minor", labelsize=TYPE["cap"]["size"])
```

ALLO

ALLO — VWAP trajectory across 322 fills (143 buy, 179 sell)

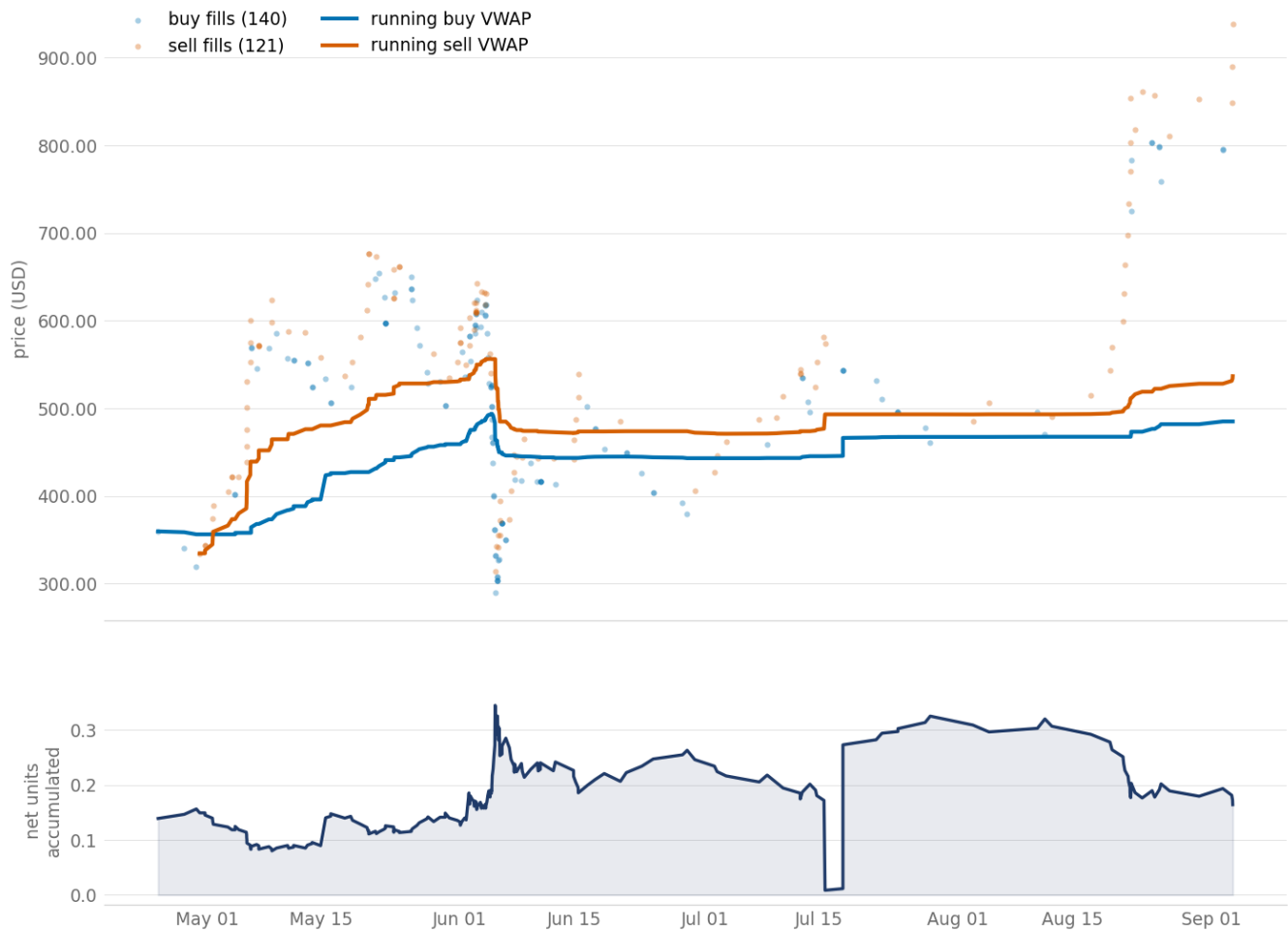


ALLO, 322 fills, with more sells than buys and both lines rising.

One of eleven charts with more sells than buys. Both lines dip to about 0.22 in early June and climb after it. Net units step from about 120 to about 500 through late July.

ZEC

ZEC — VWAP trajectory across 261 fills (140 buy, 121 sell)

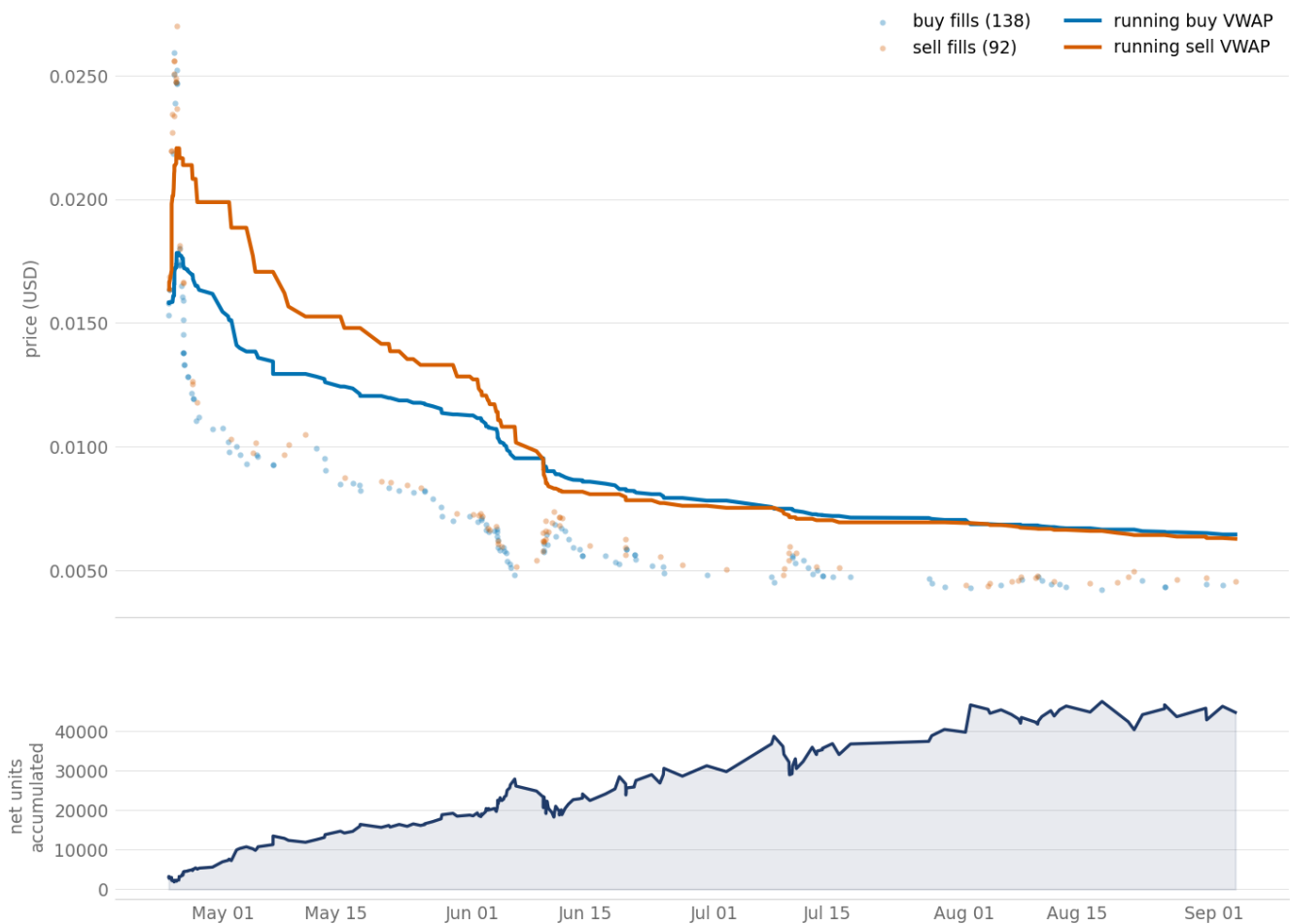


ZEC, 261 fills, the highest final index of the eight most-traded assets.

The buy line rises from 359.67 to 485.05, the largest climb among the eight in the combined chart. Net units fall to near zero in mid-July, then rebuild to about 0.20.

KAT

KAT — VWAP trajectory across 230 fills (138 buy, 92 sell)

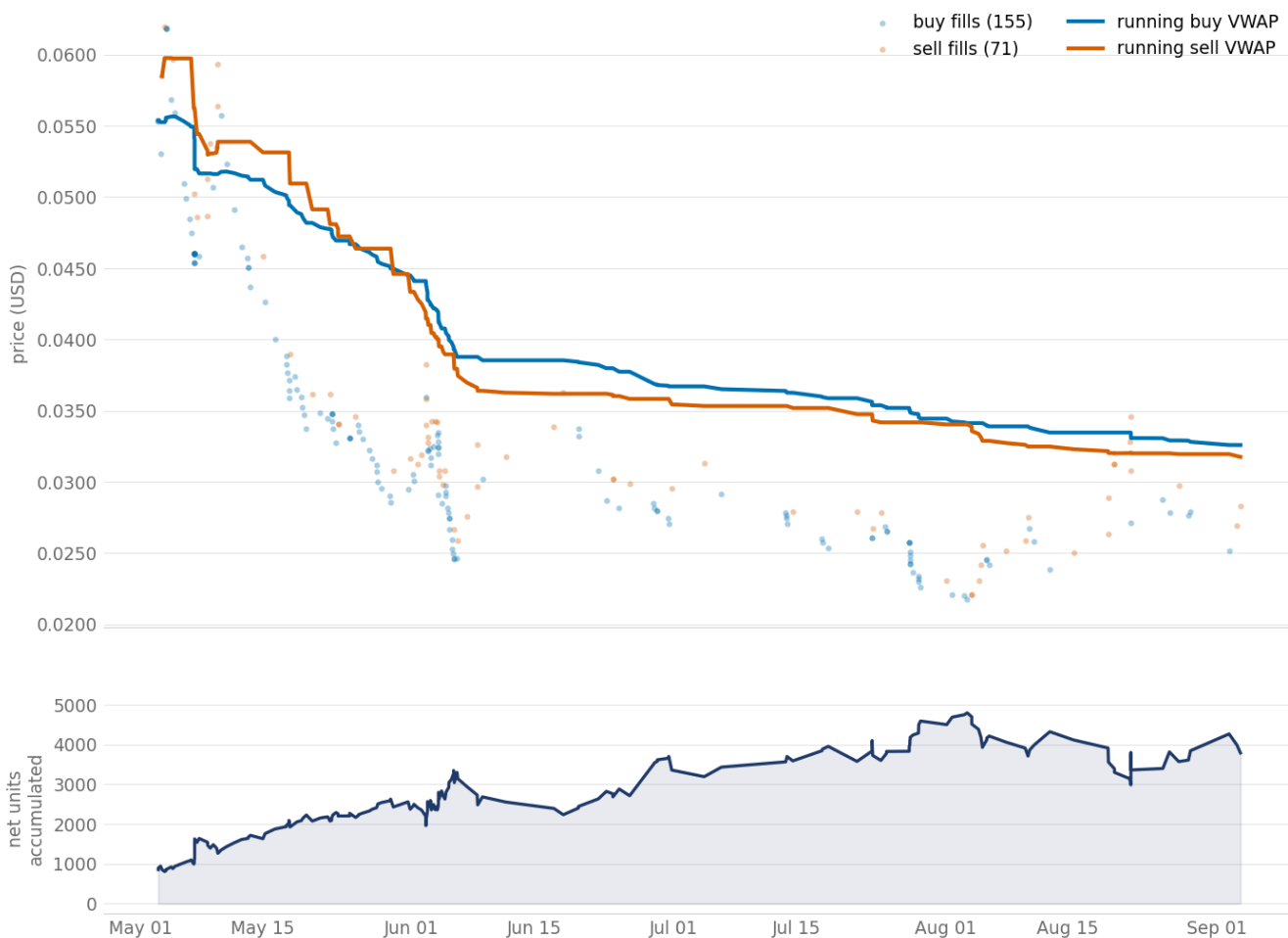


KAT, 230 fills, with the two lines converging from mid-June.

The sell line runs above the buy line until mid-June, then the two converge and the sell line finishes just below. Net units grow to 47,574 and end at 44,795.

BIO

BIO — VWAP trajectory across 226 fills (155 buy, 71 sell)

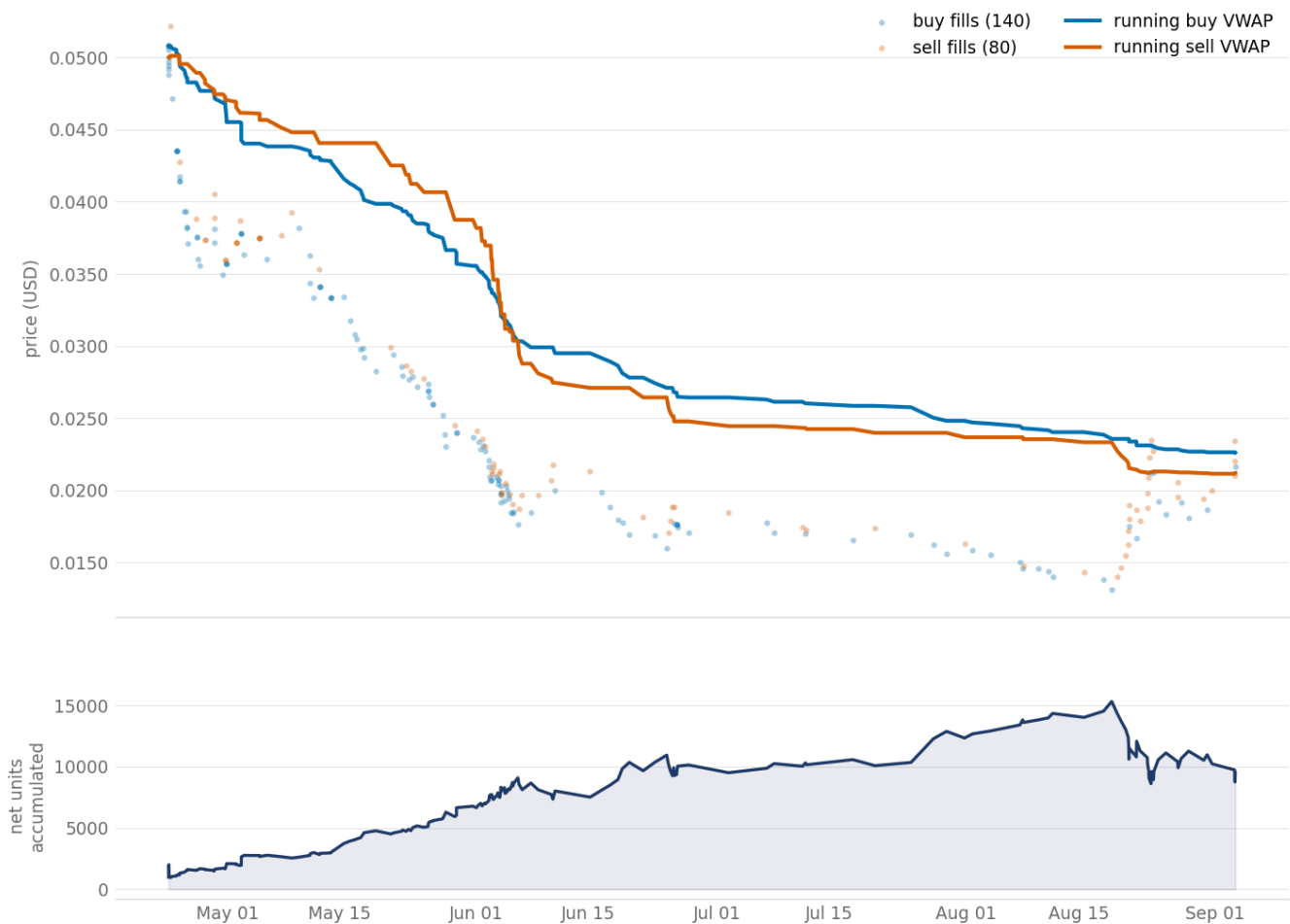


BIO, 226 fills, with a step down at the start of June.

One step down at the start of June takes the buy line from about 0.044 to about 0.039, and a slow decline follows. The sell line crosses below the buy line at that same step.

SPK

SPK — VWAP trajectory across 220 fills (140 buy, 80 sell)

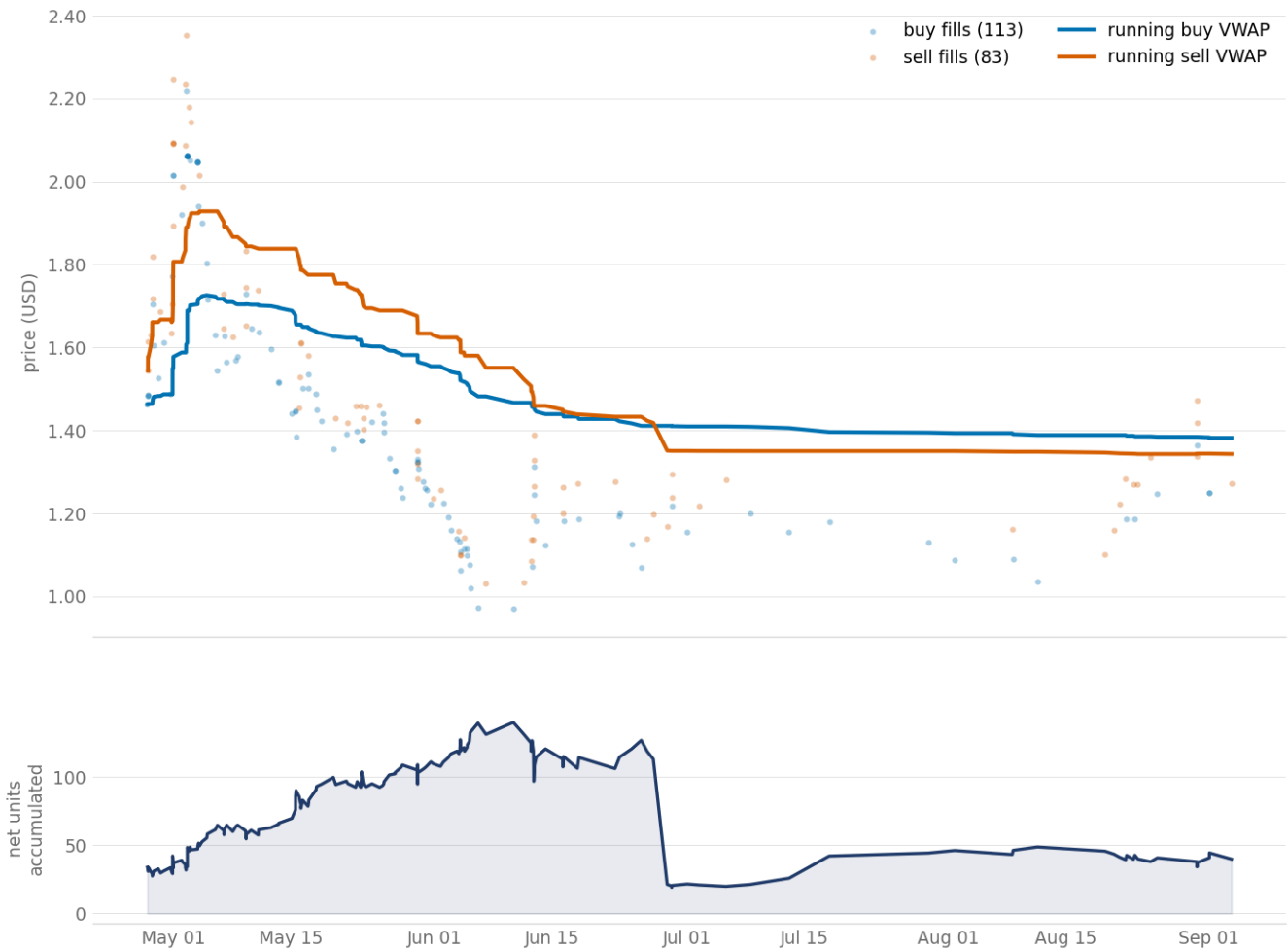


SPK, 220 fills, with the sell line crossing below the buy line in mid-June.

The crossing holds for the rest of the window. Net units peak at 15,301 in mid-August, then fall to 9,483.

ORCA

ORCA — VWAP trajectory across 196 fills (113 buy, 83 sell)

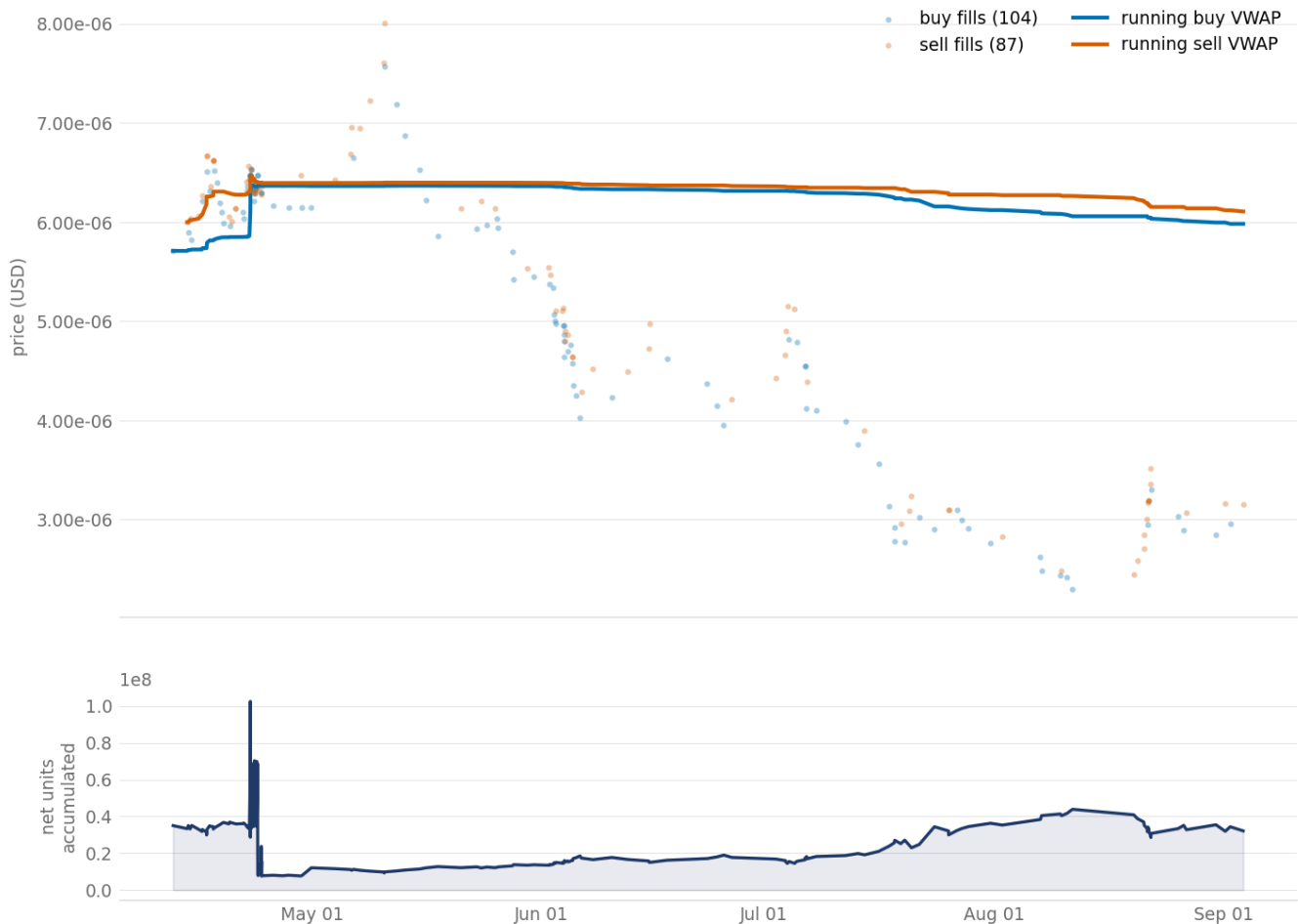


ORCA, 196 fills, with both lines flat from July onward.

Both lines rise to a peak near 1 May, then fall to a flat run from July. Net units drop from 140 to about 20 at the end of June and hold near 40 after that.

BONK

BONK — VWAP trajectory across 191 fills (104 buy, 87 sell)



BONK, 191 fills, priced near six millionths of a dollar.

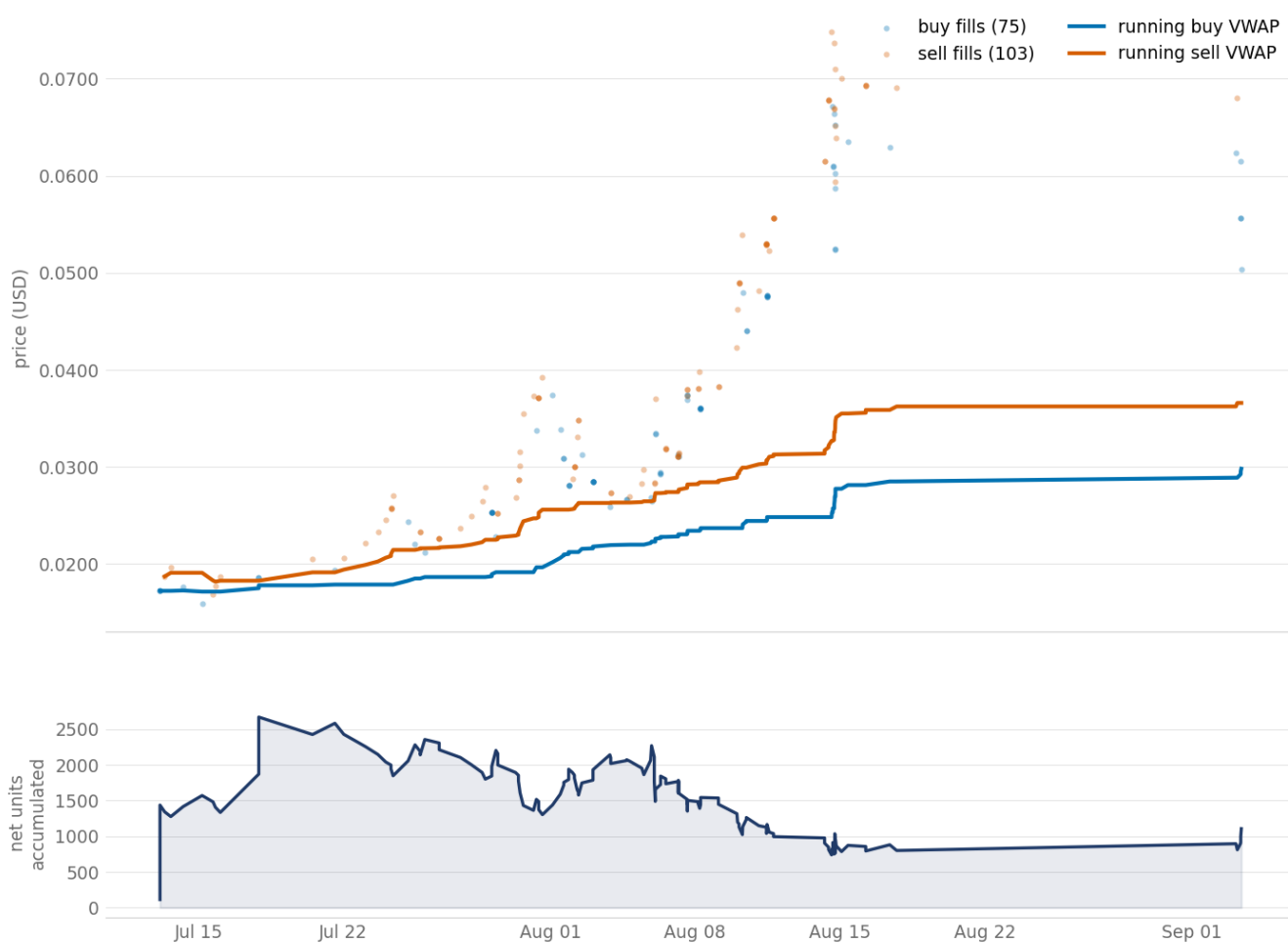
The lowest-priced base in the record. The buy line holds near 6.3×10^{-6} from late April while the fill dots fall from 8.0×10^{-6} to 2.3×10^{-6} . Net units spike to 1.02×10^8 in late April and end at 3.22×10^7 .

BONK is the only base whose fills fall under a tenth of a cent. The next lowest is PUMP at 0.001479, so BONK alone reaches the last branch of the tick formatter, and this is the only chart of the 38 labelled in scientific notation.

```
def fmt_price(v, _pos=None) -> str:
    """Format a y-axis price tick, keeping a micro-cap significant visible."""
    if v == 0:
        return "0"
    if abs(v) >= 1000:
        return f"{v:,.0f}"
    if abs(v) >= 1:
        return f"{v:,.2f}"
    if abs(v) >= 0.001:
        return f"{v:.4f}"
    return f"{v:.2e}"
```

CAP

CAP — VWAP trajectory across 178 fills (75 buy, 103 sell)

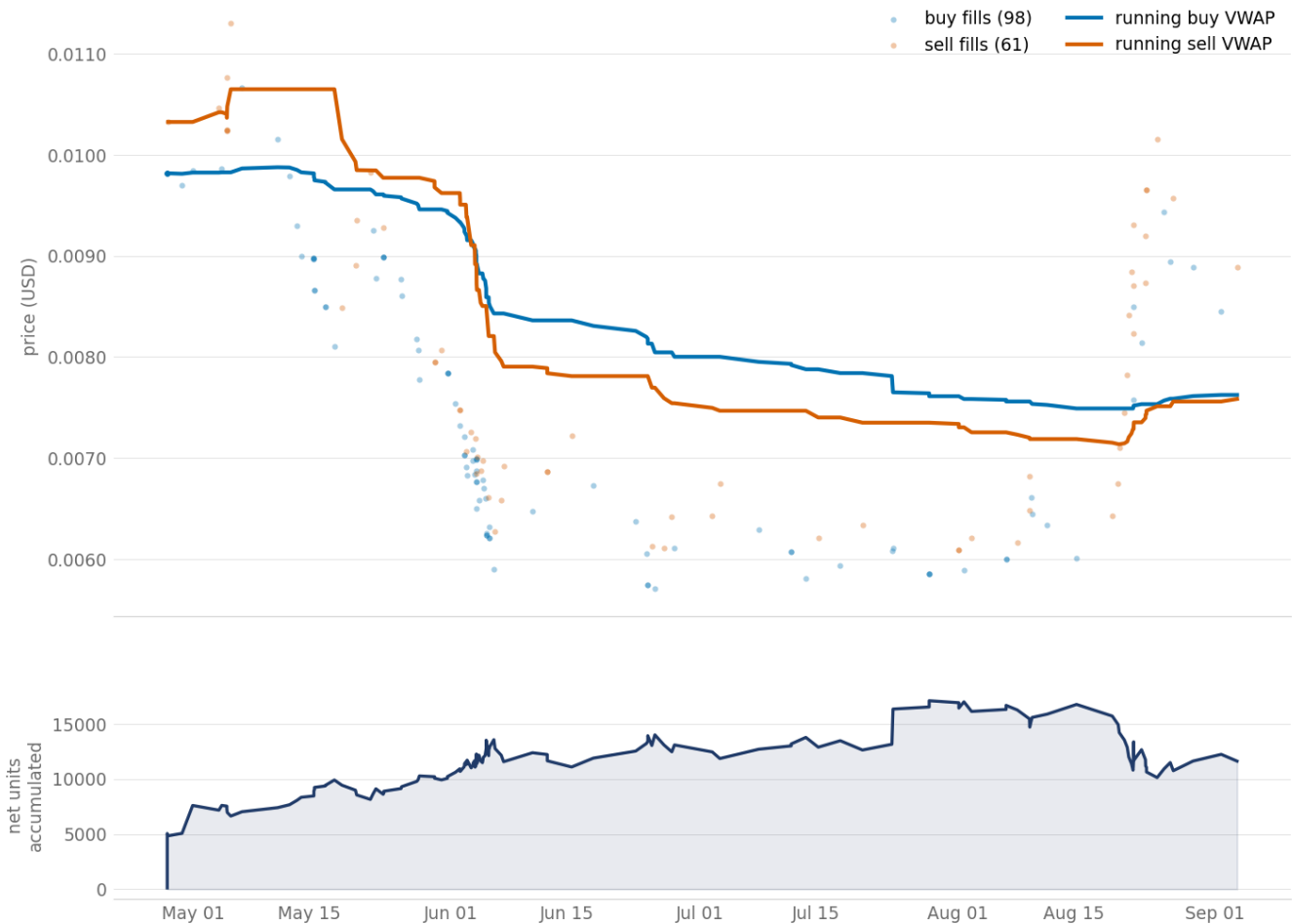


CAP, 178 fills, and the largest rise of the 38.

The buy line rises from 0.01722 to 0.02979, an index of 1.730 and the largest of the 38. Both lines climb through August. Net units drop across that same climb, 2,669 down to 1,107.

PENGU

PENGU — VWAP trajectory across 159 fills (98 buy, 61 sell)

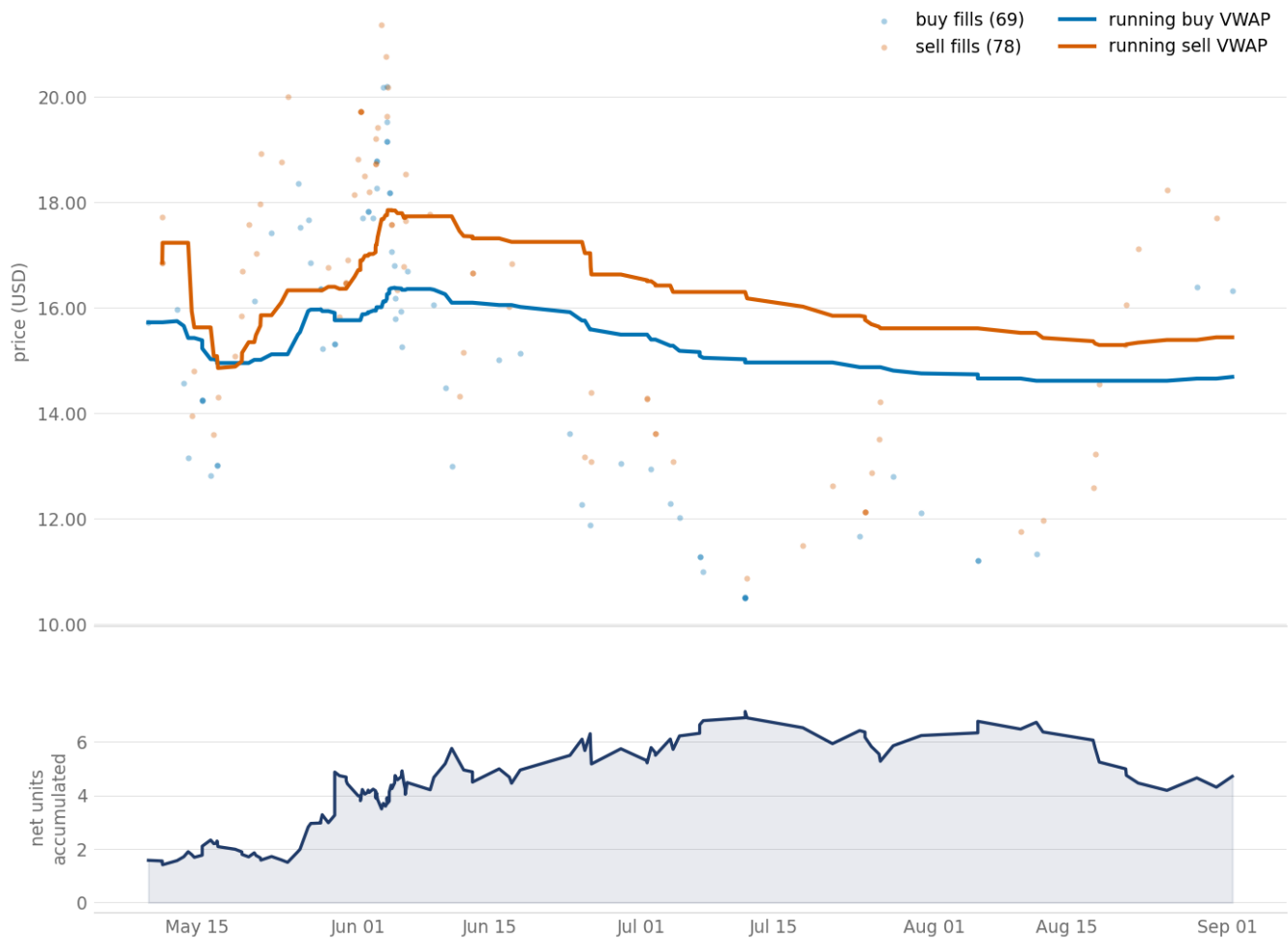


PENGU, 159 fills, with the sell line dropping below the buy line at the start of June.

The buy line holds near 0.0098 through May and the sell line near 0.0105, then both step down at the start of June. Net units reach 17,089 in early August and end at 11,612.

VVV

VVV — VWAP trajectory across 147 fills (69 buy, 78 sell)

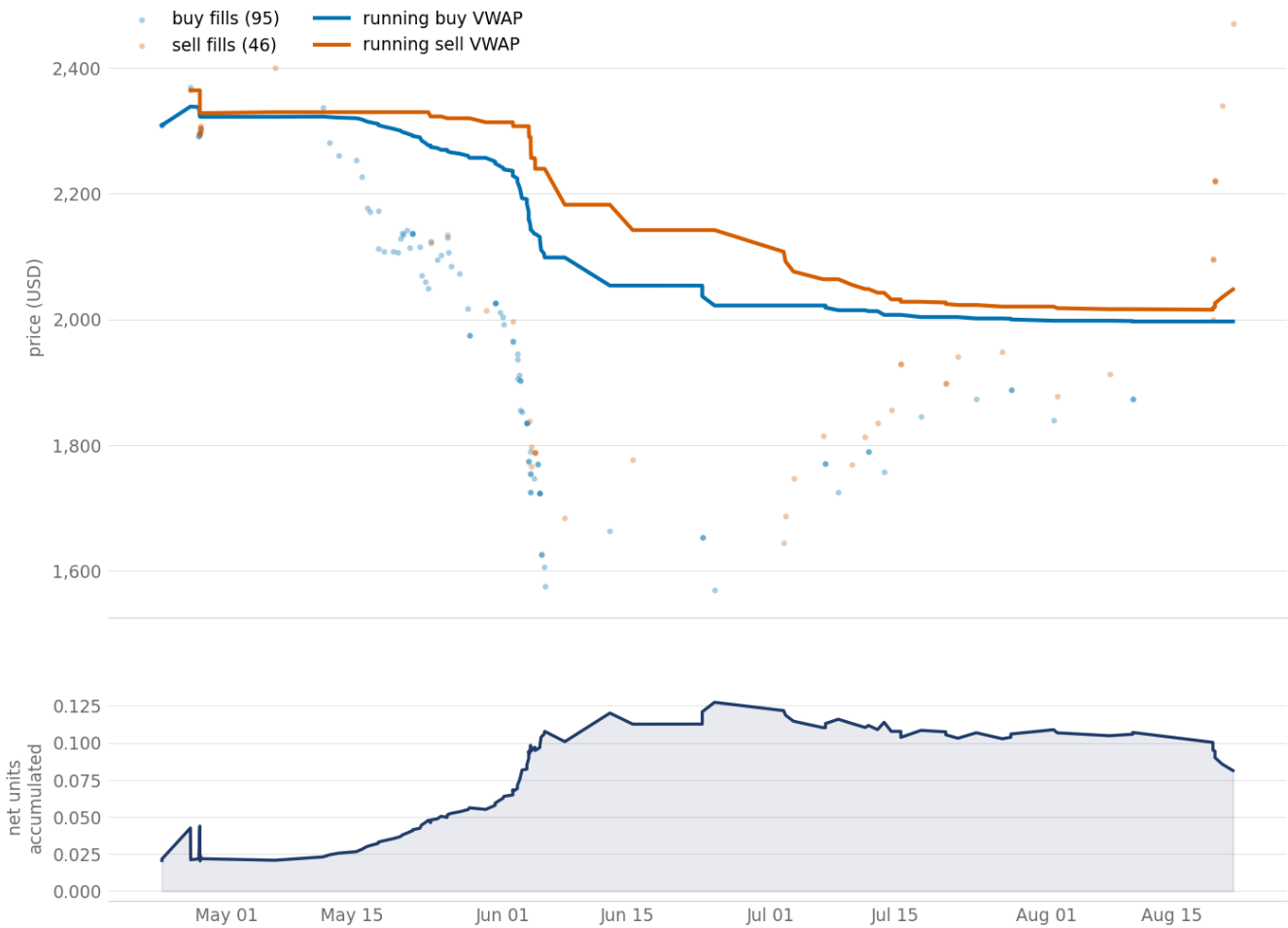


VVV, 147 fills, with the sell line above the buy line throughout.

The sell line stays above the buy line for the whole window, ending near 15.4 against 14.7. Net units peak at 7.13 and end at 4.71.

ETH

ETH — VWAP trajectory across 141 fills (95 buy, 46 sell)

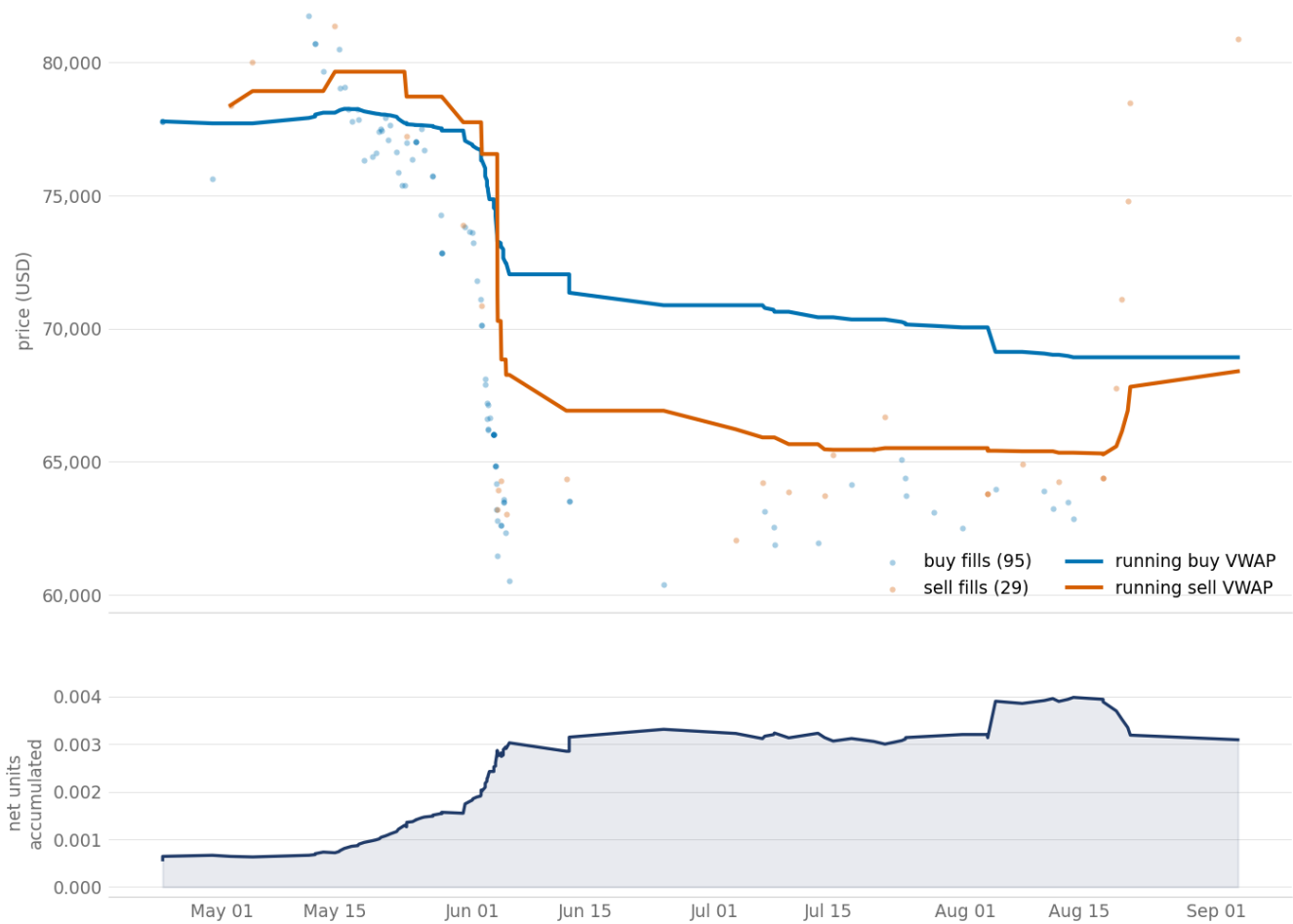


ETH, 141 fills, ending 21 August.

The window closes on 21 August, the earliest close of the 38 apart from WLF1. The two lines run together until the start of June, then split, with the sell line about 100 dollars above through June and July. They meet again in August.

BTC

BTC — VWAP trajectory across 124 fills (95 buy, 29 sell)

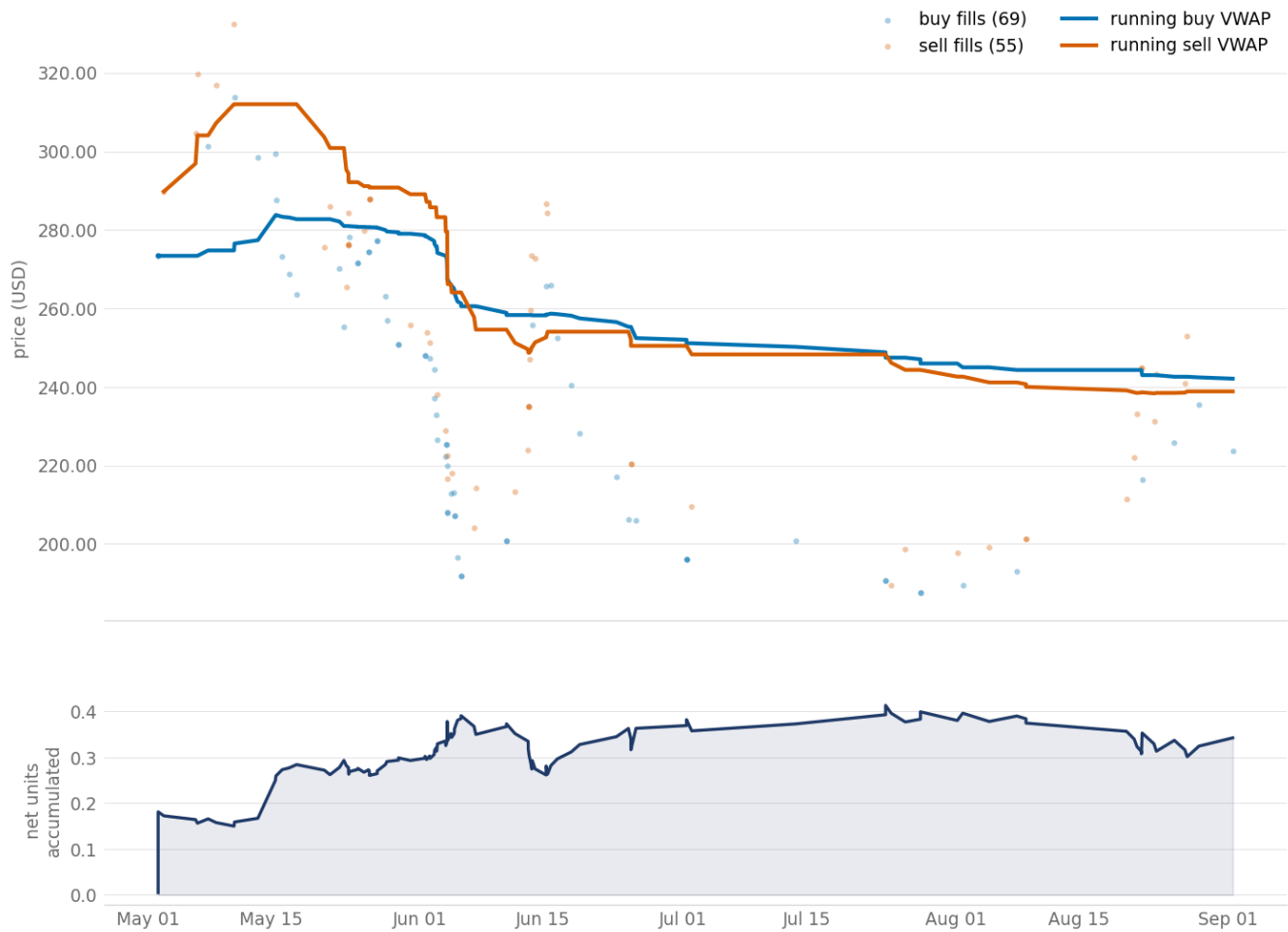


BTC, 124 fills, with the two lines splitting at the start of June.

The highest-priced base in the record. Both lines hold near 78,000 through May. At the start of June the sell line drops to about 68,000 and the buy line to about 72,000, and the gap holds until late August. Net units end at 0.0031.

TAO

TAO — VWAP trajectory across 124 fills (69 buy, 55 sell)

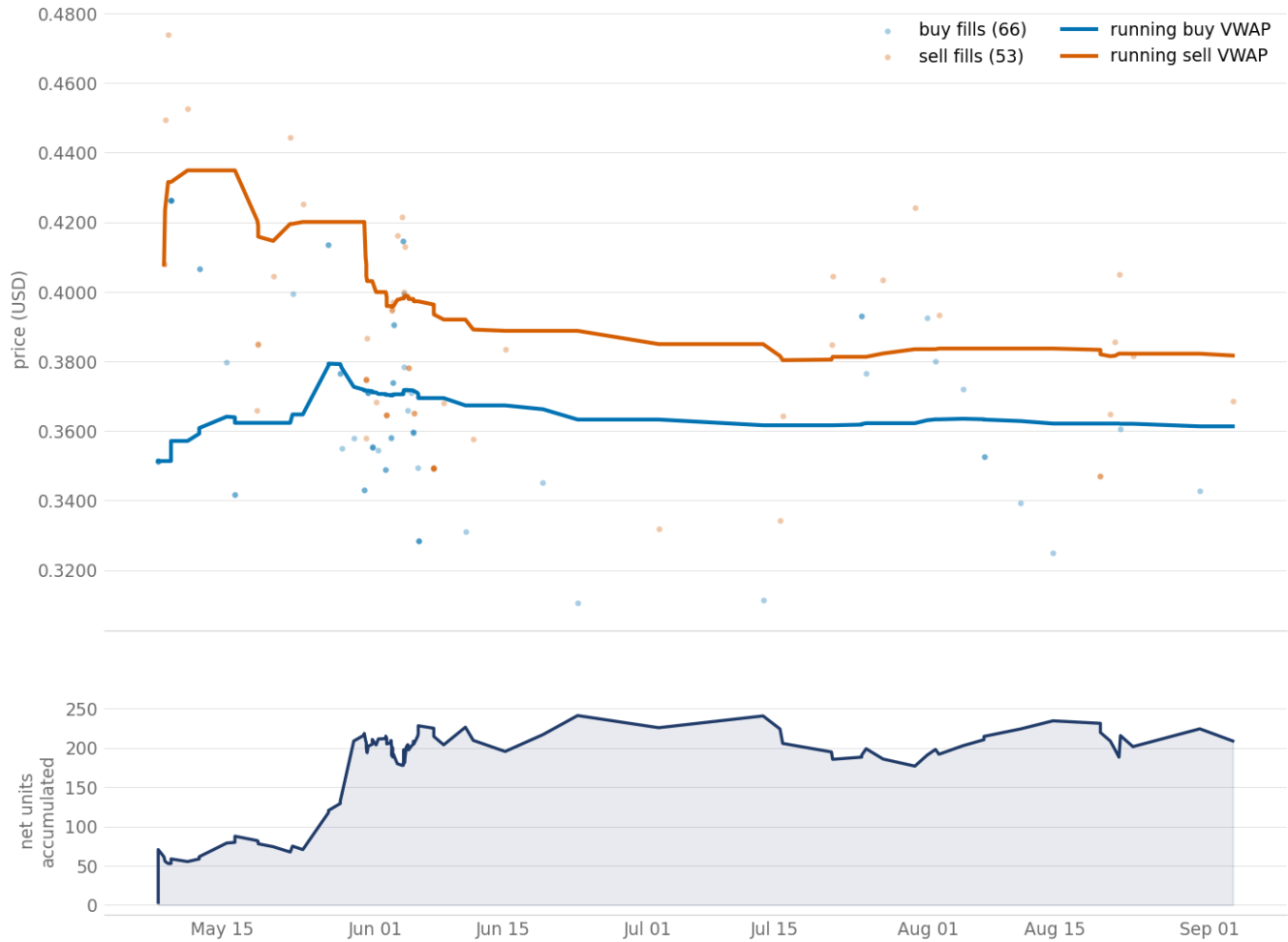


TAO, 124 fills, with the sell line above the buy line until mid-June.

The sell line peaks near 312 in mid-May, about 30 dollars above the buy line, and crosses below it in mid-June. Net units end at 0.34.

ONDO

ONDO — VWAP trajectory across 119 fills (66 buy, 53 sell)

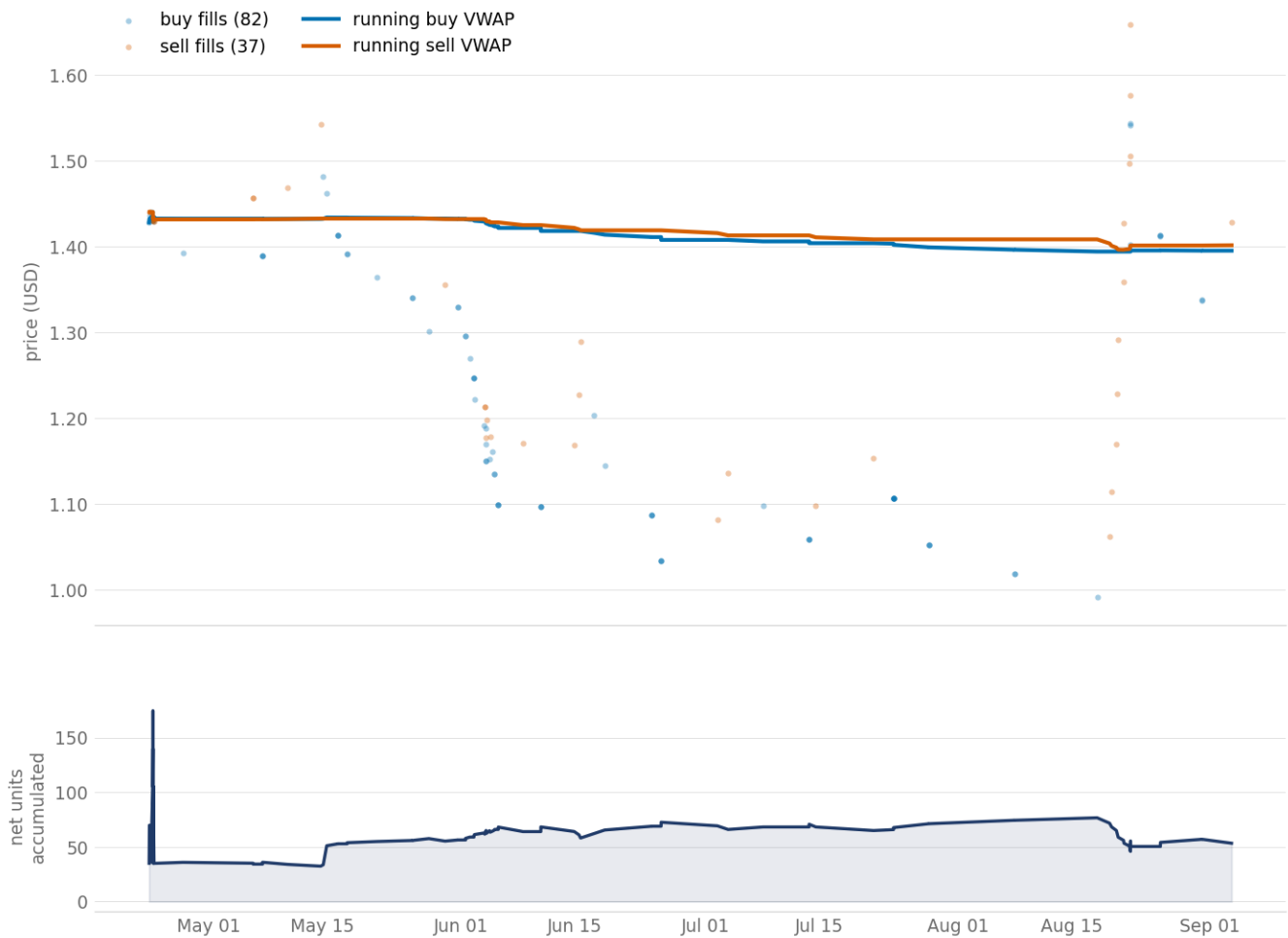


ONDO, 119 fills, with both lines nearly flat after June.

The buy line sits near 0.362 and the sell line near 0.382 from mid-June to the close, a gap of about 0.02 held across eleven weeks.

XRP

XRP — VWAP trajectory across 119 fills (82 buy, 37 sell)

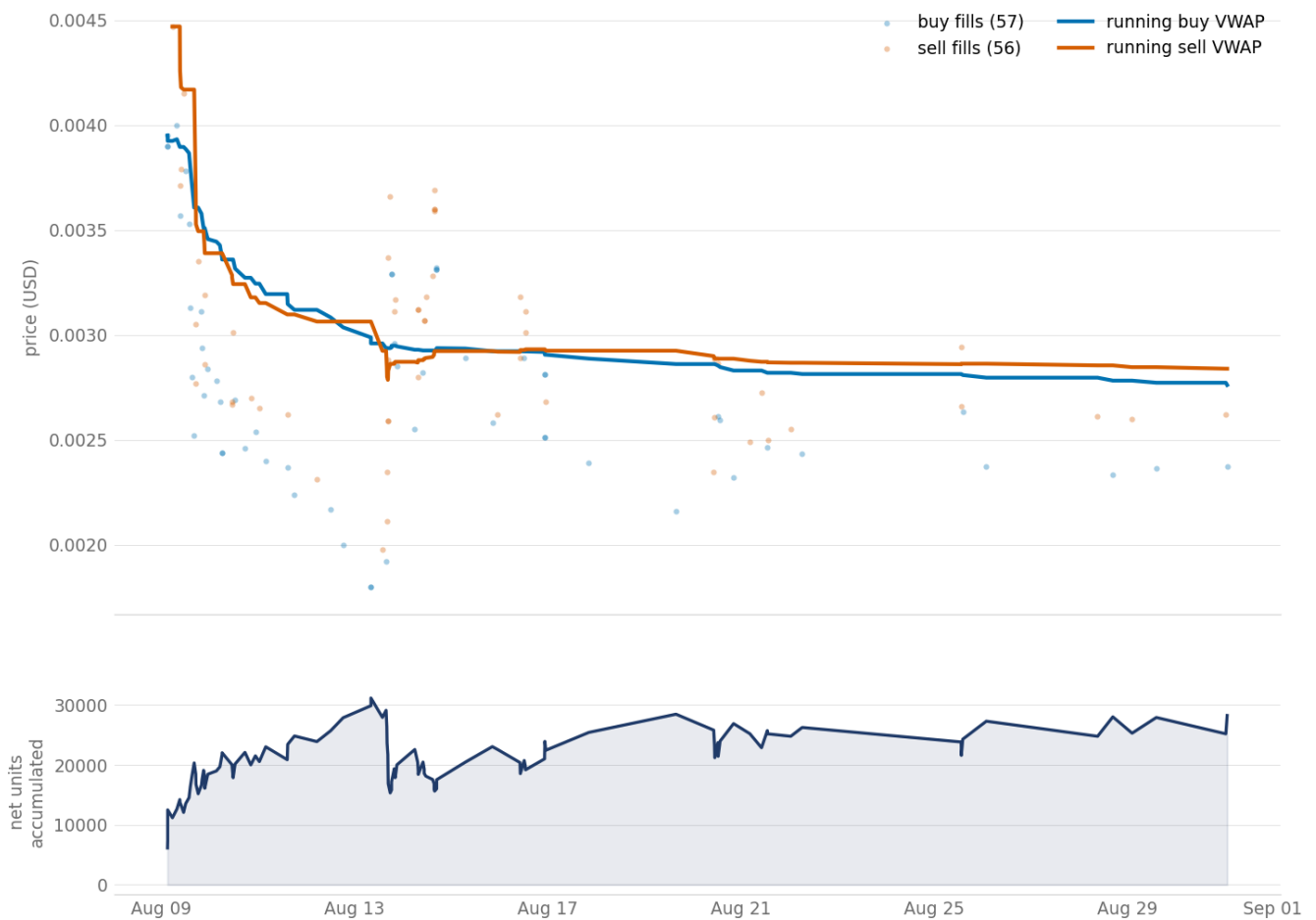


XRP, 119 fills, with the two lines almost touching across the window.

The two lines run within about 0.01 of each other for the whole window. Net units spike to 175 at the start, then hold near 55.

IMU

IMU — VWAP trajectory across 113 fills (57 buy, 56 sell)

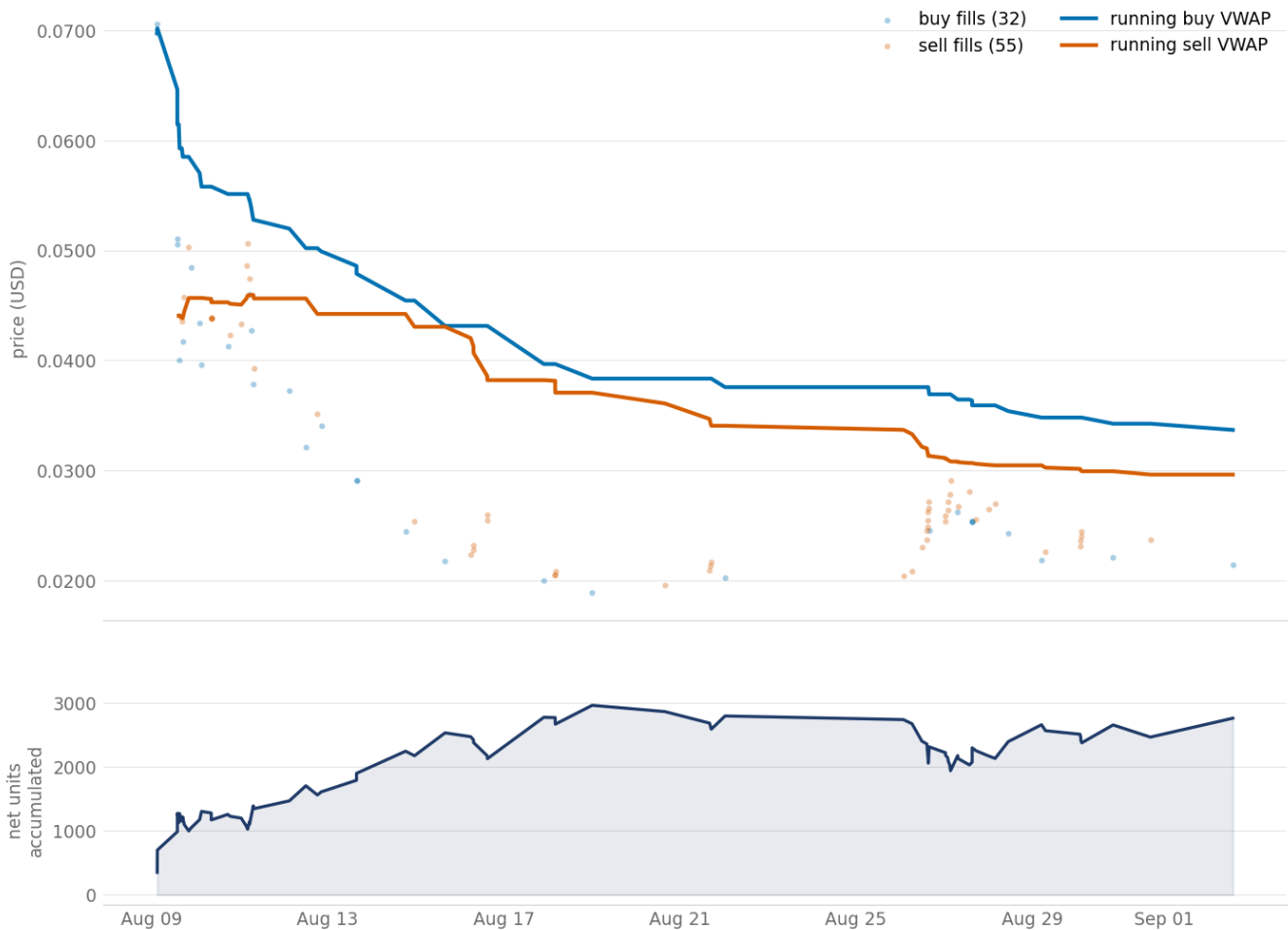


IMU, 113 fills across 22 days in August.

The second-shortest window of the 38. The buy line falls from 0.00395 to about 0.0030 in the first four days, then flattens. The two lines nearly overlap after 17 August.

BICO

BICO — VWAP trajectory across 87 fills (32 buy, 55 sell)

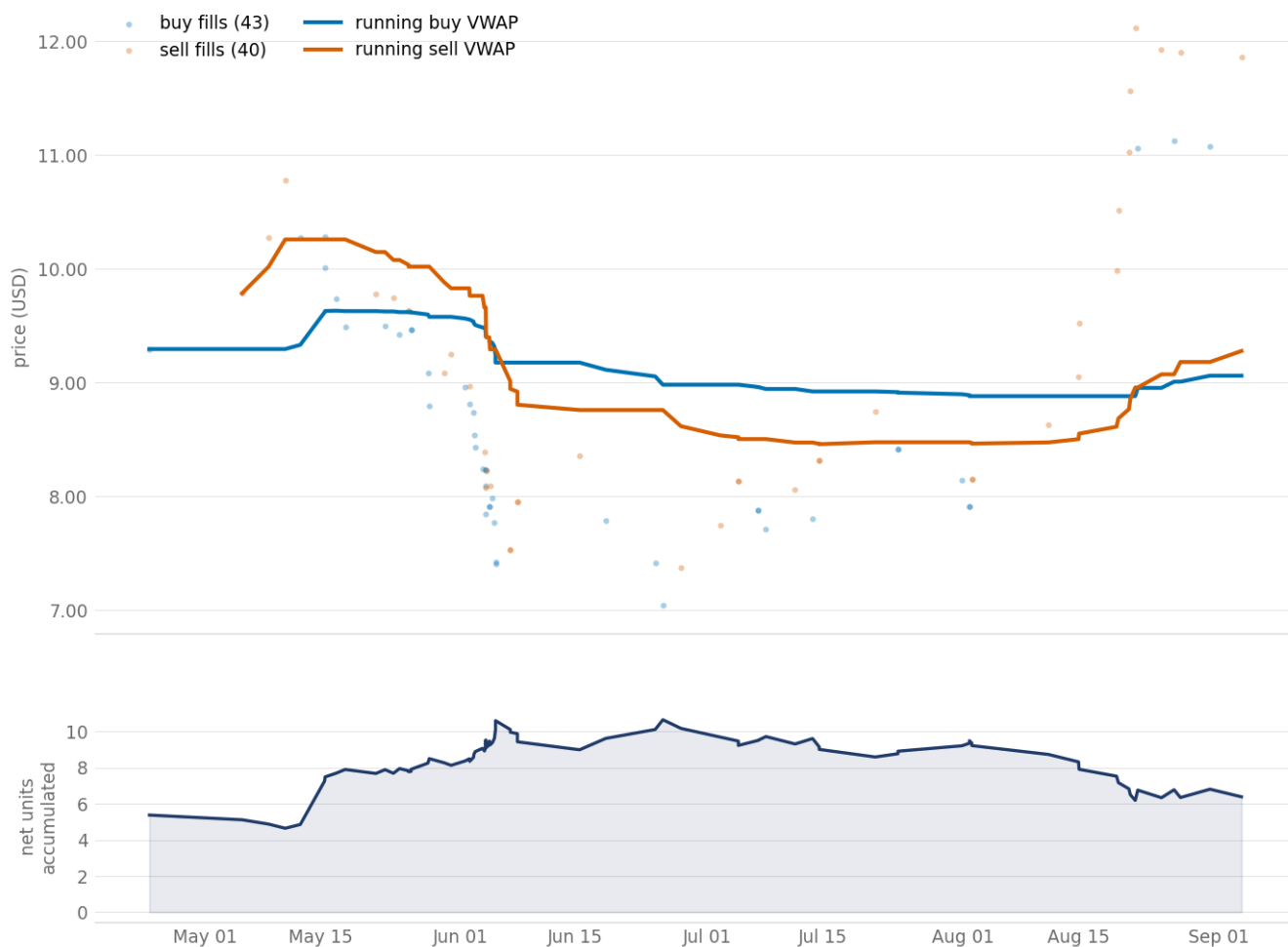


BICO, 87 fills, with the sell line crossing below the buy line in mid-August.

Buys number 32 against 55 sells. The buy line starts at 0.0698 and falls to 0.0337 across 24 days, the steepest fall of the three assets that start in August.

LINK

LINK — VWAP trajectory across 83 fills (43 buy, 40 sell)

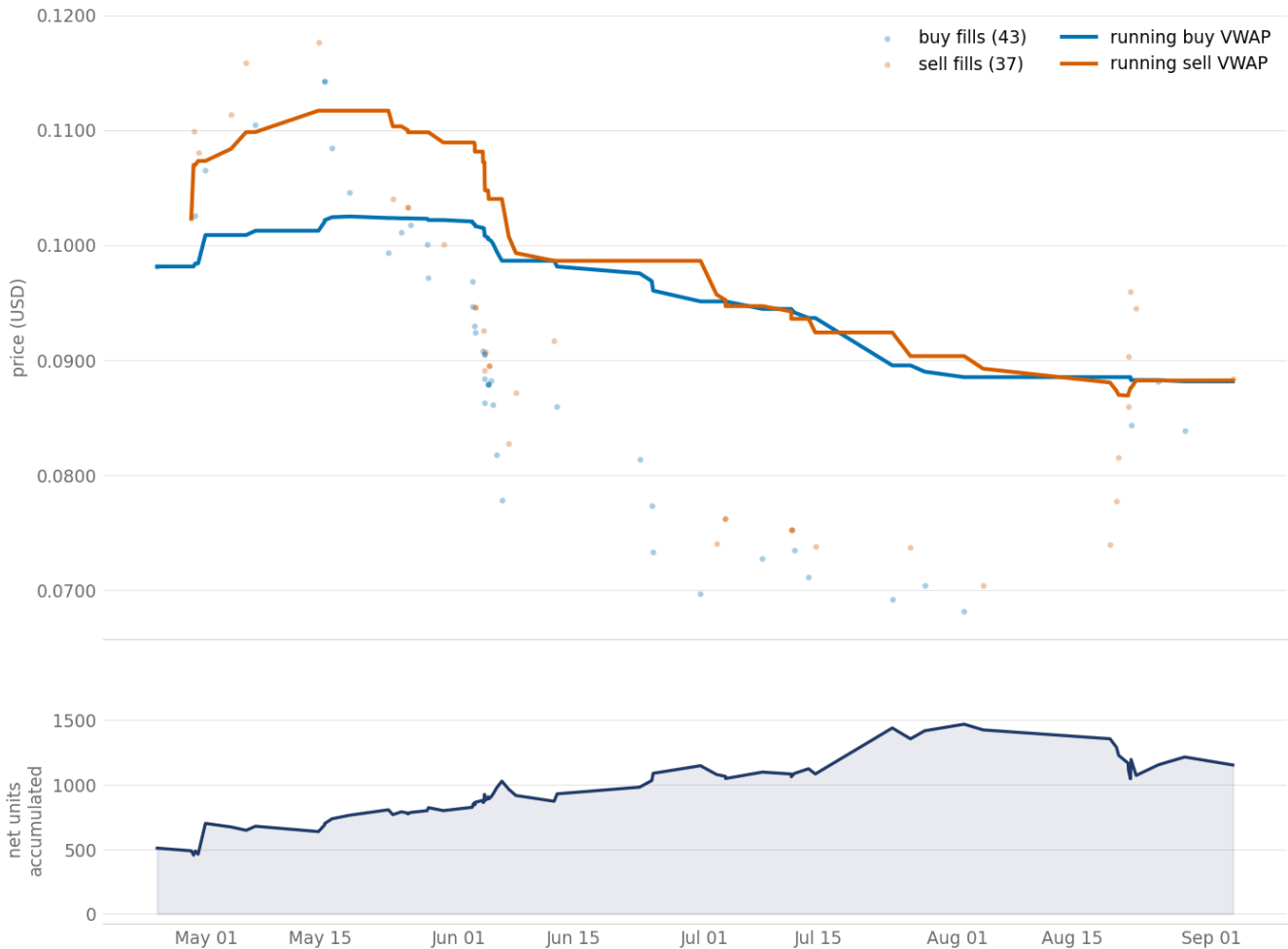


LINK, 83 fills, with both lines turning up in the last two weeks.

The sell line falls below the buy line at the start of June and stays there until late August, when both turn up and cross again.

DOGE

DOGE — VWAP trajectory across 80 fills (43 buy, 37 sell)

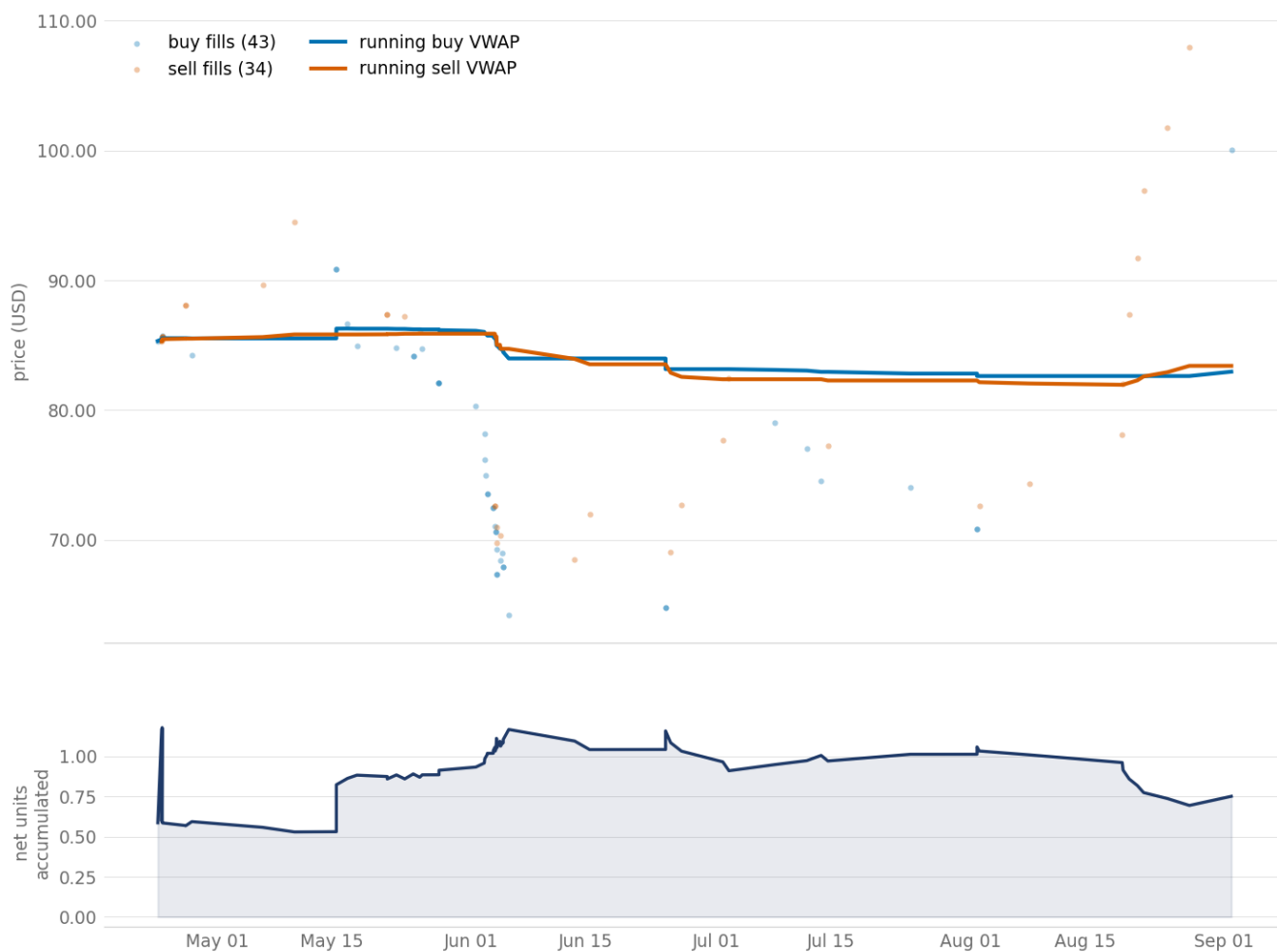


DOGE, 80 fills, with the two lines meeting in mid-August.

The sell line runs above the buy line from May to mid-August, then the two converge near 0.088 and finish together. Net units end at 1,153.

SOL

SOL — VWAP trajectory across 77 fills (43 buy, 34 sell)

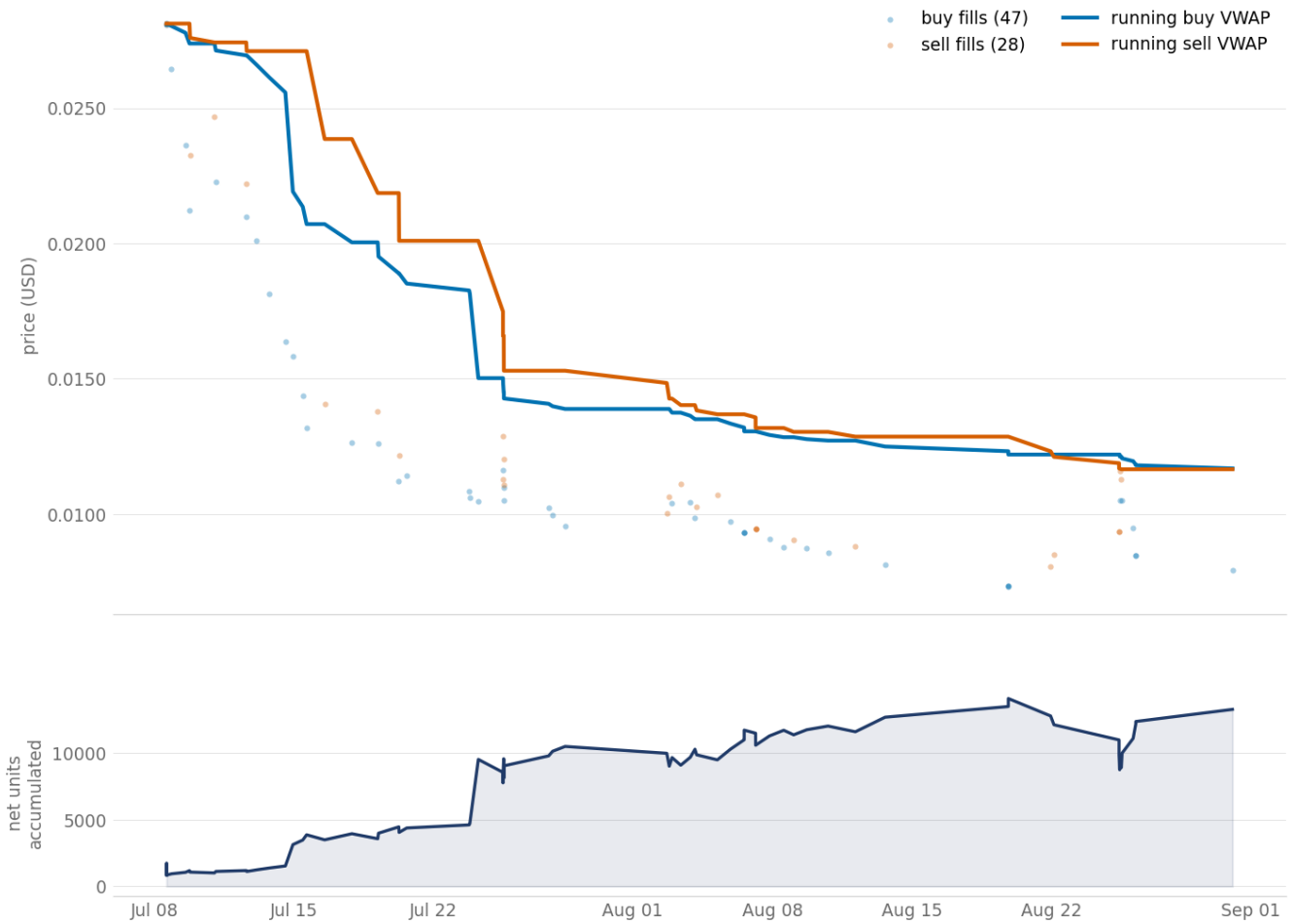


SOL, 77 fills, with the two lines within about a dollar of each other.

Both lines hold near 85 through May, step to about 83 at the start of June, and end within a dollar of each other. Net units end below one whole unit, at 0.75.

GROVE

GROVE — VWAP trajectory across 75 fills (47 buy, 28 sell)

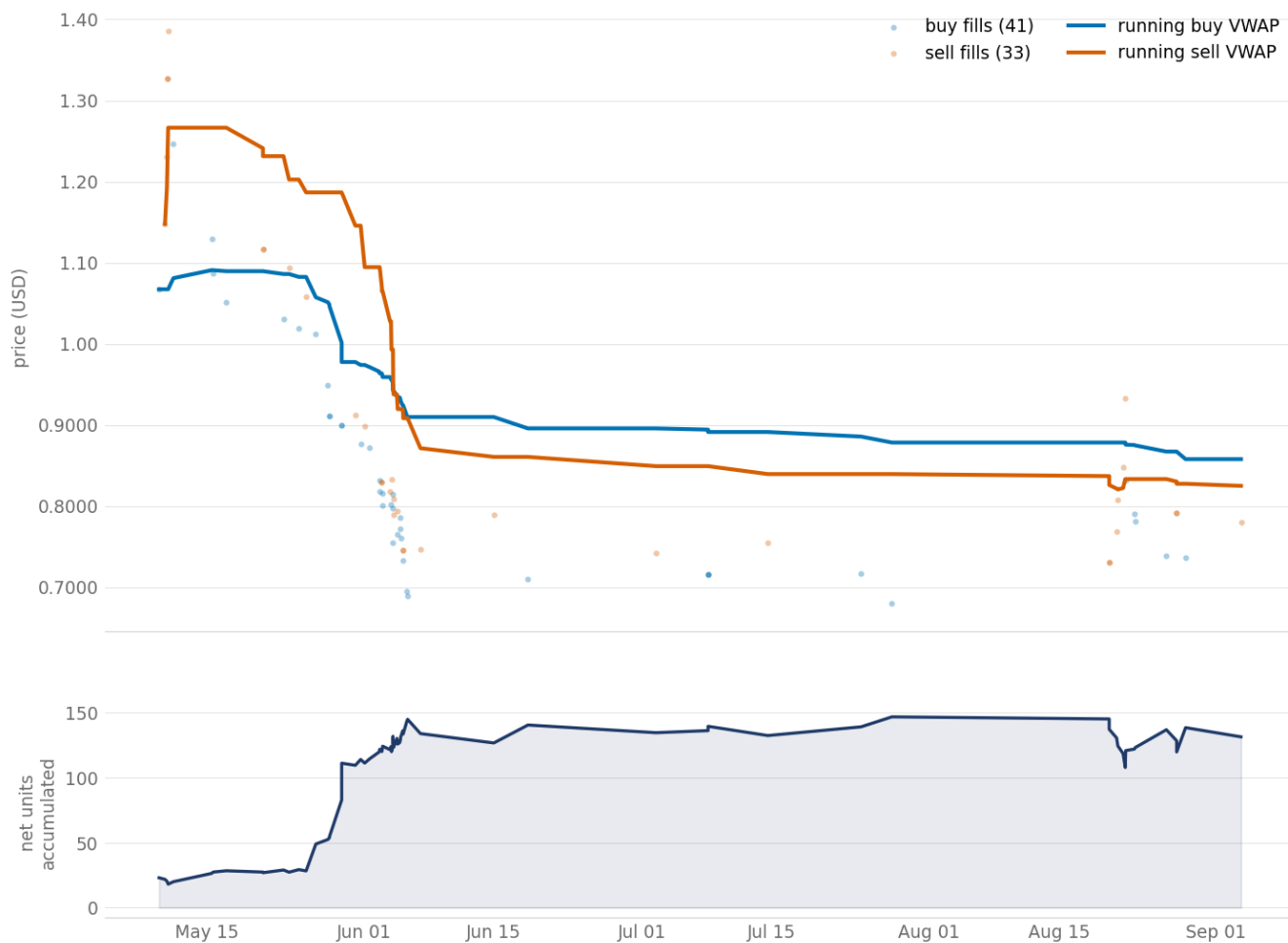


GROVE, 75 fills, with both lines falling by more than half.

A July starter. The buy line falls from 0.02809 to 0.01168 in eight weeks, and the sell line converges onto it by the close. Net units end at 13,268.

SUI

SUI — VWAP trajectory across 74 fills (41 buy, 33 sell)

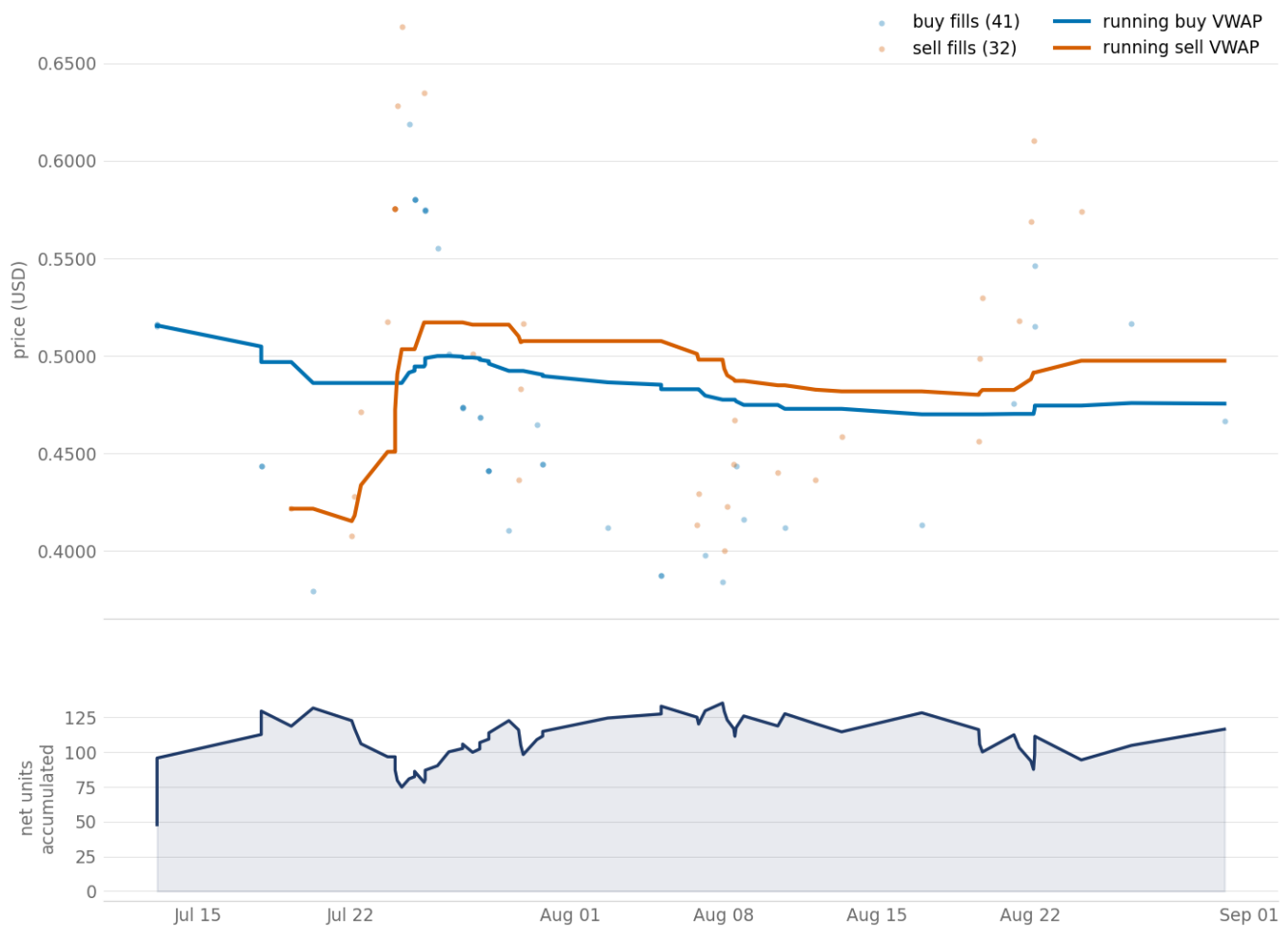


SUI, 74 fills, with a step down at the start of June.

The sell line falls from 1.27 to 0.87 across late May and early June, crossing below the buy line, and the two hold about 0.04 apart afterwards.

RE

RE — VWAP trajectory across 73 fills (41 buy, 32 sell)

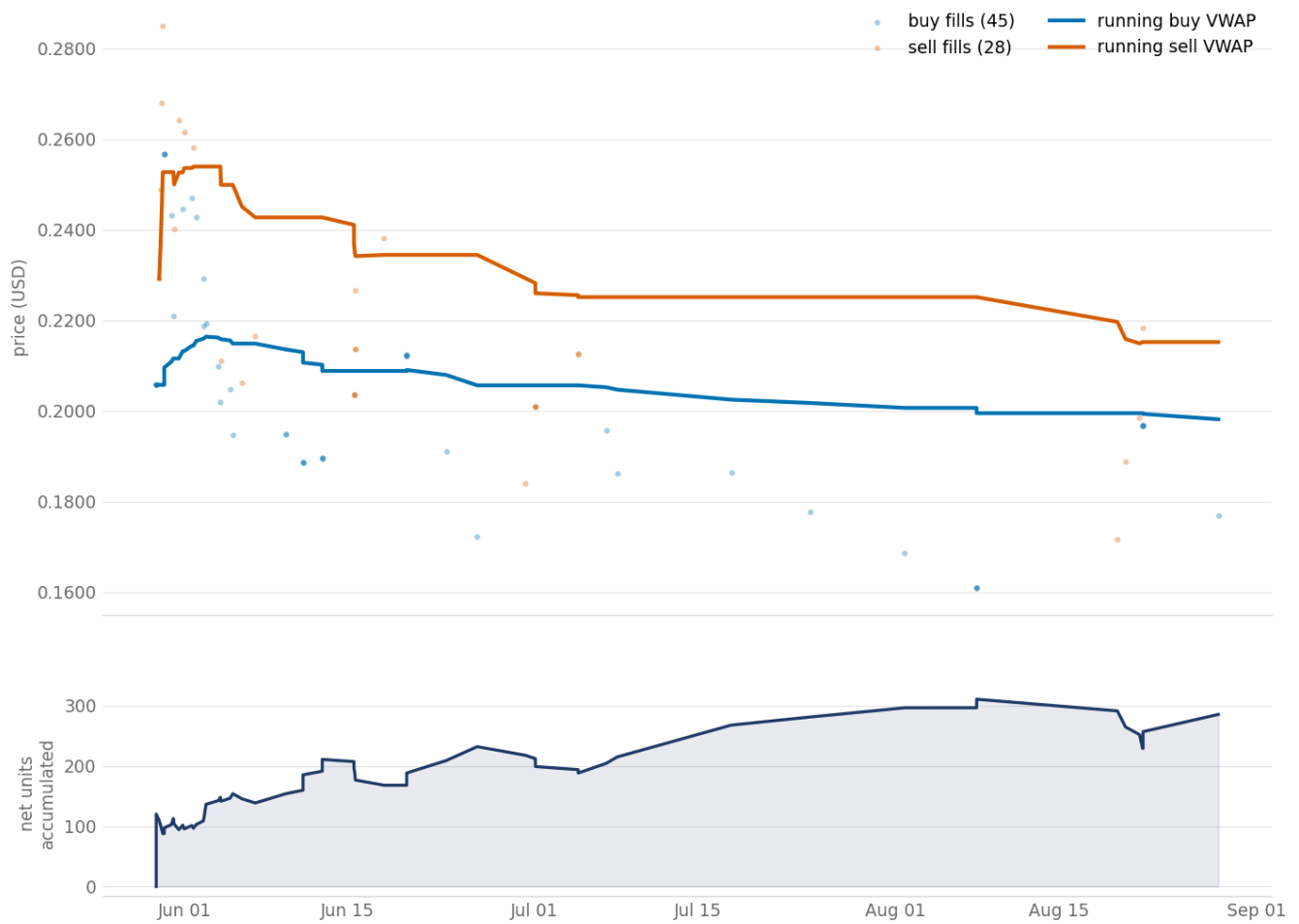


RE, 73 fills, with the sell line above the buy line from late July.

The sell line jumps from 0.42 to 0.52 in the last week of July, then settles near 0.50 against a buy line near 0.476.

XLM

XLM — VWAP trajectory across 73 fills (45 buy, 28 sell)

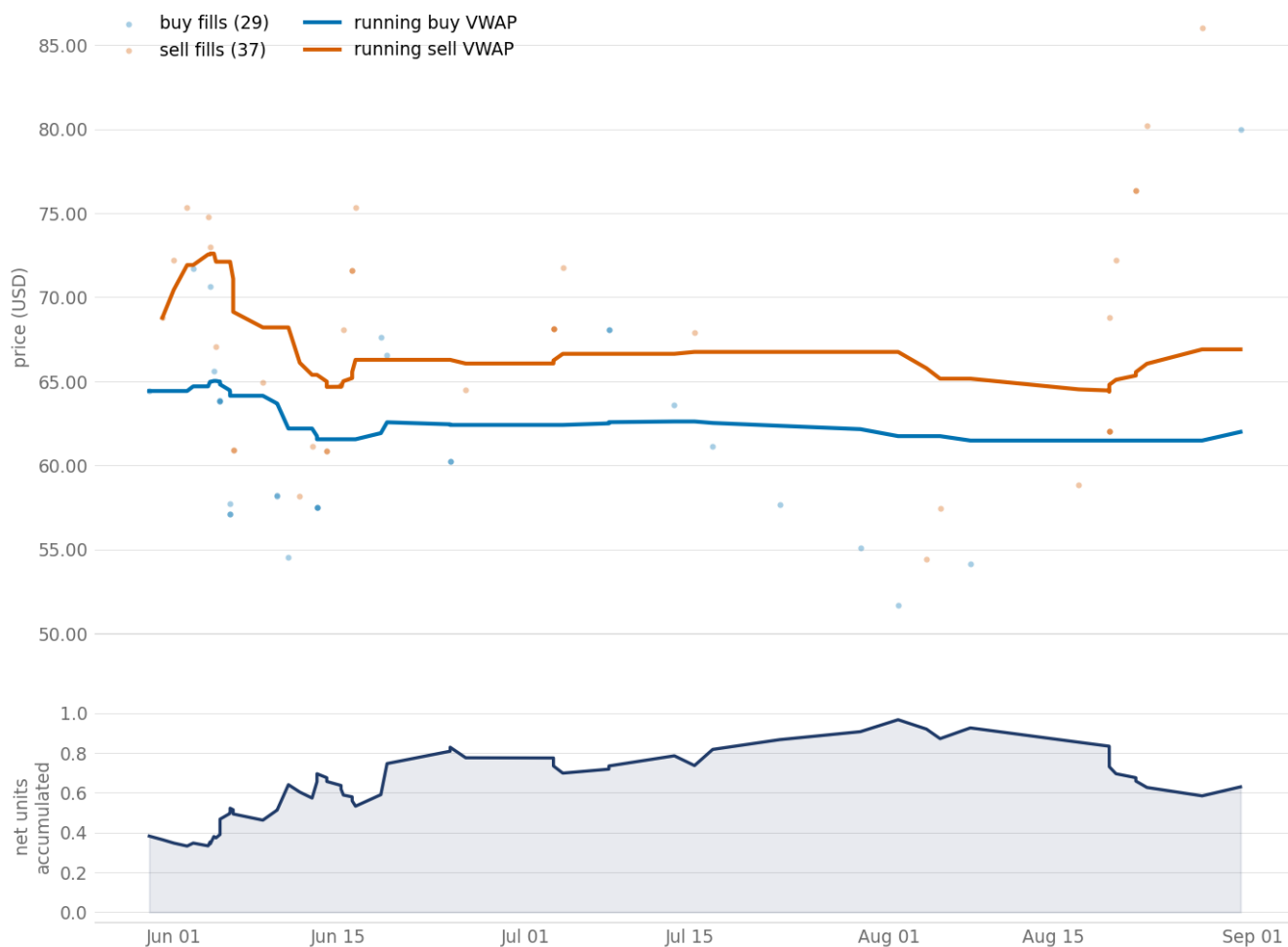


XLM, 73 fills, with the sell line about 0.02 above the buy line throughout.

The gap between the two lines holds across the whole window, and both drift down. Net units grow to 311 and end at 285.

HYPE

HYPE — VWAP trajectory across 66 fills (29 buy, 37 sell)

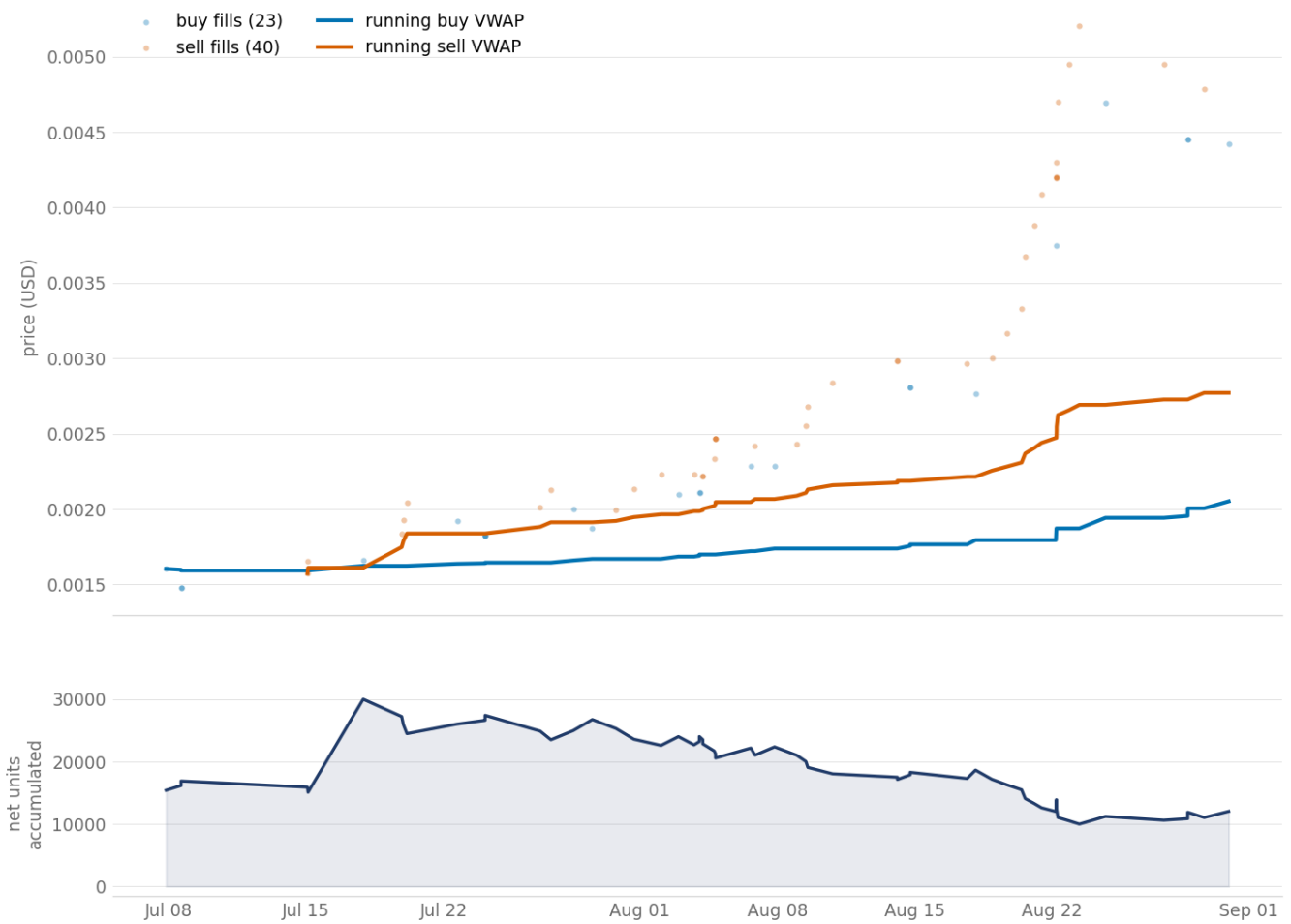


HYPE, 66 fills, with more sells than buys.

29 buys against 37 sells. The sell line peaks near 72.6 in early June and ends near 67, and the buy line holds between 61 and 65 across the window.

PUMP

PUMP — VWAP trajectory across 63 fills (23 buy, 40 sell)

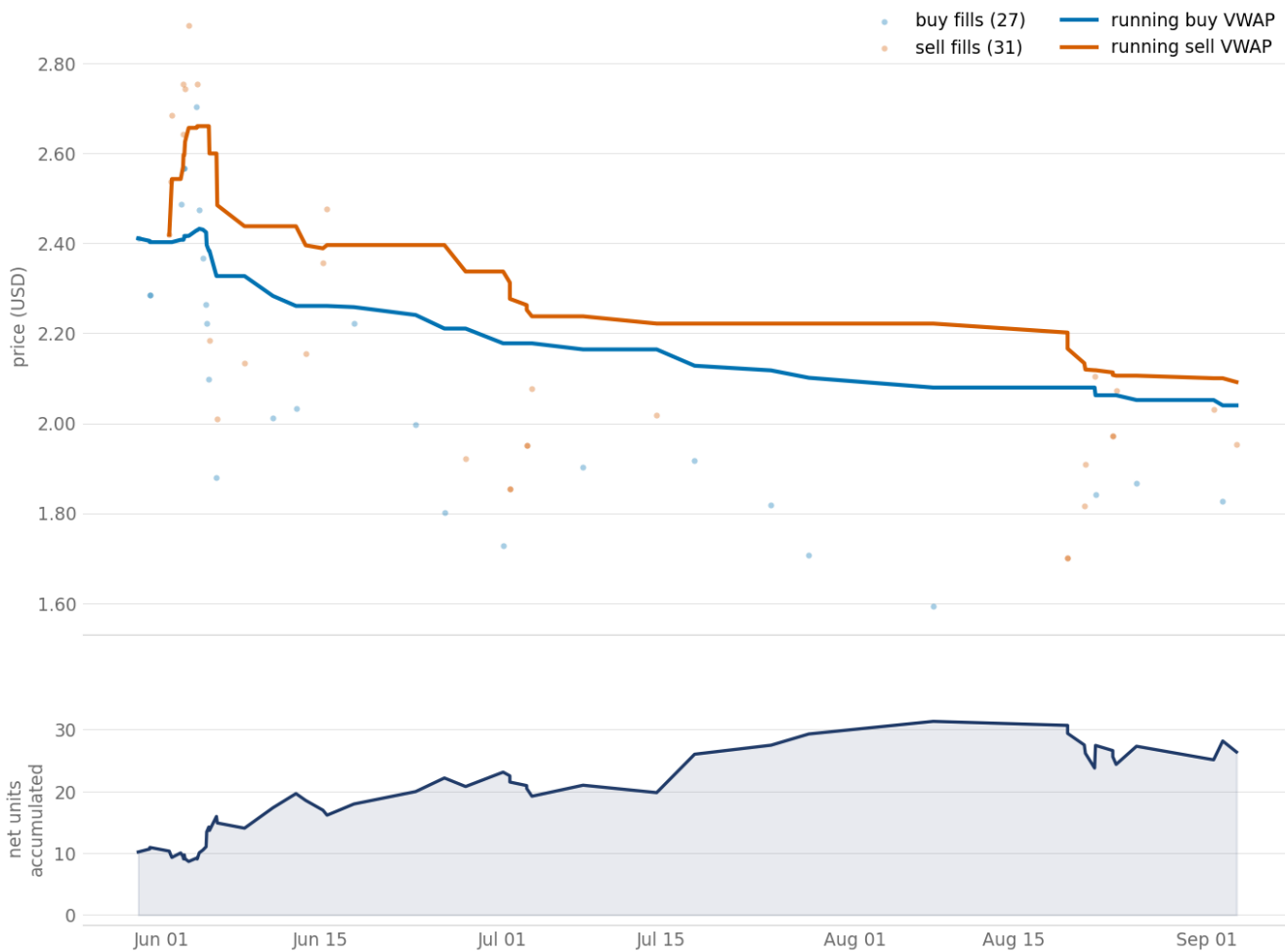


PUMP, 63 fills, with both lines rising through August.

23 buys against 40 sells. The buy line rises from 0.001603 to 0.002051, and the sell line rises faster, ending about 0.0007 above it. Net units drop across the window, 29,987 down to 12,032.

NEAR

NEAR — VWAP trajectory across 58 fills (27 buy, 31 sell)

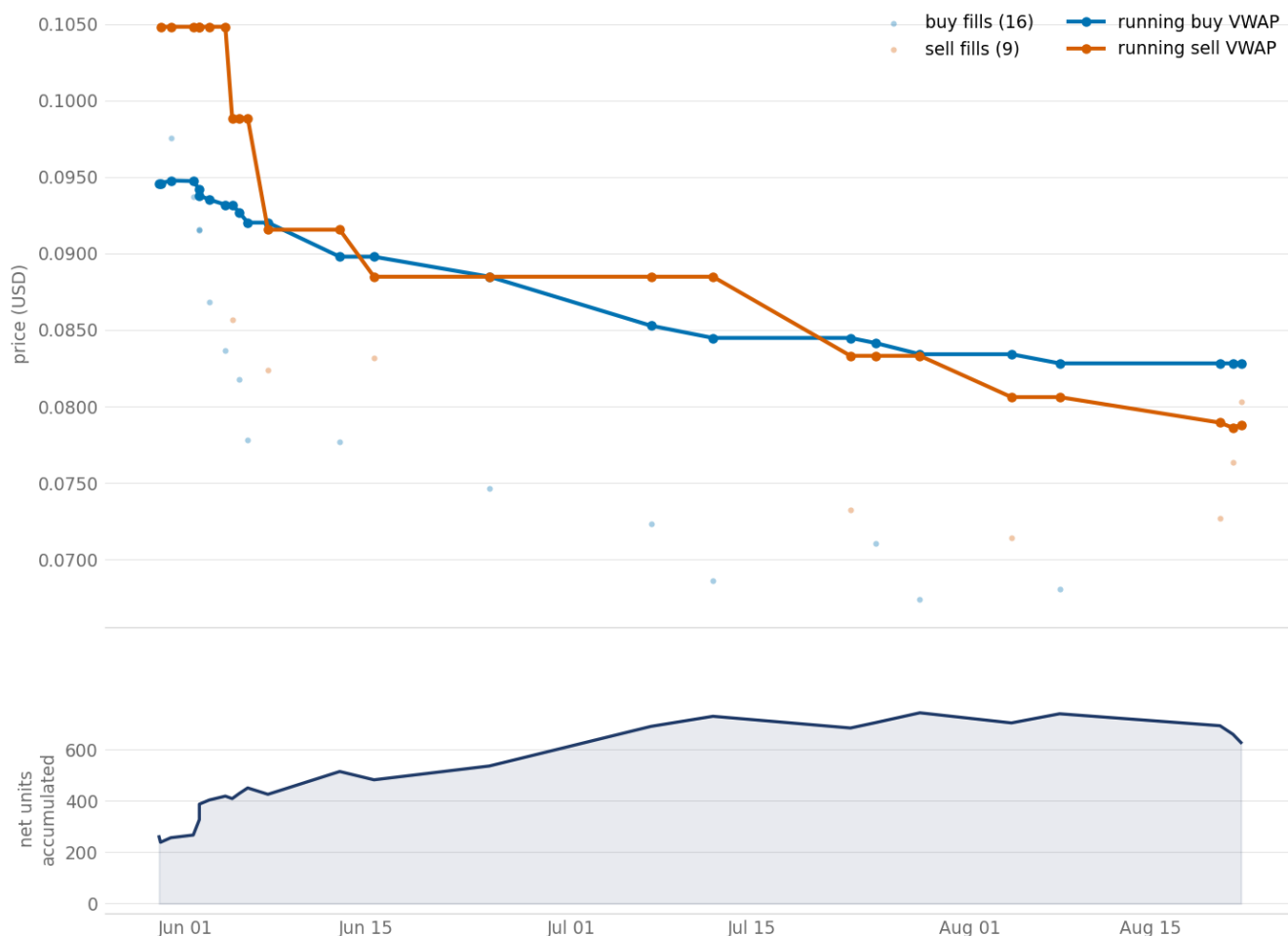


NEAR, 58 fills, with both lines falling steadily.

The sell line runs about 0.1 above the buy line from June to late August, then the two converge near 2.05. Net units end at 26.

HBAR

HBAR — VWAP trajectory across 25 fills (16 buy, 9 sell)



HBAR, 25 fills, thin enough for the two lines to carry a dot per step.

The first of the eight thin charts. The sell line starts at 0.1048, steps down through June and July, and crosses below the buy line in late July. Net units end at 626.

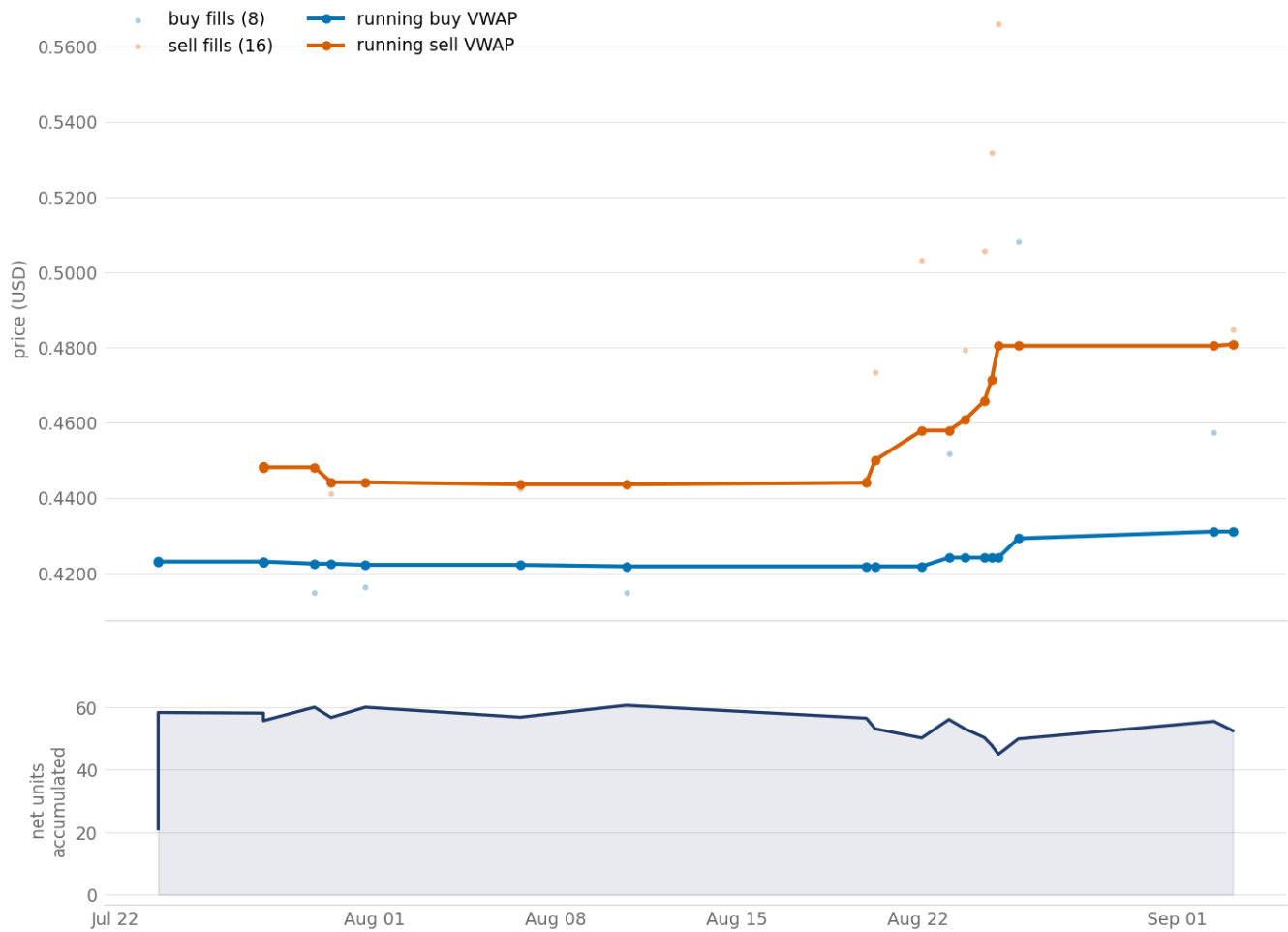
25 fills is under the module's threshold of 30, so both lines carry a marker at every step. Sixteen dots sit on the buy line and nine on the sell line. The seven charts after this one are drawn the same way.

```
THIN_FILLS = 30
```

```
dot = "o" if len(fills) <= THIN_FILLS else None
ax.plot(t, buy_v, color=COLORS["series"][0], lw=2.0, marker=dot,
        markersize=4, label="running buy VWAP")
```

AERO

AERO — VWAP trajectory across 24 fills (8 buy, 16 sell)

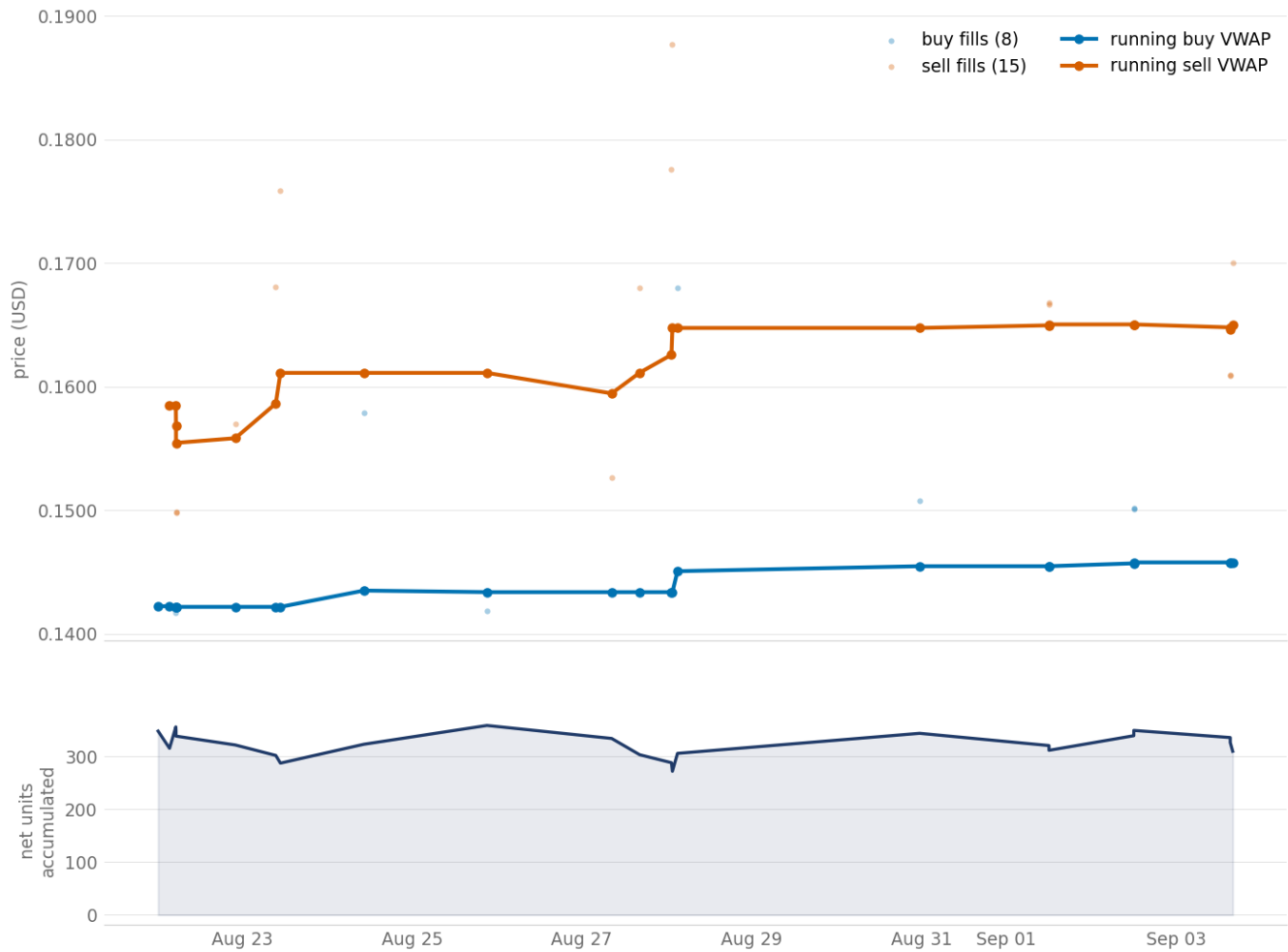


AERO, 24 fills, with eight buys and sixteen sells.

A thin chart. The buy line holds near 0.4225 until late August, then rises to 0.4310. The sell line jumps from 0.444 to 0.481 in the same week.

ENA

ENA — VWAP trajectory across 23 fills (8 buy, 15 sell)

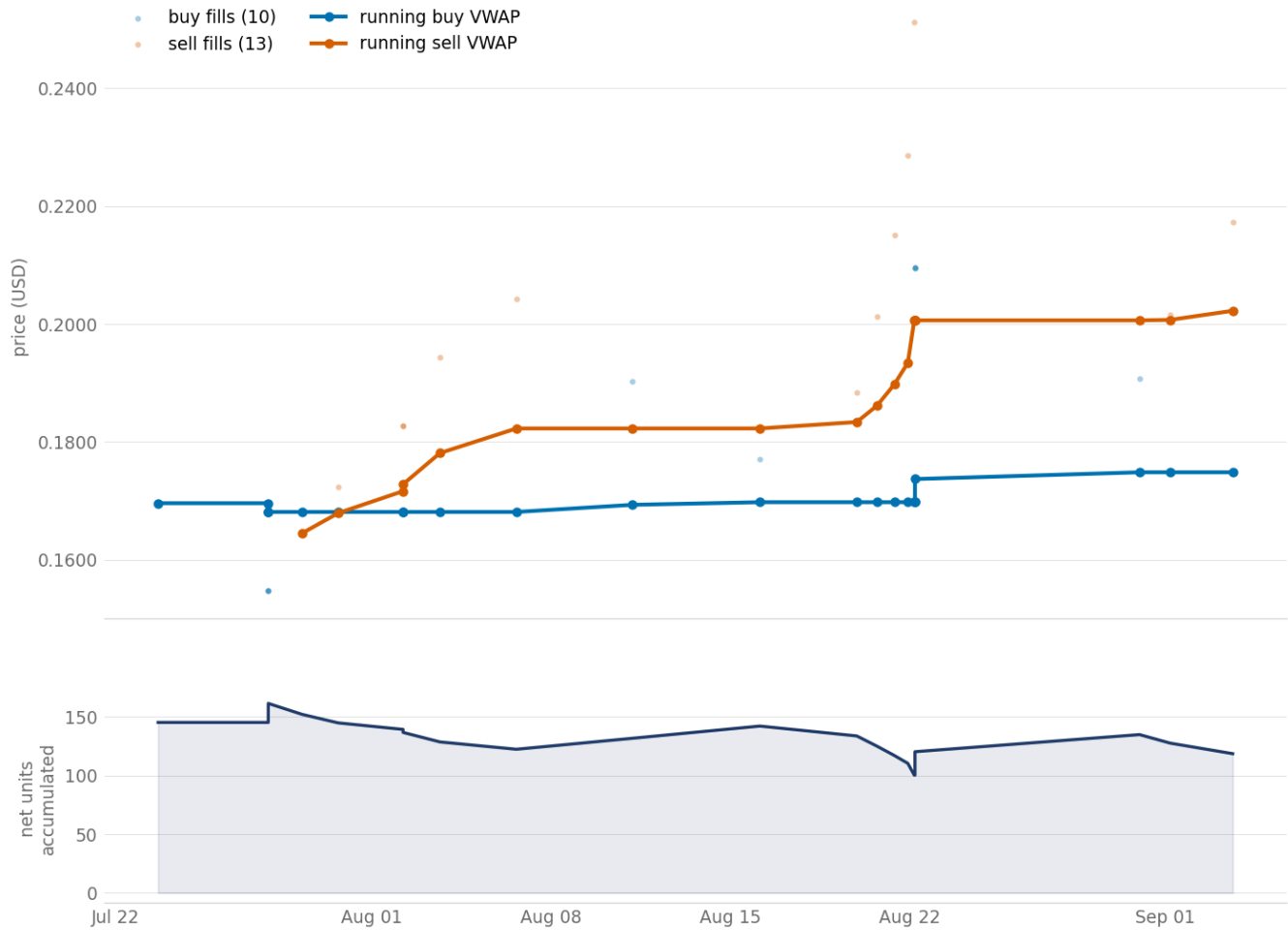


ENA, 23 fills across 12 days, the shortest window of the 38.

A thin chart, and the shortest window here. Eight buys against fifteen sells. Both lines step up once, on 28 August.

ADA

ADA — VWAP trajectory across 23 fills (10 buy, 13 sell)

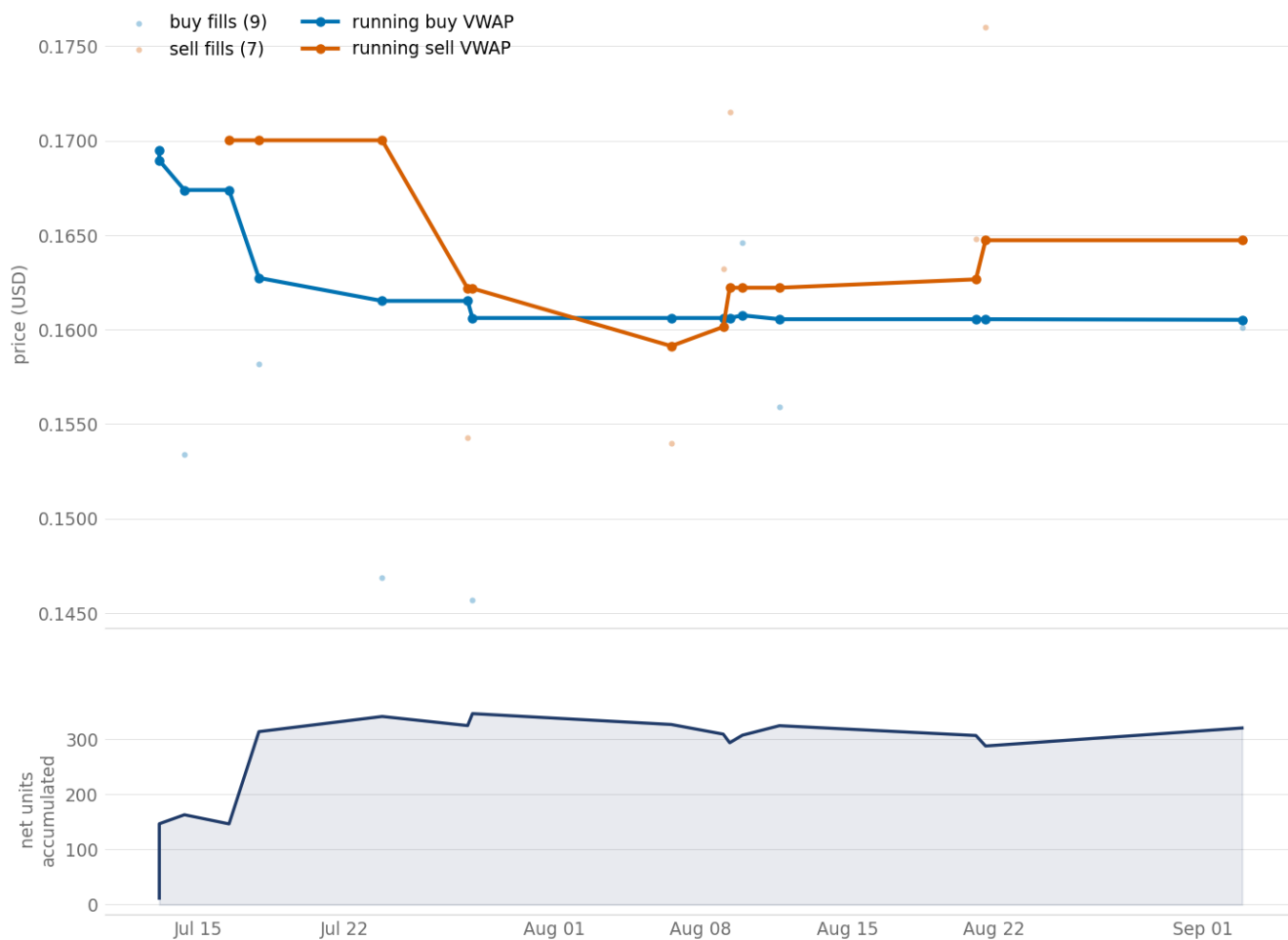


ADA, 23 fills, with the sell line stepping up sharply on 22 August.

A thin chart. The sell line climbs from 0.1645 to about 0.202, while the buy line moves from 0.1696 to 0.1748 across the same six weeks.

AGLD

AGLD — VWAP trajectory across 16 fills (9 buy, 7 sell)

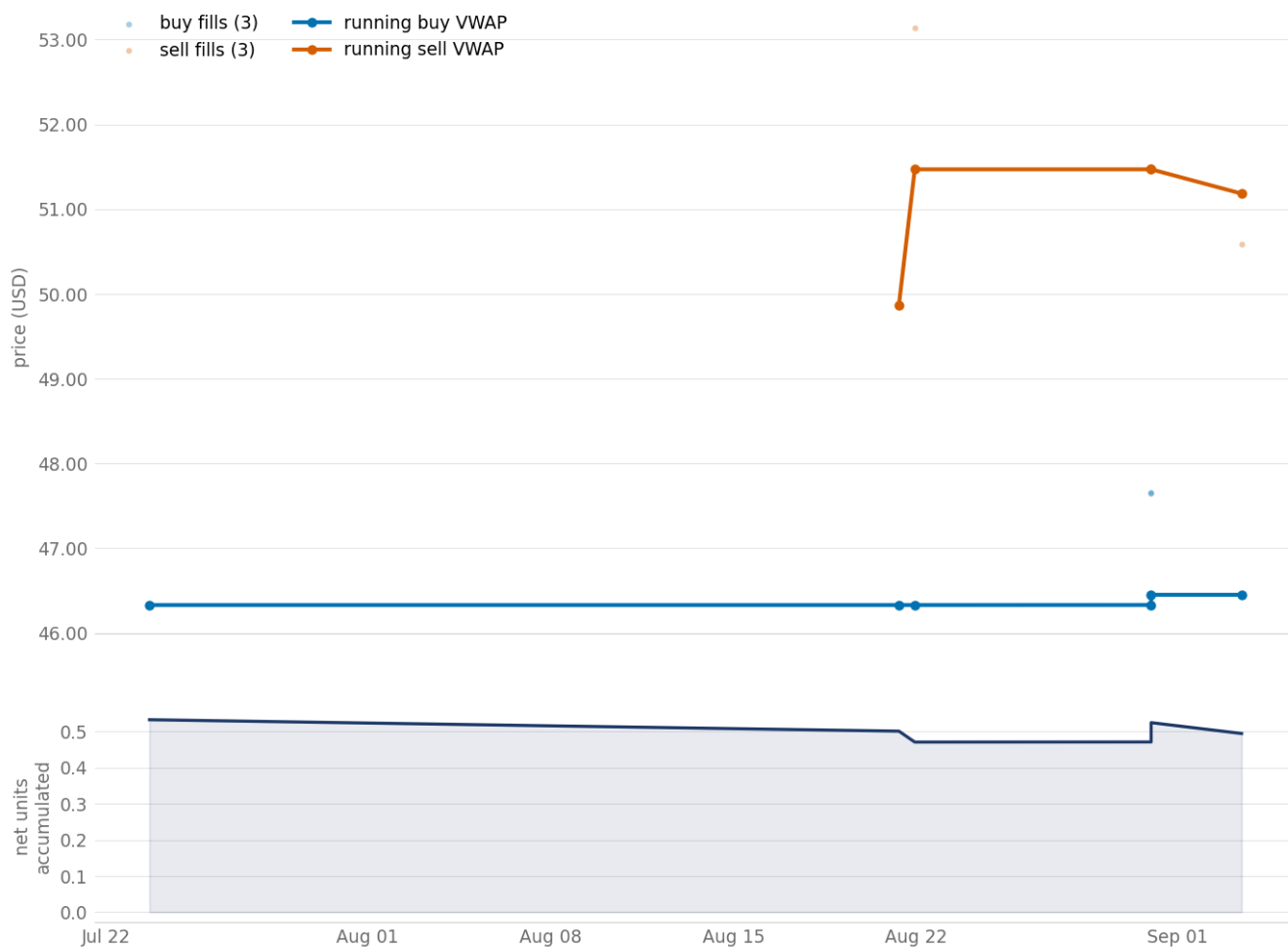


AGLD, 16 fills, with the buy line flat from late July.

A thin chart. The buy line settles at 0.1605 in late July and holds. The sell line dips to 0.159 in early August, then steps back to 0.1648.

LTC

LTC — VWAP trajectory across 6 fills (3 buy, 3 sell)

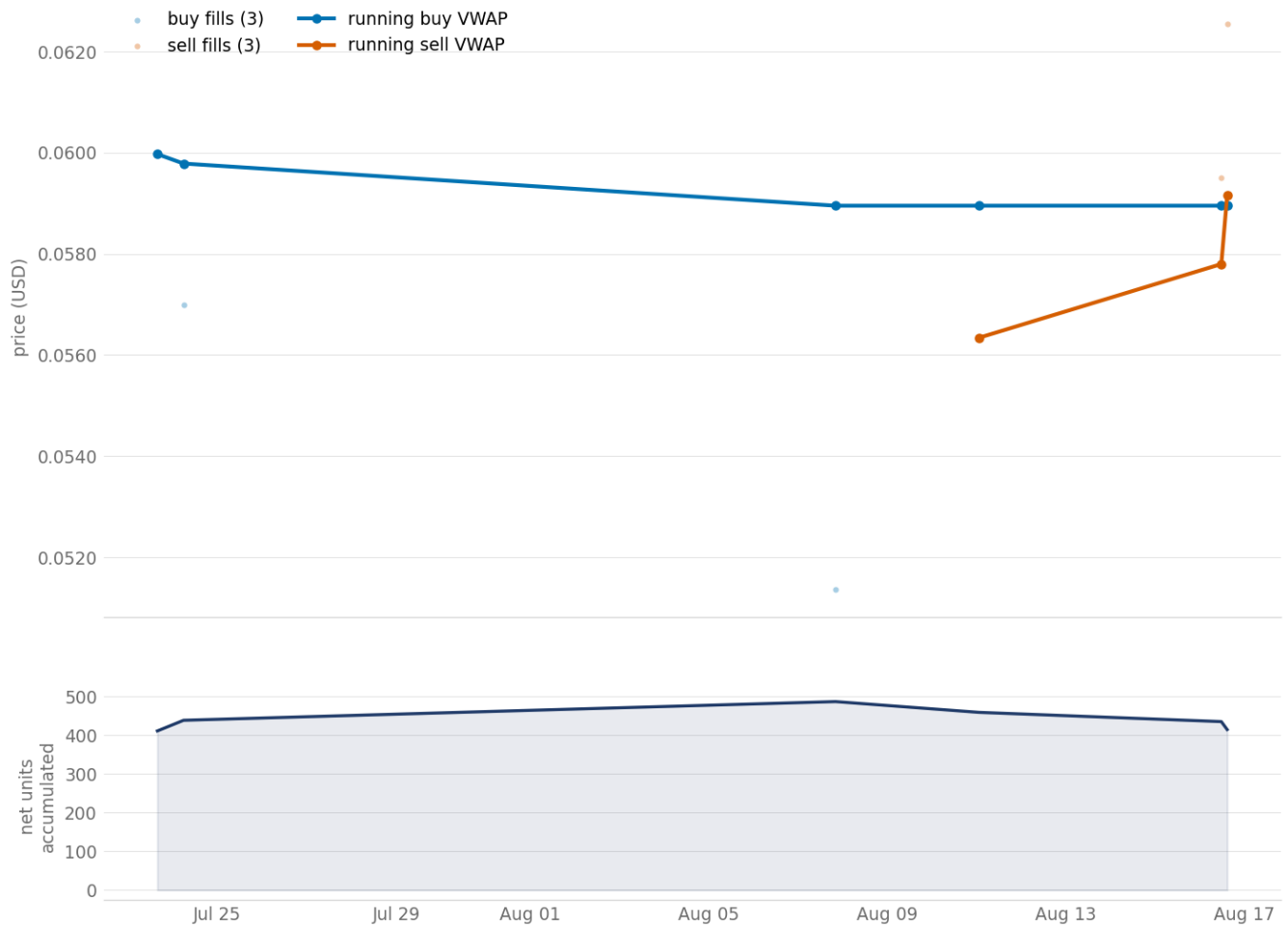


LTC, 6 fills, three buys and three sells.

The buy line is nearly flat at 46.33, since all three buys land near one price. The sell line runs about 5 dollars above it. Net units end at 0.49.

WLFI

WLFI — VWAP trajectory across 6 fills (3 buy, 3 sell)

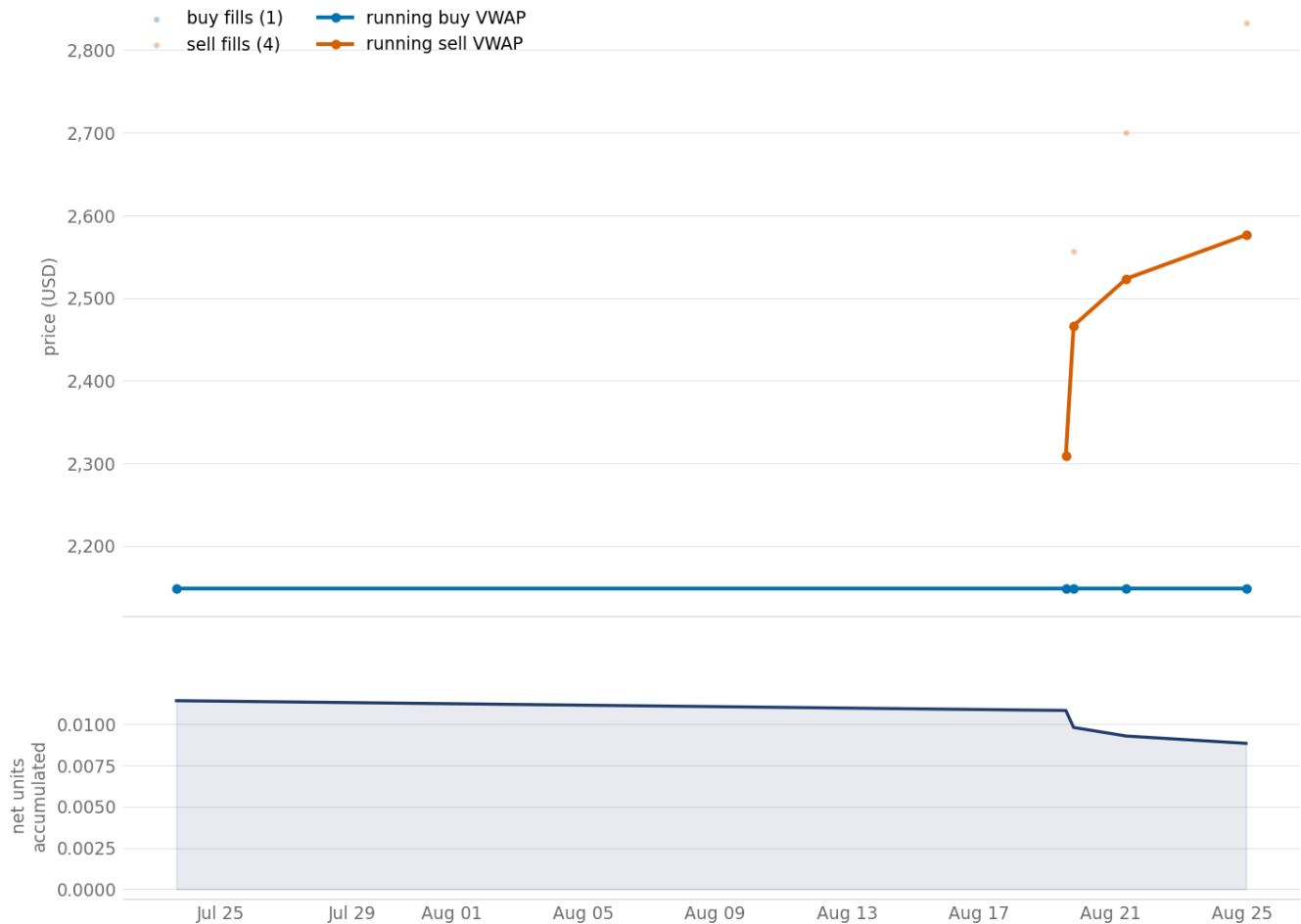


WLFI, 6 fills, closing 16 August.

The earliest close of the 38. Three buys and three sells, all inside four weeks. The buy line drifts from 0.0600 to 0.0590.

LSETH

LSETH — VWAP trajectory across 5 fills (1 buy, 4 sell)



LSETH, 5 fills, one buy and four sells.

The only base in the record with a single buy. One buy makes the buy line a single horizontal line at 2,148.63 and its index exactly 1.000. The sell line rises across the four sells, 2,307 up to 2,577. Net units drop across the window, 0.0114 down to 0.0088.

Trade grading

One function in `src/trading/trade_grader.py` grades a completed fill, on four axes and no more. Execution scores against a reference price at the decision, timing across the prices after the trade, strategy across the shift in the rolling S/B figure, and outcome against realised profit per unit. An axis whose price inputs are absent is left out, and the overall figure is the unweighted mean of the axes that scored — 0.5 when none of them could.


```
def grade_trade(record: TradeRecord, ctx: PriceContext) -> TradeGrade: ...

def _score_execution(...): ...
def _score_timing(...): ...
def _score_strategic(...): ...
def _score_outcome(...): ...
```

A second function maps that mean to a letter.

```
def _letter_from_numeric(num: float) -> str:
    if num >= 0.93:
        return "A+"
    elif num >= 0.85:
        return "A"
    elif num >= 0.70:
        return "B"
    elif num >= 0.55:
        return "C"
    elif num >= 0.40:
        return "D"
    else:
        return "F"
```

The market regime rides beside the grade rather than inside it. `PriceContext.regime_tag` reaches `TradeGrade.regime` and the rationale text, and it carries no sub-score of its own, which is why a rationale can read five terms while the grade rests on four.

`grade_trades` runs the batch.

The screen these grades reach is in the History tab reference.

Gate logs behind each trade

A fill says what happened. The gate log says which decision produced it.

`gate_light_row` in `src/trading/gate_vocabulary.py` returns nineteen lights for one decision, the scrum bank first and the fold bank second, each entry carrying its bank, label, state and colour. The History table and the Simulator draw their lights from that one function, so both surfaces name the same gates in the same order. The labels themselves, and the indicator readings behind them, are in the indicator reference.

`src/trading/gate_coverage.py` pairs each fill with the nearest gate entry for the same bot and gives it one status out of six. The tolerance is 300 seconds, one five-minute candle either side of the fill, and a fill that finds an entry inside that window has a gate. The reported coverage counts only those.

```

DEFAULT_TOLERANCE_S: float = 300.0
LOG_GAP_THRESHOLD_S: float = 1800.0

class GateStatus:
    HAS_GATE = "has_gate"
    BEFORE_LOGGING = "before_logging"
    LOG_GAP = "log_gap"
    NO_GATE_FOR_BOT = "no_gate_for_bot"
    NO_GATE_DATA = "no_gate_data"
    NO_GATE_IN_TOLERANCE = "no_gate_in_tolerance"

```

`classify_trades` returns the pairings and `GateCoverageReport.coverage_pct` gives the percentage.

The five statuses that are not a gate each name a different reason a fill has none. A fill the log could not have seen and a fill the log should have seen are different findings, and the report keeps them apart rather than folding both into one coverage shortfall.

Status	What it marks
before_logging	a fill older than the first gate entry
log_gap	a fill inside a silence longer than 1,800 seconds, six candles
no_gate_in_tolerance	a bot logging either side of the fill but not inside the window
no_gate_for_bot	a bot with no entries at all
no_gate_data	a run given no gate log

`format_coverage_lines` renders the counts, one line per status.